Cloud Practitioner

Contents

Traditional IT Overview 8
What is Cloud Computing 12
The Different Types of Cloud Computing 17
Shared Responsibility Model & AWS Acceptable Policy 22
IAM – Identity and Access Management 24
IAM Introduction: Users, Group, Polices 24
IAM Policies 26
IAM MFA Overview 28
AWS Access Key, CLI and SDK 31
IAM Roles for AWS Services 34
IAM Security Tools 36
Shared Responsibility Model for IAM 37
EC2 – Elastic Compute Cloud 39
AWS Budget Setup 39
EC2 Basics 41
EC2 Instance Type Basics 43
Security Groups & Classic Ports Overview 45
SSH Overview 47
EC2 Instance Connect 49
EC2 Instance Roles 51
EC2 Purchasing Options 53
Shared Responsibility Model – EC2 57
EC2 Summary 58
EC2 Instance Storage 60
EBS Overview 60
EBS Snapshot Overview 61
AMI Overview 62
EC2 Image Builder Overview 63
EC2 Instance Store 64
EFS Overview 65
Shared Responsibility Model for EC2 Storage 67

Cloud Practitioner

Amazon Fsx Overview 68
EC2 Instance Storage Summary 70
ELB & ASG – Elastic Load Balancing & Auto Scaling Groups 72
High Availability, Scalability, Elasticity 72
Elastic Load Balancing Overview 74
Auto Scaling Group Overview 77
Auto Scaling Group Strategies 79
ELB & ACG Summary 81
Amazon S3 83
S3 Overview 83
S3 Security: Bucket Policy 86
S3 Website Overview 89
S3 Versioning Overview 91
S3 Replication Overview 93
S3 Storage Classes Overview 95
S3 Express One Zone 99
S3 Encryption 101
IAM Access Analyzer for S3 102
Shared Responsibility Model for S3 104
AWS Snow Family Overview 105
AWS Snowball Edge – Pricing 108
Storage Gateway Overview 110
S3 Summary 112
Databases & Analytics 115
Database Introduction 115
RDS & Aurora Overview 118
RDS Deployment Option 125
ElastiCache Overview 129
DynamoDB Overview 133
DynamoDB Global Tables 139
Redshift Overview 140
EMR Overview 143
Athena Overview 145

Quicksight Overview 147
DocumentDB Overview 149
Neptune Overview 151
Timestream Overview 153
Managed Blockchain Overview 155
Glue Overview 157
DMS Overview 159
Databases & Analytics Summary 161
Other Compute Services: ECS, Lambda, Batch, LightSail 165
What is Docker? 165
ECS, Fargate & ECR Overview 167
Amazon EKS 169
Serverless Introduction 171
Lambda Overview 173
API Gateway Overview 176
Batch Overview 178
Lightsail Overview 181
Other Compute – Summary 183
Deployment & Managing Infrastructure at Scale 186
CloudFormation Overview 186
CDK Overview 189
Beanstalk Overview 192
Code Deploy Overview 198
Code Commit Overview 201
Code Pipeline Overview 208
Code Artifact Overview 213
System Manager Overview 217
SSM Manager Overview 222
SSM Parameter Store 230
Deployment Summary 236
Leveraging the AWS Global Infrastructure 244
Why Global Applications? 244
Route 53 Overview 250

CloudFront Overview 258
S3 Transfer Acceleration 267
AWS Global Accelerator 272
AWS Outposts 279
AWS Wavelength 286
AWS Local Zone 293
Global Application Architecture 298
Leveraging the AWS Global Infrastructure Summary 304
Cloud Integrations 310
Cloud Integrations Overview 310
SQS Overview 313
Kinesis Overview 315
SNS Overview 318
Amazon MQ Overview 322
Cloud Integration Summary 326
Cloud Monitoring 330
CloudWatch Metrics & CloudWatch Alarms Overview 330
CloudWatch Log Overview 337
EventBridge Overview 342
CloudTrail Overview 349
X-Ray Overview 354
AWS Health Dashboard 360
Cloud Monitoring Summary 364
VPC & Networking 368
VPC Overview 368
IP Address in AWS 370
VPC, Subnet, Internet Gateway & NAT Gateways 373
Security Groups & Network Access Control List (NACL) 378
VPC Flow Logs & VPC Peering 382
VPC Endpoints – Interface & Gateway (S3 & DynamoDB) 386
Private Link 390
Direct Connect & Site-to-site VPN 394
Client VPN 397

Transit Gateway Overview 400
VPC & Networking Summary 403
Security & Compliance 409
Shared Responsibility Model: Remainders & Examples 409
DDoS Protection: WAF & Shield 412
AWS Network Firewall 418
AWS Firewall Manager 421
Penetration Testing 424
Encryption with KMS and CloudHSM 428
AWS Certificate Manager 434
Secret Manager Overview 438
Artifact Overview 442
GuardDuty Overview 445
Inspector Overview 452
Config Overview 458
Macie Overview 465
Security Hub Overview 469
Amazon Detective Overview 476
Amazon Abuse 481
Root User Privileges 484
IAM Access Analyzer 488
Security & Compliance Summary 494
Machine Learning 496
Recognition Overview 496
Transcribe Overview 498
Polly Overview 501
Translate Overview 503
Lex & Connect Overview 505
Comprehend Overview 508
SageMaker AI overview 512
Kendra Overview 515
Personalize Overview 518
Textract Overview 522

Machine Learning Summary 526
Account Management, Billing & Support 531
Organization Overview 531
Organizations Consolidated Billing 536
AWS Control Tower Overview 538
AWS Resource Access Manager 541
AWS Service Catalog – Overview 544
Pricing Models of the Cloud 548
Saving Plan Overview 558
Compute Optimizer Overview 564
Billing & Costing Tool Overview 568
Estimate Costs in the Cloud – Pricing Calculator 573
Tracking Costs in the Cloud – Billing Dashboard, Cost Allocation Tags, Reports 577
Monitoring Cost in the Cloud – Billing Alarm & AWS Budgets 584
AWS Cost Anomaly Detection 590
AWS Service Quotas 595
AWS Trusted Advisor 599
Support Plans for AWS 605
Account Best Practices Summary 612
Billing Summary 613
Advanced Utility 615
Security Token Services (STS) Overview 615
Cognito Overview 617
Directory Services Overview 620
AWS IAM Identity Center 623
Advanced Identity Summary 626
Other Services 629
Other Servies - Section Intro 629
Workspaces Overview 630
AppStream 2.0 Overview 632
IoT Core Overview 635
AppSync 638
Amplify 641

AWS Infrastructure Composer 644
Device Farm Overview 647
AWS Backup Overview 650
Disaster Recovery Strategies 653
The exam mainly expects you to know the **cost order** and **basic characteristics** of each DR strategy. 653
AWS Elastic Disaster Recovery (RDS) 655
AWS Data Sync 659
Cloud Migration Strategies – The 7Rs 663
Application Discovery Service & Application Migration Service 669
AWS Migration Evaluator 675
AWS Migration Hub 679
AWS Fault Injection Simulator 683
AWS Step Function 688
AWS Ground Station 694
AWS Pinpoint 697
AWS Architecture & Ecosystem 703
AWS Whitepapers Well-Architected Framework 703
Pillar 1: Operational Excellence 706
Pillar 2: Security 710
Pillar 3: Reliability 716
Pillar 4: Performance Efficiency 723
Pillar 5: Cost Optimization 729
Pillar 6: Sustainability 735
AWS Well-Architected Tool 741
AWS Customer Carbon Footprint Tool 745
AWS Cloud Adoption Framework (CAF) 749
Right Sizing 757
AWS Ecosystem 761
AWS re:Post 769
AWS Knowledge Center 773
AWS Managed Services (AMS) 776

Cloud Practitioner

What is Cloud Computing?

Traditional IT Overview

1. What is Cloud & Cloud Computing

- Cloud computing provides **computing, storage, and networking resources on demand**
- No hands-on initially understanding **concepts and context** is important
- Cloud helps solve problems of **traditional IT infrastructure**

2. How Websites Work

- A client (browser) requests data from a server
- Communication happens over a network
- Data is transmitted as packets
- Server processes the request and sends a response back to the client
- Browser displays the website
- This is a client–server architecture

3. Role of IP Addresses

- Both **clients and servers have IP addresses**
- IP addresses help:
    - Client locates the server
    - Server locate the client to send response
- IP address = **unique identifier on the network**

4. Networking Analogy

- Sending data is like **sending a letter by post**
    - Letter → Data
    - Address → IP address
    - Post office network → Internet
- The reply uses the **sender's return address**

This analogy helps understand **how the internet works**

5. What is Inside a Server?

A server contains multiple components:

a) Compute (CPU)

- Performs calculations and processing
- Like the **brain thinking**

b) Memory (RAM)

- Fast, temporary storage
- Used to quickly access active data
- Like **short-term memory**

c) Storage

- Long-term data storage
- Stores:
    - Files
    - Databases (structured data)

d) Networking

- Routers
- Switches
- DNS
- Enables communication with other systems

✅ Cloud provides **all these components as services**

6. Basic IT Terminology

Network

- Collection of:
    - Cables
    - Routers
    - Switches
    - Servers

Router

- Forwards data packets between networks
- Decides **where packets should go**

Switch

- Sends data to the **correct device within a network**

✅ Data flow:

Client → Router → Switch → Server

7. Traditional IT Infrastructure

Before clouds, companies:

- Bought physical servers
- Hosted them:
    - At home
    - In garages
    - Later in offices and data centers
- Manually added servers as demand increased

Example: Google started in a garage

8. Problems with Traditional IT

Major challenges:

1. **High Cost**
    - Rent
    - Electricity
    - Cooling
    - Maintenance
2. **Slow Scaling**
    - Servers must be ordered and installed
    - Not instant
3. **Limited Capacity**
    - Space and time constraints
    - Cannot handle sudden traffic spikes easily
4. **Operational Overhead**
    - 24/7 monitoring team required
5. **Risk of Disasters**
    - Fire
    - Power failure
    - Earthquakes
    - Hardware failure

9. Need for Cloud Computing

- Businesses want to externalize infrastructure
- Avoid managing:
  - Physical servers
  - Power
  - Cooling
  - Maintenance
- Focus on application and business, not infrastructure

✅ This need leads to Cloud Computing

Cloud Practitioner

What is Cloud Computing

1️⃣ What Is Cloud Computing?

- **Cloud computing** is the **on-demand delivery** of:
  - Compute power
  - Database storage
  - Applications
  - Other IT resources
- Delivered **over the internet**
- Uses a **pay-as-you-go pricing model**
- You pay:
  - Only for what you use
  - Only while you are using it

🔑 Key keyword for exam: **on-demand**

2️⃣ Key Features of Cloud Computing

- Resources are available **instantly**
- No advance notice required (no weeks/day to order servers)
- You can:
  - Choose exact size and type of resources
  - Scale up or down at any time
- Access resources via:
  - Web console (UI)
  - APIs
  - Management tools

Cloud Practitioner

3️⃣ Role of AWS (Cloud Provider)

- AWS **owns and maintains**:
  - Data centers
  - Servers
  - Networking hardware
- Customers:
  - Provision resources via a web interface
  - Do not manage physical infrastructure

✅ AWS handles infrastructure

✅ Customer handles usage and configuration

4️⃣ Traditional IT vs Cloud Computing

Traditional IT

- Build and manage own data center
- Buy servers in advance
- Pay for:
  - Space
  - Power
  - Cooling
  - Maintenance
- Limited scalability
- Time-consuming setup

Cloud Computing

- Use provider's data centers
- Provision servers when needed
- Pay only for usage
- Instant scalability

5️⃣ Real-Life Examples of Cloud Usage

- **Gmail** → Cloud email service
- **Dropbox / Google Drive / iCloud** → Cloud storage
- **Netflix** → Built entirely on AWS (video streaming)

✅ You use cloud services **without provisioning servers**

Cloud Practitioner

6️⃣ Types of Cloud Deployment Models

1. Private Cloud

- Used by **one organization**
- Not shared with public users
- More control and security
- Managed by third party (example: Rackspace)
- Used for sensitive or specialized workloads

---

2. Public Cloud ✅ (High exam importance)

- Owned and operated by third-party providers
- Resources shared among multiple customers
- Examples:
  - Amazon Web Services (AWS)
  - Microsoft Azure
  - Google Cloud
- Accessed over the internet
- Highly scalable and cost-effective

3. Hybrid Cloud ✅ (Exam important)

- Combination of:
  - On-premises infrastructure (private)
  - Public cloud (AWS)
- Sensitive data stays on-premises
- Use public cloud for flexibility & scalability

Cloud Practitioner

7️ Five Characteristics of Cloud Computing ✅ ✅

1. **On-Demand Self-Service**
    - Users provision resources without provider interaction
2. **Broad Network Access**
    - Accessible via network
    - Supports multiple devices and interfaces
3. **Resource Pooling (Multi-Tenancy)**
    - Multiple customers share infrastructure
    - Security and isolation maintained
4. **Rapid Elasticity & Scalability**
    - Scale resources up or down quickly
    - Automatic scaling based on demand
5. **Measured Service**
    - Usage is monitored and metered
    - Pay exactly for what you use
    -

8️ Six Advantages of Cloud Computing ✅ ✅ ✅

1. **Trade CAPEX for OPEX**
    - No upfront hardware purchase
    - Pay as you go
2. **Benefit from Economies of Scale**
    - Lower prices due to massive usage by many customers
3. **Stop Guessing Capacity**
    - Scale automatically based on real demand
4. **Increase Speed and Agility**
    - Launch resources in minutes
5. **Stop Spending on Data Centers**
    - No cost for power, cooling, or maintenance
6. **Go Global in Minutes**
    - Deploy applications worldwide using AWS infrastructure

Cloud Practitioner

Business Problems Solved by Cloud

- ✅ Flexibility
- ✅ Cost effectiveness
- ✅ Scalability
- ✅ Elasticity (scale in & out)
- ✅ High availability
- ✅ Fault tolerance
- ✅ Faster development & deployment

Cloud Practitioner

The Different Types of Cloud Computing

1️⃣ Infrastructure as a Service (IaaS)

Definition

- **IaaS provides the basic building blocks of cloud IT**
- Offers resources in **raw form**

What It Provides

- Networking
- Virtual machines (compute)
- Storage

Customer Responsibility

- Applications
- Data
- Runtime
- Middleware
- Operating system

Key Characteristics

- Highest level of **flexibility & control**
- Closest to **traditional on-premises IT**
- Easy migration from on-prem to cloud

AWS Example

- ✅ **Amazon EC2**

Other Providers

- Google Cloud
- Microsoft Azure
- Rackspace
- DigitalOcean
- Linode

🔑 Exam keyword: **“Most control for customer”**

2️⃣ Platform as a Service (PaaS)

Definition

- Removes the need to manage infrastructure
- Focus only on **application development and deployment**

What AWS Manages

- Runtime
- Middleware
- Operating system
- Virtualization
- Servers
- Storage
- Networking

What Customer Manages

- Application
- Data

Key Characteristics

- Less operational overhead
- Faster development
- No infrastructure management

AWS Example

- ✅ **Elastic Beanstalk**

Other Providers

- Heroku
- Google App Engine
- Microsoft Azure (App Services)

🔑 Exam keyword: **“Focus on application, not infrastructure”**

3️⃣ Software as a Service (SaaS)

Definition

- **Fully managed product**
- Offered directly to end users

What Provider Manages

- Everything:
  - Application
  - Data
  - Runtime
  - Middleware
  - OS
  - Infrastructure
  - Networking

Customer Responsibility

- ONLY usage and configuration

AWS Examples

- Amazon Rekognition
- Many managed AWS services

Real-World Examples

- Gmail
- Dropbox
- Zoom

🔑 Exam keyword: **“No infrastructure or application management”**

4 Responsibility Comparison (Very Important for Exam)

On-Premises

Customer manages:

- Applications
- Data
- Runtime
- Middleware
- OS
- Virtualization
- Servers
- Storage
- Networking

IaaS

Customer manages:

- Applications
- Data
- Runtime
- Middleware
- OS

Provider manages:

- Virtualization
- Servers
- Storage
- Networking

PaaS

Customer manages:

- Applications
- Data

Provider manages:

- Everything else

SaaS

Provider manages:

- **Everything**

Cloud Practitioner

5️⃣ Cloud Pricing Fundamentals ✅✅✅

Cloud pricing is very different from traditional IT.

AWS Pricing Model: Pay-As-You-Go

1. Compute

- Pay for **exact compute time used**
- Example: EC2 running hours, Lambda execution time

2. Storage

- Pay for **actual data stored**
- Example: GB per month in S3

3. Networking

- Pay only for **data leaving the cloud**
- ✅ Data **into** AWS is FREE
- ❌ Data **out** of AWS is CHARGED

6️⃣ Key Benefits of Cloud Pricing

- No upfront hardware cost
- No wasted capacity
- Pay only for what you use
- Massive cost savings compared to traditional IT

Cloud Practitioner

Shared Responsibility Model & AWS Acceptable Policy

AWS Shared Responsibility Model (Important Concept)

- AWS uses a **Shared Responsibility Model**
- Defines **what the customer is responsible for** vs **what AWS is responsible for**
- Very important for **AWS Certified Cloud Practitioner exam**

👤 **Customer Responsibility: *Security IN the Cloud***

You (customer) are responsible for:

- Security of what you configure and use in AWS
- Your **data**
- **Operating systems** (patching, updates)
- **Network configuration**
- **Firewall rules**
- Access control and permissions
- Application-level security

🔑 Keyword: **Security in the cloud**

☁️ **AWS Responsibility: *Security OF the Cloud***

AWS is responsible for:

- Physical data centers
- Hardware (servers, storage, networking)
- Underlying infrastructure
- AWS-managed software
- Physical and environmental security
- Internal AWS security controls

🔑 Keyword: **Security of the cloud**

🤝 **Why It's Called "Shared" Responsibility**

- Security responsibility is **split** between:
  - AWS (infrastructure)
  - Customer (configuration and usage)
- Responsibilities depend on the **AWS service used** (IaaS, PaaS, SaaS)

📝 **Exam Relevance**

- Exam questions often ask:
  - "Is this AWS's responsibility or the customer's?"
- Understanding this model helps answer **security-related questions correctly**

📜 **AWS Acceptable Use Policy (AUP)**

When using AWS, customers must:

- Follow AWS **Acceptable Use Policy**
- Must NOT:
  - Perform illegal activities
  - Use harmful or offensive content
  - Violate security policies
  - Abuse networks or systems
  - Send spam or message abuse

Cloud Practitioner

IAM – Identity and Access Management

IAM Introduction: Users, Group, Polices

🔐 **AWS IAM (Identity and Access Management)**

What is IAM?

- **IAM stands for Identity and Access Management**
- Used to **manage users, groups, and permissions**
- IAM is a **global service** (not region-specific)

👤 **Root User**

- Created **by default** when an AWS account is created
- Has **full access to everything**
- Best practice:
  - Use root user **only for initial setup**
  - Do **not** use it for daily tasks
  - Do **not** share root credentials

👥 **IAM Users**

- An **IAM user represents one person** in an organization
- Each user can log in and access AWS based on permissions
- Best practice: **Create individual users instead of sharing accounts**

👫 **IAM Groups**

- A **group is a collection of IAM users**
- Groups make permission management easier
- Policies attached to a group apply to **all users in that group**

Group Rules

- ✅ Groups can contain **users only**
- ❌ Groups **cannot contain other groups**
- ✅ A user can belong to **multiple groups**
- ✅ A user **does not have to belong to a group** (not recommended)

Cloud Practitioner

📝 **IAM Policies**

- Permissions are defined using **IAM policies**
- Policies are written in **JSON format**
- Policies define:
  - What actions are allowed or denied
  - On which AWS services
- Policies can be attached to:
  - Users
  - Groups
  - Roles

🔑 Policies define **who can do what in AWS**

🔒 **Principle of Least Privilege** ✅ ✅ ✅

- Very important AWS security concept
- Users should only have **permissions they need**
- Avoid giving full access unnecessarily
- Prevents:
  - Security risks
  - Accidental high costs
  - Misuse of AWS resources

⚠️ **Why IAM Is Critical**

- Prevents unauthorized access
- Controls costs and security
- Ensures safe use of AWS services

🎯 **Exam Focus Takeaways**

- IAM is **global**
- Never use root user for daily work
- Users = people
- Groups = collection of users
- Policies = permissions
- Follow **least privilege principle**

IAM Policies

1. IAM Groups and Policy Inheritance

Policies attached to a group are automatically applied to all users in that group.

- Example:
  - Developers group → Alice, Bob, Charles all inherit the same developer policy.
  - Operations group → David and Edward inherit a different operations policy.

2. Users Can Belong to Multiple Groups

- A user can be a member of more than one group.
- The user inherits all policies from all groups they belong to.
- Example:
  - Charles → Developers + Audit team → gets both policies.
  - David → Operations + Audit team → gets both policies.

3. Inline Policies

- Inline policies are policies attached directly to a user.
- They apply only to that specific user.
- A user:
  - May or may not belong to a group.
  - Can still have an inline policy.
- Example:
  - Fred does not belong to any group but can still have permission via an inline policy.

4. IAM Policy Evaluation (Combined Permissions)

- A user's final permission is the sum of all policies:
  - Group policies
  - Inline user policies
- AWS evaluates all applicable policies together.

IAM Policy Structure (High Level)

5. IAM Policies Are JSON Documents

- IAM policies are written in JSON format.
- You will frequently see and work with this structure in AWS.

6. Core Components of an IAM Policy

- Version
  - Specifies the policy language version.
  - Common value: "2012-10-17"
- Id (Optional)
  - Identifies the policy.
- Statement
  - One or more statements define permissions.

Statement Elements

Each statement can include:

- Sid (Optional)
  - Statement ID used for identification.
- Effect
  - Determines whether access is:
    - "Allow" or "Deny"
- Principal
  - Specifies who the policy applies to (user, role, account).
  - Often seen in resource-based policies.
- Action
  - List of AWS API actions that are allowed or denied.
  - Example: s3:GetObject, ec2:StartInstances
- Resource
  - Specifies which AWS resources the actions apply to.
  - Example: S3 bucket, EC2 instance, etc.
- Condition (Optional)
  - Define conditions under which the statement is applied.
  - Example: time, IP address, MFA, etc.

IAM MFA Overview

1. Why User Protection Is Important

- IAM users can perform critical actions in AWS:
  - Change configurations
  - Delete resources
  - Manage security
- If credentials are compromised, the **entire AWS account is at risk**.
- AWS provides **two main defense mechanisms**:
  - Password Policy
  - Multi-Factor Authentication (MFA)

Password Policy (First Line of Defense)

2. Purpose of a Password Policy

- Stronger passwords improve account security.
- Helps protect against **brute-force attacks**.

3. Password Policy Options in AWS

AWS allows you to define:

- **Minimum password length**
- **Character requirements**, such as:
  - Uppercase letters
  - Lowercase letters
  - Numbers
  - Special (non-alphanumeric) characters (e.g., ?, @)
- **Allow or deny users changing their own password**
- **Password expiration**
  - Force users to change password after a set time (e.g., every 90 days)
- **Password reuse prevention**
  - Prevent users from reusing old passwords

✅ Password policies apply **account-wide** and help enforce consistent security standards.

Multi-Factor Authentication (MFA) – Second Line of Defense

4. What Is MFA?

- MFA combines:
  - **Something you know** → password
  - **Something you have** → physical or virtual MFA device
- Provides **much stronger security than password alone**.

5. Why MFA Is Critical

- Even if a password is:
  - Stolen
  - Hacked
- The account **cannot be accessed without the MFA device**.
- AWS **strongly recommends MFA**, especially for:
  - Root account
  - Administrator IAM users

How MFA Works (Example)

- User (e.g., Alice) logs in using:
  - Her password
  - A temporary MFA code from her device
- Login succeeds **only if both are correct**.

MFA Device Options in AWS (Exam Important)

6. Virtual MFA Devices

- Software-based authentication apps
- Examples:
  - **Google Authenticator** (single device)
  - **Authy** (supports multiple tokens on one device)
- Advantages:
  - Easy to use
  - Supports multiple IAM users and AWS accounts
  - Most commonly used option

7. U2F Security Key (Physical Device)

- Universal 2nd Factor (U2F)
- Example:
  - **YubiKey** by Yubico (third-party)
- Features:
  - Physical hardware device
  - Can support **multiple root and IAM users**
  - Convenient (keychain-style)

8. Hardware Key Fob MFA Devices

- Physical token device
- Example:
  - **Gemalto** hardware MFA key fob (third-party)
- Generates one-time passwords

9. MFA for AWS GovCloud (US)

- Special MFA device for AWS GovCloud (US) region
- Provided by:
  - **SurePassID** (third-party)

Cloud Practitioner

AWS Access Key, CLI and SDK

1. Three Ways to Access AWS

AWS provides **three main access methods**:

1. **AWS Management Console**
    - Web-based interface.
    - Accessed using:
        - Username & password
        - Optional Multi-Factor Authentication (MFA)
    - Most commonly used by beginners and for manual operations.
2. **AWS Command Line Interface (CLI)**
    - Used from a **terminal or command prompt**.
    - Protected using **Access Keys**.
    - Enables interaction with AWS services using commands.
    - Example commands start with aws, such as:
        - aws s3 cp
        - aws ec2 describe-instances
3. **AWS Software Development Kit (SDK)**
    - Used to access AWS **programmatically from application code**.
    - Uses the **same access keys** as the CLI.
    - Embedded inside applications to call AWS APIs.

Access Keys – Security Basics (Very Important)

2. What Are Access Keys?

- Access keys are **credentials** used by:
    - AWS CLI
    - AWS SDK
- Consist of two parts:
    - **Access Key ID** → like a username
    - **Secret Access Key** → like a password

3. How Access Keys Are Managed

- Generated through the **AWS Management Console**.
- Each IAM user:
    - Generates and manages **their own access keys**.
- Access keys are:
    - **Secret**
    - **Private**
    - **Downloadable only once at creation time**

⚠️ **Security Rule**:

- NEVER share your access keys.
- Treat:
  - Access Key ID like a username
  - Secret Access Key like a password

AWS CLI – Important Concepts

4. What Is the AWS CLI?

- CLI = **Command Line Interface**
- A tool to interact with AWS services using text commands.
- Gives direct access to AWS **public APIs**.

5. Benefits of Using the CLI

- Automate tasks using scripts.
- Manage resources efficiently.
- Faster and more powerful for advanced users.
- Can completely replace the Management Console for some users.

6. AWS CLI Characteristics

- All commands start with aws
- Open-source
  - Source code available on GitHub
- Built on top of AWS SDKs

AWS SDK – Important Concepts

7. What Is the AWS SDK?

- SDK = **Software Development Kit**
- A set of libraries for interacting with AWS services **from application code**.
- Not used in a terminal like the CLI.
- Embedded inside software applications.

8. Language Support

AWS SDKs support many programming languages, including:

- JavaScript
- Python
- Java
- PHP
- .NET
- Ruby
- Go
- C++
- Node.js

Also includes:

- **Mobile SDKs** (Android, iOS)
- **IoT SDKs** (for connected devices like sensors, locks, etc.)

Relationship Between CLI and SDK

9. CLI Is Built Using an SDK

- The AWS CLI used in this course is built on:
  - **AWS SDK for Python (Boto / Boto3)**
- This shows:
  - CLI and SDK are closely related
  - Both use AWS APIs in different forms

Cloud Practitioner

IAM Roles for AWS Services

1. IAM Roles Are the Final Core Component of IAM

- IAM has three main components:
  - Users
  - Groups
  - **Roles**
- IAM Roles are essential for enabling **AWS services to act on your behalf**.

2. Why IAM Roles Are Needed

- Some AWS services need to:
  - Access other AWS services
  - Perform actions inside your AWS account
- Since these services are **not human users**, they cannot use:
  - Username/password
  - Access keys like a human IAM user
- Therefore, AWS uses **IAM Roles** to grant permissions to services securely.

3. What Is an IAM Role?

- An IAM Role is:
  - Similar to an IAM user **in terms of permissions**
  - **Not tied to a physical person**
  - Intended to be **assumed by AWS services**
- Roles **do not have long-term credentials** like passwords or access keys.

4. Example: EC2 Instance Role

- An **EC2 instance** (virtual server) may need to:
  - Access AWS APIs
  - Read/write data from AWS services
- To enable this:
  1. Create an **IAM Role**
  2. Attach permissions (policies) to the role
  3. Attach the role to the EC2 instance
- The EC2 instance:
  - Uses the IAM Role automatically
  - Gets access **only if the role allows the requested action**
- ✅ EC2 + IAM Role work together as **one security entity**.

5. How IAM Roles Work (High Level)

- AWS service assumes the role
- The role provides **temporary credentials**
- AWS checks:
  - Does the role's policy allow this action?
- If yes → request succeeds
- If no → access is denied

6. Common AWS Services That Use IAM Roles

- Examples of services that require IAM Roles:
  - **EC2 Instance Roles**
  - **Lambda Function Roles**
  - **CloudFormation Roles**
  - Many other AWS services that make API calls

7. Key Concept to Remember for Exams ✅

- **IAM Roles are used by AWS services, not users**
- Roles provided:
  - Secure
  - Temporary
  - Permission-based access
- Preferred over embedding credentials inside services

Cloud Practitioner

IAM Security Tools

1. IAM Provides Built-in Security Tools

- IAM includes **security and auditing tools** to help manage and review user permissions.
- These tools support the **principle of least privilege** and improve account security.

2. IAM Credentials Report (Account-Level)

- Generated at the **AWS account level**.
- Contains information about **all IAM users** in the account.
- Shows the **status of user credentials**, such as:
  - Password enabled/disabled
  - Password age
  - Access keys enabled/disabled
  - Access key age
  - MFA status

✅ Useful for:

- Auditing user security
- Identifying users with weak or unused credentials
- Meeting compliance requirements

3. IAM Access Advisor (User-Level)

- Works at the **individual user level**.
- Shows:
  - Which AWS services a user has permissions for
  - **When those services were last accessed**

✅ Helps administrators:

- Identify **unused permissions**
- Remove unnecessary access
- Apply the **principle of least privilege**

4. How These Tools Support Least Privilege

- Least privilege = users should have **only the permissions they actually need**
- Access Advisor helps detect:
  - Services never used
  - Permissions that can be safely removed
- Results in a **more secure and controlled environment**

Cloud Practitioner

Shared Responsibility Model for IAM

1. Shared Responsibility Model Is Very Important for the CCP Exam

- Many **AWS Certified Cloud Practitioner (CCP)** questions are based on the **Shared Responsibility Model**.
- You must clearly understand:
  - **What AWS is responsible for**
  - **What the customer (you) is responsible for**

AWS Responsibilities (Security *OF* the Cloud)

2. What AWS Is Responsible For

AWS is responsible for the **underlying cloud infrastructure**, including:

- Physical **data centers**
- **Global network security**
- Hardware, servers, and storage
- Infrastructure availability and reliability
- **Configuration and vulnerability management** of AWS-managed services
- **Compliance** related to their infrastructure and services
  - Certifications and audits (e.g., ISO, SOC, etc.)

✅ AWS handles everything **behind the scenes**.

Customer Responsibilities (Security *IN* the Cloud)

3. IAM Falls Under Customer Responsibility

For IAM, **AWS does NOT manage security for you**. You are responsible for:

- Creating and managing:
  - IAM users
  - IAM groups
  - IAM roles
  - IAM policies
- Assigning and maintaining correct permissions
- Monitoring and auditing IAM usage

4. Authentication & Credential Management (Customer Responsibility)

You are responsible for:

- **Enabling and enforcing MFA**
  - Especially for the root account and IAM users
- **Rotating access keys** regularly
- Ensuring secure handling of credentials
- Removing unused or compromised credentials

AWS will **not** do these automatically.

5. Access Control & Monitoring

Customer responsibilities also include:

- Applying the **principle of least privilege**
- Using IAM tools to:
  - Review permissions
  - Analyze access patterns
- Regularly reviewing and adjusting permissions
- Ensuring users have **only what they need**, nothing more

Core Concept to Remember ✅

6. Simple Rule for the Exam

- **AWS is responsible for the cloud**
- **You are responsible for how you use the cloud**

Or said another way:

- AWS → Infrastructure, hardware, global security
- Customer → Users, permissions, MFA, access keys, monitoring

EC2 – Elastic Compute Cloud

AWS Budget Setup

Purpose: Set up AWS billing controls (budgets and alerts) to avoid spending money or overspending during the course.

Accessing Billing:

- Go to **Billing and Cost Management** from the top-right menu.
- IAM users may see **Access Denied** even with admin permissions.

Fix Billing Access for IAM Users:

- Log in using the **root account**.
- Go to **Account settings**.
- Enable **IAM user and role access to billing information**.
- This allows admin IAM users to view billing data.

Billing Dashboard Overview:

- Shows **month-to-date cost**, **forecasted monthly cost**, and **last month's cost**.
- Forecast may be unavailable for new accounts due to insufficient data.

Viewing Detailed Bills:

- Go to **Bills** → select a month.
- Scroll to **Charges by service** to see:
    - Costs by AWS service (e.g., EC2).
    - Regional costs (e.g., EU Ireland).
    - Detailed breakdown (NAT Gateway, EBS, Elastic IP, etc.).

Free Tier Monitoring:

- The **Free Tier** page shows current and forecasted usage.
- If usage exceeds free tier limits, it turns **red**, meaning charges will apply.
- Important to turn off resources that may incur costs.

Setting Up Budgets:

- Go to **Budgets** → **Create budget**.
- **Zero Spend Budget**:
    - Alerts you as soon as spending reaches **$0.01**.
    - Useful to detect any accidental spending.
- **Monthly Cost Budget**:
    - Example: **$10/month limit**.
    - Sends email alerts at:
        - 85% actual spend

    - 100% actual spend
    - 100% forecasted spend

Email Notifications:

- Budget alerts are sent via email.
- Alerts may trigger immediately if spending already occurred.

Best Practice:

- Even if the course should not cost money, budgets prevent unexpected bills.
- Regularly check **Bills**, **Free Tier**, and **Budgets**.

Key Skill:

- Understanding AWS billing and cost management is an essential AWS skill.

Cloud Practitioner

EC2 Basics

Amazon EC2 (Elastic Compute Cloud):

- One of the most popular and widely used AWS services.
- It provides **Infrastructure as a Service (IaaS)**.
- Allows you to rent virtual servers on demand.

What EC2 Includes (not just one service):

- **EC2 instances** → virtual machines.
- **EBS volumes** → virtual storage disks.
- **Elastic Load Balancer (ELB)** → distributes traffic across instances.
- **Auto Scaling Groups (ASG)** → automatically scale instances up or down.

Why EC2 is Important:

- Fundamental to understanding how cloud computing works.
- Demonstrates the core cloud principle: renting compute resources when needed.

Choices When Creating an EC2 Instance:

- **Operating System**: Linux (most popular), Windows, or macOS.
- **Compute power**: number of CPUs and cores.
- **Memory (RAM)**: amount of RAM required.
- **Storage options**:
    - Network-attached storage (EBS, EFS).
    - Hardware-attached storage (EC2 Instance Store).
- **Networking options**:
    - Type of network interface.
    - Public IP configuration.
- **Security**:
    - Firewall rules managed using **Security Groups**.

EC2 User Data (Bootstrap Script):

- A script that runs **only once at first launch** of the instance.
- Used for **bootstrapping**, meaning automating startup tasks.
- Common use cases:
    - Installing updates and software.
    - Downloading files.
    - Performing initial configuration.
- Runs as the **root user** (has sudo/admin privileges).
- More tasks in the script = longer instance startup time.

Key Takeaway:

- EC2 allows you to fully customize how your virtual machine behaves.
- You can launch and configure servers **within minutes**, which highlights the power and flexibility of the cloud.

EC2 Instance Type Basics

- **EC2 Instance Types**:
  - AWS provides different EC2 instance types optimized for different workloads.
  - Each type is designed with specific performance characteristics (CPU, memory, storage, networking).
- **AWS Instance Type Categories**:
  - **General Purpose**
  - **Compute Optimized**
  - **Memory Optimized**
  - **Storage Optimized**
  - (Other categories also exist, and they evolve over time.)
- **Instance Naming Convention (Example: m5.2xlarge)**:
  - **m** → Instance class (e.g., general purpose).
  - **5** → Generation of the instance (newer generations = improved hardware).
  - **2xlarge** → Size of the instance (determines CPU, memory, performance).
- **General Purpose Instances**:
  - Balanced compute, memory, and networking.
  - Suitable for diverse workloads:
    - Web servers
    - Code repositories
  - Example used in the course:
    - **t2.micro** (Free Tier eligible).
- **Compute Optimized Instances**:
  - Designed for compute-intensive workloads.
  - Use cases:
    - Batch processing
    - Media transcoding
    - High-performance web servers
    - High Performance Computing (HPC)
    - Machine learning
    - Gaming servers
  - Typically use the **C series** (e.g., C5, C6).
- **Memory Optimized Instances**:
  - Optimized for workloads requiring large amounts of RAM.

  - Use cases:
    - In-memory databases
    - Relational and NoSQL databases
    - Redis / ElastiCache
    - Business Intelligence (BI)
    - Real-time big data processing
  - Common families:
    - **R series**, X1, High Memory, Z1.
- **Storage Optimized Instances**:
  - Optimized for high-speed access to large datasets on local storage.
  - Use cases:
    - OLTP systems
    - Relational and NoSQL databases
    - Data warehousing
    - Distributed file systems
  - Instance families include:
    - **I**, **D**, **H1**.
- **Comparing Instance Types**:
  - Example:
    - t2.micro: 1 vCPU, 1 GB RAM.
    - r5.16xlarge: High memory emphasis (large RAM).
    - c5d.4xlarge: High CPU, lower RAM comparatively.
  - Different instances also vary in:
    - Network performance
    - EBS bandwidth
    - Storage performance
- **Useful Reference Resources**:
  - **AWS EC2 Instance Types documentation** (official reference).
  - **ec2instances.info**:
    - Compare all EC2 instances.
    - View CPU, memory, pricing, and performance.
    - Filter, sort, and search easily.

Cloud Practitioner

Security Groups & Classic Ports Overview

- **Security Groups**:
  - Act as **firewalls** for EC2 instances.
  - Control **inbound (incoming)** and **outbound (outgoing)** traffic.
  - Fundamental to **network security** in AWS.
- **Rule Characteristics**:
  - Security groups contain **only ALLOW rules** (no deny rules).
  - Rules can reference:
    - **IP address ranges** (IPv4 or IPv6)
    - **Other security groups**
  - If traffic is not explicitly allowed, it is blocked.
- **Inbound & Outbound Traffic**:
  - **Inbound rules**: Control traffic from the internet (or other sources) *to* the EC2 instance.
  - **Outbound rules**: Control traffic *from* the EC2 instance to the outside.
  - **Default behavior**:
    - All inbound traffic ❌ blocked
    - All outbound traffic ✅ allowed
- **Firewall Behavior**:
  - Security groups operate **outside the EC2 instance**.
  - Blocked traffic never reaches the instance.
  - If connection **times out** → security group issue.
  - If **connection refused** → security group allows traffic, but application isn't responding.
- **Rule Components**:
  - **Type** (e.g., SSH, HTTP)
  - **Protocol** (e.g., TCP)
  - **Port number**
  - **Source** (IP range, e.g., 0.0.0.0/0 for anywhere)
- **Relationship with EC2 Instances**:
  - One security group can be attached to **multiple instances**.
  - One EC2 instance can have **multiple security groups**.
  - Security groups are **region- and VPC-specific** (must be recreated in new regions/VPCs).

- **Best Practice**:
  - Use a **separate security group for SSH access** to reduce complexity and risk.
  - SSH is often the most sensitive and error-prone access method.
- **Referencing Security Groups**:
  - Security groups can allow inbound traffic from **other security groups**.
  - Enables secure instance-to-instance communication **without relying on IP addresses**.
  - Common and useful when working with **load balancers**.
  - If a security group is not referenced, traffic is denied even if the instance exists.
- **Common Ports to Know (Exam-Important)**:
  - **22** → SSH (Linux login, SFTP)
  - **21** → FTP
  - **80** → HTTP (unsecured web traffic)
  - **443** → HTTPS (secured web traffic)
  - **3389** → RDP (Windows login)
- **Key Takeaway**:
  - Security groups are a core AWS concept.
  - Understanding traffic flow, default rules, and common ports is essential for both real-world use and exams.

SSH Overview

- **Challenge in the Cloud**:
  - One of the trickiest aspects of cloud computing is **connecting to servers** to perform maintenance or actions.
- **SSH (Secure Shell)**:
  - Primary method to connect to **Linux EC2 instances**.
  - Uses a **command-line interface** to securely access servers.
- **Connection Methods by Operating System**:
  - **Mac & Linux**:
    - Use **SSH** directly from the terminal.
  - **Windows (Version 10 and above)**:
    - Can use **SSH** from the command prompt or terminal.
  - **Windows (Before Version 10)**:
    - Use **PuTTY**.
    - PuTTY performs the same function as SSH and uses the SSH protocol.
    - Works on all Windows versions.
- **EC2 Instance Connect**:
  - A newer method that connects via a **web browser**.
  - No terminal or additional software required.
  - Works on **Mac, Linux, and all Windows versions**.
  - Only supported on **Amazon Linux 2** instances.
  - Chosen for demonstrations in this course due to simplicity.
- **Which Method to Use**:
  - Mac/Linux users → SSH lecture for Mac/Linux.
  - Windows users:
    - Use PuTTY, or
    - Use SSH if on Windows 10+.
  - EC2 Instance Connect is recommended for beginners and simplicity.
- **Common Issues with SSH**:
  - SSH is the **most common source of problems** for learners.
  - Issues may be caused by:
    - Missing or incorrect security group rules.
    - Command mistakes.
    - Typographical errors.

- **Troubleshooting Support**:
  - Rewatch the SSH lecture if problems occur.
  - Use the provided troubleshooting guide.
  - EC2 Instance Connect can often bypass SSH-related issues.
- **Practical Advice**:
  - You do **not** need all connection methods to work.
  - If **one method works**, you are good to proceed.
  - If none work, it's still okay—this is an introductory course and SSH is not heavily required.
- **Key Takeaway**:
  - Multiple ways exist to connect to EC2 instances.
  - Choose the method that works best for your operating system and comfort level.

EC2 Instance Connect

- **EC2 Instance Connect**:
  - An **alternative to traditional SSH**.
  - Allows **browser-based SSH access** to EC2 instances.
  - No need to use terminal, PuTTY, or install any software.
- **How to Access**:
  - Select the EC2 instance → click **Connect** → choose **EC2 Instance Connect**.
  - Opens a new browser tab with a terminal session.
- **Username Handling**:
  - AWS automatically sets the correct username (**ec2-user**) for **Amazon Linux 2**.
  - Changing the username will cause connection failure.
- **SSH Key Management**:
  - **No SSH key required** from the user.
  - AWS temporarily uploads and uses an SSH key behind the scenes.
  - Simplifies setup and avoids key management issues.
- **Supported AMI**:
  - Works **only with Amazon Linux 2**.
  - This is why Amazon Linux 2 is used throughout the course.
- **Behind-the-Scenes Behavior**:
  - EC2 Instance Connect still relies on **SSH (port 22)**.
  - If SSH traffic is blocked, Instance Connect will fail.
- **Security Group Requirement**:
  - **Inbound rule for port 22 (SSH) must be open**.
  - Sometimes both rules are required:
    - SSH from **Anywhere IPv4**
    - SSH from **Anywhere IPv6**
  - Without these, connection will fail.
- **Troubleshooting Example**:
  - Removing SSH inbound rules causes Instance Connect to break.
  - Re-adding port 22 rules restores access.
- **What You Can Do Once Connected**:
  - Run Linux commands (e.g., whoami, ping google.com).
  - Fully interact with the instance from your browser.
- **Flexibility**:

    - When the course says “use SSH,” you can choose:
        - SSH from terminal
        - PuTTY (Windows)
        - Windows SSH
        - **EC2 Instance Connect**
- **Instructor Preference**:
    - EC2 Instance Connect is used heavily due to ease of use.
    - Especially helpful for beginners and troubleshooting.
- **Key Takeaway**:
    - EC2 Instance Connect is the **easiest and fastest way** to access EC2 instances.
    - Still requires correct **security group configuration**.
    - Great replacement for traditional SSH in most learning scenarios.

EC2 Instance Roles

- **Goal**:
  - Provide AWS permissions to an EC2 instance **securely**, without storing credentials on the server.
- **Connecting to EC2**:
  - You can connect using:
    - EC2 Instance Connect
    - SSH
    - PuTTY
  - Regardless of the method, once connected, you are at the same Linux terminal (ec2-user@<private-ip>).
- **AWS CLI on EC2**:
  - Amazon Linux AMI comes with the **AWS CLI pre-installed**.
  - You can run AWS commands directly from the EC2 instance.
- **Why AWS CLI Initially Fails**:
  - Running commands like aws iam list-users fails because:
    - No AWS credentials are configured on the instance.
- **What NOT to Do (Very Important)**:
  - ❌ **Do NOT use aws configure on an EC2 instance**
  - ❌ Do NOT enter:
    - Access Key ID
    - Secret Access Key
  - Reason:
    - Credentials stored on the instance can be stolen by anyone who gains access.
    - This is a **major security risk**.
- **Correct Approach: Use IAM Roles**:
  - EC2 instances should **only receive AWS permissions via IAM roles**.
  - IAM roles provide **temporary credentials automatically and securely**.
- **IAM Role Setup**:
  - A role (e.g., DemoRoleForEC2) exists with a policy attached (e.g., IAMReadOnlyAccess).
  - This role must be **attached to the EC2 instance**.
- **How to Attach an IAM Role to EC2**:

  - EC2 → Instances → Actions → Security → Modify IAM Role
  - Select the IAM role and save.
  - After attachment, the role appears under the instance's **Security** tab.
- **Effect of Attaching the IAM Role**:
  - AWS CLI commands now work **without configuring credentials manually**.
  - Example:
    - aws iam list-users executes successfully once the role is attached.
- **Proof IAM Roles Control Access**:
  - If you:
    - Detach the policy from the role → **Access denied**
    - Reattach the policy → Access restored
  - Confirms permissions flow:
  - IAM Policy → IAM Role → EC2 Instance
- **IAM Propagation Delay**:
  - Permission changes may take a short time to propagate.
  - Re-running the command after a few seconds usually resolves it.
- **Key Rule (Exam + Real World)**:
  - ✅ **Always use IAM Roles for EC2**
  - ❌ Never store IAM user credentials on EC2 instances
- **Core Takeaway**:
  - IAM roles are the **only secure and recommended way** to grant AWS permissions to EC2 instances.
  - This is a **must-know concept for AWS exams and real-world usage**.

EC2 Purchasing Options

1. On-Demand Instances

- Run instances **immediately, pay for what you use**.
- **No upfront payment**, no long-term commitment.
- **Highest cost** compared to other options.
- Billed:
  - Per second (Linux/Windows, after first minute)
  - Per hour (other OS)
- **Best for**:
  - Short-term workloads
  - Unpredictable or spiky usage
  - Applications that must not be interrupted

2. Reserved Instances (RI)

- Commit to **1 year or 3 years**.
- Up to **~72% discount** compared to On-Demand.
- You reserve:
  - Instance type
  - Region
  - OS
  - Tenancy
- Payment options:
  - No upfront
  - Partial upfront
  - All upfront (maximum discount)
- Can be **sold on the RI marketplace**.
- **Best for**:
  - Steady-state workloads
  - Databases or long-running services

Convertible Reserved Instances

- More flexibility:
  - Can change instance family, type, OS, tenancy, scope
- Slightly lower discount (up to **~66%**).
- **Best when future needs may change**.

3. Savings Plans

- Commit to **a fixed dollar spend per hour** (e.g., $10/hour).
- Duration: **1 or 3 years**.
- Discount similar to Reserved Instances (~70%).
- Flexibility:
  - Instance size ✅
  - OS ✅
  - Tenancy ✅
- Locked to:
  - Instance family
  - Region
- Usage beyond the plan → billed as On-Demand.
- **Best for**:
  - Long-term workloads
  - When you want flexibility without choosing exact instance types

4. Spot Instances

- Use **spare AWS capacity**.
- Up to **90% discount**.
- Can be **terminated at any time** if spot price exceeds your bid.
- **Best for fault-tolerant jobs**:
  - Batch processing
  - Data analysis
  - Image processing
  - Distributed workloads
- **Not suitable for**:
  - Databases
  - Critical or stateful applications
- Very exam-important ⚠️

5. Dedicated Hosts

- **Entire physical server** dedicated to you.
- Full visibility into hardware (sockets, cores).
- Payment:
  - On-demand
  - 1- or 3-year reservation (up to ~70% discount)
- **Most expensive option**.
- **Best for**:
  - Compliance / regulatory requirements
  - Bring-Your-Own-License (BYOL)
  - Licenses tied to cores or sockets

6. Dedicated Instances

- Run on **dedicated hardware**, not shared with other customers.
- You **do not control instance placement**.
- Less control than Dedicated Hosts.
- Exam tip:
  - Dedicated Instance ≠ Dedicated Host

7. Capacity Reservations

- Reserve **On-Demand capacity in a specific AZ**.
- No discount → billed at On-Demand rate.
- Charged **whether or not you use the instances**.
- Can be created or canceled anytime.
- Can be combined with:
  - Regional Reserved Instances
  - Savings Plans (for cost savings)
- **Best for**:
  - Short-term, uninterrupted workloads
  - Applications requiring guaranteed capacity in a specific AZ

Easy Analogy (Great for Remembering)

- **On-Demand** → Walk-in hotel stay (full price).
- **Reserved Instances** → Long hotel stay discount.
- **Savings Plans** → Commit to monthly hotel spending, flexible room types.
- **Spot Instances** → Last-minute cheap rooms, but you can be kicked out.
- **Dedicated Host** → Rent the entire hotel building.
- **Capacity Reservation** → Reserve a room whether you stay or not.

Shared Responsibility Model – EC2

AWS Responsibilities (Security *of* the Cloud)

AWS is responsible for:

- **Physical data centers** and global infrastructure.
- **Physical security** of servers and hardware.
- **Isolation on physical hosts** (including Dedicated Hosts).
- **Replacing faulty hardware** if servers fail.
- **Compliance** with industry and regulatory standards AWS has agreed to.

Customer Responsibilities (Security *in* the Cloud)

As an EC2 user, **you are responsible for everything inside the virtual machine**, including:

- **Security Groups**
  - Define inbound and outbound firewall rules.
  - Control who can access your EC2 instances and on which ports.
- **Operating System Management**
  - OS choice (Linux or Windows).
  - Installing patches and security updates.
  - OS hardening and configuration.
- **Software & Applications**
  - Installing, updating, and securing applications and utilities.
  - Managing vulnerabilities in installed software.
- **IAM Roles & Permissions**
  - Properly assigning IAM roles to EC2 instances.
  - Ensuring least-privilege access.
  - Avoiding hard-coded credentials.
- **Data Security**
  - Protecting data stored on EC2 instances.
  - Managing encryption, backups, and access controls.

EC2 Summary

EC2 Instance Components

- An **EC2 instance** is built using:
  - **AMI (Amazon Machine Image)** → Defines the operating system (Linux, Windows, etc.).
  - **Instance Type** → Defines CPU power and RAM.
  - **Storage** → Defines disk type and size attached to the instance.
  - **Security Group** → Acts as a firewall for the instance.
  - **EC2 User Data** → A bootstrap script that runs at first launch.

Security Groups

- Security groups are **firewalls outside the EC2 instance**.
- Control **inbound and outbound traffic**.
- Rules define:
  - Allowed **ports**
  - Allowed **IP addresses**
- Only **allow rules** (no deny rules).

EC2 User Data

- A **bootstrap script** that runs **once at first startup**.
- Used to:
  - Install software
  - Configure the instance
  - Set up services (e.g., web server saying “Hello World”)

Connecting to EC2

- **SSH** is used to connect from your local computer to EC2.
- Uses **port 22**.
- Allows you to run commands inside the instance.

IAM Roles for EC2

- EC2 instances can use **IAM roles** (instance roles).
- Allows EC2 to securely make AWS API calls.
- No need to store access keys on the instance.

- Permissions are inherited from the attached IAM role.

EC2 Purchasing Options (Exam-Important)

You should know the following EC2 purchasing options:

- **On-Demand**
- **Spot Instances**
- **Reserved Instances**
  - Standard Reserved
  - Convertible Reserved
- **Dedicated Hosts**
- **Dedicated Instances**

Core Takeaway

- EC2 lets you fully define and control:
  - Compute
  - Storage
  - Networking
  - Security
- Understanding EC2 is **fundamental to AWS and cloud computing**.
- These concepts are **frequently tested in AWS exams**.

EC2 Instance Storage

EBS Overview

- **EBS (Elastic Block Store)** is the primary storage option for EC2 instances.
- EBS volumes are **network-attached block storage** devices.
- They can be **attached to EC2 instances while running**.
- **Data persists even after an EC2 instance is terminated**.
- An EBS volume can be **re-attached to a new EC2 instance** to recover data.
- At the **Cloud Practitioner (CCP) level**, an EBS volume can be **attached to only one instance at a time**.
- EBS volumes are **bound to a specific Availability Zone (AZ)**.
- An EBS volume **cannot be attached to an instance in a different AZ**.
- You can move EBS data across AZs **by creating snapshots**.
- EBS volumes are **network drives**, not physical disks.
- Because they use the network, **slight latency may occur**.
- EBS volumes can be **detached and reattached quickly**, making them useful for **failover scenarios**.
- Storage capacity and **IOPS (input/output operations per second)** must be **provisioned in advance**.
- Users are **billed for the provisioned capacity**, not actual usage.
- **Volume size and performance can be increased later**.
- A single EC2 instance can have **multiple EBS volumes attached**.
- EBS volumes can exist **without being attached** to any instance.
- **Delete on termination** controls whether an EBS volume is deleted when the EC2 instance is terminated.
- By default:
    - **Root volume** → delete on termination is **enabled**
    - **Additional EBS volumes** → delete on termination is **disabled**
- This setting can be modified to **preserve data**, especially for exam scenarios.

EBS Snapshot Overview

- **EBS Snapshots** are **backups** of EBS volumes that can be taken at any time.
- A snapshot captures the **state of the EBS volume** at the time it is created.
- Even if an EBS volume is **deleted or terminated**, data can be **restored from its snapshot**.
- Detaching the EBS volume before taking a snapshot is **not required**, but it is **recommended** for data consistency.
- Snapshots are commonly used for:
  - **Backup and recovery**
  - **Moving data across Availability Zones (AZs)**
  - **Copying data across AWS Regions**
- Snapshots allow transfer of EBS data to other AZs by:
  - Creating a snapshot of the source EBS volume
  - Restoring a **new EBS volume** from that snapshot in a different AZ
  - Attaching the new volume to an EC2 instance in that AZ
- **EBS snapshots are regional resources**, but can be copied to other regions if needed.
- Stopping an EC2 instance before taking a snapshot helps ensure a **clean backup**, but snapshots can also be taken while the instance is running.
- **EBS Snapshot Archive**:
  - Allows moving snapshots to a **lower-cost archive tier**
  - Archive tier is **up to 75% cheaper**
  - Restoring from archive takes **24 to 72 hours**
  - Best for **infrequently accessed backups**
- **Recycle Bin for EBS Snapshots**:
  - Prevents permanent loss from **accidental deletion**
  - Deleted snapshots move to the recycle bin instead of being immediately removed
  - Retention period can be set from **1 day to 1 year**
  - Snapshots are permanently deleted after the retention period expires
- These features help with **cost optimization**, **data protection**, and **disaster recovery**.

AMI Overview

- **AMI (Amazon Machine Image)** powers EC2 instances.
- An AMI represents a **customized template** for launching EC2 instances.
- An AMI includes:
  - Operating system
  - Software configuration
  - Installed applications
  - Monitoring and configuration tools
- Using a custom AMI results in **faster boot and configuration time**.
- AWS provides **default/public AMIs**, such as **Amazon Linux 2**, which are widely used.
- Users can **create and maintain their own AMIs** with required software preinstalled.
- AMIs are **region-specific**, but they can be **copied across regions**.
- There are **three main types of AMIs**:
  - **AWS-provided AMIs** (public)
  - **Custom AMIs** created by users
  - **AWS Marketplace AMIs** created and sold by third-party vendors
- Marketplace AMIs save time by providing **preconfigured software stacks**.
- Users can also **sell their own AMIs** on the AWS Marketplace as a business.
- **AMI creation process**:
  - Launch an EC2 instance
  - Customize it (install software, configure settings)
  - Stop the instance to ensure data integrity
  - Create an AMI (this automatically creates EBS snapshots)
- Once created, the AMI can be used to **launch identical EC2 instances**.
- AMIs enable **copying EC2 instances across Availability Zones** (e.g., from us-east-1a to us-east-1b).
- This makes AMIs useful for **scalability, consistency, and disaster recovery**.

EC2 Image Builder Overview

- **EC2 Image Builder** is an AWS service used to **automate the creation of virtual machine images (AMIs) and container images**.
- It helps **automate building, maintaining, validating, and testing AMIs** for EC2 instances.
- EC2 Image Builder automatically creates a **temporary "Builder EC2 instance"**.
- This builder instance:
  - Installs required software (e.g., Java, CLI tools)
  - Applies OS updates
  - Configures security tools (firewalls, patches)
  - Installs applications as defined by you
- Once customization is complete, **an AMI is automatically created**.
- EC2 Image Builder can then **launch a test EC2 instance** from the AMI.
- Automated tests can be run to verify:
  - AMI functionality
  - Security posture
  - Application correctness
- Tests are **optional** and can be skipped if not required.
- After validation, the AMI can be **distributed to multiple AWS Regions**.
- This supports **global deployments and consistency** across regions.
- EC2 Image Builder can be:
  - **Scheduled** (e.g., weekly)
  - Triggered when **packages are updated**
  - Run **manually**
- **EC2 Image Builder itself is free** to use.
- You only pay for **underlying resources**, such as:
  - EC2 instances created during image builds
  - Storage costs for AMIs in each region
- This service reduces **manual effort**, improves **consistency**, and enhances **security and compliance**.
- EC2 Image Builder is a **common exam topic** under automation and infrastructure management.

EC2 Instance Store

- **EC2 Instance Store** provides **local, hardware-attached storage** to EC2 instances.
- It is attached to the **physical server** hosting the EC2 instance, not over the network.
- Instance Store offers **very high I/O performance and throughput**.
- It is ideal when **extremely fast disk performance** is required.
- Storage is **ephemeral (temporary)** in nature.
- **Data is lost** if the EC2 instance is **stopped or terminated**.
- Instance Store is **not suitable for long-term or durable storage**.
- Typical use cases include:
  - Caches
  - Buffers
  - Scratch space
  - Temporary data
- It should **not be used for critical or persistent data**.
- For durable storage, **EBS volumes** are a better choice.
- If the **underlying physical server fails**, all Instance Store data is lost.
- **Backup and replication are the user's responsibility** when using Instance Store.
- Some EC2 instance types (e.g., **I3 instances**) come with Instance Store storage.
- Instance Store can achieve **millions of IOPS**, far exceeding EBS performance.
- Example comparison:
  - Instance Store: **millions of IOPS**
  - EBS (e.g., GP2): **tens of thousands of IOPS**
- **Exam tip**:
  When you see **very high-performance, local, hardware-attached storage** for EC2, think **EC2 Instance Store**.

EFS Overview

Elastic File System (EFS)

- **EFS (Elastic File System)** is a **managed network file system (NFS)** for EC2 instances.
- It allows a **single file system to be mounted to hundreds of EC2 instances simultaneously**.
- Unlike EBS, EFS provides **shared storage** across multiple instances.
- **Supports Linux EC2 instances only**.
- EFS works **across multiple Availability Zones (AZs)**.
- EC2 instances in different AZs can access the **same EFS file system at the same time**.
- EFS is:
    - **Highly available**
    - **Highly scalable**
    - **More expensive** (≈ 3× the cost of GP2 EBS)
- EFS is **pay-as-you-use**:
    - You do **not provision capacity in advance**
    - You only pay for the **actual storage used**
- Example: Storing 20 GB means you pay **only for 20 GB**.

EBS vs EFS – Core Differences

EBS (Elastic Block Store):

- Can be attached to **only one EC2 instance at a time**
- Bound to a **single Availability Zone**
- To move EBS across AZs:
    - Create a **snapshot**
    - Restore it in another AZ
    - This creates a **copy**, not a live shared volume

EFS (Elastic File System):

- Can be mounted to **multiple EC2 instances simultaneously**
- Instances across **multiple AZs** can share the same file system
- All instances see the **same files in real time**
- Acts as a **true shared file system**

EFS Storage Classes

- **EFS Standard**
  - For frequently accessed files
- **EFS Infrequent Access (EFS-IA)**
  - Cost-optimized for files accessed less often
  - Can reduce storage costs by **up to 92%**
  - Designed for files not accessed every day

EFS Lifecycle Management

- EFS supports **lifecycle policies**.
- Files not accessed for a defined period (e.g., **60 days**) can be:
  - Automatically moved from **EFS Standard → EFS-IA**
- When files are accessed again:
  - They are automatically moved back to **EFS Standard**
- This process is:
  - **Automatic**
  - **Transparent to applications**
  - Requires **no code changes**

Use Cases for EFS

- Shared application files
- Content management systems
- Web servers with shared assets
- Multi-AZ workloads needing shared storage

Shared Responsibility Model for EC2 Storage

AWS Responsibilities (Security *of* the Cloud)

- AWS is responsible for the **underlying infrastructure**.
- AWS handles **data replication** for storage services like **EBS and EFS**.
- Data is automatically replicated across **multiple hardware components**.
- If hardware fails, AWS ensures **high availability** so customers are not impacted.
- AWS is responsible for **replacing faulty storage hardware**.
- AWS ensures that **its employees cannot access customer data**.
- Physical data center security and infrastructure reliability are **AWS-managed**.

Customer Responsibilities (Security *in* the Cloud)

- **Setting up backups and snapshots** (EBS snapshots, EFS backups) is the customer's responsibility.
- **Data encryption** must be enabled and managed by the customer.
- All **data written to the storage** is owned and managed by the customer.
- Customers must **manage and protect their own data**.
- Understanding storage behavior is critical:
    - Especially when using **EC2 Instance Store**
- For **EC2 Instance Store**:
    - Data is lost if the instance is **stopped or terminated**
    - Data can be lost if **hardware fails**
    - Backups must be handled **manually by the customer**
- Proper **backup, recovery, and redundancy strategies** are the customer's responsibility.

Amazon Fsx Overview

Amazon FSx – Key Points

- **Amazon FSx** is a **managed service** that provides **third-party, high-performance file systems** on AWS.
- It is used when **EFS or S3 are not suitable** for your use case.
- FSx allows you to run **native file systems** without managing the underlying infrastructure.

FSx Offerings (Exam Relevant)

1. Amazon FSx for Windows File Server

- Fully managed, **highly reliable and scalable Windows native file system**
- Built on **Windows File Server**
- Designed specifically for **Windows-based workloads**
- Supports **Windows-native protocols**:
  - **SMB (Server Message Block)**
  - **NTFS**
- Typically deployed **across multiple Availability Zones** for high availability
- Integrates with **Microsoft Active Directory (AD)** for authentication and security
- Can be accessed:
  - From **Windows EC2 instances**
  - From **on-premises Windows environments**
- Ideal replacement for **on-prem Windows file servers**

✅ **Exam hint:**
If the scenario mentions **Windows**, **SMB**, **NTFS**, or **Active Directory**, think **FSx for Windows File Server**.

2. Amazon FSx for Lustre

- Fully managed **high-performance file system** designed for **High-Performance Computing (HPC)**.
- Lustre = **Linux + Cluster**
- Optimized for **extremely fast workloads**
- Supports:
  - **Millions of IOPS**
  - **Hundreds of GB/s throughput**

    - **Sub-millisecond latency**
- Common use cases:
    - Machine learning
    - Big data analytics
    - Video processing
    - Financial modeling
- Can be accessed by:
    - EC2 compute instances
    - On-premises environments
- Can store data **directly in Amazon S3** as the backend

✅ **Exam hint:**
If you see **HPC**, **machine learning**, or **very high performance**, think **FSx for Lustre**.

Other FSx Option (Mentioned)

- **FSx for NetApp ONTAP**
- Not a primary focus for the exam in this lesson

General FSx Characteristics

- Fully **managed by AWS**
- Eliminates the need to manage servers and file system software
- Designed for **specialized workloads**
- No hands-on practical demo required for the exam

EC2 Instance Storage Summary

EBS (Elastic Block Store)

- Network-attached storage for EC2 instances
- Can be attached to **one EC2 instance at a time**
- **Bound to a specific Availability Zone (AZ)**
- Persistent storage (data survives instance termination)
- **EBS Snapshots** are used for:
    - Backups
    - Moving data across AZs
- Snapshots create a **copy**, not a shared drive

AMIs (Amazon Machine Images)

- Preconfigured templates used to launch EC2 instances
- Contain OS, software, configurations, and monitoring tools
- Enable fast and consistent instance launch
- AMIs are **region-specific** but can be copied across regions
- **EC2 Image Builder**:
    - Automates AMI creation
    - Builds, tests, validates, and distributes AMIs automatically

EC2 Instance Store

- **Local, hardware-attached storage**
- Extremely **high performance**
- **Ephemeral** storage:
    - Data is lost when instance is stopped or terminated
- Suitable for:
    - Cache
    - Buffers
    - Temporary/scratch data
- Not suitable for long-term or critical data storage

EFS (Elastic File System)

- Managed **Network File System (NFS)** for Linux instances
- Can be mounted to **hundreds of EC2 instances simultaneously**
- Works at the **region level** (no AZ lock-in)
- Highly available and scalable
- Pay-as-you-use (no pre-provisioning required)
- **EFS-IA (Infrequent Access)**:
    - Lower-cost storage tier
    - Automatically moves infrequently accessed files via lifecycle policies

Amazon FSx

- Managed service for **third-party file systems**
- Used when EBS/EFS/S3 are not sufficient
- **FSx for Windows File Server**:
    - Native Windows file system
    - Supports SMB, NTFS, and Active Directory
- **FSx for Lustre**:
    - Designed for **High-Performance Computing (HPC)**
    - Linux-based
    - Ideal for machine learning, analytics, and large-scale processing

ELB & ASG – Elastic Load Balancing & Auto Scaling Groups

High Availability, Scalability, Elasticity

Scalability

- Scalability means the ability of an application to **handle increased load by adapting resources**.
- Two types of scalability exist in AWS:
    - **Vertical scalability (Scale Up / Scale Down)**
    - **Horizontal scalability (Scale Out / Scale In)**

Vertical Scalability

- Involves **increasing or decreasing the size of an EC2 instance**.
- Example:
    - Upgrade from t2.micro → t2.large
- Comparable to upgrading a **junior operator to a senior operator** in a call center.
- Common for **non-distributed systems**, such as databases.
- Has **hardware limits**, even though modern limits are very high.

Horizontal Scalability (Elasticity)

- Involves **adding or removing EC2 instances**.
- Example:
    - Increase from 1 to 6 operators in a call center.
- Requires a **distributed system**.
- Very common for **web and modern applications**.
- Implemented in AWS using:
    - **Auto Scaling Groups**
    - **Load Balancers**

High Availability

- Ensures applications remain operational even if part of the system fails.
- Achieved by running applications in **at least two Availability Zones (AZs)**.
- Example:
    - Call centers in **New York and San Francisco**

- Protects against:
  - Data center failures
  - Power outages
  - Natural disasters
- Closely related to **horizontal scaling**.

EC2 Scaling Summary

- **Vertical Scaling**
  - Increase/decrease instance size
  - Scale up / scale down
  - Example range:
    - t2.nano → u-12tb1.metal
- **Horizontal Scaling**
  - Increase/decrease number of instances
  - Scale out / scale in
  - Uses **Auto Scaling Groups + Load Balancers**
- **High Availability**
  - Instances spread across **multiple AZs**
  - Enabled by multi-AZ Auto Scaling Groups and Load Balancers

Key Exam Definitions

- **Scalability**
  - Ability to handle increased load by **scaling up or scaling out**
- **Elasticity**
  - Automatic scaling **based on demand**
  - Matches capacity to load
  - Optimizes **cost (pay per use)**
- **Agility** (Exam distractor)
  - Ability to **quickly provision IT resources**
  - Measured in minutes instead of weeks
  - Not related to scaling or elasticity

Elastic Load Balancing Overview

What is a Load Balancer?

- A **load balancer** is a managed AWS service that **distributes incoming traffic** across multiple backend servers (EC2 instances).
- It acts as a **single-entry point** for users (via a DNS name).
- Backend servers are called **targets** (commonly EC2 instances).

Why Use Elastic Load Balancing?

- **Distributes load** across multiple instances to improve scalability.
- Provides a **single DNS endpoint** for the application.
- Performs **health checks** on backend instances.
- Automatically **stops sending traffic** to unhealthy instances.
- Helps **hide instance failures** from users.
- Supports **SSL/TLS termination** (HTTPS).
- Works **across multiple Availability Zones**, enabling **high availability**.

Managed by AWS

- ELB is a **fully managed service**:
  - No servers to provision
  - AWS handles scaling, maintenance, upgrades, and availability
- Easier and more reliable than running your **own load balancer on EC2**.
- You only need to **configure behavior**, not infrastructure.

Types of AWS Load Balancers (Exam Important)

1. Application Load Balancer (ALB)

- **Layer 7** (HTTP/HTTPS, gRPC)
- Used for **web applications**
- Supports **advanced routing** (based on URL path, host, headers)
- Uses a **static DNS name** (not a static IP)
- ✅ Think ALB when you see:
  - HTTP / HTTPS
  - Web apps
  - Routing rules

2. Network Load Balancer (NLB)

- **Layer 4** (TCP, UDP)
- **Ultra-high performance** (millions of requests per second)
- Provides a **static IP address** (via Elastic IP)
- Used for **low-latency or high-throughput workloads**
- ✅ Think NLB when you see:
  - TCP / UDP
  - Static IP
  - Very high performance

3. Gateway Load Balancer (GWLB)

- Operates at **Layer 3**
- Uses the **GENEVE protocol**
- Designed to route traffic to **security appliances**
  - Firewalls
  - Intrusion detection/prevention
  - Deep packet inspection
- Does **not** balance traffic to applications directly
- Sits **in the middle** to inspect network traffic
- ✅ Think GWLB when you see:
  - Firewalls
  - Security inspection
  - Third-party virtual appliances

4. Classic Load Balancer (CLB) *(Legacy)*

- Older generation (Layer 4 + Layer 7)
- Being retired and **not relevant for the exam**
- Replaced by **ALB and NLB**

Architecture Summary

- Users send traffic → **Load Balancer**
- Load Balancer routes traffic → **Healthy backend EC2 instances**
- For GWLB:
  - Traffic → GWLB → Security EC2 appliances → GWLB → Application

Exam Keyword Mapping

- **HTTP / HTTPS / gRPC → ALB**
- **TCP / UDP / static IP → NLB**
- **Firewall / traffic inspection → GWLB**
- **Highly available, scalable entry point → ELB**

Auto Scaling Group Overview

Auto Scaling Groups (ASG)

- **Auto Scaling Groups** are used to **automatically manage the number of EC2 instances** behind a load balancer.
- They help applications **handle variable traffic** (e.g., more users during the day, fewer at night).
- ASG supports:
  - **Scale out** → add EC2 instances when load increases
  - **Scale in** → remove EC2 instances when load decreases
- This enables **elasticity**, a core cloud principle.

Why Use Auto Scaling Groups?

- Traffic to applications **changes over time**.
- AWS allows servers to be **created and removed quickly**.
- ASG ensures:
  - Enough capacity during high load
  - Cost savings during low load
- You only run **what you need**, optimizing cost.

Key ASG Configuration Settings

- **Minimum capacity**
  - Smallest number of instances that must always run
- **Desired capacity**
  - Target number of instances (usually the current running count)
- **Maximum capacity**
  - Upper limit on how many instances ASG can create

ASG and Load Balancer Integration

- Auto Scaling Groups work **hand-in-hand with Load Balancers**.
- When ASG:
  - **Adds instances** → they are automatically **registered** with the load balancer
  - **Removes instances** → they are **deregistered** from the load balancer
- Load balancer then:
  - Distributes traffic across all healthy instances

Health Management

- ASG continuously monitors instance health.
- If an instance becomes **unhealthy**:
    - It is **deregistered**
    - **Terminated**
    - Automatically **replaced by a new healthy instance**
- Ensures **high availability** and reliability.

End-to-End Flow

- Users send traffic → **Load Balancer**
- Load balancer forwards traffic → **EC2 instances in ASG**
- As traffic increases:
    - ASG scales out (adds instances)
    - Load balancer distributes traffic to all instances
- As traffic decreases:
    - ASG scales in (removes instances)

Benefits of Auto Scaling Groups

- Automatic scaling based on demand
- Improved **availability and fault tolerance**
- **Cost optimization** (pay only for needed capacity)
- Seamless integration with load balancers
- Core implementation of **elasticity** in AWS

Auto Scaling Group Strategies

1. Manual Scaling

- ASG size is **changed manually** by the user.
- Example:
  - Increase capacity from 1 → 2 instances
  - Decrease capacity from 2 → 1 instance
- No automation involved.

2. Dynamic Scaling

Automatically adjusts capacity based on demand.

a. Simple Scaling / Step Scaling

- Based on **CloudWatch alarms**.
- You define:
  - **Trigger condition** (e.g., CPU utilization)
  - **Action** (add or remove instances)
- Example:
  - CPU > 70% for 5 minutes → **Add 2 instances**
  - CPU < 30% for 10 minutes → **Remove 1 instance**
- Simple Scaling → fixed adjustment
- Step Scaling → different adjustments based on alarm severity

b. Target Tracking Scaling

- Easiest and most commonly used dynamic scaling type.
- You define a **target metric value**.
- ASG automatically adjusts capacity to maintain that target.
- Example:
  - Keep average CPU utilization at **40%**
- ASG calculates how many instances are needed automatically.

✅ **Exam-friendly and recommended approach**

3. Scheduled Scaling

- Used when **traffic patterns are known in advance**.
- Scaling happens at a **specific date and time**.
- Example:
  - Every Friday at 5 PM → set minimum capacity to 10 instances

- Useful for predictable, time-based events.

4. Predictive Scaling

- Uses **Machine Learning** to analyze historical traffic.
- Predicts future demand and **provisions capacity in advance**.
- Ideal for:
  - Regular daily or weekly traffic patterns
  - Time-based spikes
- Requires minimal user intervention.
- Appears frequently in **exam questions**.

Summary Table (Conceptual)

- **Manual Scaling** → User-controlled, no automation
- **Simple/Step Scaling** → Alarm-based, fixed adjustments
- **Target Tracking** → Maintain a target metric automatically
- **Scheduled Scaling** → Time-based, predictable loads
- **Predictive Scaling** → ML-based, forecast-driven scaling

Cloud Practitioner

ELB & ACG Summary

Core Cloud Concepts

- **High Availability**
  - Applications run across **multiple Availability Zones (AZs)**
  - Protects against data center or AZ failures
- **Scalability**
  - **Vertical Scaling**: Increase or decrease **instance size**
  - **Horizontal Scaling**: Increase or decrease **number of instances**
- **Elasticity**
  - Ability to **automatically scale up or down based on demand**
  - Ensures optimal resource usage and cost efficiency
- **Agility**
  - Ability to **quickly create and delete resources**
  - Enables faster development and deployment cycles
  - Not directly related to scaling (exam distractor)

Elastic Load Balancing (ELB)

- Distributes incoming traffic across **multiple backend EC2 instances**
- Provides a **single access point (DNS name)** for applications
- Supports **health checks** to ensure only healthy instances receive traffic
- Can span **multiple Availability Zones** to improve availability
- Fully **managed by AWS** (no server maintenance needed)

Types of Load Balancers

- **Application Load Balancer (ALB)**
  - Layer 7 (HTTP/HTTPS)
  - Used for web applications
- **Network Load Balancer (NLB)**
  - Layer 4 (TCP/UDP)
  - Designed for **very high performance**
- **Gateway Load Balancer (GWLB)**
  - Layer 3
  - Routes traffic through **virtual appliances** (firewalls, inspection)
- **Classic Load Balancer**
  - Legacy and retired (not exam-relevant)

Auto Scaling Groups (ASG)

- Automatically **add or remove EC2 instances** based on demand
- Implements **elasticity** for applications
- Maintains:
  - Minimum capacity
  - Desired capacity
  - Maximum capacity
- Replaces **unhealthy instances automatically**
- Works closely with **Load Balancers**:
  - New instances are automatically registered
  - Removed instances are deregistered

ELB + ASG Together

- Form a powerful combination to achieve:
  - **High availability**
  - **Scalability**
  - **Elasticity**
  - **Agility**
- ELB distributes traffic
- ASG ensures the correct number of healthy EC2 instances are running

✅ **Exam Takeaway**

- Multi-AZ deployment → **High Availability**
- Bigger instance → **Vertical Scaling**
- More instances → **Horizontal Scaling**
- Automatic scaling → **Elasticity**
- Fast provisioning → **Agility**
- Traffic distribution → **Load Balancer**
- Automatic instance management → **Auto Scaling Group**

Amazon S3

S3 Overview

- **Amazon S3 (Simple Storage Service)** is a core AWS service and a **foundational building block** of the cloud.
- It provides **infinitely scalable object storage**.
- Widely used by:
  - Websites
  - Many AWS services
  - Large enterprises across the internet

Common Use Cases of Amazon S3

- Backup and file storage
- Disaster recovery (cross-region data storage)
- Archival storage (low-cost long-term storage with S3 Glacier)
- Hybrid cloud storage (on-premises + cloud)
- Hosting:
  - Static websites
  - Media files (images, videos)
  - Software distribution
- Data lakes for big data analytics
- Business intelligence and analytics
- Real-world examples:
  - **NASDAQ** stores 7 years of data in **S3 Glacier**
  - **Sysco** runs analytics on data stored in S3

S3 Buckets

- Files (objects) are stored inside **buckets**.
- Buckets:
  - Are created in a **specific AWS region**
  - Are visible globally from the S3 console
- Buckets act like **directories in the cloud**.

Bucket Naming

- Previously: bucket names had to be **globally unique**
- Now: **Account-level regional namespace** allows reuse of names
  - AWS automatically adds a unique suffix
- Naming rules:
  - No uppercase letters

  - No underscores (_)
  - Cannot look like an IP address
  - Must start with a lowercase letter or number
  - Must not start with xn--
  - Must not end with -s3-alias
- Best practice: use **simple lowercase names with letters and numbers**

S3 Objects

- Objects are the **files** uploaded to S3.
- Each object consists of:
  - **Key** (full file path)
  - **Value** (file content)

Object Keys

- The object key is the **full path** of the file.
- Example:
  - my_file.txt
  - my_folder1/another_folder/my_file.txt
- Keys have:
  - **Prefix** → folder path
  - **Object name** → file name
- S3 does **not truly have folders**—folders are part of the key name.

Object Characteristics

- **Maximum object size:** 50 TB
- **Multipart upload** is required for files larger than **5 GB**
- Objects can include:
  - **Metadata** (system or user-defined key-value pairs)
  - **Tags** (up to 10 Unicode key-value pairs)
    - Used for security and lifecycle rules
  - **Version ID** (if versioning is enabled)

Key Exam Takeaways

- S3 is **object storage**, not block or file storage
- Buckets are **region-specific**
- Objects are identified by **keys**, not directories
- S3 is massively scalable and highly durable
- Ideal for:
  - Backup

- Archival
- Static content
- Data lakes

✅ **Exam Tip**

If the question mentions:

- Infinite storage
- Object storage
- Backup or archival
- Static websites
- Data lakes

👉 **Think Amazon S3**

S3 Security: Bucket Policy

1. User-Based Security (IAM Policies)

- Security can be managed using **IAM policies**.
- IAM policies define **which S3 API calls** (e.g., GetObject, PutObject) are allowed or denied for:
    - IAM users
    - IAM groups
    - IAM roles
- Commonly used to grant:
    - Users access to S3
    - EC2 instances access to S3 via **IAM roles** (not IAM users)

2. Resource-Based Security (Bucket Policies)

- **S3 Bucket Policies** are attached directly to buckets.
- They apply **bucket-wide rules**.
- Used to:
    - Grant access to a specific user
    - Grant **cross-account access**
    - Make an S3 bucket **public**
    - Enforce encryption requirements
- Bucket policies are written in **JSON**.

3. Access Control Lists (ACLs)

- **Object ACLs**:
    - Fine-grained permissions at the object level
    - Can be **disabled**
- **Bucket ACLs**:
    - Less commonly used
    - Can also be disabled
- Modern best practice:
    - **Use Bucket Policies instead of ACLs**

4. How Access Is Granted to S3 Objects

An IAM principal can access an S3 object if:

- **IAM policy allows it**, OR
- **Bucket policy allows it**
- AND there is **no explicit deny**
- Encryption does not block access

5. S3 Bucket Policy Structure

- Bucket policies are **JSON-based**
- Key elements include:
  - **Resource**: bucket and objects the policy applies to
  - **Effect**: Allow or Deny
  - **Action**: S3 API calls (e.g., GetObject)
  - **Principal**: user, account, or * (everyone)
- Example use case:
  - Allow public read access using:
    - Principal: *
    - Action: s3:GetObject

6. Common S3 Security Use Cases

a. Public Access

- Achieved using **Bucket Policies**
- Allows website visitors to access objects
- Example: Static website hosting

b. IAM User Access (Same Account)

- Assign **IAM policy** to the user
- No bucket policy required

c. EC2 Instance Access

- Use an **IAM Role** attached to EC2
- IAM users should NOT be used on EC2

d. Cross-Account Access

- Must use **Bucket Policy**
- IAM user from another AWS account is specified in the policy

7. Encryption for S3 Objects

- Objects can be encrypted for additional security
- Encryption adds a **second protection layer**
- Can be enforced through bucket policies

8. Block Public Access (Critical Exam Topic)

- **Block Public Access** is an S3 security feature to prevent data leaks.
- When enabled:
  - Buckets **cannot be made public**
  - Even if a bucket policy allows public access
- Can be configured:
  - At **bucket level**
  - At **account level**
- Recommended to keep enabled unless public access is explicitly required

✅ **Best practice:**

If your buckets should never be public, **leave Block Public Access ON**.

Exam Tips

- IAM → user-based security
- Bucket Policy → resource-based security
- Public access → bucket policy + block public access disabled
- EC2 access → IAM Role
- Cross-account access → bucket policy
- Prevent data leaks → Block Public Access ON

S3 Website Overview

Amazon S3 Static Website Hosting – Key Points

- **Amazon S3 can host static websites** (HTML, CSS, JavaScript, images).
- Websites hosted on S3 are **accessible over the internet**.
- The website endpoint URL depends on the **AWS region** where the bucket is created.
    - URL formats may vary slightly (dash - vs dot .), but this detail is **not critical for the exam**.

How S3 Static Website Hosting Works

- You create an **S3 bucket**.
- Upload website files such as:
    - HTML files
    - Images
    - Other static assets
- Enable **Static Website Hosting** on the bucket.
- S3 generates a **website endpoint URL**.
- Users access the website directly via this URL.

Public Access Requirement

- For a static website to work, the S3 bucket **must allow public read access**.
- If the bucket or objects are **not public**, users will receive a **403 Forbidden error**.
- To allow public access:
    - Attach an **S3 Bucket Policy** that allows GetObject for everyone.
- This reinforces why understanding **S3 bucket policies** is important.

Common Error & Fix

- **403 Forbidden error** after enabling static website hosting means:
    - The bucket is **not public**
- Solution:
    - Add or fix the **bucket policy** to allow public reads
    - Ensure **Block Public Access** settings are appropriately configured

Use Cases

- Hosting:
  - Static websites
  - Landing pages
  - Documentation
  - Media-heavy content
- Cost-effective solution with **no servers to manage**

Exam Tips

- Static content + public access → **Amazon S3**
- 403 error → **Bucket policy / public access issue**
- No backend processing → **Static website**
- Hosting websites without EC2 → **S3 Static Website Hosting**

S3 Versioning Overview

Amazon S3 Versioning – Key Points

- **S3 Versioning** allows you to keep **multiple versions of the same object** in a bucket.
- Versioning is enabled **at the bucket level**.
- Once enabled, **every upload creates a new version** of an object instead of overwriting the previous one.

How Versioning Works

- Uploading a file creates **Version 1**.
- Re-uploading the same file (same key) creates **Version 2, Version 3**, and so on.
- Each version is uniquely identifiable.

Benefits of S3 Versioning

- Protects against **unintended or accidental deletes**:
  - Deleting an object adds a **delete marker** instead of permanently deleting the data.
- Enables **easy rollback**:
  - You can restore a **previous version** of a file if needed.
- Helps maintain **data integrity** when updating websites or files.
- Considered a **best practice** for important buckets.

Important Notes

- Objects uploaded **before versioning is enabled** have a version ID of **null**.
- **Suspending versioning**:
  - Does **not delete existing versions**
  - Is a **safe operation**
- Versioning does **not automatically reduce costs**:
  - Multiple versions mean **more storage usage**

Common Use Cases

- Safely updating **static websites**
- Data protection and rollback
- Accident recovery
- Auditing changes made to objects

✅ **Exam Tips**

- Enable versioning → **bucket-level setting**
- Delete does not remove data → **delete marker**
- Rollback needed → **versioning**
- Older objects before versioning → **version = null**

S3 Replication Overview

- **Amazon S3 Replication** automatically copies objects between S3 buckets.
- Replication is **asynchronous** (happens in the background).

Types of S3 Replication

1. Cross-Region Replication (CRR)

- Source and destination buckets are in **different AWS regions**
- Used for:
  - **Compliance requirements**
  - **Lower latency access** for users in different regions
  - **Cross-account replication**
  - Disaster recovery

2. Same-Region Replication (SRR)

- Source and destination buckets are in the **same AWS region**
- Used for:
  - **Aggregating logs** from multiple buckets
  - **Live replication** between production and test accounts
  - Data separation within the same region

Replication Requirements

- **Versioning must be enabled** on:
  - Source bucket
  - Destination bucket
- Buckets can be in:
  - The **same AWS account**
  - **Different AWS accounts**
- Proper **IAM permissions** must be granted so that:
  - Amazon S3 can **read from the source bucket**
  - Amazon S3 can **write to the destination bucket**

How Replication Works

- Objects are copied **after replication is enabled**.
- Replication does **not happen instantly** (asynchronous).
- S3 handles replication **automatically once configured**.

Use Case Summary

- **CRR** → compliance, disaster recovery, global access, cross-account
- **SRR** → log aggregation, test environments, same-region duplication

✅ **Exam Tips**

- Replication always requires **versioning**
- Cross-region → **CRR**
- Same-region → **SRR**
- Happens **asynchronously**
- Uses **IAM permissions** for S3

S3 Storage Classes Overview

Amazon S3 provides **multiple storage classes** to optimize **cost, performance, availability, and retrieval time** based on data access patterns.

Key Concepts

Durability

- Measures how unlikely data loss is.
- **Amazon S3 durability = 99.999999999% (11 nines)**.
- Same durability across **all S3 storage classes**.
- Example:
    - Out of 10 million objects, you may lose **1 object every 10,000 years**.

Availability

- Measures how often the service is accessible.
- Varies by storage class.
- Example:
    - **S3 Standard availability = 99.99%**
    - Approx. **53 minutes of downtime per year**.

Main S3 Storage Classes

1. S3 Standard – General Purpose

- Default storage class.
- For **frequently accessed data**.
- **Low latency, high throughput**.
- Availability: **99.99%**
- Use cases:
    - Big data analytics
    - Mobile & gaming applications
    - Content distribution
- Can withstand **two concurrent AZ failures**.

2. S3 Standard – Infrequent Access (IA)

- For **less frequently accessed data**.
- Lower cost than S3 Standard.
- **Retrieval fee applies**.

- Availability: **99.9%**
- Use cases:
    - Disaster recovery
    - Backups

3. S3 One Zone – Infrequent Access

- Data stored in **one Availability Zone only**.
- **Lower availability** than Standard-IA.
- Availability: **99.5%**
- Lower cost than Standard-IA.
- Data is lost if the AZ is destroyed.
- Use cases:
    - Secondary backups
    - Re-creatable or non-critical data

S3 Glacier Storage Classes (Archive Storage)

Glacier is **low-cost archival storage**.
You pay for **storage + retrieval**.

4. Glacier Instant Retrieval

- **Milliseconds retrieval**
- Minimum storage: **90 days**
- Use cases:
    - Quarterly accessed backups

5. Glacier Flexible Retrieval

- Formerly “S3 Glacier”
- Retrieval options:
    - **Expedited**: 1–5 minutes (paid)
    - **Standard**: 3–5 hours
    - **Bulk**: 5–12 hours (lowest cost)
- Minimum storage: **90 days**
- Use cases:
    - Archives where delay is acceptable

6. Glacier Deep Archive

- **Lowest-cost** storage class
- Retrieval:
  - **Standard**: ~12 hours
  - **Bulk**: ~48 hours
- Minimum storage: **180 days**
- Use cases:
  - Long-term retention
  - Regulatory archives

7. S3 Intelligent-Tiering

- Automatically moves data between tiers based on access patterns.
- Includes:
  - Frequent Access tier (default)
  - Infrequent Access tier (after ~30 days)
  - Archive Instant Access tier (after ~90 days)
  - Optional Archive & Deep Archive Access tiers
- Requires:
  - **Small monthly monitoring & automation fee**
- **No retrieval charges**
- Best for:
  - Unknown or changing access patterns
  - "Set it and forget it" storage optimization

Storage Class Movement

- Objects can be:
  - Assigned a storage class at upload
  - Changed manually
  - Automatically transitioned using **S3 Lifecycle Rules**

Exam-Friendly Summary

Requirement	Storage Class
Frequently accessed data	S3 Standard
Less frequent + quick access	S3 Standard-IA
Cheapest non-critical data	One Zone-IA
Archive, fast access	Glacier Instant
Archive, flexible timing	Glacier Flexible
Long-term archives	Glacier Deep Archive
Automatic optimization	Intelligent-Tiering

✅ **Exam Tips**

- All S3 classes have **11-nines durability**
- Availability decreases as cost decreases
- Archive classes have **retrieval delays**
- Intelligent-Tiering = **automatic + monitoring fee**
- Glacier = **archive + retrieval cost**

S3 Express One Zone

- **S3 Express One Zone** is a **high-performance Amazon S3 storage class**.
- Data is stored in a **single Availability Zone (AZ)**.
- It uses a special type of bucket called a **Directory Bucket** (not a standard S3 bucket).
- You **choose the specific AZ** where the data is stored.

Performance Characteristics

- Designed for **very low latency and very high throughput**.
- Supports **hundreds of thousands of requests per second**.
- Delivers **single-digit millisecond latency**.
- Provides **~10× higher performance** than S3 Standard.
- Costs about **50% less than S3 Standard** for performance-intensive workloads.

Durability & Availability

- **Good durability**, but:
  - **Lower availability** than standard S3 storage classes
  - Data exists in **only one AZ**
- If the chosen AZ has an outage, **data becomes unavailable**.

Key Design Idea

- Optimized to **co-locate compute and storage in the same AZ**.
- Helps:
  - Reduce latency
  - Reduce network traffic and network costs
- Best when **performance matters more than multi-AZ availability**.

Common Use Cases

- Latency-sensitive applications
- Data-intensive workloads
- **AI / ML training**
- Financial modeling
- Media processing
- High-Performance Computing (HPC)

AWS Service Integrations

Works especially well with:

- **Amazon SageMaker** (model training)
- **Amazon Athena**
- **Amazon EMR**
- **AWS Glue**
- Other big-data and analytics services

Exam Tips

- Single AZ + ultra-low latency → **S3 Express One Zone**
- Highest performance S3 storage class → **S3 Express One Zone**
- Needs directory bucket → **S3 Express One Zone**
- Not suitable when **multi-AZ availability** is required

✅ **Quick Comparison**

- **S3 Standard** → Multi-AZ, high availability
- **S3 Express One Zone** → Single-AZ, ultra-high performance

S3 Encryption

- **Amazon S3 encrypts data by default** when objects are uploaded.
- Encryption helps **protect data at rest**.

Two Types of Encryption in S3

1. Server-Side Encryption (SSE)

- **Enabled by default** in Amazon S3.
- The user uploads an object **unencrypted**.
- **Amazon S3 encrypts the object after it reaches the bucket**.
- Encryption and decryption are fully **managed by AWS**.
- Called *server-side* because encryption happens **on the AWS servers**.

✅ **Exam Tip:**
If encryption is not specified, assume **server-side encryption is enabled**.

2. Client-Side Encryption

- The user **encrypts the file before uploading it to S3**.
- Encryption keys are **managed by the user**.
- Amazon S3 only stores the **already encrypted object**.
- S3 does **not perform encryption** in this case.

Key Differences

Feature	Server-Side Encryption	Client-Side Encryption
Who encrypts the data	AWS (S3)	User
When encryption happens	After upload	Before upload
Default behavior	✅ Yes	❌ No

Exam Takeaways

- Default S3 encryption → **Server-Side Encryption**
- AWS handles encryption → **Server-Side Encryption**
- User encrypts data manually → **Client-Side Encryption**
- Both models are supported in AWS.

IAM Access Analyzer for S3

- **IAM Access Analyzer for S3** is a **monitoring and security feature**.
- It helps ensure that **only intended users and accounts** have access to your S3 buckets.
- It is commonly tested with **one conceptual question** in the exam.

What IAM Access Analyzer Does

- Analyzes S3 access configurations such as:
  - **S3 Bucket Policies**
  - **S3 Access Control Lists (ACLs)**
  - **S3 Access Point Policies**
- Identifies buckets that are:
  - **Publicly accessible**
  - **Shared with other AWS accounts**
- Surfaces these findings clearly so you can review them.

Why It's Important

- Helps detect **unintended data exposure**.
- Allows you to confirm whether access is:
  - **Expected and intentional**
  - Or a **potential security issue**
- Enables you to **take corrective action** if a bucket is accidentally shared.

Underlying Technology

- Powered by **IAM Access Analyzer**.
- IAM Access Analyzer identifies **resources shared outside your AWS account**.
- Works across AWS to improve **visibility into access permissions**.

Use Cases

- Security reviews and audits
- Preventing accidental public access
- Identifying cross-account sharing
- Improving S3 security posture

Exam Tips

- Monitoring S3 bucket access → **IAM Access Analyzer**
- Identify public or cross-account S3 access → **IAM Access Analyzer**
- Detect unintended sharing → **IAM Access Analyzer**
- Prevent data leaks → **Security analysis + remediation**

✅ **Key takeaway:**

IAM Access Analyzer **does not enforce permissions**—it **analyzes and reports** them.

Shared Responsibility Model for S3

- AWS is responsible for the **underlying S3 infrastructure**.
- This includes:
  - **Durability** of S3 data (11 nines).
  - **Availability** of the S3 service.
  - Ability to **withstand multiple facility failures**.
- AWS manages:
  - Hardware
  - Networking
  - Physical data centers
  - Internal configuration and patching
- AWS performs:
  - **Vulnerability analysis**
  - **Compliance validation**
  - Internal security controls
- AWS ensures the **S3 service itself works securely and reliably**.

Customer Responsibilities (Security *in* the Cloud)

- Properly configuring **S3 Versioning** to protect against accidental deletions.
- Defining and maintaining **S3 Bucket Policies** to control access.
- Enabling **encryption** for data stored in S3 (if required).
- Setting up **logging and monitoring** (optional but recommended).
- Choosing the **most cost-effective S3 storage class** for your data.
- Managing data access, permissions, and lifecycle rules.
- Ensuring data inside buckets is **secure and compliant**.

Key Exam Takeaways

- AWS → infrastructure, durability, availability, compliance.
- Customer → configuration, access control, encryption, monitoring, cost optimization.
- AWS does **not** manage your data or bucket configuration.
- Misconfigured bucket policies or missing backups are **customer responsibility**.

✅ **Exam Tip**

If a question asks:

- *Who is responsible for S3 durability or infrastructure?* → **AWS**
- *Who is responsible for versioning, encryption, access control, or data protection?* → **Customer**

AWS Snow Family Overview

What is AWS Snowball?

- AWS Snowball is a **highly secure, portable physical device**.
- Used to **migrate large amounts of data** in and out of AWS.
- Also supports **edge computing** (processing data at remote locations).
- Ideal when migrating **petabytes of data**.

Why Use Snowball Instead of the Internet?

- Transferring large data volumes over the internet can be:
  - Very **slow**
  - **Expensive**
  - Limited by **bandwidth**
  - Unstable or unreliable
- Example:
  - Transferring **100 TB** over a **1 Gbps** connection takes ~**12 days**
- Recommendation:
  - If a transfer takes **more than a week**, use **Snowball**

Types of Snowball Edge Devices

1. Snowball Edge – Storage Optimized

- Designed for **large-scale data migration**
- Storage capacity: **~210 TB**
- Best for:
  - Bulk data transfer
  - Backup
  - Archival migrations

2. Snowball Edge – Compute Optimized

- Designed for **edge computing**
- Storage capacity: **~28 TB**
- Includes **more compute power**
- Best for:
  - Processing data at the edge
  - Running applications locally

Data Migration Use Case

- Process:
    1. AWS ships a **Snowball device** to your location
    2. You **copy data locally** to the device
    3. Ship the device back to AWS
    4. AWS loads data into services like **Amazon S3**
- Benefits:
    - Saves time
    - Avoids network congestion
    - Lower cost for large transfers

Edge Computing Use Case

- Used in **remote or disconnected locations**, such as:
    - Ships
    - Trucks
    - Mining stations
    - Industrial sites
- Benefits:
    - Works with **limited or no internet**
    - Provides **local compute + storage**
- Capabilities:
    - Run **EC2 instances**
    - Run **AWS Lambda functions**
- Use cases:
    - Data pre-processing
    - Machine learning at the edge
    - Media transcoding at the edge
- Processed data can later be **sent back to AWS**

Key Exam Takeaways

- Large data migration (TBs–PBs) → **AWS Snowball**
- Slow or limited internet → **Snowball**
- Physical data transfer → **Snowball**
- Edge computing with local processing → **Snowball Edge**
- Migration + compute → **Snowball Edge devices**

✅ **Exam Tip**

If a question mentions:

- Shipping physical devices
- Offline or remote locations
- Massive data transfer
- Edge processing

👉 **Answer: AWS Snowball**

AWS Snowball Edge – Pricing

What You Pay For

- **Snowball Edge device usage**
- **Data transfer *out* of AWS**
- Pricing depends on **how you use the device** and **duration of usage**

Data Transfer Costs

- ✅ **Data transferred *into* Amazon S3 is FREE**
  - $0 per GB when importing data into S3 using Snowball
- ❌ **Data transferred *out* of AWS is charged**
  - Example: exporting data from AWS onto a Snowball device

✅ **Exam Tip:**
Uploading data to Amazon S3 is always **free**.

Device Pricing Options

1. On-Demand Pricing

- Pay a **one-time service fee per Snowball job**
- Includes **free usage days**:
  - **80-TB device** → 10 days of usage
  - **210-TB device** → 15 days of usage
- **Shipping is free**
  - Shipping days do **not** count toward usage limits
- Extra usage beyond free days is **charged per additional day**

2. Committed Upfront Pricing

- Used mainly for **edge computing workloads**
- Pay in advance for:
  - Monthly
  - 1-year
  - 3-year commitments
- Provides **up to 62% cost savings**
- Best for **long-term Snowball Edge usage**

Key Exam Takeaways

- ✅ Data **into S3** = **$0**
- 💰 Data **out of AWS** = **charged**
- 📦 You pay for:
  - Device usage
  - Extra usage days (if exceeded)
- 🚚 Shipping is **free**
- 💸 Long-term commitments = **big discounts**

✅ **One-Line Exam Memory Aid**

*Everything costs money except putting data **into Amazon S3*** ✅

Storage Gateway Overview

Hybrid Cloud Concept

- A **hybrid cloud** combines:
  - **On-premises infrastructure**
  - **AWS cloud services**
- Common reasons for hybrid cloud:
  - Gradual or long-term cloud migration
  - Security or compliance requirements
  - Existing on-prem infrastructure investments
  - Business or regulatory strategy

Why Amazon S3 Needs a Bridge

- **Amazon S3 is proprietary object storage**.
- It **cannot be directly accessed** from on-premises systems like:
  - NFS
  - EFS-style file systems
- To expose S3 data to on-premises environments, AWS provides **AWS Storage Gateway**.

AWS Storage Gateway – Core Idea

- **AWS Storage Gateway** bridges:
  - **On-premises storage systems**
  - **AWS cloud storage**
- Enables on-prem applications to **seamlessly use AWS storage**.
- Common use cases:
  - Backup and restore
  - Disaster recovery
  - Storage tiering
  - Hybrid cloud architectures

How Storage Gateway Fits with AWS Storage Types

- **Block Storage**:
  - Amazon EBS
  - EC2 Instance Store
- **File Storage**:

    - Amazon EFS
- **Object Storage**:
    - Amazon S3
    - Amazon S3 Glacier
- **Storage Gateway**:
    - Connects **on-premises storage** to AWS services above

Types of AWS Storage Gateway

(High-level knowledge only for CCP exam)

- **File Gateway**
- **Volume Gateway**
- **Tape Gateway**

✅ **Exam Note:**

You do **not** need in-depth details of each type for CCP—just know **what Storage Gateway does**.

Behind the Scenes

- AWS Storage Gateway uses:
    - **Amazon S3**
    - **Amazon Glacier**
    - **Amazon EBS**
- These services power hybrid storage without users managing the internals.

CCP Exam Takeaways

- Hybrid cloud = **on-prem + AWS**
- S3 cannot be directly mounted on-prem → need **Storage Gateway**
- Storage Gateway enables:
    - Hybrid backup
    - Hybrid disaster recovery
    - Cloud storage extension
- Storage Gateway = **bridge between on-prem storage and AWS cloud**

✅ **One-Line Exam Memory Tip**

*Hybrid storage with on-prem systems accessing AWS storage → **AWS Storage Gateway*** ✅

S3 Summary

Buckets & Objects

- **Buckets**:
  - Containers for objects (files)
  - Tied to a **specific AWS region**
  - Bucket names are **globally unique**
- **Objects**:
  - Files stored inside buckets
  - Identified by a **key (full path name)**

S3 Security

- **IAM Policies**:
  - Attached to IAM users or roles
  - Control who can access S3 and what actions are allowed
- **S3 Bucket Policies**:
  - Resource-based policies attached directly to buckets
  - Used to:
    - Make buckets public
    - Enable cross-account access
- **S3 Encryption**:
  - Used to protect data at rest
  - Can be enabled to secure objects inside buckets

Static Website Hosting

- Amazon S3 can **host static websites** (HTML, CSS, images).
- Bucket must have:
  - **Static website hosting enabled**
  - **Public read access**
- Widely used for low-cost website hosting without servers.

S3 Versioning

- Enables **multiple versions of objects** in a bucket.
- Protects against:
  - Accidental deletion (delete markers)

    - Unwanted overwrites
- Allows easy **rollback to previous versions**.

S3 Replication

- **Same-Region Replication (SRR)**:
    - Replicates data between buckets in the same region
- **Cross-Region Replication (CRR)**:
    - Replicates data across different regions
- **Versioning must be enabled** on both source and destination buckets.

S3 Storage Classes

- **S3 Standard** – frequently accessed data
- **S3 Infrequent Access (IA)** – less frequent access
- **S3 One Zone-IA** – low cost, single AZ
- **S3 Intelligent-Tiering** – automatic cost optimization
- **Glacier Storage Classes** (for archival):
    - Glacier Instant Retrieval
    - Glacier Flexible Retrieval
    - Glacier Deep Archive

AWS Snowball

- Uses **physical devices** to transfer large amounts of data into S3.
- Ideal for:
    - Massive data migrations
    - Limited or slow network connectivity
- Supports **edge computing** on the device itself.

AWS Storage Gateway

- Enables **hybrid cloud storage**.
- Bridges **on-premises storage** with AWS storage services.
- Commonly used to:
    - Extend on-prem storage to Amazon S3
    - Enable backup and disaster recovery in AWS

✅ **Key Exam Takeaway**

Amazon S3 is a **foundational AWS service** offering:

- Secure, durable object storage
- Multiple storage classes
- Built-in versioning, replication, and encryption
- Integration with hybrid and migration services like **Snowball** and **Storage Gateway**

Cloud Practitioner

Databases & Analytics

Database Introduction

What is a Database?

- A **database** is used to store data in a **structured format**.
- Unlike raw storage (EBS, EC2 Instance Store, S3, EFS), databases:
    - Allow **efficient querying and searching**
    - Support **indexing**
    - Can define **relationships between data**

🔹 **Why Use Databases Instead of File Storage?**

- File storage → basic file operations only
- Databases →
    - Structured data organization
    - Faster queries
    - Relationships between datasets
    - Better performance and scalability

🔹 **Types of Databases**

1. ✅ Relational Databases (SQL)

- Data stored in **tables (like Excel sheets)**
- Tables are **linked using relationships (keys)**
- Use **SQL (Structured Query Language)** for queries

Example:

- Students table → StudentID, DeptID
- Departments table → DeptID
- Relationship through DeptID

Key Points:

- Structured schema (fixed format)
- Strong relationships between tables
- SQL-based querying
- Harder to scale horizontally (usually vertical scaling)

2. ✅ NoSQL Databases (Non-relational)

- Designed for **modern, scalable applications**
- Use **flexible schema** (data structure can change)
- Optimized for **specific data models**

Types of NoSQL:

- Key-value
- Document
- Graph
- In-memory
- Search databases

Key Benefits:

- Flexible schema
- Easy to evolve data model
- High scalability (**horizontal scaling**)
- High performance for specific use cases

🔹 **JSON in NoSQL**

- Common format used in NoSQL databases
- Stands for **JavaScript Object Notation**

Features:

- Supports **nested data**
- Fields can change over time
- Supports **arrays**

Example characteristics:

- Data is not tabular (unlike SQL)
- Can include nested objects (e.g., address inside user)
- Flexible structure

🔹 **Scaling Concepts**

- **Relational DBs → Vertical scaling** (increase size of one server)
- **NoSQL DBs → Horizontal scaling** (add more servers)

🔹 **AWS Managed Databases (Important for Exams)**

✅ **Benefits of Managed Databases:**

- Easy and fast **provisioning**
- Built-in **high availability**
- Supports **vertical & horizontal scaling**
- Automated:
  - Backups
  - Restores
  - OS patching
  - Upgrades
- Integrated **monitoring & alerting**

✅ **AWS Responsibility:**

- Handles:
  - OS patching
  - Maintenance
  - Availability
- You focus on:
  - Data
  - Application logic

🔹 **Self-Managed Databases on EC2**

If you run DB on EC2, you must manage:

- Backups
- Patching
- High availability
- Fault tolerance
- Scaling

👉 **Conclusion:** Managed databases are highly recommended.

RDS & Aurora Overview

Amazon RDS (Relational Database Service)

✅ **Definition**

- **RDS = Managed relational database service** on AWS
- Uses **SQL** (supports relational databases)

✅ **Supported Database Engines**

- PostgreSQL
- MySQL
- MariaDB
- Oracle
- Microsoft SQL Server
- IBM DB2
- Amazon Aurora (AWS proprietary) [aws.plainenglish.io]

✅ **Why Use RDS (instead of EC2 DB)?**

- Fully **managed by AWS**
- Automatic:
  - Provisioning
  - OS patching
  - Backups & restore (Point-in-Time Restore)
- Monitoring dashboards (CloudWatch)
- Easy scaling
- High availability support

👉 AWS handles infrastructure, you focus on application

✅ **Key Features**

- **Automated backups + restore**
- **Read Replicas** → improve read performance
- **Multi-AZ deployment** → disaster recovery
- **Maintenance windows** → controlled updates
- **Scaling**:
  - Vertical (instance size)

    - Horizontal (read replicas)
- Storage backed by **EBS**
- **No SSH access** → fully managed service [aws.plainenglish.io]

✅ **Architecture (Important)**

Typical 3-tier architecture:

1. Load Balancer → handles requests
2. EC2 instances → application logic
3. RDS → database (shared storage)

🔹 **Amazon Aurora**

✅ **Definition**

- AWS **proprietary relational database**
- Fully managed like RDS
- Compatible with:
    - MySQL
    - PostgreSQL [docs.aws.amazon.com]

✅ **Key Advantages**

- **High performance**
    - Up to **5× faster than MySQL**
    - Up to **3× faster than PostgreSQL** [docs.aws.amazon.com]
- Cloud-optimized architecture
- Higher scalability

✅ **Storage Features**

- **Auto-scaling storage**
- Grows automatically (no manual provisioning)
- Up to very large sizes (hundreds of TB range) [docs.aws.amazon.com]

✅ **Cost**

- ~20% more expensive than RDS
- More efficient → often **cost-effective for high workloads**

✅ **Key Comparison (Exam Tip)**

- **RDS** → traditional managed DB (many engines)
- **Aurora** → cloud-native, high-performance DB

🔹 **Aurora Serverless**

✅ **Definition**

- **Serverless version of Aurora**
- Fully automated database scaling

✅ **Key Features**

- **No capacity planning required**
- **Autoscaling** based on usage
- Pay-per-second billing
- No server management

✅ **Best Use Cases**

- Infrequent workloads
- Intermittent usage
- Unpredictable traffic

✅ **How It Works**

- Clients connect to a **proxy layer**
- Aurora automatically:
  - Starts DB instances
  - Scales up/down as needed
- Shared storage across instances

✅ **Key Exam Clue** ⭐

- If question says:
  - "No management"
  - "Auto scaling DB"
  - "Unpredictable workload"

👉 **Answer = Aurora Serverless**

🔹 **Quick Summary Table**

Feature	RDS	Aurora	Aurora Serverless
Type	Managed relational DB	Cloud-native relational DB	Serverless DB
Engines	Many (MySQL, Oracle, etc.)	MySQL & PostgreSQL	Same as Aurora
Performance	Standard	Very high	High (auto-scale)
Scaling	Manual + replicas	Auto storage scaling	Fully auto scaling
Management	Managed	Managed	Fully serverless
Use case	General apps	High-performance apps	Unpredictable workloads

---

🔹 **Final Exam Tips** ⭐

- **RDS → default choice for relational DB**
- **Aurora → high performance + cloud optimization**
- **Aurora Serverless → auto-scaling + no admin**

Creating an RDS Database (Console Steps)

✅ **Basic Steps**

- Go to **RDS/Aurora Console → Databases → Create database**
- Choose:
  - **Standard (Full configuration)** → more control
  - Easy create → auto settings (not used here) [docs.aws.amazon.com]

✅ **Engine Selection**

- Choose database engine:
  - Aurora (MySQL/Postgres)
  - MySQL, PostgreSQL, etc.
- Example used: **MySQL (common & simple)**

✅ **Templates (Very Important)**

- **Production**:
  - Multi-AZ deployment
  - High availability (multiple instances)
- **Dev/Test**

- **Free Tier**:
  - Single-AZ only (1 instance)
  - Used for practice

👉 Exam Tip:

- **Production → Multi-AZ**
- **Free tier → Single instance**

✅ **Credentials**

- Options:
  - Self-managed password
  - AWS Secrets Manager (more secure but costs extra)
- Authentication:
  - Password-based (used)
  - IAM authentication (optional)

✅ **Instance & Storage Configuration**

- Instance type:
  - Free tier → **db.t4g.micro / db.t3.micro**
- Storage:
  - Default ~20 GB
- Feature:
  - **Storage autoscaling** (can increase storage automatically)

✅ **Connectivity Settings**

- Choose:
  - VPC (default)
  - Public access (Yes for testing)
- Security group:
  - Create new
  - Allow inbound traffic (e.g., port **3306 for MySQL**) [linkedin.com]

✅ **Monitoring & Settings**

- Enable:
  - Monitoring dashboards
  - Logs (optional)
- RDS provides built-in:
  - CPU metrics

- Logs
- Performance insights

✅ **Database Creation**

- Click **Create database**
- Status changes:
  - Creating → Available [docs.aws.amazon.com]

🔹 **After Creation (Important)**

✅ **Connection Details**

- Get:
  - **Endpoint + Port**
- Used to connect your application/database client

✅ **Security**

- Security group allows:
  - Access via port (e.g., 3306 for MySQL)

✅ **Monitoring Benefits**

- Built-in:
  - CPU utilization
  - Logs
  - Performance metrics

👉 Managed service advantage: no manual setup needed

🔹 **Snapshots (Very Important Topic)**

✅ **What is Snapshot?**

- Backup of database state
- Used for restore or migration

✅ **Snapshot Operations**

- **Create snapshot** → backup database
- **Restore snapshot** → create new DB from backup
- **Copy snapshot** → move to another region (disaster recovery)
- **Share snapshot** → share with other AWS accounts [docs.amazonaws.cn]

✅ **Use Cases**

- Backup & recovery
- Create duplicate databases
- Cross-region disaster recovery

🔹 **Deleting Resources**

✅ **Delete Database**

- Option to:
  - Create final snapshot (backup)
  - Retain automated backups
- Confirm deletion manually

🔹 **Key Concepts for Exams** ⭐

✅ **RDS Setup**

- Engine selection is critical
- Templates define architecture (Single-AZ vs Multi-AZ)

✅ **Ports**

- MySQL → **3306**

✅ **Security**

- Controlled via **Security Groups**

✅ **Snapshots**

- Core AWS backup mechanism:
  - Restore
  - Copy
  - Share

✅ **Managed Service Advantage**

- Easy provisioning
- Built-in monitoring
- Integrated backup
- No infrastructure management

RDS Deployment Option

RDS Deployment Types (Very Important for Exams)

There are **3 main architectures**:

1. Read Replicas
2. Multi-AZ
3. Multi-Region

🔹 **1. RDS Read Replicas**

✅ **Purpose**

- Used to **scale read operations**

✅ **How It Works**

- Create **copies (replicas)** of the main database
- Applications can **read from replicas**
- **Writes only go to the main DB**

✅ **Key Features**

- Up to **15 read replicas**
- Helps distribute heavy read traffic
- Improves application performance

✅ **Important Notes**

- Read replicas are **read-only**
- Replication is typically **asynchronous**
- Used for **performance scaling**, NOT failover

✅ **Exam Tip** ⭐

👉 If question says:

- "Increase read performance"
- "Heavy read workload"

→ **Answer = Read Replica**

🔹 **2. Multi-AZ Deployment**

✅ **Purpose**

- **High availability + disaster recovery (within region)**

✅ **How It Works**

- Primary DB in one AZ
- **Standby (replica) in another AZ**
- Data is **synchronously replicated**

✅ **Failover Behavior**

- If primary DB fails:
  - Automatic **failover to standby**
- Applications switch seamlessly to new DB

✅ **Key Features**

- Standby DB is **not accessible**
- Used only during failure
- Only **one standby AZ**

✅ **Important Notes**

- No performance improvement
- Used for **failover protection**
- Same endpoint used after failover [rapyder.com]

✅ **Exam Tip** ⭐

👉 If question says:

- "High availability"
- "Failover"
- "AZ failure"

→ **Answer = Multi-AZ**

🔹 **3. Multi-Region Deployment**

✅ **Purpose**

- **Disaster recovery across regions**
- **Low latency for global users**

✅ **How It Works**

- Primary DB in one region
- Read replicas in **other regions**
- Applications read from **local region replica**

✅ **Key Features**

- Faster local reads (low latency)
- Writes still go to **primary region**
- Supports global applications

✅ **Advantages**

- **Disaster recovery (region failure)**
- Better **global performance**

✅ **Limitation**

- **Replication cost** (cross-region data transfer)

✅ **Exam Tip** ⭐

👉 If question says:

- “Global users”
- “Cross-region DR”
- “Low latency worldwide”

→ **Answer = Multi-Region Read Replica**

- **Quick Comparison Table**

Feature	Read Replica	Multi-AZ	Multi-Region
Purpose	Scale reads	High availability	Global performance + DR
Writes	Primary only	Primary only	Primary region
Reads	From replicas	From primary only	From local replicas
Failover	No	Yes (automatic)	Manual / architecture dependent
Location	Same region	Different AZ	Different regions
Use case	Performance	Reliability	Global apps

- **Final Exam Summary** ⭐
  - **Read Replica → Scaling (reads)**
  - **Multi-AZ → High Availability (failover)**
  - **Multi-Region → Global apps + DR**

ElastiCache Overview

🔹 **What is Amazon ElastiCache?**

✅ **Definition**

- **ElastiCache = Managed in-memory database service**
- Supports:
  - **Redis**
  - **Memcached** [docs.aws.amazon.com]

✅ **Key Concept**

- Data is stored in **memory (RAM)** → not disk
- Result → **Very fast performance (low latency)**

👉 Exam keyword:

- **"In-memory database" → ElastiCache**

🔹 **Why Use ElastiCache?**

✅ **Purpose**

- **Reduce load on primary database (RDS/DynamoDB)**
- Improve **performance for read-heavy workloads**

✅ **How It Works**

- Frequently accessed data is stored in **cache**
- Instead of querying RDS repeatedly:
  - Application first checks **ElastiCache**
  - If data exists → return quickly
  - If not → fetch from RDS and store in cache

👉 This reduces repeated queries to RDS [aws.plainenglish.io]

🔹 **Key Benefits**

✅ **High Performance**

- **Microsecond / sub-millisecond latency**
- Much faster than disk-based databases [aws.amazon.com]

✅ **Reduces Database Load**

- Offloads read traffic from RDS
- Improves scalability and efficiency

✅ **Fully Managed**

AWS handles:

- OS patching
- Maintenance
- Monitoring
- Backups
- Failure recovery [aws.amazon.com]

✅ **Scalability**

- Scales to handle **high throughput**
- Supports distributed caching

🔹 **Architecture (Very Important)**

✅ **Typical Flow**

1. User request → Load Balancer
2. EC2 instances process requests
3. Try to read from **ElastiCache (fast)**
4. If not found → read from **RDS (slower)**
5. Store result in cache

✅ **Key Idea**

- **Cache = shortcut for repeated queries**
- Moves load from:
  - ❌ Slow (RDS)
  - ✅ Fast (ElastiCache)

🔹 **Cache Characteristics**

- Stores:
  - Frequently accessed data
  - Temporary data (often)
- Works best for:
  - Repeated queries
  - Read-heavy workloads

🔹 **Engines (Exam Point)**

Engine	Description
Redis	Advanced features, persistence, data structures
Memcached	Simple, high-speed key-value cache

🔹 **Key Exam Triggers** ⭐

👉 If question mentions:

- "Improve performance"
- "Reduce database load"
- "Repeated queries"
- "Low latency"
- "In-memory caching"

➡️ **Answer = ElastiCache**

🔹 **RDS vs ElastiCache (Quick View)**

Feature	RDS	ElastiCache
Storage	Disk-based	In-memory
Speed	Slower	Very fast
Purpose	Main database	Cache layer
Use case	Data storage	Speed optimization

- **Final Summary** ⭐
  - **ElastiCache = In-memory cache**
  - Used with RDS to:
    - Speed up applications
    - Reduce database load
  - Best for:
    - **Read-heavy workloads**
    - **Repeated queries**

DynamoDB Overview

🔹 **Amazon DynamoDB**

✅ **Definition**

- **DynamoDB = Fully managed, serverless NoSQL database**
- Designed for **high performance and scalability** [docs.aws.amazon.com]

✅ **Key Characteristics**

- **Serverless**
  - No server provisioning or management
- **NoSQL**
  - Non-relational (key-value / document model)
- **Fully managed by AWS**
- **Highly available**
  - Data replicated across multiple AZs [geeksforgeeks.org]

✅ **Performance & Scale**

- Handles:
  - **Millions of requests per second**
  - Very large data (TBs, billions of items)
- **Low latency**
  - Single-digit millisecond response time [docs.aws.amazon.com]

✅ **Key Features**

- **Auto-scaling** (adjusts capacity automatically)
- **IAM integration** (security & access control)
- **Cost-efficient**
- Storage classes:
  - Standard
  - Infrequent Access (IA)

✅ **Exam Keywords** ⭐

👉 If you see:

- "Serverless database"
- "NoSQL"
- "Single-digit millisecond latency"
- "Auto-scaling"

➡️ **Answer = DynamoDB**

🔹 **DynamoDB Data Model**

✅ **Structure**

- **Table → Items → Attributes**
- Item = row
- Attributes = columns

✅ **Primary Key**

- **Partition Key (required)**
- **Sort Key (optional)**

👉 Key-value model (simple + flexible) [docs.aws.amazon.com]

✅ **Important Notes**

- Schema-less (flexible structure)
- Each record stored independently

🔹 **When to Use DynamoDB**

- Real-time applications
- Large-scale workloads
- Low-latency requirements
- Serverless architectures

🔹 **DynamoDB Accelerator (DAX)**

✅ **Definition**

- **DAX = In-memory cache for DynamoDB**
- Fully managed caching layer [docs.aws.amazon.com]

✅ **Purpose**

- Improve **read performance**
- Reduce repeated reads on DynamoDB

✅ **Key Features**

- **Microsecond latency** (vs milliseconds)
- Up to **~10x faster performance**
- Works as **cache in front of DynamoDB**

✅ **How It Works**

- Cache frequently accessed data
- Applications read from:
  - DAX → fast response
  - DynamoDB → if cache miss

✅ **Benefits**

- Faster reads
- Reduced cost (less direct DB reads)
- Handles burst traffic

✅ **Usage Conditions**

- Best for:
  - Read-heavy workloads
  - Repeated queries
- Less useful for:
  - Write-heavy workloads [docs.aws.amazon.com]

🔹 **DAX vs ElastiCache (Important Difference)**

Feature	DAX	ElastiCache
Scope	Only DynamoDB	Any database
Purpose	DynamoDB cache	General-purpose cache
Integration	Built-in with DynamoDB	External integration

🔹 **Quick Summary (Exam Revision)**

- **DynamoDB**
  - Serverless NoSQL DB
  - Massive scale + low latency
- **DAX**
  - In-memory cache for DynamoDB
  - Microsecond performance

🔹 **Final Exam Tips** ⭐

- **DynamoDB → serverless + scalable NoSQL**
- **DAX → caching for DynamoDB (ultra-fast reads)**
- If question includes:
  - "Repeated reads + ultra low latency" → **DynamoDB + DAX**

✅ **DynamoDB – Important Points (Quick Revision)**

🔹 **Table Creation**

- In DynamoDB, you create a **table directly (no database creation needed)**.
- DynamoDB is **serverless** → no need to manage servers or infrastructure.
- You specify only the **primary key** during table creation.

🔹 **Primary Key (Must Know)**

- A **partition key** is mandatory (e.g., user_id).
- It uniquely identifies items (when no sort key is used). [dynamodbguide.com]
- DynamoDB uses this key to **store and retrieve data efficiently using hashing**. [docs.aws.amazon.com]

👉 For exam:

- Partition key = required
- Sort key = optional (often ignored in Cloud Practitioner level)

🔹 **Serverless Concept**

- DynamoDB is a **fully managed NoSQL service**.
- AWS handles:
  - Storage
  - Scaling
  - Performance
- You just say:

  👉 "Create table" → AWS does the rest

🔹 **Data Structure (Core Concepts)**

- **Table** → collection of data
- **Item** → single record (like a row)
- **Attributes** → fields in a record (like columns) [dynamodbguide.com]

🔹 **Flexible Schema (VERY IMPORTANT)**

- DynamoDB is **schema-less (NoSQL)**.
- Only the **primary key is required**.
- Other attributes can vary between items.

✅ Example:

- Item 1 → user_id, first_name, last_name, favorite_number
- Item 2 → user_id, first_name only

👉 Both are valid ✅

🔹 **Data Types**

- Supports multiple attribute types:
  - String
  - Number
  - etc.

🔹 **NoSQL vs RDS (Key Difference)**

DynamoDB:

- NoSQL database
- **No joins between tables**
- Data must be stored together (denormalized)

RDS:

- Relational database

- Supports joins and relationships

🔹 **Single Table Concept**

- Data is often stored in **one main table**.
- You design carefully because:
  - No joins later
  - All required data should be in the same table

🔹 **Key Characteristics of DynamoDB**

- ✅ Serverless
- ✅ Highly scalable
- ✅ Flexible schema
- ✅ Fast access using keys
- ❌ No joins
- ❌ No fixed schema

DynamoDB Global Tables

⭐ **DynamoDB Global Tables – Key Points**

- **Definition**
  DynamoDB Global Tables allow you to create a **multi-region, fully replicated database** for low-latency access.
- **Purpose**
  Designed to provide **low latency reads and writes** for users across different geographic regions.
- **Multi-Region Setup**
  - You can replicate a DynamoDB table across **1 to 10 AWS regions**.
  - Example:
    - Primary table in **us-east-1 (N. Virginia)**
    - Replica in **eu-west-3 (Paris)**
- **Data Replication**
  - **Automatic, two-way (bidirectional) replication** between regions
  - Ensures **same data is available everywhere**
- **Active-Active Architecture**
  - Users can **read and write in any region**
  - Changes are **automatically synced across all regions**
  - No primary/secondary concept — all regions are active
- **Performance Benefit**
  - Users access the **nearest region**, reducing latency significantly
- **Scalability & Availability**
  - Helps achieve **high availability and fault tolerance**
  - If one region fails, others continue to serve traffic

📌 **One-Line Summary**

DynamoDB Global Tables provide **fully managed, multi-region, active-active replication**, enabling **low-latency access and high availability worldwide**.

Redshift Overview

⭐ **Amazon Redshift – Key Points**

📌 **Purpose & Type**

- **Redshift = OLAP database** (Online Analytical Processing)
- Used for:
  - **Data warehousing**
  - **Analytics & reporting**
- ❌ Not suitable for OLTP (transactional workloads like RDS)

📌 **Core Characteristics**

- Based on **PostgreSQL**
- Designed for **analyzing large datasets**
- Handles **complex queries and computations efficiently**

📌 **Performance & Scalability**

- Provides **high performance (up to 10x faster)** than traditional warehouses
- Scales to **petabytes of data**
- Uses:
  - **Columnar storage** (stores data by columns, not rows)
  - **MPP (Massively Parallel Processing)** engine for fast query execution

📌 **Data Loading**

- Data is **loaded periodically** (e.g., hourly batch loads)
- Not typically used for continuous real-time writes

📌 **Querying & Integration**

- Supports **SQL interface**
- Integrates with BI tools like:
  - **Amazon QuickSight**
  - **Tableau**
- Used for:
  - **Dashboards**
  - **Reporting**

- **Data visualization**

📌 **Pricing**

- **Pay-as-you-go**
- Based on **provisioned instances**

⭐ **Redshift Serverless – Key Points**

📌 **Definition**

- **Serverless version of Redshift**
- No need to manage infrastructure or capacity

📌 **Key Benefits**

- ✅ **Automatic scaling** based on workload
- ✅ **No provisioning needed**
- ✅ **Cost-efficient (pay only for usage)**

📌 **Use Cases**

- Reporting applications
- Dashboarding
- Real-time analytics
- Ad-hoc queries

📌 **How It Works**

- Enable **Redshift Serverless**
- Connect using:
  - Query Editor
  - BI tools
- Redshift:
  - Automatically **provisions compute**
  - Runs queries
  - Scales resources dynamically

📌 **Pricing Model**

- Pay only for:
  - **Compute used during queries**
  - **Storage**

📌 **One-Line Summary**

Amazon Redshift is a columnar, OLAP data warehouse for analytics, while Redshift Serverless removes infrastructure management and charges only for actual usage.

🎯 **Exam Tip**

- **Analytics + Warehouse → Redshift**
- **Columnar + MPP → Redshift**
- **No infrastructure + auto scaling → Redshift Serverless**

EMR Overview

⭐ **Amazon EMR – Key Points**

📌 **Definition**

- **EMR (Elastic MapReduce)** is **not a database**
- It is a service to create and manage **Hadoop clusters on AWS**

📌 **Purpose**

- Used for **big data processing and analysis**
- Helps process **large-scale datasets efficiently**

📌 **Hadoop & Cluster Concept**

- Based on **Hadoop (open-source framework)**
- Uses **multiple servers (EC2 instances)** working together as a cluster
- Can scale to **hundreds of EC2 instances**

📌 **Ecosystem Tools**

EMR supports popular big data frameworks:

- **Apache Spark**
- **HBase**
- **Presto**
- **Flink**

📌 **What EMR Manages**

- ✅ Provisions EC2 instances
- ✅ Configures the cluster automatically
- ✅ Ensures all nodes work together

📌 **Key Features**

- **Auto scaling** (adjusts cluster size based on workload)
- Integration with **Spot Instances** (cost savings)

📌 **Use Cases**

- Big data processing

- Machine learning
- Data analytics
- Web indexing

🎯 **Exam Tip**

- **Hadoop / Big Data / Cluster → Amazon EMR**
- If question mentions:
  - Spark, HBase, Presto, Flink → ✅ **EMR**
  - Processing large datasets using clusters → ✅ **EMR**

📌 **One-Line Summary**

Amazon EMR is a managed service to run Hadoop clusters on AWS for large-scale data processing and analytics.

Athena Overview

⭐ Amazon Athena – Key Points

📌 Definition

- **Amazon Athena** is a **serverless query service**
- Used to **analyze data directly from Amazon S3**

📌 Core Concept

- Query data using **SQL**
- No need to:
  - Load data into a database
  - Manage infrastructure

📌 Data Sources & Formats

- Works with data stored in **Amazon S3**
- Supports multiple file formats:
  - **CSV**
  - **JSON**
  - **ORC**
  - **Avro**
  - **Parquet**

📌 Architecture & Engine

- Built on **Presto (distributed SQL query engine)**

📌 How It Works

1. Store data in **S3**
2. Use **Athena to run SQL queries**
3. Analyze results directly
4. Optionally visualize using **Amazon QuickSight**

📌 Pricing

- Charges based on **data scanned**
- Approx: **$5 per TB scanned**

- Cost optimization:
  - ✅ Use **compressed data**
  - ✅ Use **columnar formats** (e.g., Parquet, ORC)

📌 **Use Cases**

- Business intelligence (BI)
- Data analytics
- Reporting
- Log analysis:
  - **VPC Flow Logs**
  - **ELB Logs**
  - **CloudTrail logs**
  - Application logs

🎯 **Exam Tip**

- **Serverless + SQL + S3 data → Athena**
- **Analyze logs in S3 → Athena**

📌 **One-Line Summary**

Amazon Athena is a serverless service that lets you run SQL queries directly on data stored in S3 without managing infrastructure.

Quicksight Overview

⭐ Amazon QuickSight – Key Points

📌 Definition

- **Amazon QuickSight** is a **serverless Business Intelligence (BI) service**
- Used to create **interactive dashboards and visualizations**
- Powered by **machine learning**

📌 Purpose

- Helps **visualize data** and generate **business insights**
- Enables users to create:
  - Charts
  - Graphs
  - Dashboards

📌 Key Features

- ✅ **Serverless** (no infrastructure to manage)
- ✅ **Automatically scalable**
- ✅ **Fast performance**
- ✅ **Embeddable dashboards** (can embed into apps)
- ✅ **Per-session pricing** (cost-effective)

📌 Use Cases

- Business analytics
- Data visualization
- Ad-hoc analysis
- Dashboard creation
- Extracting insights from data

📌 Integrations

Works with multiple AWS data sources:

- **Amazon RDS**
- **Amazon Aurora**
- **Amazon Athena**

- **Amazon Redshift**
- **Amazon S3**

🎯 **Exam Tip**

- **Dashboards + Visualization + BI → QuickSight**
- If question asks:
  - "Create visual reports from data" → ✅ QuickSight
  - "Business insights through charts" → ✅ QuickSight

📌 **One-Line Summary**

Amazon QuickSight is a serverless BI tool used to create interactive dashboards and visualize data from various AWS data sources.

DocumentDB Overview

⭐ **Amazon DocumentDB – Key Points**

📌 **Definition**

- **Amazon DocumentDB** is a **fully managed NoSQL database**
- Compatible with **MongoDB**
- Considered AWS's **cloud-native alternative to MongoDB** (similar to how Aurora relates to MySQL/PostgreSQL)

📌 **Database Type**

- **NoSQL database**
- Stores data in **JSON-like documents**

📌 **Core Features**

- ✅ **MongoDB compatibility** (use same tools & APIs)
- ✅ Stores, queries, and indexes **JSON data**
- ✅ Designed for **high performance and scalability**

📌 **Architecture & Availability**

- **Fully managed service**
- **Highly available**
- Data is:
  - Automatically replicated across **3 Availability Zones (AZs)**
- Ensures **durability and fault tolerance**

📌 **Scalability**

- Storage **auto-scales** in increments of **10 GB**
- Can handle **millions of requests per second**

📌 **Similarity with Aurora**

- Like Aurora:
  - Managed service
  - High availability
  - Multi-AZ replication

- Auto-scaling storage

🎯 **Exam Tip**

- **MongoDB → DocumentDB**
- **NoSQL (JSON document store) → DocumentDB / DynamoDB**
- If question mentions:
  - MongoDB compatibility → ✅ **DocumentDB**
  - JSON document storage → ✅ **DocumentDB**

📌 **One-Line Summary**

Amazon DocumentDB is a fully managed, MongoDB-compatible NoSQL database designed for storing and querying JSON document data at scale.

Neptune Overview

⭐ **Amazon Neptune – Key Points**

📌 **Definition**

- **Amazon Neptune** is a **fully managed graph database service**

📌 **Purpose**

- Designed for **highly connected datasets**
- Best suited for relationships and connections between data

📌 **Graph Data Concept**

- Data is stored as a **graph (nodes + relationships)**
- Example use case:
  - Social networks (users, friends, likes, comments, shares)

📌 **Key Features**

- ✅ Optimized for **complex graph queries**
- ✅ Handles **billions of relationships**
- ✅ Provides **millisecond query performance**
- ✅ Fully managed service (no infrastructure management)

📌 **High Availability & Scaling**

- **Data replicated across 3 Availability Zones (AZs)**
- Supports up to **15 read replicas**
- Ensures **high availability and fault tolerance**

📌 **Use Cases**

- Social networking applications
- Recommendation engines
- Fraud detection
- Knowledge graphs (e.g., Wikipedia link structure)

🎯 **Exam Tip**

- **Graph database / relationships → Neptune**

- If question mentions:
  - Social connections or networks → ✅ Neptune
  - Recommendation engine → ✅ Neptune
  - Fraud detection using relationships → ✅ Neptune

📌 **One-Line Summary**

Amazon Neptune is a fully managed graph database designed to efficiently handle highly connected data with fast, complex relationship queries.

Timestream Overview

⭐ **Amazon Timestream – Key Points**

📌 **Definition**

- **Amazon Timestream** is a **fully managed, serverless time series database**
- Designed specifically for **time-based data**

📌 **What is Time Series Data?**

- Data that **changes over time**
- Example:
  - Metrics tracked across timestamps (e.g., temperature, CPU usage, stock prices)

📌 **Core Features**

- ✅ **Serverless** (no infrastructure management)
- ✅ **Automatic scaling** (up/down based on workload)
- ✅ **High performance** (up to **1000x faster** than relational databases)
- ✅ **Cost-efficient** (about **1/10th the cost** of relational databases)

📌 **Performance & Capacity**

- Can handle **trillions of events per day**
- Optimized for **fast ingestion and querying of time-series data**

📌 **Analytics Capabilities**

- Supports **real-time analytics**
- Provides built-in functions to:
  - Detect patterns
  - Analyze trends over time

📌 **Use Cases**

- IoT applications (sensor data)
- Application monitoring (CPU, memory metrics)
- DevOps metrics
- Real-time analytics on time-based events

🎯 **Exam Tip**

- **Time series data → Timestream**
- If question mentions:
  - Data evolving over time → ✅ Timestream
  - Metrics/logs with timestamps → ✅ Timestream

📌 **One-Line Summary**

Amazon Timestream is a serverless database designed to store and analyze time series data efficiently at scale.

Managed Blockchain Overview

⭐ Amazon Managed Blockchain – Key Points

📌 Definition

- **Amazon Managed Blockchain** is a **fully managed service** to create and manage **blockchain networks**

📌 What is Blockchain?

- A **decentralized system** where:
  - Multiple parties can **execute transactions**
  - No need for a **central authority**
- Ensures **trust, transparency, and security**

📌 Core Features

- ✅ Create **private blockchain networks** on AWS
- ✅ Join **public blockchain networks**
- ✅ Fully managed (no infrastructure to manage)
- ✅ Scalable and secure

📌 Supported Frameworks

- **Hyperledger Fabric**
- **Ethereum**

📌 Purpose / Use Cases

- Applications requiring:
  - **Decentralized transactions**
  - **Multi-party trust systems**
  - Secure and transparent data sharing

🎯 Exam Tip

- **Blockchain → Amazon Managed Blockchain**
- If question mentions:
  - Hyperledger Fabric → ✅ Managed Blockchain
  - Ethereum → ✅ Managed Blockchain

- Decentralized system → ✅ Managed Blockchain

📌 **One-Line Summary**

Amazon Managed Blockchain is a fully managed service for building and running decentralized blockchain networks using Hyperledger Fabric or Ethereum.

Glue Overview

⭐ **AWS Glue – Key Points**

📌 **Definition**

- **AWS Glue** is a **fully managed ETL service**
- ETL = **Extract, Transform, Load**

📌 **What is ETL?**

- **Extract** → Get data from sources (S3, RDS, etc.)
- **Transform** → Clean, format, and prepare data
- **Load** → Store processed data into a destination (e.g., Redshift)

📌 **Core Purpose**

- Used to **prepare and transform data for analytics**
- Converts raw/unstructured data into **usable format**

📌 **Key Features**

- ✅ **Serverless** (no infrastructure management)
- ✅ Automatically handles **scaling and resources**
- ✅ Focus only on **data transformation logic**
- ✅ Supports multiple data sources and destinations

📌 **How It Works**

1. **Extract data** from:
    - Amazon S3
    - Amazon RDS (or other sources)
2. **Transform data** using Glue scripts
3. **Load data** into:
    - Amazon Redshift (for analytics)
    - Other storage/services

📌 **Glue Data Catalog**

- Central **metadata repository** for datasets
- Stores:

    - Table definitions
    - Column names
    - Data types
- Used by:
    - **Athena**
    - **Redshift**
    - **EMR**
- Helps services **discover and understand data structure**

📌 **Use Cases**

- Data preparation for analytics
- Data warehousing pipelines
- Data integration between services

🎯 **Exam Tip**

- **ETL (Extract, Transform, Load) → AWS Glue**
- If question mentions:
    - Data transformation before analytics → ✅ Glue
    - Preparing data for Redshift → ✅ Glue

📌 **One-Line Summary**

AWS Glue is a serverless ETL service used to extract, transform, and load data for analytics.

DMS Overview

⭐ **AWS DMS (Database Migration Service) – Key Points**

📌 **Definition**

- **AWS DMS** is a service used to **migrate databases to AWS**
- Enables **quick, secure, and reliable migration**

📌 **Core Functionality**

- Migrates data from:
  - **Source database → Target database**
- Uses an **EC2 instance running DMS software** to:
  - Extract data
  - Transfer it
  - Load it into the target

📌 **Key Features**

- ✅ **Source database remains available** during migration (no downtime)
- ✅ **Resilient and self-healing** process
- ✅ Supports **continuous data replication**

📌 **Types of Migration**

1. Homogeneous Migration

- Source and target use the **same database engine**
- Example:
  - Oracle → Oracle

2. Heterogeneous Migration

- Source and target use **different database engines**
- Example:
  - SQL Server → Amazon Aurora
- DMS automatically **converts data formats**

📌 **Use Cases**

- Migrating on-premises databases to AWS

- Cloud-to-cloud database migration
- Database engine upgrades or replacements

🎯 **Exam Tip**

- **Database migration → AWS DMS**
- If question mentions:
  - Minimal downtime migration → ✅ DMS
  - Moving databases to AWS → ✅ DMS
  - Converting database types → ✅ DMS

📌 **One-Line Summary**

AWS DMS is a managed service that migrates databases to AWS with minimal downtime, supporting both same-type and different-type database migrations.

Cloud Practitioner

Databases & Analytics Summary

⭐ **AWS Databases & Analytics – Final Summary**

📌 **1. Relational Databases (OLTP)**

- **Amazon RDS & Aurora**
- Use case: **Online Transaction Processing**
- Supports **SQL**
- Key concepts:
  - **Multi-AZ** → High availability
  - **Read Replicas** → Scaling reads
  - **Multi-Region** → Global performance

📌 **2. In-Memory Database / Cache**

- **Amazon ElastiCache**
- Use case: **Caching, high-speed access**
- Improves application performance

📌 **3. NoSQL Databases**

- **Amazon DynamoDB**
  - Key-value, serverless, highly scalable
- **DAX**
  - Cache specifically for DynamoDB
- **Amazon DocumentDB**
  - MongoDB-compatible
  - JSON-based document storage

📌 **4. Data Warehousing (OLAP)**

- **Amazon Redshift**
- Use case:
  - Analytics
  - Data warehousing
- Features:
  - Columnar storage
  - SQL queries

📌 **5. Big Data Processing**

- **Amazon EMR**
- Use case:
  - Hadoop clusters
  - Big data analysis
- Supports tools like:
  - Spark, HBase, Presto, Flink

📌 **6. Query Data in S3**

- **Amazon Athena**
- Use case:
  - Run **SQL queries directly on S3**
- Serverless and cost-effective

📌 **7. Data Visualization / BI**

- **Amazon QuickSight**
- Use case:
  - Dashboards
  - Reports
  - Business insights

📌 **8. Blockchain**

- **Amazon Managed Blockchain**
- Supports:
  - Hyperledger Fabric
  - Ethereum
- Use case: **Decentralized applications**

📌 **9. ETL (Data Processing)**

- **AWS Glue**
- Use case:
  - Extract, Transform, Load data

    - Prepare data for analytics
- Includes **Glue Data Catalog**

📌 **10. Database Migration**

- **AWS DMS**
- Use case:
    - Migrate databases to AWS
    - Supports:
        - Homogeneous
        - Heterogeneous migrations

📌 **11. Graph Database**

- **Amazon Neptune**
- Use case:
    - Highly connected data
    - Social networks, recommendations

📌 **12. Time-Series Database**

- **Amazon Timestream**
- Use case:
    - Time-based data (metrics, IoT, logs)

🎯 **Ultimate Exam Tips**

👉 Match keywords to services:

- **OLTP / Relational → RDS, Aurora**
- **Cache → ElastiCache**
- **Key-value / Serverless → DynamoDB**
- **MongoDB / JSON → DocumentDB**
- **OLAP / Warehouse → Redshift**
- **Hadoop / Big Data → EMR**
- **SQL on S3 → Athena**
- **Dashboards / BI → QuickSight**
- **ETL → Glue**

- **Migration → DMS**
- **Graph / Relationships → Neptune**
- **Time-series → Timestream**
- **Blockchain → Managed Blockchain**

📌 **One-Line Master Summary**

Choose AWS services based on workload type: transactions (RDS), analytics (Redshift/Athena), big data (EMR), visualization (QuickSight), and specialized databases (DynamoDB, Neptune, Timestream, etc.).

Other Compute Services: ECS, Lambda, Batch, LightSail

What is Docker?

- **What is Docker?**
  - Docker is a **software platform used to build, package, and deploy applications**.
  - Applications are packaged into **containers**.
- **Key Concept: Containers**
  - A **container** includes the app and all its dependencies.
  - Containers can run **consistently across any environment** (any OS, machine, or platform).
  - Benefits:
    - No compatibility issues
    - Predictable behavior
    - Easier deployment and maintenance
- **Advantages of Docker**
  - Works with **any programming language and technology**.
  - Enables **fast scaling** (containers can be started/stopped in seconds).
  - Reduces setup effort compared to traditional installation.
- **Docker on EC2**
  - Multiple containers can run on a **single EC2 instance**, such as:
    - Java app
    - Node.js app
    - MySQL database
  - Makes application deployment on EC2 **simple and efficient**.
- **Docker Images & Repositories**
  - Applications are stored as **Docker images**.
  - Images are hosted in **Docker repositories**:
    - **Docker Hub** (public repository)
    - **Amazon ECR** (private repository in AWS)
- **Docker vs Virtual Machines (EC2)**
  - **EC2 Instances (VMs):**
    - Each instance has its own **full operating system**.
    - More resource-heavy.
  - **Docker Containers:**
    - Share the **host OS**.

- Use a **Docker daemon** to run containers.
- Lightweight and faster to scale.
- No need for a full OS per container.

- **Why Docker is Powerful**
  - Lightweight and efficient
  - Easy to scale and manage
  - Ensures consistent application behavior everywhere

Cloud Practitioner

ECS, Fargate & ECR Overview

🔹 **What is ECS (Elastic Container Service)?**

- ECS is an **AWS service used to run Docker containers**.
- You must **provision and manage infrastructure (EC2 instances)** yourself.
- AWS handles:
  - Starting/stopping containers
  - Load balancing integration (via Application Load Balancer)

✅ **Key Idea:**

- You **create EC2 instances first**, then ECS runs containers on them.
- ECS decides **which EC2 instance** will run each container.

🔹 **ECS Architecture**

- Multiple **EC2 instances (servers)** are created in advance.
- ECS service:
  - Schedules and places containers on these instances.
  - Runs multiple containers across instances efficiently.

✅ **Exam Tip:**

- If you see **"run Docker containers with EC2 management" → Think ECS**.

🔹 **What is Fargate?**

- Fargate is another AWS service to **run Docker containers**.
- It is **serverless**:
  - No need to create or manage EC2 instances.
- AWS automatically manages infrastructure.

✅ **Key Features:**

- You only specify:
  - CPU
  - RAM
- AWS runs the containers **behind the scenes**.

🔹 **ECS vs Fargate**

Feature	ECS	Fargate
Infrastructure	You manage EC2	No management needed
Complexity	More setup	Simpler
Control	More control	Less control
Type	Not serverless	Serverless

✅ **Key Difference:**

- **ECS = manage servers**
- **Fargate = no servers (serverless)**

🔹 **What is ECR (Elastic Container Registry)?**

- ECR is a **private Docker image repository on AWS**.
- Used to **store Docker images**.

✅ **Usage:**

- Store images in ECR
- ECS or Fargate pulls images from ECR
- Containers are created and run from these images

🔹 **How Everything Works Together**

- **ECR** → stores Docker images
- **ECS** → runs containers on EC2 instances
- **Fargate** → runs containers without managing servers

✅ **Final Summary**

- **ECS**: Run containers with EC2 (you manage infrastructure)
- **Fargate**: Run containers serverlessly (AWS manages everything)
- **ECR**: Store Docker images

✅ **Exam Shortcut:**

- Docker on AWS → think **ECS / Fargate**
- Need servers → **ECS**
- No servers → **Fargate**
- Store images → **ECR**

Amazon EKS

- **What is Amazon EKS?**
  - **EKS (Elastic Kubernetes Service)** is an AWS service to **run and manage Kubernetes clusters**.
  - It simplifies the process of deploying Kubernetes on AWS by making it **fully managed**.

- **What is Kubernetes?**
  - Kubernetes is an **open-source system** used for:
    - Deployment of containerized applications
    - Management of containers
    - Scaling applications automatically
  - Works mainly with **Docker containers**, but supports other container types too.

- **How EKS Works**
  - EKS runs a **Kubernetes cluster** on AWS.
  - Containers are deployed as **pods** (basic units in Kubernetes).
  - Pods run on:
    - **EC2 instances (nodes)** OR
    - **Fargate (serverless option)**

- **EKS Architecture**
  - You create an **EKS cluster**.
  - The cluster contains **nodes (EC2 instances)** or uses Fargate.
  - When you deploy an app:
    - Kubernetes creates **pods**
    - Pods run your containers automatically on nodes

- **Why Use EKS?**
  - Kubernetes setup is **complex** → EKS makes it easier by managing it.
  - Provides:
    - High availability
    - Scalability
    - Managed infrastructure

🔹 **Why Use Kubernetes?**

- **Cloud-agnostic**: works across:
  - AWS
  - Azure
  - GCP
  - On-premises
- Lets you **run containers anywhere**, not just AWS.

🔹 **Key Concept (Exam Tip)**

- If you see **Kubernetes** → think **Amazon EKS**

✅ **Final Summary**

- **EKS**: Managed Kubernetes service on AWS
- **Kubernetes**: Tool to deploy & manage containers
- **Pods**: Units that run containers
- **Runs on**: EC2 or Fargate
- **Main advantage**: Works across multiple cloud platforms

✅ **Simple Memory Trick:**

- Docker → Containers
- ECS → Containers with AWS control
- Fargate → Containers without servers
- EKS → Kubernetes on AWS

Cloud Practitioner

Serverless Introduction

- **What is Serverless?**
  - **Serverless** is a cloud computing model where:
    - Developers **do not manage servers**
    - They only **deploy code or functions**
  - Servers still exist, but they are **fully managed by the cloud provider (AWS)**.

- **Key Idea of Serverless**
  - Focus on:
    - Writing code
    - Deploying functions
  - No need to:
    - Provision servers
    - Maintain infrastructure
    - Monitor hardware

- **Origin of Serverless**
  - Started with **AWS Lambda** (Function-as-a-Service).
  - Developers deploy **functions**, and AWS runs them automatically.

- **Modern Meaning of Serverless**
  - Includes any **fully managed service** where:
    - You don’t manage servers
  - Examples:
    - Compute (Lambda, Fargate)
    - Storage (S3)
    - Databases (DynamoDB)
    - Messaging services

- **Important Clarification**
  - Serverless **does NOT mean no servers**
  - It means:
    - Servers are **hidden and managed by AWS**

- Users don't interact with them

🔹 **Examples of Serverless Services**

1. **Amazon S3**
   - Storage service
   - No servers to manage
   - Scales automatically
2. **DynamoDB**
   - Serverless database
   - Auto-scaling
   - No infrastructure setup needed
3. **Fargate**
   - Runs Docker containers
   - No EC2 management required
4. **AWS Lambda**
   - Runs code as independent functions
   - Executes automatically when triggered

🔹 **Why Serverless is Useful**

- Eliminates infrastructure management
- Automatic scaling
- Faster development and deployment
- Pay only for usage

✅ **Final Summary**

- **Serverless = No server management**
- AWS handles everything behind the scenes
- Key services:
  - **Lambda** → run functions
  - **S3** → storage
  - **DynamoDB** → database
  - **Fargate** → containers

✅ **Simple Memory Trick:**

- Serverless = *"Just write code, AWS handles the rest"*

Cloud Practitioner

Lambda Overview

🔹 What is AWS Lambda?

- AWS Lambda is a **serverless compute service**.
- Instead of servers, you run **functions** in the cloud.
- Functions are:
  - **On-demand**
  - **Short-lived (time-limited)**

🔹 EC2 vs Lambda

✅ **EC2 (Traditional)**

- Fixed CPU and memory
- Runs continuously (even when not used)
- Requires manual scaling (Auto Scaling groups)
- More complex to manage

✅ **Lambda (Serverless)**

- No servers to manage
- Runs **only when triggered**
- **Automatically scales**
- You pay only when it runs

🔹 Key Features of Lambda

- **Event-driven**:
  - Runs when an event occurs (e.g., file upload, API call)
- **Auto-scaling**:
  - Handles scaling automatically
- **No idle cost**:
  - No charges when not running
- **Integrated with AWS services**
- **Monitoring via CloudWatch**

🔹 Pricing Model

- Pay based on:
  1. **Number of requests (calls)**
  2. **Execution duration**

✅ **Free Tier:**

- **1 million requests/month**
- **400,000 GB-seconds of compute time**

✅ **After Free Tier:**

- ~$0.20 per 1 million requests
- Pay for compute duration

👉 **Exam Tip**:

Lambda pricing = Requests + Duration

🔹 **Supported Languages**

- Popular:
  - Node.js (JavaScript)
  - Python
  - Java
  - C# (.NET / PowerShell)
  - Ruby
- Supports others (like Go, Rust) via **custom runtime**

🔹 **Lambda with Containers**

- Lambda can run **container images**
- But:
  - Prefer **ECS/Fargate** for Docker workloads (exam perspective)

🔹 **Resource Limits**

- Up to **10 GB RAM per function**
- More RAM → better CPU & network performance

🔹 **Common Use Cases**

✅ **1. Image Processing (Thumbnail Creation)**

- Flow:
  1. Image uploaded to **S3**
  2. S3 triggers **Lambda**
  3. Lambda:

- Creates thumbnail
- Stores output in S3
- Saves metadata in **DynamoDB**

👉 Fully **serverless & event-driven**

✅ **2. Scheduled Tasks (Serverless CRON)**

- Use **CloudWatch Events / EventBridge**
- Example:
  - Run function every hour/day
- No EC2 needed

👉 Fully serverless scheduled execution

🔹 **Why Lambda is Popular**

- No infrastructure management
- Automatic scaling
- Very low cost
- Works well with other AWS services
- Ideal for **event-based workloads**

✅ **Final Summary**

- **Lambda = Serverless functions**
- Runs **only when triggered**
- **Auto-scales & cost-efficient**
- Pricing = **Requests + Execution time**
- Common uses:
  - Event-driven apps
  - Automation
  - Image processing
  - Scheduled jobs

✅ **Simple Memory Trick:**

- EC2 → Servers always running
- Lambda → *Run code only when needed*

API Gateway Overview

- **What is Amazon API Gateway?**
  - Amazon API Gateway is a **fully managed AWS service** used to:
    - **Create, publish, maintain, monitor, and secure APIs**
  - It is a **serverless service** and **fully scalable**

- **Purpose of API Gateway**
  - Used to **expose backend services (like Lambda)** to external clients
  - Helps build **serverless HTTP APIs**

- **Why API Gateway is Needed**
  - **Lambda functions are not directly accessible via HTTP**
  - API Gateway acts as a **front door** for applications

- **How It Works (Architecture)**
  1. **Client (user/app)** sends request
  2. **API Gateway** receives request
  3. API Gateway forwards request to **Lambda function**
  4. Lambda processes data (e.g., CRUD operations on DynamoDB)
  5. Response is sent back to the client

- **Integration with Serverless Services**
  - Works commonly with:
    - **AWS Lambda** (compute)
    - **DynamoDB** (database)
  - Enables a fully **serverless application architecture**

- **Key Features**
  - Supports:
    - **REST APIs**
    - **WebSocket APIs** (real-time communication)
  - Built-in capabilities:
    - **Authentication & security**

- **API keys**
- **Throttling (limit requests)**
- **Monitoring**

🔹 **Common Use Case**

- Build a **serverless CRUD API**:
  - API Gateway → Lambda → DynamoDB

✅ **Exam Tip**

- If you see:
  - **"Create a serverless API"** 👉 Think:
- **API Gateway + Lambda**

✅ **Final Summary**

- **API Gateway** = Entry point for APIs
- Connects **clients → Lambda → backend services**
- Fully **serverless, scalable, and secure**

✅ **Simple Memory Trick:**

- API Gateway = *"Door to your serverless application"* 🚪

Cloud Practitioner

Batch Overview

- **What is AWS Batch?**
  - AWS Batch is a **fully managed service** used for **batch processing jobs** at any scale.
  - It allows you to run **hundreds of thousands of batch jobs efficiently**.

- **What is a Batch Job?**
  - A **batch job**:
    - Has a **start and end time**
    - Runs at a specific time (e.g., 1:00 AM to 3:00 AM)
  - Different from:
    - **Streaming/continuous jobs** (which run indefinitely)

- **How AWS Batch Works**
  - You **submit or schedule jobs** into a **batch queue**
  - AWS Batch automatically:
    - Provisions compute resources
    - Scales capacity based on workload
  - Uses:
    - **EC2 instances**
    - **Spot instances** (for cost savings)

- **Key Features**
  - Automatic scaling of compute resources
  - Efficient handling of large workloads
  - Cost optimization using Spot instances
  - Fully managed infrastructure (no manual scaling needed)

- **Docker Integration**
  - Batch jobs are defined using **Docker images**
  - Jobs can run on:
    - **ECS**
    - **EKS**
    - **Fargate**

🔹 **Example Use Case**

Image Processing Pipeline:

1. User uploads image to **S3**
2. Triggers a **Batch job**
3. AWS Batch:
    - Launches EC2/Spot instances
    - Runs Docker containers
4. Processed images stored back in **S3**

🔹 **AWS Batch vs Lambda**

Feature	AWS Batch	AWS Lambda
Execution Time	No limit	Max 15 minutes
Infrastructure	Uses EC2 (managed)	Fully serverless
Runtime	Any (via Docker)	Limited languages
Storage	Large (EBS/instance store)	Limited temporary storage
Use Case	Long-running jobs	Short event-driven tasks

🔹 **Key Differences**

- **Batch**:
    - Best for **long-running, large-scale jobs**
    - Uses EC2 but AWS manages them
- **Lambda**:
    - Best for **short, event-driven tasks**
    - Fully serverless

✅ **Final Summary**

- **AWS Batch** = Managed service for batch processing
- Runs jobs with **start & end time**
- Uses **EC2/Spot instances** automatically
- Supports **Docker-based workloads**
- Ideal for **large-scale, long-duration processing**

✅ **Exam Tip:**

- Long-running processing jobs → **AWS Batch**
- Short event-driven functions → **Lambda**

✅ **Simple Memory Trick:**

- Batch = *"Big jobs, long time"*
- Lambda = *"Small jobs, quick time"*

Lightsail Overview

- **What is Amazon Lightsail?**
  - Lightsail is a **simplified, standalone AWS service**.
  - Provides everything in one place:
    - **Virtual servers**
    - **Storage**
    - **Databases**
    - **Networking**

- **Key Features**
  - **Simple and easy to use**
  - **Low and predictable pricing**
  - Minimal configuration required
  - Includes basic **monitoring and notifications**

- **Purpose of Lightsail**
  - Designed for users with **little or no cloud experience**
  - Offers a **simpler alternative** to complex AWS services like:
    - EC2 (compute)
    - RDS (database)
    - ELB (load balancing)
    - EBS (storage)
    - Route 53 (DNS)

- **Common Use Cases**
  - Deploy simple applications quickly:
    - **LAMP stack**
    - **Nginx / Node.js / MEAN stack**
  - Host websites:
    - **WordPress**
    - **Magento**
    - **Joomla**
    - **Plesk**

- Create **development and test environments**

🔹 **Limitations**

- **No auto-scaling**
- Limited integration with other AWS services
- Not suitable for complex or large-scale applications

🔹 **High Availability**

- Supports basic **high availability**
- But lacks advanced scaling features

🔹 **When to Use Lightsail (Exam Tip)**

- Use when:
  - User has **no cloud experience**
  - Needs **quick setup**
  - Wants **simple and predictable pricing**

🔹 **When NOT to Use Lightsail**

- For:
  - Advanced architectures
  - Deep AWS integrations
  - Scalable production systems

✅ **Final Summary**

- **Lightsail = Simple AWS for beginners**
- All-in-one service with easy setup
- Best for **small apps, websites, and testing**

✅ **Exam Trick:**

- Beginner + simple setup + fixed pricing → **Lightsail**
- Advanced, scalable architecture → **NOT Lightsail**

Other Compute – Summary

- **Docker (Foundation)**
  - Docker is a **container technology** to package and run applications.
  - Ensures **consistent behavior across environments**.

- **ECS (Elastic Container Service)**
  - Runs Docker containers on **EC2 instances**.
  - You must **provision and manage EC2 infrastructure**.
  - ECS handles **container placement and orchestration**.

- **Fargate**
  - Runs Docker containers **without managing servers**.
  - Fully **serverless container service**.
  - AWS handles infrastructure automatically.

- **ECR (Elastic Container Registry)**
  - Stores **Docker images**.
  - Private repository used by:
    - ECS
    - Fargate

- **AWS Batch**
  - Runs **batch jobs (start & end tasks)**.
  - Automatically provisions **EC2/Spot instances**.
  - Works on top of **ECS**.
  - Ideal for **large-scale, long-running jobs**.

- **Amazon Lightsail**
  - **Simple, beginner-friendly AWS service**.
  - Provides:
    - Servers, storage, databases, networking
  - Has:
    - **Low & predictable pricing**

- Limited features:
  - No auto-scaling
  - Limited integrations
- Often a **distractor in exams**

AWS Lambda

- **Serverless compute service (Function-as-a-Service)**.
- Runs **functions on demand**.
- Features:
  - Auto-scaling
  - Event-driven execution
  - Up to **15 minutes runtime**
- Pricing based on:
  - **Number of invocations**
  - **Execution duration & memory**

Lambda Key Details

- Supports multiple programming languages
- Can run **custom container images** (with limitations)
- Not ideal for arbitrary Docker → use **ECS/Fargate instead**

Lambda Use Cases

- **Image processing (e.g., thumbnails from S3 uploads)**
- **Scheduled jobs (serverless CRON)**

API Gateway

- Used to **expose Lambda functions as HTTP APIs**.
- Features:
  - Security
  - API keys
  - Throttling
- Fully **serverless**

✅ **Final Big Picture**

- **Docker** → Package apps
- **ECS** → Containers with EC2
- **Fargate** → Containers without servers
- **ECR** → Store Docker images
- **Batch** → Run large jobs
- **Lightsail** → Simple beginner setup
- **Lambda** → Run functions serverlessly
- **API Gateway** → Expose APIs for Lambda

✅ **Ultimate Exam Memory Trick:**

- Containers → ECS / Fargate
- Serverless functions → Lambda
- Store images → ECR
- APIs → API Gateway
- Big jobs → Batch
- Beginner → Lightsail

Deployment & Managing Infrastructure at Scale

CloudFormation Overview

What is CloudFormation?

- **AWS CloudFormation** is an **Infrastructure as Code (IaC)** service.
- It allows you to **define AWS infrastructure declaratively** using templates.
- You specify the desired resources, and CloudFormation automatically creates them in the correct order.

Example

Using a CloudFormation template, you can define:

- Security Groups
- EC2 Instances
- S3 Buckets
- Load Balancers
- Databases
- Other AWS resources

CloudFormation automatically provisions and configures them.

Benefits of CloudFormation

1. Infrastructure as Code (IaC)

- Infrastructure is defined in code instead of being created manually.
- Changes can be tracked, version-controlled, and reviewed through code reviews.
- Ensures consistency and reduces human errors.

2. Cost Management

- Resources created within a stack receive common tags.
- Easier cost tracking and cost allocation.
- CloudFormation templates can help estimate infrastructure costs.

3. Cost Saving Strategy

- Entire environments can be deleted and recreated automatically.
- Example:
    - Delete development environment at **5:00 PM**
    - Recreate it at **8:00 AM**
- Reduces costs by avoiding unnecessary resource usage.

4. Improved Productivity

- Quickly create, update, delete, and recreate infrastructure.

- No need to manually provision resources.
- Accelerates deployment processes.

5. Declarative Programming

- You describe **what** infrastructure you want.
- CloudFormation determines **how** and **in what order** resources should be created.
- No need to manage dependencies manually.

6. Reusable Templates

- Existing CloudFormation templates can be reused.
- Large community support and documentation available.
- Avoids reinventing the wheel.

7. Broad AWS Resource Support

- Supports almost all AWS services.
- For unsupported resources, **Custom Resources** can be used.

Infrastructure Composer

- CloudFormation templates can be visualized using **Infrastructure Composer**.
- Provides architecture diagrams automatically.
- Shows:
  - Resources
  - Relationships between resources
  - Architecture dependencies
- Helps in understanding and documenting infrastructure.

Important Exam Points

✅ CloudFormation = **Infrastructure as Code (IaC)**

✅ Uses **declarative templates**

✅ Automatically handles **resource dependencies and creation order**

✅ Enables **repeatable deployments**

✅ Can deploy the same architecture across:

- Multiple environments
- Multiple AWS Regions
- Multiple AWS Accounts

✅ Supports **cost management and automation**

- ✅ Generates **architecture visualizations**
- ✅ Nearly all AWS resources are supported
- ✅ **Custom Resources** can extend support for unsupported resources

CDK Overview

What is AWS CDK?

- **AWS CDK (Cloud Development Kit)** is an **Infrastructure as Code (IaC)** framework.
- It allows developers to define cloud infrastructure using familiar programming languages instead of writing CloudFormation YAML/JSON directly.
- Supported languages include:
    - TypeScript
    - JavaScript
    - Python
    - Java
    - .NET (C#)

How CDK Works

1. Write infrastructure code using a programming language.
2. CDK converts (synthesizes) the code into a **CloudFormation template**.
3. CloudFormation deploys the resources to AWS.

Flow:

CDK Code (Python/JavaScript/Java/.NET)
↓
CDK CLI Synthesizes
↓
CloudFormation Template (JSON/YAML)
↓
AWS CloudFormation Deploys
↓
AWS Infrastructure

Key Benefits of AWS CDK

1. Use Familiar Programming Languages

- No need to learn CloudFormation YAML/JSON.
- Developers can use languages they already know.

2. Infrastructure and Application Together

- Infrastructure code and application code can coexist in the same project.

- Particularly useful for:
  - AWS Lambda
  - Amazon ECS
  - Amazon EKS
  - Docker-based applications

3. Type Safety

- Programming languages provide:
  - Type checking
  - Auto-completion
  - Better error detection during development

4. Code Reusability

- Create reusable components and constructs.
- Share infrastructure code across projects.

5. Advanced Programming Features

- Use:
  - Loops (for, while)
  - Functions
  - Classes
  - Variables
  - Conditions
- Makes infrastructure definitions more efficient.

6. Faster Development

- Less repetitive work compared to writing large CloudFormation templates manually.
- Easier maintenance and scalability.

Example from the Lecture

The CDK code defined:

- A **VPC**
- An **ECS Cluster**
- An **Application Load Balancer (ALB)**
- A **Fargate Service**

The CDK CLI then generated a CloudFormation template, which was deployed using CloudFormation.

Relationship Between CDK and CloudFormation

CDK = Higher-Level Development Tool

↓

Generates

↓

CloudFormation Template

↓

Deploys AWS Resources

Important: CDK does not replace CloudFormation; it generates CloudFormation templates behind the scenes.

Exam Tips

- ✅ CDK = Infrastructure as Code using programming languages
- ✅ Supports Python, TypeScript, JavaScript, Java, and .NET
- ✅ Converts code into CloudFormation templates
- ✅ Uses the **CDK CLI** to synthesize and deploy infrastructure
- ✅ Enables code reuse, loops, conditions, and object-oriented programming
- ✅ Provides type safety and developer-friendly tooling
- ✅ Commonly used with Lambda, ECS, EKS, and Docker applications
- ✅ CloudFormation remains the underlying deployment engine

Beanstalk Overview

What is Elastic Beanstalk?

- **AWS Elastic Beanstalk** is a **Platform as a Service (PaaS)** offering.
- It allows developers to **deploy and manage applications without managing the underlying infrastructure**.
- Developers focus only on **application code**, while Beanstalk handles infrastructure provisioning and management.

Traditional 3-Tier Web Architecture

A typical web application consists of:

Users
↓
Load Balancer (ELB)
↓
Auto Scaling Group (ASG)
↓
EC2 Instances
↓
Database (Amazon RDS)
↓
(Optional) ElastiCache
`

Components

- **Load Balancer (ELB)** – Distributes incoming traffic.
- **Auto Scaling Group (ASG)** – Automatically adjusts the number of EC2 instances.
- **EC2 Instances** – Run the application code.
- **Amazon RDS** – Stores relational data.
- **ElastiCache** – Stores cached or session data.

Why Elastic Beanstalk?

Without Beanstalk, developers must:

- Configure EC2 instances
- Configure Load Balancers

- Configure Auto Scaling
- Configure Monitoring
- Configure Databases

With **Elastic Beanstalk**:

- Upload code
- Beanstalk automatically provisions infrastructure
- Application scales automatically

Key Benefits

1. Developer-Centric Service

- Focus on writing and deploying code.
- Infrastructure management is automated.

2. Managed Service

Beanstalk manages:

- EC2 configuration
- Operating system configuration
- Capacity provisioning
- Auto Scaling
- Load balancing
- Deployment processes
- Health monitoring

3. Easy Scaling

- Automatically integrates with:
    - Load Balancers
    - Auto Scaling Groups
- Supports scalable production workloads.

4. Consistent Deployments

- Same deployment model across multiple environments.
- Suitable for development, testing, staging, and production.

Pricing

✅ **Elastic Beanstalk itself is free**

❌ You pay for the underlying AWS resources:

- EC2

- RDS
- ELB
- EBS
- Data transfer
- Other AWS services used

Shared Responsibility

AWS / Beanstalk Manages

- Infrastructure provisioning
- OS configuration
- Load balancing
- Auto Scaling
- Application deployment process
- Monitoring and health checks

Developer Manages

- Application code
- Application configuration

Exam Tip:

Beanstalk lets developers focus mainly on their code.

Beanstalk Deployment Models

1. Single Instance Environment

Users

↓

EC2 Instance

Use Case

- Development
- Testing
- Small workloads

2. Load Balanced Environment

Users

↓

Load Balancer

↓

Auto Scaling Group

↓

Multiple EC2 Instances

Use Case

- Production applications
- High availability
- Scalability

3. Worker Environment

Auto Scaling Group

↓

Worker EC2 Instances

Use Case

- Background processing
- Queue workers
- Non-web applications

Supported Platforms

Elastic Beanstalk supports:

- Java
- .NET
- Node.js
- Python
- PHP
- Ruby
- Go
- Docker
- Multi-Container Docker

Exam Note: You do not need to memorize all supported platforms.

Health Monitoring

Built-in Monitoring

Each EC2 instance contains a **health agent** that:

- Collects application health information
- Sends metrics to CloudWatch
- Reports health status to Elastic Beanstalk

Monitoring Capabilities

- Application health status
- EC2 health status
- Environment health
- Event reporting
- Performance metrics

Cloud Service Models Comparison

Infrastructure as a Service (IaaS)

Examples:

- EC2
- EBS
- VPC

You manage:

- Servers
- OS
- Applications

Platform as a Service (PaaS)

Example:

- **Elastic Beanstalk**

You manage:

- Application code

AWS manages:

- Infrastructure

Software as a Service (SaaS)

Examples:

- Microsoft 365
- Gmail

Users simply consume the software.

Exam Tips

✅ Elastic Beanstalk = **Platform as a Service (PaaS)**

✅ Developers upload code; AWS manages infrastructure

✅ Supports automatic:

- Load Balancing
- Auto Scaling
- Monitoring
- Deployment

✅ Beanstalk itself is **free**

✅ Pay only for the underlying AWS resources

✅ Health monitoring is built-in

✅ Uses CloudWatch metrics and health agents

✅ Supports web applications and worker applications

✅ Available deployment models:

- Single Instance
- Load Balanced + ASG
- Worker Environment

Code Deploy Overview

What is AWS CodeDeploy?

- **AWS CodeDeploy** is a service that automates application deployments.
- It helps upgrade applications from one version to another (e.g., **Version 1 → Version 2**).
- Works independently of **CloudFormation** and **Elastic Beanstalk**.

Key Purpose

- Automate application deployments.
- Reduce manual deployment effort.
- Ensure consistent deployments across multiple servers.

Example

Application Version 1

↓

CodeDeploy

↓

Application Version 2

Supported Deployment Targets

1. EC2 Instances

- Deploy applications to one or many EC2 instances.
- Automatically update applications across servers.

2. On-Premises Servers

- Deploy applications to servers located in your own data center.
- Useful for organizations with hybrid environments.

Hybrid Service

CodeDeploy is considered a Hybrid Service because it supports:

AWS Environment (EC2)

+

On-Premises Environment

Exam Tip

✅ If a service works with both AWS and on-premises resources, it is often referred to as a **hybrid service**.

Requirements for Using CodeDeploy

Before using CodeDeploy:

1. Provision Servers

- Servers must already exist.
- CodeDeploy does not create servers for you.

2. Install CodeDeploy Agent

- A **CodeDeploy Agent** must be installed on target servers.
- The agent coordinates deployments and updates.

CodeDeploy Service

↓

CodeDeploy Agent

↓

EC2 / On-Premises Server

Benefits of CodeDeploy

Automated Deployments

- Eliminates manual application updates.
- Speeds up release cycles.

Consistent Deployments

- Deploy the same application version across multiple servers.

Supports Hybrid Environments

- Same deployment process for:
    - AWS EC2 instances
    - On-premises servers

Migration Support

- Helps organizations transition from on-premises environments to AWS.
- Allows managing deployments from a single platform.

CodeDeploy vs Other Services

Cloud Practitioner

Service	Main Purpose
CloudFormation	Provision AWS infrastructure using Infrastructure as Code
Elastic Beanstalk	Deploy applications while AWS manages infrastructure
CodeDeploy	Automate application deployments to existing servers

Exam Tips

- ✅ CodeDeploy automates application deployments.
- ✅ Deploys applications from **Version 1 to Version 2**.
- ✅ Works with:
  - EC2 Instances
  - On-Premises Servers
- ✅ Independent of CloudFormation and Elastic Beanstalk.
- ✅ Requires servers to be provisioned beforehand.
- ✅ Requires installation of the **CodeDeploy Agent**.
- ✅ Considered a **Hybrid Service** because it supports both AWS and on-premises environments.
- ✅ Useful for organizations migrating workloads from on-premises data centers to AWS.

Code Commit Overview

What is AWS CodeCommit?

- **AWS CodeCommit** is a **fully managed source control service** provided by AWS.
- It allows you to store and manage application source code in **Git-based repositories**.
- Acts as AWS's alternative to GitHub for code hosting and version control.

Why Do We Need CodeCommit?

Before deploying code to servers, developers need a place to store their code.

CodeCommit provides:

- Secure code storage
- Version control
- Team collaboration
- Change tracking

Developer

↓

CodeCommit Repository

↓

Store & Manage Code

↓

Deploy Applications

Git-Based Repository

- CodeCommit uses **Git** as the version control system.
- Supports standard Git operations:
    - Clone
    - Commit
    - Push
    - Pull
    - Branch
    - Merge

Exam Tip

✅ CodeCommit is a **Git-based repository service**.

Key Benefits of CodeCommit

1. Team Collaboration

- Multiple developers can work on the same project.
- Developers can share, review, and merge code changes.

2. Version Control

- Every code change is tracked.
- Maintains complete version history.
- Earlier versions can be restored if needed.

3. Rollback Capability

- If a deployment causes issues, previous code versions can be recovered and redeployed.

4. Fully Managed Service

- AWS manages:
  - Infrastructure
  - Scaling
  - Maintenance
  - Availability

5. Scalability

- Repositories can grow without requiring manual infrastructure management.

6. High Availability

- AWS automatically provides durable and highly available code storage.

7. Security and Privacy

- Repositories remain inside your AWS account.
- Provides secure access control through AWS IAM.

8. AWS Integration

- Integrates easily with other AWS Developer Tools such as:
  - CodeBuild
  - CodeDeploy
  - CodePipeline
  - CloudWatch

CodeCommit vs GitHub

Feature	CodeCommit	GitHub
Repository Type	Git-based	Git-based

Managed by	AWS	GitHub
AWS Integration	Native	External Integration
Private Repositories	Yes	Yes
Version Control	Yes	Yes

Benefits Mentioned in the Lecture

- ✅ Fully managed service
- ✅ Git-based version control repository
- ✅ Enables developer collaboration
- ✅ Tracks and versions code changes
- ✅ Supports rollback to previous versions
- ✅ Scalable and highly available
- ✅ Private and secure within AWS accounts
- ✅ Integrated with AWS services

Exam Tips

- ✅ CodeCommit = AWS-managed Git repository service
- ✅ Used to store source code before deployment
- ✅ Supports version control and collaboration
- ✅ Code changes are automatically tracked
- ✅ Previous versions can be restored (rollback)
- ✅ Fully managed by AWS
- ✅ Highly available and scalable
- ✅ Secure and private within AWS accounts
- ✅ Commonly used with:
  - CodeBuild
  - CodeDeploy
  - CodePipeline

Code Build Overview

What is AWS CodeBuild?

- **AWS CodeBuild** is a **fully managed build service**.
- It compiles source code, runs tests, and produces deployable artifacts.
- Used as part of the software development and deployment process.

Main Purpose

CodeBuild automates the build process by:

1. Retrieving source code
2. Compiling the code
3. Running tests
4. Packaging the application
5. Generating deployable artifacts

Source Code
↓
AWS CodeBuild
↓
Compile + Test + Package
↓
Deployable Artifacts

How CodeBuild Works

Example workflow:

CodeCommit Repository
↓
CodeBuild
↓
Runs Build Scripts
↓
Compiles Application
↓

Runs Automated Tests

↓

Creates Deployable Package

↓

CodeDeploy / Other Deployment Service

Process

- Source code is stored in **CodeCommit**.
- CodeBuild retrieves the source code.
- Executes build instructions defined by the developer.
- Produces application artifacts ready for deployment.

Build Output (Artifacts)

Artifacts may include:

- Compiled applications
- Executable files
- JAR files
- ZIP packages
- Docker images
- Deployment packages

These artifacts can later be deployed using services such as:

- AWS CodeDeploy
- Elastic Beanstalk
- ECS
- EKS

Key Benefits of CodeBuild

1. Fully Managed

- No infrastructure to provision or manage.
- AWS handles the build environment automatically.

2. Serverless

- No build servers required.
- Builds run on demand.

3. Continuous Scalability

- Automatically scales according to build requirements.
- Supports multiple concurrent builds.

4. High Availability

- Highly available AWS-managed service.
- Reliable build execution.

5. Secure

- Integrates with IAM for access control.
- Secure interaction with AWS services.

6. Pay-As-You-Go Pricing

- Pay only for the build time consumed.
- No charges when builds are not running.

7. Developer Focused

- Developers focus on coding.
- AWS manages the build infrastructure.

Relationship with Other AWS Developer Tools

CodeCommit
↓
CodeBuild
↓
CodeDeploy
`

Roles

Service	Purpose
CodeCommit	Store source code
CodeBuild	Compile, test, and package code
CodeDeploy	Deploy application to servers

Exam Tips

✅ CodeBuild is a **build service**

✅ Retrieves source code from repositories (e.g., CodeCommit)

- ✅ Compiles code and runs tests
- ✅ Produces deployable artifacts
- ✅ Fully managed and serverless
- ✅ No servers to provision or manage
- ✅ Automatically scalable
- ✅ Highly available and secure
- ✅ Uses pay-as-you-go pricing
- ✅ Commonly integrated with:
  - CodeCommit
  - CodeDeploy
  - CodePipeline

AWS Developer Tools Flow

CodeCommit
(Store Code)
↓
CodeBuild
(Build & Test Code)
↓
CodeDeploy
(Deploy Application)

Code Pipeline Overview

What is AWS CodePipeline?

- **AWS CodePipeline** is a **continuous integration and continuous delivery (CI/CD) orchestration service**.
- It automates the software release process by coordinating different stages such as:
  - Source
  - Build
  - Test
  - Deploy

Main Purpose

CodePipeline connects AWS Developer Tools and automates the workflow from code commit to deployment.

Developer
↓
CodeCommit
↓
CodePipeline
↓
CodeBuild
↓
Testing
↓
CodeDeploy
↓
Production Environment

What Does CodePipeline Do?

CodePipeline orchestrates multiple steps automatically:

1. Retrieve code from repository
2. Build the application
3. Run tests

4. Provision infrastructure (if required)
5. Deploy the application
6. Deliver updates to production

Example Pipeline

CodeCommit
(Store Code)
↓
CodeBuild
(Build & Test)
↓
CloudFormation
(Provision Infrastructure)
↓
CodeDeploy
(Deploy Application)
↓
Elastic Beanstalk / EC2 / Production

CI/CD Concept

Continuous Integration (CI)

- Developers frequently push code changes.
- Code is automatically:
  - Built
  - Tested
  - Validated

Continuous Delivery (CD)

- Validated code is automatically prepared for deployment.
- Releases become faster and more reliable.

Exam Tip

✅ CodePipeline is a core AWS service for **CI/CD automation**.

Integration with AWS Services

CodePipeline works with many AWS services:

Source Stage

- CodeCommit
- GitHub
- Third-party repositories

Build Stage

- CodeBuild

Deployment Stage

- CodeDeploy
- Elastic Beanstalk
- ECS
- CloudFormation

Other Integrations

- Third-party services
- Custom plugins

Key Benefits

1. Workflow Orchestration

- Automates and coordinates all deployment stages.
- Removes manual intervention.

2. Fully Managed

- AWS manages the pipeline infrastructure.
- No servers to maintain.

3. Fast Delivery

- Accelerates software releases.
- Enables continuous deployments.

4. Rapid Updates

- Automatically processes code changes.
- Reduces deployment delays.

5. Service Integration

- Seamlessly integrates with AWS Developer Tools and third-party tools.

6. Reliable Automation

- Ensures each deployment follows the same process.

Relationship with Other AWS Developer Tools

CodeCommit
(Store Code)
↓
CodePipeline
(Orchestrates Workflow)
↓
CodeBuild
(Build & Test)
↓
CodeDeploy
(Deploy)
↓
Production

Service Roles

Service	Purpose
CodeCommit	Store source code
CodeBuild	Compile and test code
CodeDeploy	Deploy application
CodePipeline	Orchestrate the entire CI/CD process

Exam Tips

✅ CodePipeline = **CI/CD orchestration service**

✅ Automates:

- Build
- Test
- Deploy workflows

✅ Connects services such as:

- CodeCommit
- CodeBuild

- CodeDeploy
- Elastic Beanstalk
- CloudFormation
- GitHub

✅ Fully managed by AWS

✅ Enables continuous integration and continuous delivery

✅ Provides fast and automated software releases

✅ Used whenever the question mentions:

- Pipeline
- Workflow automation
- CI/CD
- Release automation
- Orchestration

Code Artifact Overview

What is AWS CodeArtifact?

- **AWS CodeArtifact** is a **fully managed artifact repository service**.
- It helps developers securely **store, publish, and retrieve software packages and dependencies**.
- Eliminates the need to build and manage your own artifact repository infrastructure.

What Problem Does CodeArtifact Solve?

Most applications depend on external libraries and packages.

Examples:

- Java applications use Maven dependencies.
- Python applications use pip packages.
- Node.js applications use npm packages.

These packages are called **code dependencies**.

Managing and storing these dependencies is known as **Artifact Management**.

Traditional Approach

Without CodeArtifact, organizations often:

- Store artifacts in Amazon S3.
- Manage custom artifact repositories on EC2.
- Maintain third-party dependency management tools.

This increases:

- Operational overhead
- Infrastructure management
- Maintenance complexity

How CodeArtifact Helps

CodeArtifact provides a centralized repository for:

✅ Software packages

✅ Libraries

✅ Build artifacts

✅ Application dependencies

Developers

↓

AWS CodeArtifact

↓

Store & Retrieve Packages

↓

Build Applications

Supported Package Managers

CodeArtifact integrates with popular dependency management tools:

Java

- Maven
- Gradle

JavaScript

- npm
- yarn

Python

- pip
- twine

.NET

- NuGet

Exam Tip: You don't need to memorize every package manager, but know that CodeArtifact supports common development tools.

Integration with AWS Developer Tools

Example CI/CD Workflow

CodeCommit

↓

CodeBuild

↓

CodeArtifact

(Retrieve Dependencies)

↓

Build Application

↓

CodeDeploy

↓

Deploy Application

Workflow

1. Developers store code in CodeCommit.
2. CodeBuild starts the build process.
3. CodeBuild retrieves required dependencies from CodeArtifact.
4. Application is compiled and packaged.
5. CodeDeploy deploys the application.

Key Benefits

1. Fully Managed

- No infrastructure to manage.
- AWS handles scaling, availability, and maintenance.

2. Secure

- Integration with AWS IAM.
- Fine-grained access control.

3. Scalable

- Supports teams of any size.
- Automatically scales as repositories grow.

4. Cost-Effective

- No need to run dedicated artifact servers.
- Reduced operational costs.

5. Centralized Dependency Storage

- Single location for storing software packages and dependencies.

Common Use Cases

Store Internal Packages

- Share company-developed libraries across teams.

Manage Third-Party Dependencies

- Maintain approved versions of external libraries.

Support CI/CD Pipelines

- Build systems can retrieve dependencies automatically.

Organization-Wide Package Management

- Central repository for all development teams.

CodeArtifact vs Other AWS Developer Tools

Service	Purpose
CodeCommit	Store source code
CodeBuild	Build and test source code
CodeArtifact	Store software packages and dependencies
CodeDeploy	Deploy applications
CodePipeline	Automate CI/CD workflows

Exam Tips

✅ **CodeArtifact = Artifact Repository Service**

✅ Used for storing and retrieving software packages and dependencies

✅ Eliminates the need to manage custom artifact repositories

✅ Fully managed by AWS

✅ Secure, scalable, and cost-effective

✅ Integrates with:

- Maven
- Gradle
- npm
- yarn
- pip
- twine
- NuGet

✅ Commonly used together with CodeBuild

✅ Helps manage **code dependencies**

System Manager Overview

What is AWS Systems Manager (SSM)?

- **AWS Systems Manager (SSM)** is a service used to **manage, monitor, and automate operations** across AWS and on-premises infrastructure.
- Helps manage large fleets of servers from a single console.
- Supports both:
  - **EC2 Instances**
  - **On-Premises Servers / VMs**
- Because it works with AWS and on-premises systems, it is a **Hybrid AWS Service**.

Key Functions of SSM

1. Fleet Management

- Manage thousands of servers from one place.
- Provides operational visibility across infrastructure.

2. Automated Patching

- Automatically apply operating system patches.
- Helps maintain security and compliance.

Exam Tip:

✅ If the question mentions **patching EC2 instances or on-premises servers**, think **SSM**.

3. Run Commands at Scale

- Execute commands across multiple servers simultaneously.
- No need to log in to each server individually.

Examples:

- Restart services
- Install software
- Update configurations

Exam Tip:

✅ Running commands consistently across a fleet of servers = **SSM Run Command**.

4. Parameter Store

- Secure storage for:
  - Configuration values
  - Application settings

- Secrets and parameters

Exam Tip:

✅ Configuration management → **SSM Parameter Store**

Supported Operating Systems

SSM supports:

- Linux
- Windows
- macOS
- Raspberry Pi

How SSM Works

Step 1: Install SSM Agent

A small program called the **SSM Agent** must run on managed systems.

EC2 Instance
↓
SSM Agent

On-Premises Server
↓
SSM Agent

Step 2: Agent Communicates with AWS SSM

EC2 / On-Prem VM
↓
SSM Agent
↓
AWS Systems Manager

The agent reports status and receives instructions from Systems Manager.

SSM Agent

Important Points

- Required for SSM management.
- Runs in the background.
- Enables communication between servers and AWS Systems Manager.

Default Installation

The SSM Agent is usually pre-installed on:

- Amazon Linux AMIs
- Many Ubuntu AMIs

Common Operations Performed with SSM

Patch Management

SSM
↓
Patch All Servers

- Update operating systems.
- Maintain compliance.

Run Command

SSM
↓
Execute Command
↓
All Servers

- Execute scripts remotely.
- Perform administrative tasks.

Configuration Management

SSM Parameter Store
↓

Store Configurations

↓

Applications Retrieve Values

Why SSM is Important

Centralized Management

- Single interface for managing infrastructure.

Automation

- Reduces manual operational work.

Hybrid Support

- Same management tools for:
  - AWS resources
  - On-premises resources

Compliance

- Helps enforce patching standards.

Exam Tips

✅ **SSM = AWS Systems Manager**

✅ Used to manage:

- EC2 instances
- On-premises servers

✅ Considered a **Hybrid AWS Service**

✅ Requires an **SSM Agent**

✅ Agent often comes pre-installed on:

- Amazon Linux
- Some Ubuntu AMIs

✅ Key features:

- Patch Management
- Run Command
- Parameter Store

✅ Used for:

- Automated patching

- Fleet management
- Configuration management
- Running commands remotely

✅ Supports:

- Linux
- Windows
- macOS
- Raspberry Pi

Common Exam Scenarios

Scenario 1

"Automatically patch hundreds of EC2 instances."

✅ Answer: **AWS Systems Manager (Patch Manager)**

Scenario 2

"Run the same command on all servers simultaneously."

✅ Answer: **AWS Systems Manager Run Command**

Scenario 3

"Store application configuration values securely."

✅ Answer: **SSM Parameter Store**

Scenario 4

"Manage both EC2 instances and on-premises servers from one service."

✅ Answer: **AWS Systems Manager (Hybrid Service)**

SSM Manager Overview

What is SSM Session Manager?

- **Session Manager** is a feature of **AWS Systems Manager (SSM)**.
- It allows secure remote access to:
  - EC2 Instances
  - On-Premises Servers
- Provides shell access **without SSH**, **without port 22**, **without SSH keys**, and **without a Bastion Host**.

Why Use Session Manager?

Traditional SSH Access

User
↓
Port 22 Open
↓
SSH Key
↓
EC2 Instance

Problems:

- Requires SSH keys
- Requires port 22 open
- Security risks from exposed SSH access

Session Manager Access

User
↓
SSM Session Manager
↓
SSM Agent
↓
EC2 Instance

`

Benefits:

- No SSH keys
- No open SSH port (22)
- No Bastion Host
- More secure remote access

Key Benefits

1. Improved Security

✅ No inbound SSH access required

✅ Port 22 can remain closed

✅ No SSH key management

✅ Reduces attack surface

2. Centralized Access

- Access instances directly through AWS Console, CLI, or SDK.
- No need for separate remote access solutions.

3. Session Logging

- Session activity can be logged to:
    - Amazon S3
    - CloudWatch Logs

Benefits:

- Auditing
- Compliance
- Security monitoring

4. Cross-Platform Support

Supports:

- Linux
- Windows
- macOS

Requirements for Session Manager

1. SSM Agent Installed

The target instance must have the **SSM Agent** running.

EC2 Instance
↓
SSM Agent

Good News

The SSM Agent is usually pre-installed on:

- Amazon Linux 2
- Many Ubuntu AMIs

2. IAM Role Attached

The EC2 instance needs an IAM Role with permissions to communicate with Systems Manager.

Common Managed Policy

AmazonSSMManagedInstanceCore

Exam Tip:

✅ If Session Manager isn't working, verify:

- SSM Agent installed
- IAM role attached

3. Instance Registered with Systems Manager

The instance should appear under:

Systems Manager
↓
Fleet Manager
↓
Managed Nodes

When visible there:

- SSM Agent is online
- Instance is managed by Systems Manager

How Session Manager Works

User
↓
AWS Console / CLI
↓
Session Manager
↓
SSM Agent
↓
EC2 Instance

The connection is established through the SSM service rather than SSH.

Demonstration Highlights from the Lecture

EC2 Instance Configuration

The EC2 instance was launched with:

✅ Amazon Linux 2

✅ No SSH Key Pair

✅ No inbound security group rules

Inbound Rules = 0

Yet Session Manager access still worked.

IAM Role Creation

An IAM role was created with:

AmazonSSMManagedInstanceCore

This policy allowed the instance to communicate with Systems Manager.

Fleet Manager Verification

The instance appeared under:

Systems Manager
↓
Fleet Manager
↓
Managed Nodes

This confirmed:

- SSM Agent online
- Instance managed by SSM

Starting a Session

Systems Manager
↓
Session Manager
↓
Start Session
↓
Secure Shell Access

A terminal session was opened directly from AWS Console.

Comparing Instance Access Methods

Method 1: SSH

User
↓
SSH Key
↓
Port 22 Open

↓
EC2

Requirements:

- SSH key pair
- Port 22 open

Method 2: EC2 Instance Connect

User
↓
EC2 Instance Connect
↓
Temporary SSH Keys
↓
Port 22 Open
↓
EC2

Requirements:

- Port 22 open
- No permanent SSH keys needed

Method 3: Session Manager

User
↓
Session Manager
↓
SSM Agent
↓
EC2

Requirements:

- SSM Agent
- IAM Role

Does NOT require:

- SSH Keys
- Port 22
- Bastion Host

Session History and Logging

Session Manager automatically keeps session information that can be audited.

Logs can be sent to:

✅ CloudWatch Logs

✅ Amazon S3

This improves:

- Compliance
- Monitoring
- Security investigations

Exam Tips

✅ **Session Manager is part of AWS Systems Manager (SSM)**

✅ Provides secure shell access to EC2 and on-premises servers

✅ No SSH keys required

✅ No Bastion Host required

✅ No need to open port 22

✅ Requires:

- SSM Agent
- IAM Role
- AmazonSSMManagedInstanceCore policy

✅ Supports:

- Linux
- Windows
- macOS

✅ Session logs can be stored in:

- CloudWatch Logs

- Amazon S3

✅ More secure than traditional SSH access

Common Exam Scenario

Question:

"An organization wants administrators to securely access EC2 instances without opening port 22 or managing SSH keys."

✅ **Answer: AWS Systems Manager Session Manager**

SSM Parameter Store

What is SSM Parameter Store?

- **AWS Systems Manager Parameter Store** is a service used to **securely store configuration data and secrets**.
- It is part of **AWS Systems Manager (SSM)**.
- Commonly used to store:
    - API Keys
    - Passwords
    - Database Connection Strings
    - Application Configurations
    - Environment Variables

Key Features

1. Centralized Configuration Management

- Store application configurations in one central location.
- Multiple applications can access the same parameters.

Applications
↓
SSM Parameter Store
↓
Configurations & Secrets

2. Serverless Service

- No servers to provision or manage.
- AWS handles the infrastructure automatically.

3. Scalable

- Can support a large number of parameters and requests.
- Designed for applications of any size.

4. Durable

- AWS manages storage and availability.

Security Features

IAM Integration

- Access to parameters is controlled using **IAM Policies**.
- Fine-grained permissions can be applied.

Exam Tip:

✅ Use IAM to control who can read or modify parameters.

Encryption with AWS KMS

For sensitive data, Parameter Store supports **SecureString**.

SecureString
↓
AWS KMS Encryption
↓
Encrypted Storage

Examples:

- Passwords
- API Keys
- Secrets

Exam Tip:

✅ SecureString parameters are encrypted using **AWS KMS**.

Parameter Types

1. String

Stores a single value.

Example:

DatabaseURL=prod-db.company.com

2. StringList

Stores multiple values separated by commas.

Example:

server1,server2,server3

3. SecureString

Stores encrypted values.

Example:

DatabasePassword=*******

Encrypted with KMS.

Version Tracking

Parameter Store automatically supports **versioning**.

Benefits:

- Track parameter changes
- Roll back to previous versions
- Audit configuration updates

Version 1

↓

Version ** ↓

Version 3

Exam Tip:

✅ Parameter Store supports parameter version history.

Typical Workflow

Administrat**

↓

Create Parameter

↓**tore Configuration / Secret

**

Application Retrieves Parameter

**

Example from the Lecture

Parameter Creation

Parameter Name:

demo-parameter

``**

Type:

String

Value:

my configuration paramet**

After creation:

- Stored in Parameter Store
- Easily retrieved
- Can be updated with new versions

Common Use Cases

Store Application Configuration

Environment = Production

Region **us-east-1

Store Secrets

API Keys

Passwords

To**ns

Certificates

Use **SecureString**.

Configuration Sharing

Multiple applications can use the same centrally managed settings.

Benefits of Parameter Store

- ✅ Centralized configuration storage
- ✅ Secure secret storage
- ✅ IAM-based access control
- ✅ KMS encryption support
- ✅ Serverless
- ✅ Scalable
- ✅ Durable
- ✅ Version tracking
- ✅ Easy application integration

Exam Tips

- ✅ **Parameter Store is part of AWS Systems Manager (SSM)**
- ✅ Used to store:
  - Configurations
  - Application settings
  - Secrets
- ✅ Parameter types:
  - String
  - StringList
  - SecureString
- ✅ **SecureString uses AWS KMS encryption**
- ✅ Access controlled through IAM
- ✅ Supports parameter versioning
- ✅ Fully managed and serverless
- ✅ Frequently used to store:
  - API keys

- Passwords
- Connection strings
- Environment variables

Common Exam Scenarios

Scenario 1

"Store application configuration centrally and securely."

✅ Answer: **SSM Parameter Store**

Scenario 2

"Store database passwords with encryption."

✅ Answer: **SSM Parameter Store SecureString + AWS KMS**

Scenario 3

"Maintain versions of configuration values over time."

✅ Answer: **SSM Parameter Store Versioning**

Deployment Summary

1. AWS CloudFormation

Purpose: Infrastructure as Code (IaC)

- ✅ AWS-only service
- ✅ Define infrastructure using YAML/JSON templates
- ✅ Automatically provisions AWS resources
- ✅ Supports almost all AWS services
- ✅ Templates can be reused across:
  - Regions
  - AWS Accounts
  - Environments
- ✅ Creates infrastructure in the correct dependency order

Use When:

- You need Infrastructure as Code.
- You want repeatable deployments.

Exam Keyword: *Infrastructure as Code (IaC)*

2. AWS Elastic Beanstalk

Purpose: Application Deployment Platform

- ✅ AWS-only service
- ✅ Platform as a Service (**PaaS**)
- ✅ Developers upload code; AWS manages infrastructure
- ✅ Handles:
  - EC2
  - Load Balancer
  - Auto Scaling
  - Monitoring
- ✅ Supports:
  - Java
  - Python
  - Node.js
  - PHP
  - .NET

- Ruby
- Go
- Docker

Use When:

- Developers want to focus on code, not infrastructure.

Exam Keyword: *PaaS*

3. AWS CodeDeploy

Purpose: Automated Application Deployment

✅ Deploys application updates automatically

✅ Deploys to:

- EC2 Instances
- On-Premises Servers

✅ Upgrades applications (V1 → V2)

✅ Requires CodeDeploy Agent

✅ Works independently of CloudFormation and Beanstalk

Use When:

- Deploying applications across servers.

Exam Keyword: *Hybrid Service*

4. AWS Systems Manager (SSM)

Purpose: Server Management at Scale

✅ Hybrid service

✅ Works with:

- EC2 Instances
- On-Premises Servers

✅ Features:

- Patch Management
- Run Command
- Fleet Management
- Parameter Store
- Session Manager

✅ Requires SSM Agent

Use When:

- Managing large fleets of servers.

Exam Keyword: *Patching and Managing Servers*

Important SSM Features

SSM Session Manager

✅ Secure shell access

✅ No SSH keys required

✅ No Bastion Host required

✅ No Port 22 required

✅ Supports:

- Linux
- Windows
- macOS

✅ Session logs can be sent to:

- CloudWatch Logs
- Amazon S3

Exam Scenario: "Access EC2 securely without opening port 22."

✅ Answer: **Session Manager**

SSM Parameter Store

✅ Stores:

- Configurations
- Secrets
- API Keys
- Passwords

✅ Serverless

✅ IAM-controlled access

✅ Version tracking

✅ Supports KMS encryption

Parameter Types:

- String
- StringList

- SecureString

Exam Scenario: "Store encrypted passwords and application configurations."

✅ Answer: **SSM Parameter Store**

Developer Services

5. AWS CodeCommit

Purpose: Source Code Repository

✅ Git-based repository

✅ Fully managed

✅ Secure and private

✅ Version control

✅ Developer collaboration

✅ Rollback capability

Use When:

- Storing application source code.

Exam Keyword: *Git Repository*

6. AWS CodeBuild

Purpose: Build and Test Code

✅ Serverless build service

✅ Retrieves source code

✅ Compiles code

✅ Runs tests

✅ Produces deployable artifacts

✅ Pay only while builds run

Flow:

Source Code

↓

CodeBuild

↓

Compile + Test

↓

Artifacts

Exam Keyword: *Build Service*

7. AWS CodePipeline

Purpose: CI/CD Orchestration

✅ Automates software delivery process

✅ Connects:

- CodeCommit
- CodeBuild
- CodeDeploy
- Other AWS Services

Flow:

Source
↓
Build
↓
Test
↓
Deploy

Exam Keyword: *CI/CD Pipeline Orchestration*

8. AWS CodeArtifact

Purpose: Dependency & Package Management

✅ Fully managed artifact repository

✅ Stores:

- Software Packages
- Libraries
- Dependencies

✅ Integrates with:

- Maven

- Gradle
- npm
- yarn
- pip
- NuGet

✅ Used by CodeBuild during builds

Exam Keyword: *Artifact Repository*

9. AWS CDK (Cloud Development Kit)

Purpose: Infrastructure as Code using Programming Languages

✅ Define infrastructure using:

- Python
- TypeScript
- JavaScript
- Java
- .NET

✅ Generates CloudFormation templates

✅ Supports:

- Loops
- Functions
- Classes
- Type Safety

Flow:

CDK Code
↓
CDK CLI
↓
CloudFormation Template
↓
AWS Resources

Exam Keyword: *CloudFormation using Programming Languages*

Cloud Practitioner

Quick Exam Cheat Sheet

Service	Main Purpose	Key Exam Keyword
CloudFormation	Infrastructure as Code	IaC
CDK	IaC using programming languages	Generates CloudFormation
Elastic Beanstalk	Application deployment platform	PaaS
CodeDeploy	Deploy applications	Hybrid Service
Systems Manager	Manage/Patch servers	Fleet Management
Session Manager	Access EC2 without SSH	No Port 22
Parameter Store	Store configs & secrets	KMS Encryption
CodeCommit	Store source code	Git Repository
CodeBuild	Build and test code	Serverless Build
CodePipeline	CI/CD orchestration	Automation Pipeline
CodeArtifact	Store dependencies	Artifact Repository

Most Important Exam Mappings

- **Infrastructure as Code** → CloudFormation / CDK
- **Deploy web app without managing infrastructure** → Elastic Beanstalk
- **Deploy application to EC2 and on-prem servers** → CodeDeploy
- **Patch EC2 fleet** → Systems Manager
- **Run commands on many servers** → Systems Manager Run Command
- **Access EC2 without SSH/Port 22** → Session Manager
- **Store passwords securely** → Parameter Store (SecureString + KMS)
- **Store source code** → CodeCommit
- **Build and test code** → CodeBuild
- **Store software dependencies** → CodeArtifact
- **Automate CI/CD process** → CodePipeline
- **Use Python/Java/TypeScript for IaC** → CDK

Ultimate Memory Trick

CodeCommit → Store Code

CodeBuild → Build Code

CodeArtifact → Store Dependencies

CodeDeploy → Deploy Code

CodePipeline → Automate Everything

CloudFormation → IaC

CDK → Programming Language IaC

Beanstalk → PaaS

SSM → Manage Servers

Session Manager→ Secure Access

Parameter Store→ Configurations & Secrets

Leveraging the AWS Global Infrastructure

Why Global Applications?

Why Build a Global Application?

A **global application** is deployed across multiple AWS Regions or Edge Locations to serve users worldwide.

1. Low Latency

- **Latency** = Time taken for data to travel between a user and a server.
- Users experience faster response times when applications are deployed closer to them.

Example:

User in India → Server in India/Asia → Low Latency

User in India → Server in USA → Higher Latency

✅ Deploy applications in multiple regions to improve user experience.

2. Disaster Recovery (DR)

- Protect applications against regional failures.
- If one region becomes unavailable, traffic can be redirected to another region.

Causes of regional failures may include:

- Natural disasters
- Power outages
- Network disruptions

✅ Multi-region deployments improve availability and business continuity.

3. Protection Against Attacks

- Distributed applications are harder to take down.
- Traffic can be spread across multiple regions and AWS infrastructure.

✅ Improves resilience against large-scale attacks.

AWS Global Infrastructure Components

1. Regions

- Geographic locations where AWS data centers are deployed.
- Used to deploy applications and infrastructure.

Examples:

- N. Virginia
- Ohio
- Mumbai
- Singapore
- Frankfurt
- Paris
- London

Region Characteristics

✅ Separate geographic locations

✅ Contain multiple Availability Zones

✅ Independent failure boundaries

2. Availability Zones (AZs)

- Each Region contains multiple AZs.
- An AZ consists of one or more data centers.

Characteristics

✅ Physically separated

✅ Connected through high-speed private networks

✅ Provide High Availability

Example

Region (Paris)

├── AZ-A

├── AZ-B

└── AZ-C

Exam Tip:

Every AWS Region contains **multiple Availability Zones**.

3. Edge Locations (Points of Presence)

- Small AWS sites located closer to users.
- Used primarily for content delivery.

Characteristics

- ✅ Not used for deploying applications
- ✅ Used by CloudFront and other networking services
- ✅ Reduce latency
- ✅ Cache content close to users

User
↓
Edge Location
↓
AWS Region

Exam Tip:
Edge Locations are also called **Points of Presence (PoPs)**.

4. AWS Global Network

AWS operates a large private global network.

Features

- ✅ Connects Regions
- ✅ Connects AZs
- ✅ Connects Edge Locations
- ✅ High-speed and highly reliable
- ✅ Includes undersea cables and private infrastructure

AWS Services for Global Applications

1. Amazon Route 53

Purpose

Global DNS Service

Features

- ✅ Routes users to applications
- ✅ Supports latency-based routing
- ✅ Supports failover routing
- ✅ Helps implement disaster recovery

Use Cases

- Direct users to the nearest region
- Redirect traffic during outages

Exam Keyword:

Global DNS Service

2. Amazon CloudFront

Purpose

Content Delivery Network (CDN)

Features

✅ Uses Edge Locations

✅ Caches content closer to users

✅ Reduces latency

✅ Improves application performance

✅ Reduces load on origin servers

User

↓

CloudFront Edge Location

↓

Origin (S3/EC2/ALB)

Exam Keyword:

CDN (Content Delivery Network)

3. S3 Transfer Acceleration

Purpose

Faster global uploads and downloads to Amazon S3

Features

✅ Uses AWS Edge Locations

✅ Improves transfer performance

✅ Especially useful for geographically distant users

Example

User

↓

Nearest Edge Location

↓

AWS Global Network

↓

S3 Bucket

Exam Keyword:

Accelerate S3 uploads/downloads worldwide

4. AWS Global Accelerator

Purpose

Improve global application performance and availability

Features

- ✅ Uses AWS global private network
- ✅ Directs users to healthy endpoints
- ✅ Improves latency
- ✅ Supports disaster recovery
- ✅ Provides static IP addresses

Exam Keyword:

Improve performance and availability using AWS global network

Exam Cheat Sheet

Component/Service	Purpose
Region	Geographic AWS location
Availability Zone (AZ)	One or more data centers within a Region
Edge Location (PoP)	Content delivery location close to users
Route 53	Global DNS and traffic routing
CloudFront	Content Delivery Network (CDN)
S3 Transfer Acceleration	Faster global S3 transfers
Global Accelerator	Improve application performance and availability

Common Exam Scenarios

Scenario 1

Users worldwide need the lowest possible latency.

✅ Use:

- Multiple AWS Regions
- CloudFront
- Route 53 Latency Routing

Scenario 2

Need a disaster recovery solution across regions.

✅ Use:

- Route 53 Failover Routing
- Multi-Region Deployment

Scenario 3

Need faster file uploads to S3 from global users.

✅ Use:

- S3 Transfer Acceleration

Scenario 4

Need to improve application availability and performance globally using AWS's private network.

✅ Use:

- AWS Global Accelerator

Route 53 Overview

What is Amazon Route 53?

- **Amazon Route 53** is AWS's **managed DNS (Domain Name System)** service.
- It helps users access applications using human-readable domain names instead of IP addresses.
- It is a key service for building **global applications**.

Exam Keyword

✅ **Route 53 = Managed DNS Service**

What is DNS?

DNS (Domain Name System) acts like a **phone book of the Internet**.

Instead of remembering:

54.210.10.25

Users type:

www.example.com

DNS translates the domain name into an IP address.

DNS Flow

User Browser
↓
DNS Query
↓
Route 53
↓
Returns IP Address
↓
Connects to Server

Route 53 Record Types

1. A Record (IPv4)

Maps a hostname to an IPv4 address.

Example:

www.google.com
↓
142.250.xx.xx

✅ Used for IPv4 addresses

2. AAAA Record (IPv6)

Maps a hostname to an IPv6 address.

Example:

www.google.com
↓
2001:4860:4860::8888

✅ Used for IPv6 addresses

3. CNAME Record

Maps one hostname to another hostname.

Example:

search.example.com
↓
www.google.com

✅ Hostname → Hostname mapping

4. Alias Record (AWS-Specific)

Maps a hostname directly to an AWS resource.

Example:

www.example.com
↓
Application Load Balancer

Can point to:

- Elastic Load Balancer (ELB)
- CloudFront
- S3 Website
- API Gateway
- Other AWS resources

Exam Tip

✅ **Alias Record = AWS Resource Mapping**

Route 53 Example

Suppose:

myapp.mydomain.com

must connect users to an EC2 instance.

Process

Browser
↓
DNS Query:
myapp.mydomain.com
↓
Route 53
↓
Returns Public IP
↓
Browser Connects to EC2

Route 53 Routing Policies

These are extremely important for exams.

1. Simple Routing Policy

Purpose

- Route traffic to a single resource.

Characteristics

✅ Single resource

✅ Single DNS response

❌ No health checks

Diagram

User

↓

Route 53

↓

Server

Use Case

- Single website
- Simple applications

Exam Tip

✅ **Simple Routing = No Health Check**

2. Weighted Routing Policy

Purpose

Distribute traffic across multiple resources.

Example Weights:

Server A = 70

Server B = 20

Server C = 10

Traffic Distribution:

70% → Server A
20% → Server B
10% → Server C

Characteristics

✅ Traffic splitting
✅ Supports health checks
✅ Useful for testing new deployments

Use Cases

- Load distribution
- A/B testing
- Blue/Green deployments

Exam Tip

✅ **Weighted Routing = Traffic Distribution by Percentage**

3. Latency Routing Policy

Purpose

Route users to the resource with the lowest latency.

Example

Server 1 → California
Server 2 → Australia

Users are automatically directed to the closest/fastest server.

US User
↓
California

Australian User

↓

Australia

Characteristics

✅ Minimizes latency

✅ Improves user experience

✅ Supports health checks

Use Case

- Global applications

Exam Tip

✅ **Latency Routing = Best Performance for Global Users**

4. Failover Routing Policy

Purpose

Disaster Recovery (DR)

Architecture

Primary Server

↓

Health Check

If Healthy

↓

Users → Primary

If Failed

↓

Users → Secondary

`

Characteristics

✅ Primary and Secondary resources

✅ Uses health checks

✅ Supports disaster recovery

Use Case

- Business continuity
- High availability

Exam Tip

✅ **Failover Routing = Disaster Recovery**

Health Checks

Supported By

✅ Weighted Routing

✅ Latency Routing

✅ Failover Routing

Not Supported By

❌ Simple Routing

Quick Comparison

Routing Policy	Purpose	Health Check
Simple	Single resource routing	❌ No
Weighted	Distribute traffic by percentage	✅ Yes
Latency	Lowest latency routing	✅ Yes
Failover	Disaster recovery	✅ Yes

Common Exam Scenarios

Scenario 1

"Route all users to a single server."

✅ Answer: **Simple Routing Policy**

Scenario 2

"Send 70% traffic to one server and 30% to another."

✅ Answer: **Weighted Routing Policy**

Scenario 3

"Users should connect to the nearest AWS Region."

✅ Answer: **Latency Routing Policy**

Scenario 4

"Automatically switch to a backup server when the primary server fails."

✅ Answer: **Failover Routing Policy**

Memory Trick

Simple = One Server

Weighted = Split Traffic

Latency = Closest Server

Failover = Backup Server

CloudFront Overview

What is Amazon CloudFront?

- **Amazon CloudFront** is AWS's **Content Delivery Network (CDN)** service.
- It improves website and application performance by **caching content at Edge Locations worldwide**.
- Delivers content to users with **low latency** and **high transfer speed**.

Exam Keyword

✅ **CDN = CloudFront**

Why Use CloudFront?

1. Lower Latency

- Content is cached closer to users.
- Users access nearby Edge Locations instead of the origin server.

User

↓

Nearest Edge Location

↓

Origin Server (only if needed)

✅ Faster website and application performance

2. Improved Read Performance

- Frequently requested content is served from cache.
- Reduces requests to the origin server.

✅ Faster response times

✅ Reduced backend load

3. Global Distribution

- Uses hundreds of Edge Locations and caches worldwide.
- Makes content available globally.

✅ Better user experience worldwide

4. DDoS Protection

CloudFront integrates with:

- AWS Shield
- AWS Web Application Firewall (WAF)

Benefits:

- DDoS protection
- Security against web attacks

✅ Increased application security

Key CloudFront Components

Edge Locations (Points of Presence)

- Global locations where CloudFront caches content.
- Located closer to users.

Characteristics

✅ Cache content

✅ Reduce latency

✅ Improve performance

✅ Not used to run applications

User

↓

Edge Location

↓

Origin

What is an Origin?

An **Origin** is the backend source from which CloudFront retrieves content.

When content is not available in the cache:

CloudFront

↓

Origin

↓

Retrieve Content

Supported Origins

1. Amazon S3 Bucket

Use Cases

- Static websites
- Images
- Videos
- Downloads
- Documents

Benefits

✅ File distribution

✅ Global caching

✅ Secure access using OAC

OAC (Origin Access Control)

Used to secure communication between:

CloudFront

↓

S3 Bucket

Benefits:

✅ Restricts direct S3 access

✅ Forces access through CloudFront

Exam Tip

✅ **CloudFront + S3 security = Origin Access Control (OAC)**

2. VPC Origins

CloudFront can access resources inside a VPC.

Examples:

- Private Application Load Balancer (ALB)
- Private Network Load Balancer (NLB)
- Private EC2 Instances

✅ Deliver private applications through CloudFront

3. Custom Origins

Any public HTTP/HTTPS endpoint.

Examples:

- Public ALB
- Public EC2
- External web servers
- S3 Static Website Endpoint

✅ Supports external and custom backends

How CloudFront Works

Step 1

User requests content.

User

↓

Edge Location

Step 2

CloudFront checks cache.

Cache Hit

Content Found

↓

Serve Immediately

✅ Fastest response

Cache Miss

Content Not Found

↓

Origin

↓

Retrieve Content

↓

Store in Cache

↓

Serve User

✅ Future requests become faster

CloudFront with S3 Example

S3 Bucket (Origin)

↓

CloudFront

↓

Edge Locations Worldwide

↓

Users

Example

S3 Bucket located in Australia.

User in USA

User (USA)

↓

US Edge Location

↓

S3 Bucket (Australia)

Show more lines

First request:

- Content fetched from Australia.
- Cached at US Edge Location.

Future requests:

- Served directly from US cache.

User in Brazil

User (Brazil)

↓

Brazil Edge Location

↓

S3 Bucket (Australia)

Show more lines

Again:

- Edge Location caches content locally.

✅ Global low-latency access

CloudFront Advantages

✅ Lower latency

✅ Global caching

✅ Improved website performance

✅ Reduced origin load

✅ DDoS protection

✅ Integration with AWS Shield and WAF

✅ Secure S3 access using OAC

✅ High availability

CloudFront vs S3 Cross-Region Replication (CRR)

CloudFront

Purpose

Content Delivery Network (CDN)

Characteristics

✅ Global Edge Network

✅ Hundreds of Edge Locations

✅ Content Caching

✅ Lower latency worldwide

✅ Optimized for content delivery

S3 Bucket

↓

CloudFront Edge Locations

↓

Users Worldwide

S3 Cross-Region Replication (CRR)

Purpose

Replicate entire S3 buckets between AWS Regions.

Characteristics

✅ Regional replication

✅ Near real-time copying

✅ No caching

✅ Read-only replication

S3 Bucket Region A

↓

Replication

↓

S3 Bucket Region B

CloudFront vs S3 CRR Summary

Feature	CloudFront	S3 CRR
Purpose	CDN	Bucket Replication
Uses Edge Locations	✅ Yes	❌ No
Caching	✅ Yes	❌ No
Global Performance	✅ Excellent	Limited
Replicates Entire Bucket	❌ No	✅ Yes
Low-Latency Worldwide Access	✅ Yes	Limited
Dynamic Content Updates	Not ideal	Better

Exam Tips

✅ **CloudFront = CDN**

✅ Uses **Edge Locations (Points of Presence)**

✅ Improves application read performance

✅ Reduces latency globally

✅ Caches content close to users

✅ Supports origins:

- S3
- VPC Origins
- Custom Origins

✅ Uses **Origin Access Control (OAC)** for secure S3 access

✅ Integrates with:

- AWS Shield

- AWS WAF

✅ Helps protect against DDoS attacks

✅ Cache Hit = Content served from Edge Location

✅ Cache Miss = Retrieve from Origin and cache it

Common Exam Scenarios

Scenario 1

Users around the world experience high latency accessing a website.

✅ Answer: **Amazon CloudFront**

Scenario 2

Need a CDN service on AWS.

✅ Answer: **Amazon CloudFront**

Scenario 3

Need to cache S3 content globally at Edge Locations.

✅ Answer: **Amazon CloudFront**

Scenario 4

Need secure access between CloudFront and S3.

✅ Answer: **Origin Access Control (OAC)**

Scenario 5

Need DDoS protection and faster content delivery.

✅ Answer: **CloudFront + AWS Shield**

Memory Trick

CloudFront

=

CDN

=

Cache Content

=

Edge Locations

=

Low Latency

CloudFront → Deliver Content

S3 CRR → Replicate Buckets

S3 Transfer Acceleration

What is S3 Transfer Acceleration?

- **Amazon S3 Transfer Acceleration** speeds up **uploads and downloads** to an S3 bucket from anywhere in the world.
- It uses AWS **Edge Locations** and the AWS **Global Network** to improve transfer performance.
- Especially useful when users are geographically far from the target S3 bucket.

Exam Keyword

✅ **Faster global uploads/downloads to S3**

Why Do We Need It?

S3 buckets exist in a **single AWS Region**.

Example:

S3 Bucket → Australia

User → United States

Without Transfer Acceleration:

User

↓

Internet

↓

Australia S3 Bucket

This can result in:

- Higher latency
- Slower uploads
- Slower downloads

How S3 Transfer Acceleration Works

Instead of sending files directly to the S3 bucket:

User

↓

Nearest AWS Edge Location

↓

AWS Global Private Network

↓

S3 Bucket

Example

User (USA)

↓

US Edge Location

↓

AWS Global Network

↓

S3 Bucket (Australia)

Benefits:

- ✅ Faster transfers
- ✅ More reliable network path
- ✅ Lower latency
- ✅ Uses AWS's optimized backbone network

Architecture

Without Transfer Acceleration

User

↓

Public Internet

↓

S3 Bucket

With Transfer Acceleration

User

↓

Nearest Edge Location

↓

AWS Global Network

↓

S3 Bucket

Key Benefits

1. Faster Uploads

- Accelerates file uploads from remote locations.

2. Faster Downloads

- Improves download speeds for global users.

3. Uses AWS Global Network

- Traffic travels over AWS's optimized private network.

4. Better for Global Applications

- Ideal when users are distributed around the world.

When Should You Use It?

Use S3 Transfer Acceleration when:

✅ Users are globally distributed

✅ Files are uploaded to a bucket located far away

✅ Large files need faster transfer

✅ Performance is critical

Example Use Cases

- Media uploads
- Backup systems
- Image/video sharing applications
- Global file transfer applications

S3 Transfer Acceleration vs CloudFront

S3 Transfer Acceleration

Purpose

Accelerate uploads & downloads

to S3 buckets

Uses:

- Edge Locations
- AWS Global Network

Best for:

- File transfers

CloudFront

Purpose

Uses:

- Edge Locations
- Content caching

Best for:

- Websites
- Images
- Videos
- Static content

Comparison

Feature	S3 Transfer Acceleration	CloudFront
Main Purpose	Faster S3 transfers	Content Delivery (CDN)
Upload Acceleration	✅ Yes	❌ No
Download Acceleration	✅ Yes	✅ Yes
Content Caching	❌ No	✅ Yes
Uses Edge Locations	✅ Yes	✅ Yes
Uses AWS Global Network	✅ Yes	✅ Yes

Exam Tips

- ✅ **S3 Transfer Acceleration improves global uploads and downloads to S3**
- ✅ Uses **Edge Locations**
- ✅ Uses AWS's **private global network**
- ✅ Useful when users are far from the S3 bucket
- ✅ Ideal for global applications
- ✅ No content caching involved
- ✅ Different from CloudFront

Common Exam Scenarios

Scenario 1

A company has users worldwide uploading large files to a single S3 bucket. Uploads are slow.

✅ Answer: **S3 Transfer Acceleration**

Scenario 2

Need to speed up transfers to an S3 bucket located in another continent.

✅ Answer: **S3 Transfer Acceleration**

Scenario 3

Need a service that leverages AWS Edge Locations to accelerate uploads to S3.

✅ Answer: **S3 Transfer Acceleration**

Scenario 4

Need content caching worldwide for a website.

✅ Answer: **CloudFront** (not S3 Transfer Acceleration)

Memory Trick

CloudFront

=

Cache Content

=

CDN

S3 Transfer Acceleration

=

Transfer Files Faster

=

S3 Upload/Download Optimization

AWS Global Accelerator

What is AWS Global Accelerator?

- **AWS Global Accelerator** improves **application availability and performance** for users worldwide.
- It uses the **AWS Global Network** to route traffic efficiently.
- Traffic enters AWS through the nearest **Edge Location** and travels over AWS's private backbone network instead of relying entirely on the public internet.

Exam Keyword

✅ **Improve Global Application Performance and Availability**

Why Use Global Accelerator?

Problem

If users connect directly through the internet:

User

↓

Public Internet

↓

AWS Region

Problems:

- Variable latency
- Network congestion
- Multiple routing hops
- Unpredictable performance

Solution

With Global Accelerator:

User

↓

Nearest AWS Edge Location

↓

AWS Global Private Network

↓

Application Endpoint

Benefits:

✅ Faster performance

✅ Lower latency

✅ More reliable routing

✅ Improved availability

How Global Accelerator Works

Example

Application is hosted in:

AWS Region: India

Users are located in:

- USA
- Europe
- Australia

Instead of traversing the public internet all the way to India:

User (USA)

↓

Nearest Edge Location

↓

AWS Private Global Network

↓

Application (India)

The same applies for Europe and Australia.

Anycast Static IP Addresses

Global Accelerator provides:

✅ Two static Anycast IP addresses

Benefits

- Users always connect using the same IPs.
- AWS automatically routes users to the optimal Edge Location.

Users

↓

Static Anycast IPs

↓

Nearest Edge Location

↓

Application Endpoint

Exam Tip

✅ **Global Accelerator provides static Anycast IPs**

Main Benefits

1. Improved Performance

AWS states traffic is optimized through its private network, often resulting in significantly better performance than public internet routes.

✅ Faster response times

✅ Reduced latency

2. High Availability

Traffic can be automatically redirected to healthy endpoints.

Endpoint A (Healthy)

↓

Receives Traffic

Endpoint B (Backup)

↓

Used if A Fails

✅ Built-in failover capabilities

3. Global Reach

Users worldwide connect through the nearest Edge Location.

✅ Better user experience globally

4. DDoS Protection

Integrates with:

- AWS Shield
- AWS Global Edge Network

✅ Improved security

How Global Accelerator Differs from CloudFront

This is a very common exam question.

CloudFront

Purpose

Content Delivery Network (CDN)

How it Works

User

↓

Edge Location

↓

Cache Content

Features

✅ Caches content

✅ Stores images, videos, websites at Edge Locations

✅ Reduces latency by serving content locally

Best For

- Static websites
- Images
- Videos
- Downloads
- Streaming

Global Accelerator

Purpose

Network Traffic Optimization

How it Works

User

↓

Edge Location

↓

AWS Global Network

↓

Application Endpoint

Features

- ✅ No caching
- ✅ All requests reach the application
- ✅ Optimizes TCP and UDP traffic
- ✅ Uses static Anycast IPs

Best For

- HTTP applications
- TCP applications
- UDP applications
- Gaming applications
- APIs
- Failover requirements

CloudFront vs Global Accelerator

Feature	CloudFront	Global Accelerator
Service Type	CDN	Network Accelerator
Uses Edge Locations	✅ Yes	✅ Yes
Uses AWS Global Network	✅ Yes	✅ Yes
Caches Content	✅ Yes	❌ No
Static Anycast IPs	❌ No	✅ Yes
Improves Application Routing	Limited	✅ Yes
Supports TCP/UDP Traffic	❌ Mainly HTTP/HTTPS	✅ Yes
Helps with Regional Failover	Limited	✅ Yes

Common Use Cases

Use CloudFront When

You need:

- ✅ Content caching
- ✅ Faster website delivery
- ✅ Video delivery
- ✅ Static content distribution

Use Global Accelerator When

You need:

- ✅ Faster application access
- ✅ Global TCP/UDP performance
- ✅ Static IP addresses
- ✅ Multi-region failover
- ✅ Better network routing

Exam Scenarios

Scenario 1

Users worldwide experience inconsistent application performance due to internet routing.

✅ Answer: **AWS Global Accelerator**

Scenario 2

Need two static global IP addresses for an application.

✅ Answer: **AWS Global Accelerator**

Scenario 3

Need automatic regional failover and improved availability.

✅ Answer: **AWS Global Accelerator**

Scenario 4

Need to cache images and videos close to users.

✅ Answer: **Amazon CloudFront**

Scenario 5

Need to improve TCP/UDP application performance globally without caching.

✅ Answer: **AWS Global Accelerator**

Memory Trick

CloudFront

=

Cache Content

=

CDN

=

Edge Caching

Show more lines

Plain Text

Global Accelerator

=

Accelerate Traffic

=

Static Anycast IPs

=

AWS Private Network

=

No Caching

AWS Outposts

What is Hybrid Cloud?

A **Hybrid Cloud** is an environment where an organization uses both:

- **On-Premises Infrastructure** (own data center)
- **Cloud Infrastructure** (AWS Cloud)

On-Premises Data Center

+

AWS Cloud

=

Hybrid Cloud

Challenges of Hybrid Cloud

- Different management tools
- Different APIs
- Different skill sets
- Increased operational complexity

What is AWS Outposts?

AWS Outposts is a fully managed service that brings AWS infrastructure, services, APIs, and tools directly into your on-premises data center.

Key Idea

AWS installs and manages physical server racks in your data center.

These racks run AWS services locally while providing the same AWS experience.

Your Data Center

↓

AWS Outposts Rack

↓

AWS Services Running Locally

Exam Keyword

✅ **AWS Services On-Premises**

Why AWS Outposts?

Without Outposts:

Data Center

↓

Different Tools

↓

Different Infrastructure

With Outposts:

Data Center

↓

AWS Outposts

↓

Same AWS APIs

Same AWS Services

Same AWS Console

Benefits:

- Consistent operations
- Easier application development
- Easier migration to AWS

How AWS Outposts Works

Traditional AWS

AWS Region

↓

EC2

RDS

S3

EKS

AWS Outposts

Company Data Center

↓

AWS Outposts Rack

↓

EC2

EBS

RDS

EKS

ECS

S3

EMR

AWS installs, operates, and maintains the Outposts rack.

Responsibilities

AWS Responsibilities

✅ Deliver Outposts rack

✅ Install hardware

✅ Maintain infrastructure

✅ Update hardware/software

✅ Monitor service health

Customer Responsibilities

✅ Physical security of the data center

✅ Access control to premises

✅ Local networking

✅ Power and facility management

Exam Tip

✅ AWS manages the Outposts hardware, but **you are responsible for securing your physical data center**.

Benefits of AWS Outposts

1. Low-Latency Access

Applications can run close to local systems.

Application

↓

Local Servers

Benefits:

- Faster response times
- Reduced network latency

2. Local Data Processing

Data can remain within your premises.

Data

↓

Processed Locally

Useful when applications require local processing.

3. Data Residency

Data stays in your data center.

Benefits:

- Compliance
- Regulatory requirements
- Data sovereignty

4. Easier Cloud Migration

Migration path:

On-Premises

↓

AWS Outposts

↓

AWS Cloud

Organizations can gradually move workloads to AWS.

5. Consistent AWS Experience

Uses:

- AWS Console
- AWS APIs
- AWS CLI
- AWS Services

Same development and operational model as AWS Cloud.

6. Fully Managed

AWS manages:

- Hardware lifecycle
- Infrastructure maintenance
- Updates

AWS Services Supported on Outposts

The lecture highlighted:

Compute

✅ Amazon EC2

✅ Amazon ECS

✅ Amazon EKS

Storage

✅ Amazon EBS

✅ Amazon S3

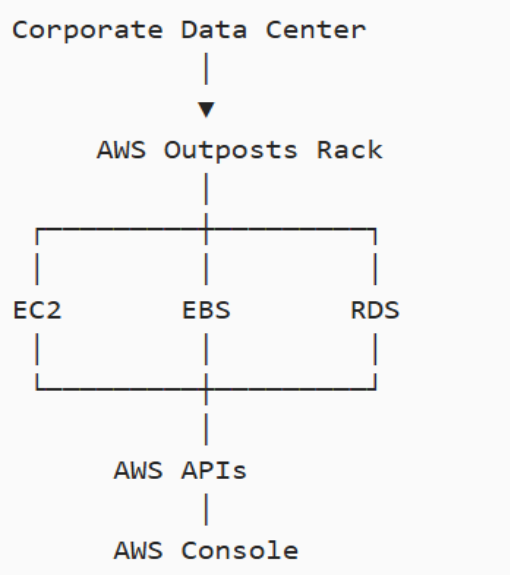
Databases

✅ Amazon RDS

Analytics

✅ Amazon EMR

AWS Outposts Architecture



AWS Outposts vs Standard AWS Region

Feature	AWS Region	AWS Outposts
Location	AWS Data Center	Customer Data Center
Managed by AWS	Yes	Yes
Physical Security	AWS	Customer
AWS APIs	Yes	Yes

AWS Console	Yes	Yes
Local Data Processing	No	Yes
Low-Latency On-Prem Access	No	Yes

Common Exam Scenarios

Scenario 1

A company must keep data and applications in its own data center while using AWS services.

✅ Answer: **AWS Outposts**

Scenario 2

An organization wants the same AWS APIs and tools on-premises.

✅ Answer: **AWS Outposts**

Scenario 3

Applications require very low latency to local manufacturing systems.

✅ Answer: **AWS Outposts**

Scenario 4

Regulations require data to remain within the company's facility.

✅ Answer: **AWS Outposts**

Exam Tips

✅ **AWS Outposts = AWS Infrastructure in Your Data Center**

✅ Supports Hybrid Cloud architectures

✅ Uses the same:

- AWS Console
- AWS APIs
- AWS CLI
- AWS Services

✅ AWS installs and manages the Outposts rack

✅ Customer secures the physical premises

✅ Benefits:

- Low latency
- Local processing
- Data residency
- Easier cloud migration

✅ Common supported services:

- EC2
- EBS
- S3
- ECS
- EKS
- RDS
- EMR

AWS Wavelength

What is AWS Wavelength?

- **AWS Wavelength** brings AWS services to the **edge of 5G networks**.
- It places AWS infrastructure inside telecommunications providers' data centers.
- Designed to deliver **ultra-low latency applications** for mobile and 5G users.

Exam Keyword

✅ **5G = AWS Wavelength**

Main Purpose

AWS Wavelength allows applications to run closer to mobile users by deploying AWS resources directly within a telecom provider's 5G network.

Mobile User

↓

5G Network

↓

AWS Wavelength Zone

↓

Application

Result:

✅ Ultra-low latency

✅ Faster response times

✅ Better user experience

What is a Wavelength Zone?

A **Wavelength Zone** is an AWS infrastructure deployment located within a telecom carrier's network.

Supported Services

You can deploy services such as:

- Amazon EC2
- Amazon EBS
- Amazon VPC

directly into a Wavelength Zone.

Wavelength Zone

├── EC2

├── EBS

└── VPC

How AWS Wavelength Works

Traditional Architecture

User

↓

Internet

↓

AWS Region

Problems:

- Higher latency
- More network hops

AWS Wavelength Architecture

5G User

↓

Telecom Provider Network

↓

Wavelength Zone

↓

Application

Benefits:

- Very low latency
- Traffic stays close to users
- Faster application performance

Connection to Parent AWS Region

Each Wavelength Zone is connected to a **Parent AWS Region**.

Wavelength Zone

↓

Parent AWS Region

This allows applications running in Wavelength to access AWS services in the parent region.

Examples:

- Amazon RDS
- Amazon DynamoDB
- Amazon S3
- Other AWS Services

Example

Mobile User

↓

Wavelength EC2

↓

Amazon RDS

(Parent Region)

Key Benefits

1. Ultra-Low Latency

Applications run extremely close to mobile users.

✅ Millisecond-level latency

✅ Improved responsiveness

2. 5G Integration

Built directly into telecom carrier networks.

✅ Optimized for mobile applications

✅ Ideal for 5G workloads

3. Reduced Network Hops

Traffic does not need to travel long distances before reaching the application.

User

↓

5G Network

↓

Application

4. Seamless AWS Integration

Applications can still use AWS services from the parent region.

✅ Compute locally

✅ Store data regionally

Common Use Cases

Smart Cities

Sensors

↓

Real-Time Processing

↓

Wavelength

✅ Fast local decision-making

Connected Vehicles

Vehicle

↓

5G

↓

Wavelength

✅ Real-time communication

✅ Faster vehicle response systems

AR / VR Applications

User

↓

AR/VR App

↓

Wavelength

✅ Smooth immersive experiences

✅ Reduced lag

Interactive Live Video Streaming

✅ Low-latency streaming

✅ Faster content delivery

Real-Time Gaming

- ✅ Reduced game latency
- ✅ Better multiplayer performance

Machine Learning Assisted Diagnostics

- ✅ Real-time processing
- ✅ Faster analysis and response

AWS Wavelength vs AWS Outposts

Feature	AWS Wavelength	AWS Outposts
Location	Telecom 5G Network	Customer Data Center
Main Goal	Ultra-low latency for 5G users	Hybrid Cloud
Typical User	Mobile/5G Applications	Enterprise Data Centers
Parent Region Connection	Yes	Yes
Key Keyword	5G	On-Premises

AWS Wavelength vs Local Zones

AWS Wavelength

- ✅ Designed for 5G networks
- ✅ Telecom provider integration
- ✅ Mobile applications

AWS Local Zones

- ✅ Bring AWS services closer to cities
- ✅ General low-latency workloads
- ✅ Not specifically tied to 5G

Exam Tips

- ✅ **AWS Wavelength = 5G Edge Computing**
- ✅ Deploy AWS services directly into telecom networks
- ✅ Supports:
  - EC2

- EBS
- VPC

✅ Connected to a Parent AWS Region

✅ Enables ultra-low latency applications

✅ Ideal for:

- AR/VR
- Gaming
- Smart Cities
- Connected Vehicles
- Live Streaming
- ML-assisted diagnostics

✅ Traffic can stay within the telecom provider network

Common Exam Scenarios

Scenario 1

An application requires ultra-low latency for mobile users on a 5G network.

✅ Answer: **AWS Wavelength**

Scenario 2

A real-time gaming application must run as close as possible to 5G users.

✅ Answer: **AWS Wavelength**

Scenario 3

A connected vehicle platform requires millisecond response times over 5G.

✅ Answer: **AWS Wavelength**

Scenario 4

Need AWS compute resources directly inside a telecom provider's network.

✅ Answer: **AWS Wavelength**

Memory Trick

Outposts

=

AWS in Your Data Center

Wavelength

=

AWS in Telecom 5G Network

5G

→ Wavelength

On-Premises

→ Outposts

AWS Local Zone

What are AWS Local Zones?

- **AWS Local Zones** are extensions of an AWS Region that place AWS resources closer to end users.
- They help run **latency-sensitive applications** by reducing the distance between users and AWS infrastructure.
- Local Zones extend a parent AWS Region into a nearby metropolitan area.

Exam Keyword

✅ **Low Latency for Specific Cities = AWS Local Zones**

Why Use Local Zones?

Normally, applications run in an AWS Region:

Users

↓

AWS Region

If users are far from the Region, latency increases.

With Local Zones:

Users

↓

Local Zone (Nearby City)

↓

Parent AWS Region

Benefits:

✅ Lower latency

✅ Better user experience

✅ Compute closer to users

How Local Zones Work

A Local Zone is an extension of an AWS Region.

Example

Parent Region:

US-East-1 (Northern Virginia)

Local Zones:

Boston

Chicago

Dallas

Houston

Miami

Applications can be deployed in these Local Zones while remaining connected to the parent Region.

Supported Services

Local Zones support many AWS services, including:

✅ EC2

✅ EBS

✅ ECS

✅ RDS

✅ ElastiCache

✅ Direct Connect

Exam Tip

You do not need to memorize every service, just know that Local Zones support compute, storage, networking, and selected database services.

Architecture Example

US-East-1 Region

│

├── AZ-A

├── AZ-B

├── AZ-C

│

└── Local Zone (Boston)

│

└── EC2 Instance

The Local Zone remains connected to the parent Region.

VPC Integration

A Local Zone can be added to an existing VPC.

Steps

1. Enable the Local Zone.
2. Create a subnet in the Local Zone.
3. Launch resources into that subnet.

Example:

VPC

├── Subnet in AZ

├── Subnet in AZ

└── Subnet in Boston Local Zone

Then EC2 instances can be launched directly in Boston.

Local Zones vs Availability Zones

Feature	Availability Zone (AZ)	Local Zone
Part of AWS Region	✅ Yes	Extension of Region
Purpose	High Availability	Low Latency
Location	Around Region	Near specific cities
Connected to Parent Region	✅ Yes	✅ Yes

Exam Tip

✅ AZs provide **High Availability**

✅ Local Zones provide **Low Latency**

AWS Local Zones vs AWS Wavelength

This is a favorite exam comparison.

AWS Local Zones

Purpose:

✅ Bring AWS services closer to cities

✅ Low-latency applications

Examples:

- Video rendering
- Gaming
- Media production

AWS Wavelength

Purpose:

✅ Bring AWS services into telecom 5G networks

✅ Ultra-low latency mobile applications

Examples:

- AR/VR
- Connected vehicles
- Real-time gaming over 5G

Quick Comparison

Service	Main Use Case
Local Zones	Low latency in a city
Wavelength	Ultra-low latency over 5G

Memory Trick

City Users

→ Local Zones

5G Mobile Users

→ Wavelength

Common Use Cases

Media & Entertainment

- Video editing
- Content rendering
- Broadcasting

Gaming

- Low-latency gameplay
- Real-time interactions

Financial Applications

- Faster transaction processing

Virtual Desktops

- Better responsiveness for local users

Exam Scenarios

Scenario 1

An application must run closer to users in Boston while still using the US-East-1 Region.

✅ Answer: **AWS Local Zones**

Scenario 2

A company wants to extend its VPC into a nearby city for lower latency.

✅ Answer: **AWS Local Zones**

Scenario 3

Users need low-latency access, but the workload is not specifically tied to a 5G network.

✅ Answer: **AWS Local Zones**

Scenario 4

An application requires ultra-low latency on a telecom provider's 5G network.

✅ Answer: **AWS Wavelength** (not Local Zones)

Exam Tips

✅ **Local Zones extend an AWS Region**

✅ Designed for **latency-sensitive applications**

✅ Bring AWS services closer to end users

✅ Can host:

- EC2
- EBS
- ECS
- RDS
- ElastiCache

✅ Connected to a parent AWS Region

✅ VPCs can be extended to Local Zones

✅ Different from Wavelength:

- Local Zones → City-based latency reduction
- Wavelength → 5G edge computing

Global Application Architecture

1. Single Region, Single AZ

Architecture

Region

↓

AZ-1

↓

EC2 Instance

Characteristics

✅ Very simple setup

✅ Lowest cost

❌ No High Availability

❌ No Disaster Recovery

❌ Poor global performance for distant users

Availability

Low

Global Latency

High

(for distant users)

Complexity

Very Low

Exam Tip

If the AZ fails, the application becomes unavailable.

2. Single Region, Multi-AZ

Architecture

Region

├— AZ-1 → EC2

└— AZ-2 → EC2

Characteristics

✅ High Availability

✅ Fault tolerance within a Region

✅ Protection against AZ failures

❌ No improvement for global latency

Availability

High

Global Latency

Still High

(for users far from the Region)

Complexity

Medium

Exam Tip

Multi-AZ improves availability, not global performance.

3. Multi-Region Active-Passive

Architecture

Region A (Active)

↓

Reads + Writes

⇅ Replication

Region B (Passive)

↓

Backup / Disaster Recovery

Characteristics

✅ Multiple Regions

✅ Disaster Recovery

✅ Data replicated to another Region

✅ Failover possible

Active Region

Reads = Yes

Writes = Yes

Passive Region

Reads = Possible

Writes = No

Benefits

✅ Better read latency globally

✅ Strong Disaster Recovery strategy

Drawback

❌ Writes still go to one central region

❌ Global write latency can remain high

Availability

Very High

Complexity

High

Exam Keyword

✅ **Active-Passive = Disaster Recovery**

4. Multi-Region Active-Active

Architecture

Region A

Reads + Writes

⇅

Data Replication

⇅

Region B

Reads + Writes

Characteristics

✅ Multiple Regions

✅ Reads and writes in all regions

✅ Continuous replication

✅ Best global user experience

Benefits

✅ Low read latency

✅ Low write latency

✅ Global scalability

✅ Highest availability

Drawback

❌ Most complex architecture

Availability

Highest

Global Latency

Lowest

Complexity

Highest

Example Service

✅ **DynamoDB Global Tables**

Supports:

Multi-Region

Active-Active Replication

Exam Keyword

✅ **Active-Active = Reads and Writes Everywhere**

Comparison Table

Architecture	High Availability	Global Read Latency	Global Write Latency	Complexity
Single Region, Single AZ	❌ No	❌ Poor	❌ Poor	⭐ Very Low
Single Region, Multi-AZ	✅ Yes	❌ Poor	❌ Poor	⭐ ⭐ Low
Multi-Region Active-Passive	✅ Yes	✅ Better	❌ Centralized Writes	⭐ ⭐ ⭐ High
Multi-Region Active-Active	✅ Yes	✅ Excellent	✅ Excellent	⭐ ⭐ ⭐ ⭐ Very High

Active-Passive vs Active-Active

Active-Passive

Region A

Read + Write

Region B

Backup

✅ Best for Disaster Recovery

✅ Simpler replication

❌ Writes only in primary region

Active-Active

Region A

Read + Write

Region B

Read + Write

✅ Reads everywhere

✅ Writes everywhere

✅ Lowest global latency

❌ More difficult to implement

Exam Memory Trick

Single AZ

=

No High Availability

Plain Text

Multi-AZ

=

High Availability

Active-Passive

=

Disaster Recovery

=

One Write Region

Active-Active

=

Global Reads + Writes

=

Best Performance

Common Exam Scenarios

Scenario 1

Need protection against AZ failure within one AWS Region.

✅ Answer: **Single Region Multi-AZ**

Scenario 2

Need a disaster recovery site in another Region.

✅ Answer: **Multi-Region Active-Passive**

Scenario 3

Need the lowest read and write latency for users worldwide.

✅ Answer: **Multi-Region Active-Active**

Scenario 4

Need a global database allowing writes in multiple AWS Regions.

✅ Answer: **DynamoDB Global Tables (Active-Active)**

Leveraging the AWS Global Infrastructure Summary

1. Amazon Route 53 (Global DNS)

Purpose

- Managed **DNS (Domain Name System)** service.
- Routes users to the best application endpoint.

Benefits

✅ Routes users to the closest region

✅ Reduces latency

✅ Supports Disaster Recovery (DR)

✅ Health checks and failover support

Routing Policies

- **Simple** → Single resource
- **Weighted** → Traffic distribution
- **Latency** → Lowest latency region
- **Failover** → Disaster Recovery

Exam Keyword

Route 53 = Global DNS

2. Amazon CloudFront (CDN)

Purpose

- AWS **Content Delivery Network (CDN)**.

Benefits

✅ Caches content at Edge Locations

✅ Reduces latency worldwide

✅ Improves website performance

✅ Supports DDoS protection (Shield/WAF)

Typical Origins

- S3 Bucket
- Load Balancers
- EC2 Instances
- Custom HTTP Servers

Exam Keyword

CloudFront = CDN + Caching

3. Amazon S3 Transfer Acceleration

Purpose

- Faster uploads and downloads to S3 buckets.

How it Works

User

↓

Nearest Edge Location

↓

AWS Global Network

↓

S3 Bucket

Benefits

✅ Faster global file transfers

✅ Uses Edge Locations

✅ Uses AWS Private Network

Exam Keyword

Accelerate global uploads/downloads to S3

4. AWS Global Accelerator

Purpose

- Improve application availability and performance globally.

How it Works

User

↓

Nearest Edge Location

↓

AWS Global Network

↓

Application Endpoint

Benefits

✅ Static Anycast IPs

✅ Faster routing

✅ Better availability

✅ Regional failover support

✅ Supports TCP and UDP

Difference from CloudFront

CloudFront

- Caches content

Global Accelerator

- Does NOT cache content
- Accelerates application traffic

Exam Keyword

Global Accelerator = Faster application traffic

5. AWS Outposts

Purpose

- Bring AWS infrastructure into your own data center.

Benefits

✅ Hybrid Cloud

✅ Same AWS APIs and tools

✅ Local data processing

✅ Data residency

✅ Low latency to on-prem systems

Supported Services

- EC2
- EBS
- S3
- ECS
- EKS
- RDS
- EMR

Exam Keyword

Outposts = AWS in your Data Center

6. AWS Wavelength

Purpose

- Bring AWS services to the edge of **5G networks**.

Benefits

✅ Ultra-low latency

✅ Runs inside telecom provider networks

✅ Great for mobile applications

Use Cases

- AR/VR
- Connected Vehicles
- Smart Cities
- Real-Time Gaming
- Live Video Streaming

Exam Keyword

5G = AWS Wavelength

7. AWS Local Zones

Purpose

- Extend AWS Regions closer to specific cities.

Benefits

✅ Lower latency

✅ Compute closer to users

✅ Supports latency-sensitive applications

Example Local Zones

- Boston
- Chicago
- Dallas
- Houston
- Miami

Supported Services

- EC2
- EBS
- ECS

- RDS
- ElastiCache

Exam Keyword

Local Zones = AWS resources closer to a city

Quick Comparison

Service	Purpose	Exam Keyword
Route 53	DNS Routing	Global DNS
CloudFront	Content Delivery	CDN
S3 Transfer Acceleration	Faster S3 Transfers	S3 Upload/Download Speed
Global Accelerator	Faster Application Traffic	Static Anycast IPs
Outposts	AWS on-premises	Hybrid Cloud
Wavelength	AWS on 5G Networks	5G
Local Zones	AWS closer to cities	Low Latency

Memory Tricks

Route 53

=

DNS

CloudFront

=

CDN

=

Cache Content

S3 Transfer Acceleration

=

Faster S3 Uploads/Downloads

Global Accelerator

=

Faster Application Traffic

=

No Caching

Outposts

=

AWS in Your Data Center

Wavelength

=

AWS + 5G

Local Zones

=

AWS Near a City

Most Important Exam Mappings

Requirement	**Service**
Route users globally	Route 53
Cache content worldwide	CloudFront
Faster uploads to S3	S3 Transfer Acceleration
Faster application traffic worldwide	Global Accelerator
Run AWS services in your data center	Outposts
Ultra-low latency over 5G	Wavelength
Low latency in a specific city	Local Zones

Cloud Practitioner

Cloud Integrations

Cloud Integrations Overview

1. Synchronous Communication

In a synchronous pattern, one application directly calls another application and waits for a response.

Example

- **Buying Service** → directly calls → **Shipping Service**
- When an order is placed, the buying service immediately contacts the shipping service.

Characteristics:

- Simple to understand and implement.
- Real-time communication.
- Tight coupling between services.
- If the shipping service is unavailable or overloaded, the buying service may also be affected.

Buying Service ---> Shipping Service

2. Asynchronous (Event-Driven) Communication

In this pattern, applications communicate through an intermediary such as a **queue**.

Example

- Buying Service places an order message into a queue.
- Shipping Service reads orders from the queue when it is ready.

Buying Service ---> Queue ---> Shipping Service

Characteristics:

- Services are **decoupled**.
- The sender does not wait for the receiver.
- Better fault tolerance.
- Easier to scale independently.

Why Asynchronous Communication Is Helpful

Consider a video encoding application:

- Normally, it processes **10 videos**.
- Suddenly, it receives **1,000 videos**.

With synchronous communication:

- The encoding service may become overwhelmed.

- Requests may fail.
- Performance can degrade significantly.

With a queue:

- Requests are stored safely.
- Workers process messages at their own pace.
- Additional workers can be added to handle spikes.

This improves:

- Scalability
- Reliability
- Fault tolerance

AWS Services Mentioned

Amazon SQS (Simple Queue Service)

A **message queue** service used for asynchronous communication.

Application A --> SQS Queue --> Application B

Use when:

- Messages need to be processed one by one.
- Applications should be decoupled.

Amazon SNS (Simple Notification Service)

A **publish/subscribe (Pub/Sub)** messaging service.

--> Email

Publisher --|--> Lambda

--> SQS

--> HTTP Endpoint

Use when:

- One message must be delivered to multiple subscribers.

Amazon Kinesis

A service for **real-time data streaming**.

Examples:

- Website click streams
- IoT sensor data
- Log processing

- Real-time analytics

Key Takeaway

Synchronous	**Asynchronous**
Direct service-to-service communication	Communication through queue/events
Tightly coupled	Loosely coupled (decoupled)
Sender waits for response	Sender does not wait
Can fail during traffic spikes	Handles spikes more gracefully
Harder to scale independently	Services scale independently

SQS Overview

Amazon SQS is a fully managed, serverless messaging service that helps **decouple applications**.

How SQS Works

1. **Producers** send messages to an SQS queue.
2. Messages are stored in the queue.
3. **Consumers** poll (retrieve) messages from the queue.
4. After processing a message, consumers **delete** it from the queue.

Producers -> SQS Queue -> Consumers

Multiple producers and multiple consumers can work with the same queue.

Why Use SQS?

- **Decouples application components**
  - Producers and consumers operate independently.
  - Each can scale at its own pace.
- **Fully managed (serverless)**
  - No servers to provision or manage.
- **Highly scalable**
  - Can handle from a few messages to tens of thousands per second.
- **Reliable message storage**
  - Default message retention: **4 days**
  - Maximum retention: **14 days**
- **Unlimited queue size**
  - No practical limit on the number of messages in a queue.
- **Low latency**
  - Typically less than 10 ms.

Real-World Example

Video Processing Architecture

Users

|

Web Servers

|

SQS Queue

|

Video Processing Workers

Instead of web servers processing videos directly:

1. Web servers place video-processing requests into SQS.
2. Worker servers read messages from the queue.
3. Workers process videos.
4. Workers delete messages after completion.

Benefits:

- Web layer and processing layer scale independently.
- Better user experience.
- Cost-efficient scaling.
- Prevents overload during traffic spikes.

FIFO Queues

FIFO = First In, First Out

Messages are delivered in the same order they are sent.

Example:

Producer sends: 1 → 2 → 3 → 4

Consumer reads: 1 → 2 → 3 → 4

Use FIFO queues when **message ordering is important**.

In contrast, a **Standard Queue** does not guarantee strict ordering.

Exam Tips

✅ **If you see "decoupling applications" → think SQS**

✅ **FIFO Queue = ordered message processing**

✅ **Consumers must delete messages after processing**

✅ **SQS is fully managed and serverless**

✅ **Supports independent scaling of application components**

Kinesis Overview

Amazon Kinesis is AWS's service for **real-time data streaming and analytics**.

Exam Tip: If you see **real-time streaming data**, **big data analytics**, or **live data ingestion**, think **Amazon Kinesis**.

What is Amazon Kinesis Data Streams?

Amazon Kinesis Data Streams is used to:

- **Collect** real-time data
- **Process** streaming data
- **Analyze** data as it arrives
- Scale to handle large volumes of streaming events

It is designed for **continuous data streams**, not batch processing.

Common Data Sources

Examples of real-time (fast) data:

- Website clickstreams
- IoT devices and sensors
- Application logs
- Server metrics
- Mobile app events
- Financial transactions
- Social media feeds

Data Sources

|

Amazon Kinesis Data Streams

|

Real-Time Processing & Analytics

Amazon Data Firehose

Amazon Data Firehose (formerly Kinesis Data Firehose) is used to **deliver streaming data to storage and analytics services**.

Common Destinations

- Amazon S3
- Amazon Redshift

- Amazon OpenSearch Service
- Other supported destinations

Data Sources

|

Kinesis Data Streams

|

Amazon Data Firehose

|

+--> Amazon S3

+--> Amazon Redshift

+--> Amazon OpenSearch

Real-World Example

Imagine an e-commerce website:

1. Users continuously click on products.
2. Click events are sent to Kinesis Data Streams.
3. The data is analyzed in real time.
4. Firehose delivers the data to S3 or Redshift for reporting and analytics.

This allows businesses to monitor user behavior instantly.

SQS vs Kinesis

Feature	Amazon SQS	Amazon Kinesis
Purpose	Message queuing	Real-time data streaming
Main Use	Application decoupling	Streaming analytics
Data Processing	Asynchronous message processing	Continuous data streams
Exam Keyword	Decouple applications	Real-time streaming

Remember

- **SQS → Decoupling**
- **Kinesis → Real-time streaming**

Exam Tips

- ✅ **Kinesis = Real-time streaming data**
- ✅ Collects, processes, and analyzes data in real time
- ✅ Common sources: logs, metrics, clickstreams, IoT devices
- ✅ Firehose delivers streaming data to destinations like:
  - S3
  - Redshift
  - OpenSearch
- ✅ Used for big data and streaming analytics

SNS Overview

Amazon SNS is a fully managed **Publish/Subscribe (Pub/Sub)** messaging and notification service.

Exam Tip: If you see **notifications**, **publish/subscribe**, **one-to-many messaging**, or **fan-out**, think **Amazon SNS**.

Why SNS?

Suppose a **Buying Service** generates an order event and needs to notify:

- Email service
- Fraud detection service
- Shipping service
- SQS queue

Without SNS (Direct Integration)

Buying Service

|

+--> Email Service

+--> Fraud Service

+--> Shipping Service

+--> SQS Queue

Problems:

- Multiple integrations to maintain
- Tightly coupled architecture
- Harder to scale and extend

With SNS (Pub/Sub)

+--> Email

|

Buying Service --> SNS Topic

|

+--> Fraud Service

|

+--> Shipping Service

|

+--> SQS Queue

The publisher sends the message **once** to the SNS topic.

SNS automatically delivers the message to all subscribers.

Key Concepts

Publisher

The application that sends messages.

Example:

Buying Service

SNS Topic

A communication channel that receives messages from publishers and distributes them to subscribers.

Subscribers

Endpoints that receive notifications from the topic.

Examples:

- SQS Queues
- AWS Lambda
- Amazon Data Firehose
- Email
- SMS
- Mobile Push Notifications
- HTTP/HTTPS Endpoints

SNS vs SQS

This is a very common exam comparison.

Amazon SQS

Producer --> Queue --> Consumers

- Messages are stored in a queue.
- Consumers poll the queue.
- Consumers share the workload.
- Each message is usually processed by one consumer.

Amazon SNS

Publisher --> Topic --> Multiple Subscribers

- Messages are pushed to subscribers.

- All subscribers receive the message.
- One message can reach many receivers.

Example

A new order is placed.

The Buying Service publishes:

"Order #123 Created"

SNS sends the same notification to:

- Email notification service
- Fraud detection system
- Shipping system
- SQS queue

Every subscriber receives a copy.

Supported SNS Destinations

SNS can send messages to:

AWS Services

- Amazon SQS
- AWS Lambda
- Amazon Data Firehose

Communication Channels

- Email
- SMS
- Mobile Push Notifications
- HTTP Endpoints
- HTTPS Endpoints

Scalability

- Over **12 million subscriptions per topic**
- Up to **100,000 topics per account** (soft limit)

SNS is designed for massive-scale notification systems.

Real-World Use Cases

Order Processing

Order Created

|

SNS Topic

|

+--> Billing

+--> Shipping

+--> Fraud Detection

+--> Customer Notification

Application Alerts

CloudWatch Alarm

|

SNS

|

+--> Email

+--> SMS

+--> Lambda

Exam Tips

- ✅ **SNS = Publish/Subscribe (Pub/Sub)**
- ✅ **One message → Many subscribers**
- ✅ **Used for notifications**
- ✅ **Subscribers receive all messages**
- ✅ **Can deliver to SQS, Lambda, Firehose, Email, SMS, HTTP/HTTPS**
- ✅ **Publisher sends message once to an SNS Topic**

Quick Comparison: SQS vs SNS

Feature	SQS	SNS
Purpose	Decouple applications	Notifications & fan-out
Pattern	Queue	Publish/Subscribe
Message Delivery	One consumer processes a message	All subscribers receive a copy
Communication	Pull (polling)	Push
Exam Keyword	Decoupling	Notifications

Amazon MQ Overview

Amazon MQ is a **managed message broker service** that supports:

- **RabbitMQ**
- **ActiveMQ**

It is mainly used when migrating existing applications that already rely on traditional messaging protocols.

Why Amazon MQ?

AWS-native messaging services such as:

- **Amazon SQS**
- **Amazon SNS**

use AWS-specific APIs and protocols.

However, many traditional on-premises applications use open-standard protocols like:

- MQTT
- AMQP
- STOMP
- OpenWire
- WebSocket (WSS)

When moving these applications to AWS, rewriting them to use SQS or SNS may require significant effort.

Amazon MQ lets you continue using these existing protocols without modifying your application.

How Amazon MQ Works

Application

|

MQTT / AMQP / STOMP

|

Amazon MQ

|

Consumers / Applications

Amazon MQ acts as a managed broker in AWS while maintaining compatibility with existing messaging clients.

Supported Technologies

RabbitMQ

Popular open-source message broker used for:

- AMQP messaging
- Event-driven architectures
- Enterprise integrations

ActiveMQ

Popular message broker supporting:

- JMS
- MQTT
- AMQP
- STOMP
- OpenWire

AWS manages the infrastructure, patching, and maintenance.

Amazon MQ vs SQS and SNS

Feature	Amazon MQ	SQS	SNS
Type	Managed Message Broker	Queue Service	Pub/Sub Service
Protocols	MQTT, AMQP, STOMP, etc.	AWS-native	AWS-native
Scaling	Limited by broker capacity	Virtually unlimited	Virtually unlimited
Server Management	Broker instances managed by AWS	Serverless	Serverless
Migration Support	Excellent	Requires application changes	Requires application changes

---

Queue and Topic Features

Amazon MQ supports both:

Queue Model

Similar to SQS:

Producer --> Queue --> Consumer

Messages are processed by consumers.

Topic Model

Similar to SNS:

Publisher --> Topic --> Multiple Subscribers

Messages are distributed to multiple subscribers.

High Availability

Unlike SQS and SNS, Amazon MQ runs on brokers (servers).

Therefore:

- Capacity must be considered.
- Failures can occur.
- Multi-AZ deployment can be configured for high availability and failover.

AZ-1 Broker

|

Failover

|

AZ-2 Broker

When to Use Amazon MQ?

✅ Migrating legacy applications to AWS

✅ Existing applications already use:

- MQTT
- AMQP
- STOMP
- OpenWire

✅ Need compatibility with RabbitMQ or ActiveMQ

When NOT to Use Amazon MQ?

If you're building a new cloud-native application:

✅ Use **SQS** for queues

✅ Use **SNS** for publish/subscribe notifications

Reason:

- Better integration with AWS
- Serverless
- Virtually unlimited scaling
- Lower operational overhead

Exam Tips

Think Amazon MQ when:

- Migration from on-premises messaging systems
- RabbitMQ
- ActiveMQ
- MQTT
- AMQP
- STOMP
- Open protocols

Think SQS when:

- Application decoupling
- Queue-based processing
- Serverless messaging

Think SNS when:

- Notifications
- Publish/Subscribe
- One-to-many messaging

Quick Service Selection

Need application decoupling?

-> SQS

Need notifications to many subscribers?

-> SNS

Need RabbitMQ/ActiveMQ or open protocols?

-> Amazon MQ

Cloud Integration Summary

1. Amazon SQS (Simple Queue Service)

Purpose: Application **decoupling** using queues.

How it works

Producers --> SQS Queue --> Consumers

- Multiple producers send messages to a queue.
- Messages are stored for up to **14 days**.
- Consumers poll and process messages.
- After processing, messages are deleted.
- Multiple consumers share the workload.

Key Characteristics

✅ Queue-based messaging
✅ Serverless and fully managed
✅ Virtually unlimited scalability
✅ Durable message storage
✅ FIFO queues available for ordered processing

Exam Keywords

- Decoupling
- Queue
- Asynchronous processing
- Producers and consumers
- FIFO

Think: SQS = Queue + Decoupling

2. Amazon SNS (Simple Notification Service)

Purpose: Publish/Subscribe (Pub/Sub) notifications.

How it works

Publisher --> SNS Topic --> Multiple Subscribers

Subscribers can be:

- Email
- SMS
- Lambda
- SQS
- HTTP/HTTPS endpoints

- Mobile notifications

Key Characteristics

✅ One-to-many messaging
✅ Push-based delivery
✅ All subscribers receive the message
✅ Fully managed and serverless

Message Behavior

Message Published
|
+--> Subscriber A
+--> Subscriber B
+--> Subscriber C

Every subscriber gets a copy.

Exam Keywords

- Notifications
- Topics
- Subscribers
- Publish/Subscribe
- Fan-out

Think: SNS = Notifications to Many Receivers

3. Amazon Kinesis

Purpose: Real-time data streaming and analytics.

How it works

Streaming Data
|
Amazon Kinesis
|
Real-Time Analytics

Data Sources

- Website clicks
- Application logs
- Metrics
- IoT devices

- Mobile applications

Typical Flow

Data Sources

|

Kinesis Data Streams

|

Data Firehose

|

+--> S3

+--> Redshift

+--> OpenSearch

Key Characteristics

✅ Real-time data ingestion

✅ Streaming analytics

✅ Big data processing

✅ Data persistence

Exam Keywords

- Real-time streaming
- Big data
- Analytics
- Clickstreams
- IoT

Think: Kinesis = Real-Time Streaming Data

4. Amazon MQ

Purpose: Managed message broker for traditional messaging systems.

Supported Technologies

- RabbitMQ
- ActiveMQ

Supported Protocols

- MQTT
- AMQP
- STOMP
- OpenWire

- WSS

Use Case

On-Premises Application

|

MQTT / AMQP

|

Amazon MQ

|

AWS Cloud

Used when migrating existing applications to AWS without changing messaging protocols.

Key Characteristics

✅ Managed RabbitMQ and ActiveMQ

✅ Supports open protocols

✅ Queue and topic capabilities

✅ Ideal for legacy migrations

Exam Keywords

- RabbitMQ
- ActiveMQ
- MQTT
- AMQP
- Migration

Think: Amazon MQ = Legacy Messaging Migration

Quick Comparison

Service	Main Purpose	Pattern	Keyword
SQS	Application decoupling	Queue	Decouple
SNS	Notifications	Publish/Subscribe	Notify
Kinesis	Real-time streaming	Data Streams	Streaming
Amazon MQ	Legacy messaging	Broker	Migration

Cloud Monitoring

CloudWatch Metrics & CloudWatch Alarms Overview

What is Amazon CloudWatch?

Amazon CloudWatch is AWS's monitoring and observability service.

It helps you:

- Monitor AWS resources
- Collect performance metrics
- Visualize data
- Create alarms and notifications
- Automate responses to events

CloudWatch Metrics

A **metric** is a numerical value that changes over time and is used to monitor a resource.

Examples:

- CPU Utilization
- Network In
- Network Out
- Disk Reads
- Disk Writes
- Billing Amount

Metrics contain timestamps and can be displayed as graphs.

CloudWatch Dashboards

A **CloudWatch Dashboard** allows you to view multiple metrics in one place.

Example dashboard:



Useful for monitoring the health of your AWS environment.

Common CloudWatch Metrics

EC2 Metrics

CloudWatch automatically collects several EC2 metrics.

CPU Utilization

Measures how busy the CPU is.

High CPU Utilization

= Heavy workload

= May need scaling

Status Check

Indicates whether the instance is healthy.

Network Metrics

- Network In
- Network Out

Shows network traffic entering and leaving the instance.

Important Exam Fact

❌ **RAM/Memory usage is NOT provided by default.**

To monitor memory, custom metrics are required.

Monitoring Frequency

Basic Monitoring

- Every **5 minutes**
- Default
- No extra charge

Detailed Monitoring

- Every **1 minute**
- Additional cost

Basic Monitoring -> 5 min

Detailed Monitoring -> 1 min

EBS Metrics

For EBS volumes, CloudWatch provides:

- Disk Reads
- Disk Writes
- Read Operations
- Write Operations

Used to monitor storage performance.

S3 Metrics

Monitor:

- Bucket Size
- Number of Objects
- Request Counts

Useful for capacity planning and usage analysis.

Billing Metric

Tracks estimated AWS charges.

Important Exam Fact

✅ Billing metrics are available only in:

us-east-1

(N. Virginia)

Even though the metric is located in us-east-1, it represents charges for the entire AWS account.

Month Start -> $0

Mid Month -> $50

Month End -> $120

Next Month -> Resets to $0

Service Limit Metrics

CloudWatch can also monitor service usage limits.

Examples:

- API calls
- Service quotas
- Resource consumption

Custom Metrics

If AWS doesn't provide a metric you need:

Application

|

Custom Metric

|

CloudWatch

You can push your own application metrics into CloudWatch.

Examples:

- Number of active users
- Orders processed
- Custom business KPIs

CloudWatch Alarms

CloudWatch Alarms monitor metrics and perform actions when thresholds are reached.

How Alarms Work

Metric

|

Threshold Reached?

|

Yes

|

Alarm Action

Example:

CPU Utilization > 90%

Trigger an alarm.

Alarm Actions

1. EC2 Actions

CloudWatch can:

- Stop instance
- Reboot instance
- Terminate instance
- Recover instance

2. Auto Scaling Actions

CloudWatch can trigger Auto Scaling.

Example:

CPU > 80%

|

Add More EC2 Instances

Or:

CPU < 20%

|

Remove EC2 Instances

3. SNS Notifications

CloudWatch can notify users through SNS.

Example:

CPU > 90%

|

CloudWatch Alarm

|

SNS Topic

|

Email Notification

Typical exam scenario:

Send an email whenever CPU exceeds 90%.

Answer:

✅ CloudWatch Alarm + SNS

Billing Alarms

A very common use case.

Example:

Monthly Cost > $20

Trigger:

CloudWatch Billing Alarm

|

SNS Email

Helps prevent unexpected AWS costs.

Alarm States

CloudWatch alarms have three possible states.

1. OK

Metric within threshold

Everything is healthy.

2. INSUFFICIENT_DATA

Not enough data points available

CloudWatch cannot determine whether the condition is good or bad.

3. ALARM

Threshold violated

Action is triggered.

Example:

CPU > 90%

Alarm enters the **ALARM** state.

Exam Tips

CloudWatch

- ✅ Monitoring service for AWS resources
- ✅ Metrics are time-based measurements
- ✅ Dashboards display multiple metrics

EC2

- ✅ CPU Utilization available
- ✅ Network metrics available
- ✅ Status Checks available
- ❌ RAM metrics not available by default

Monitoring Frequency

- ✅ Basic Monitoring = 5 minutes
- ✅ Detailed Monitoring = 1 minute

Billing

- ✅ Billing metric available only in **us-east-1**
- ✅ Can create billing alarms

Alarms

- ✅ Monitor metrics against thresholds
- ✅ Can trigger:
  - Auto Scaling

- EC2 Actions
- SNS Notifications

Alarm States

✅ OK

✅ ALARM

✅ INSUFFICIENT_DATA

Quick Cheat Sheet

Feature	CloudWatch
Monitoring Service	✅
Metrics	✅
Dashboards	✅
Alarms	✅
Auto Scaling Trigger	✅
SNS Notifications	✅
Billing Monitoring	✅
Custom Metrics	✅
Default EC2 Monitoring	5 min
Detailed Monitoring	1 min
RAM Metric by Default	❌

CloudWatch Log Overview

What is CloudWatch Logs?

Amazon CloudWatch Logs is a service that collects, stores, monitors, and analyzes log files from AWS resources and applications.

A **log file** is a record of events generated by an application, server, or service.

Examples of log entries:

User login successful

Order processed

Database connection failed

Application started

Cleanup task completed

Logs are essential for:

- Troubleshooting issues
- Monitoring application health
- Auditing activity
- Security investigations

Sources of CloudWatch Logs

CloudWatch Logs can collect logs from many AWS services, including:

AWS Services

- Amazon EC2
- AWS Lambda
- Amazon ECS
- AWS Elastic Beanstalk
- AWS CloudTrail
- Amazon Route 53

Hybrid Environments

- On-premises servers
- Virtual machines
- Physical servers

Benefits of CloudWatch Logs

Centralized Log Storage

Instead of checking logs on multiple servers:

Server A Logs

Server B Logs

Server C Logs

|

CloudWatch Logs

All logs are stored in one central location.

Real-Time Monitoring

CloudWatch Logs allows you to monitor events as they occur.

Examples:

- Application errors
- Security events
- Unexpected system behavior

Adjustable Retention

You can control how long logs are stored.

Examples:

- 1 day
- 1 week
- 30 days
- 1 year
- Forever

Retention Period

|

+--> 7 days

+--> 30 days

+--> 365 days

+--> Never expire

This helps manage storage costs and compliance requirements.

CloudWatch Logs with EC2

Important Exam Fact

By default:

❌ EC2 instances do NOT automatically send logs to CloudWatch Logs.

You must install and configure a CloudWatch agent.

Architecture

EC2 Instance

|

CloudWatch Agent

|

CloudWatch Logs

1. Application generates logs.
2. CloudWatch Agent reads the log files.
3. Agent sends logs to CloudWatch Logs.
4. Logs become searchable and monitorable in AWS.

IAM Permissions Required

The EC2 instance needs an **IAM Role** with permissions to write logs to CloudWatch.

EC2 Instance

|

IAM Role

|

Permission to send logs

|

CloudWatch Logs

Without the proper IAM permissions, log delivery will fail.

Hybrid Logging

The CloudWatch Agent can also run on:

On-Premises Server

|

CloudWatch Agent

|

CloudWatch Logs

This is useful for hybrid cloud environments where some workloads remain on-premises.

Common Use Cases

Application Troubleshooting

User reports error

|

Check CloudWatch Logs

|

Find exception/error message

Security Monitoring

Monitor:

- Failed login attempts
- Unauthorized access
- Suspicious activity

Operational Monitoring

Track:

- Application health
- Service availability
- System events

CloudWatch Logs vs CloudWatch Metrics

Feature	CloudWatch Metrics	CloudWatch Logs
Stores numerical measurements	✅	❌
Stores text log entries	❌	✅
Example	CPU Utilization	Application Error Logs
Used for alarms	✅	✅ (through log monitoring)
Time-series monitoring	✅	❌

Examples

Metric

CPU Utilization = 75%

Network In = 500 MB

Log

2026-07-06 10:00:23 User Login Successful

2026-07-06 10:01:10 Database Connection Error

Exam Tips

- ✅ **CloudWatch Logs stores and monitors log files**
- ✅ Supports logs from:
  - EC2
  - Lambda
  - ECS
  - Elastic Beanstalk
  - CloudTrail
  - Route 53
- ✅ **EC2 does NOT send logs to CloudWatch automatically**
- ✅ Must install a **CloudWatch Agent**
- ✅ EC2 requires an **IAM Role** with proper permissions
- ✅ Works with both:
  - AWS servers
  - On-premises servers
- ✅ Log retention can be customized

EventBridge Overview

What is Amazon EventBridge?

Amazon EventBridge is a **serverless event bus service** that allows AWS services, applications, and external partners to communicate through events.

Exam Tip: If you see **event-driven architecture**, **scheduled jobs**, or **reacting to AWS service events**, think **Amazon EventBridge**.

Important Naming Note

CloudWatch Events = Old Name

EventBridge = New Name

For exam purposes, treat them as the same service.

How EventBridge Works

An **event source** generates an event.

EventBridge receives the event and evaluates rules.

If a rule matches, EventBridge sends the event to a target.

Event Source

|

EventBridge

|

Target

Use Case 1: Scheduled Jobs (Cron Jobs)

EventBridge can create events on a schedule.

For example:

Every Hour

|

EventBridge Rule

|

Lambda Function

This allows you to run tasks without managing servers.

Examples:

- Daily backups
- Cleanup scripts
- Report generation
- Scheduled notifications

Exam Keyword

✅ Scheduled task

✅ Cron job

✅ Time-based automation

➡️ Think **EventBridge**

Use Case 2: React to AWS Events

EventBridge can react to actions occurring within AWS.

Example:

Root User Login

|

EventBridge Rule

|

SNS Topic

|

Email Alert

Whenever someone signs in using the AWS root account, EventBridge can automatically notify the security team.

Event Sources

EventBridge can receive events from many AWS services.

Examples:

- EC2
- S3
- IAM
- CodeBuild
- CloudTrail
- Trusted Advisor
- Lambda
- ECS

AWS Service

|

EventBridge

Event Targets

After matching a rule, EventBridge can send events to many destinations.

Common targets include:

Compute

- AWS Lambda
- ECS Tasks

Messaging & Integration

- Amazon SNS
- Amazon SQS

Workflow Automation

- Step Functions

Other AWS Services

- Various AWS integrations

EventBridge

|

+--> Lambda

+--> SNS

+--> SQS

+--> Step Functions

EventBridge Event Buses

An **Event Bus** is where events are received and routed.

There are three common types.

1. Default Event Bus

Used for AWS service events and scheduled events.

Examples:

- EC2 state changes
- S3 object uploads
- IAM actions
- Scheduled rules

AWS Services

|

Default Event Bus

2. Partner Event Bus

Allows AWS partner applications to send events into your account.

Examples:

- Zendesk
- Datadog
- Other AWS partners

Partner Service

|

Partner Event Bus

|

AWS Targets

This enables integration with systems outside AWS.

3. Custom Event Bus

Used for your own applications.

Custom Application

|

Custom Event Bus

|

AWS Targets

Allows custom event-driven architectures.

Advanced Features

Schema Registry

Helps you understand event structures.

Includes:

- Field names
- Data types
- Event formats

Useful for application development.

Event Archive

Store events for later use.

Event

|

Archive

Retention can be:

- Temporary
- Long-term

- Indefinite

Event Replay

Previously archived events can be replayed.

Archived Events

|

Replay

|

Targets

Useful for:

- Testing
- Troubleshooting
- Recovery

EventBridge vs SNS

Students often confuse these.

SNS

Publisher

|

SNS Topic

|

Multiple Subscribers

Purpose:

- Notifications
- Fan-out messaging

EventBridge

Event Source

|

EventBridge Rule

|

Targets

Purpose:

- Event routing
- Event-driven automation
- Filtering and scheduling

EventBridge vs CloudWatch Alarms

CloudWatch Alarm

Metric Threshold Exceeded

|

Alarm

|

SNS

Used for monitoring metrics.

EventBridge

AWS Event Occurs

|

EventBridge

|

Action

Used for reacting to events.

Exam Tips

✅ **EventBridge = Event-driven automation**

✅ **Formerly CloudWatch Events**

✅ Can run **scheduled jobs (cron jobs)**

✅ Can react to AWS service events

✅ Can trigger:

- Lambda
- SNS
- SQS
- Step Functions

✅ Supports:

- Default Event Bus
- Partner Event Bus
- Custom Event Bus

✅ Can archive and replay events

Quick Comparison

Service	Main Purpose	Exam Keyword
SQS	Queueing & Decoupling	Queue
SNS	Notifications	Publish/Subscribe
Kinesis	Real-Time Streaming	Streaming
Amazon MQ	Legacy Messaging	MQTT/AMQP
CloudWatch	Monitoring & Alarms	Metrics
EventBridge	Event Routing & Scheduling	Events

CloudTrail Overview

What is AWS CloudTrail?

AWS CloudTrail is a service used for:

- Governance
- Compliance
- Auditing
- Security monitoring
- Tracking API activity

Exam Tip: If you need to know **who did what, when, and from where in AWS**, think **CloudTrail**.

What Does CloudTrail Record?

CloudTrail records nearly all API activity in your AWS account, including actions performed through:

AWS Management Console

User logs into AWS Console

User creates/deletes EC2 instance

User modifies IAM policy

All actions are logged.

AWS CLI

aws s3 rm myfile.txt

The API call is recorded in CloudTrail.

AWS SDKs

Applications using:

- Python (Boto3)
- Java
- .NET
- Node.js

generate API calls that CloudTrail records.

AWS Services

Actions performed automatically by AWS services are also logged.

CloudTrail is Enabled by Default

A key exam fact:

✅ **CloudTrail is enabled by default for AWS accounts.**

This means AWS automatically records account activity for auditing purposes.

Typical Use Case

Imagine someone deletes an S3 bucket.

Questions:

- Who deleted it?
- When was it deleted?
- From which account?
- From which IP address?

Answer

✅ **CloudTrail**

CloudTrail provides a history of the API calls that performed the action.

How CloudTrail Works

AWS User / Service

|

API Call

|

CloudTrail

|

+--> CloudTrail Console

+--> CloudWatch Logs

+--> Amazon S3

Viewing Events

You can use the CloudTrail Event History to see:

- User name
- Event time
- Event source
- API action performed
- Resource affected
- IP address

Example:

User: Admin

Action: DeleteBucket

Resource: my-company-backup

Time: 10:05 AM

IP: 192.x.x.x

CloudTrail Trails

A **Trail** is used to deliver CloudTrail events to storage locations.

Trail Destinations

Amazon S3

CloudTrail

|

Amazon S3

Used for:

- Long-term storage
- Compliance
- Historical auditing

CloudWatch Logs

CloudTrail

|

CloudWatch Logs

Used for:

- Monitoring
- Real-time alerting
- Log analysis

Multi-Region Trails

You can configure a trail to:

Single Region

Monitor one AWS Region only

All Regions

Monitor every AWS Region

This is commonly used in production environments.

Example Exam Scenario

Question

A company discovers an EC2 instance was terminated. They need to determine:

- Who terminated it
- When it happened

Answer

✅ **AWS CloudTrail**

Because CloudTrail records the API call that terminated the instance.

CloudTrail vs CloudWatch

Students often confuse these services.

CloudTrail

Tracks:

Who did what?

Examples:

- Create EC2 instance
- Delete S3 bucket
- Modify IAM policy

CloudWatch

Tracks:

How is the resource performing?

Examples:

- CPU Utilization
- Memory (custom)
- Network traffic
- Billing metrics

Comparison

Feature	CloudTrail	CloudWatch
API Activity Tracking	✅	❌
Audit & Compliance	✅	❌
Performance Monitoring	❌	✅
Metrics	❌	✅
User Activity History	✅	❌
Logs Storage Integration	✅	✅

Exam Keywords

Think CloudTrail when you see:

- ✅ Audit
- ✅ Compliance
- ✅ API calls
- ✅ User activity
- ✅ Account activity
- ✅ Security investigations
- ✅ Who deleted a resource?
- ✅ Who changed a configuration?

X-Ray Overview

What is AWS X-Ray?

AWS X-Ray is a service used for **distributed tracing, application debugging, and performance analysis**.

It helps developers understand how requests travel through multiple services in a distributed application.

Exam Tip: If you see **tracing**, **service maps**, **microservices debugging**, or **request flow analysis**, think **AWS X-Ray**.

The Problem X-Ray Solves

In simple applications (monoliths), debugging is relatively easy:

Application

|

Logs

You can inspect one set of logs and find the issue.

However, modern cloud applications are often distributed:

User Request

|

API Gateway

|

Lambda

|

SQS

|

Another Service

|

Database

A single request may pass through many services.

Problems:

- Logs are scattered everywhere.
- Difficult to correlate events.
- Hard to identify bottlenecks.
- Difficult to determine which service failed.

How AWS X-Ray Helps

X-Ray traces requests as they travel through your application.

Request

|

Service A

|

Service B

|

Database

X-Ray records the entire path and provides a visual representation.

Service Map

One of X-Ray's most useful features is the **Service Map**.

Example:

User

|

API Gateway

|

Lambda

|

DynamoDB

X-Ray can visualize:

- All services involved
- Dependencies between services
- Error locations
- Latency information

This creates a common view of your application architecture.

What Can X-Ray Detect?

Performance Bottlenecks

Example:

API Gateway -> 50 ms

Lambda -> 100 ms

Database -> 4 seconds

X-Ray immediately highlights that the database is causing delays.

Application Errors

Identify where failures occur.

User Request

|

Service A ✅

|

Service B ❌

|

Database

X-Ray shows exactly which component failed.

Exceptions and Faults

Track:

- Application exceptions
- Service faults
- Failed requests
- Timeout issues

Request-Level Analysis

You can inspect an individual request and see:

- Which services it visited
- Time spent in each service
- Errors encountered

Common Use Cases

Microservices Troubleshooting

Microservice A

|

Microservice B

|

Microservice C

X-Ray helps understand how services interact and where issues occur.

Performance Optimization

Find slow components:

Request Latency

|

Slow Service Found

|

Performance Improved

Dependency Analysis

Understand relationships between services.

App

|

+--> Lambda

|

+--> SQS

|

+--> DynamoDB

Useful in large architectures.

SLA Monitoring

SLA = Service Level Agreement

Example:

Must respond within 500 ms

X-Ray helps determine whether response times meet requirements.

Detect Throttling

X-Ray can help identify where requests are being slowed down or throttled.

Example:

Request

|

Lambda Throttled

This makes troubleshooting much easier.

Typical Architecture

Application

|

Enable X-Ray

|

AWS X-Ray

|

Tracing + Service Map + Analytics

Once enabled, X-Ray collects tracing data and builds visual insights automatically.

AWS X-Ray vs CloudWatch vs CloudTrail

CloudWatch

Monitors performance metrics:

CPU Utilization

Network Traffic

Memory (custom)

Billing Metrics

Focus: Monitoring

CloudTrail

Tracks AWS API activity:

Who created an EC2 instance?

Who deleted an S3 bucket?

Focus: Auditing

X-Ray

Tracks request flow through applications:

Where did this request fail?

Why is this request slow?

Focus: Tracing and troubleshooting

Quick Comparison

Service	Main Purpose
CloudWatch	Monitoring metrics
CloudWatch Logs	Collecting logs
CloudTrail	Auditing API calls
EventBridge	Reacting to events
X-Ray	Distributed tracing

Exam Tips

✅ **AWS X-Ray = Distributed tracing**

✅ Provides a **service map**

✅ Helps debug **microservices**

✅ Tracks requests across multiple services

✅ Identifies:

- Errors
- Exceptions
- Bottlenecks
- Slow services
- Throttling

✅ Useful for performance analysis and troubleshooting

AWS Health Dashboard

What is AWS Health Dashboard?

AWS Health Dashboard helps you understand the health and availability of AWS services and whether any AWS events are impacting your resources.

There are **two views** to know for the exam:

1. **Service Health Dashboard (Public View)**
2. **AWS Health Dashboard for Your Account (Personalized View)**

1. AWS Service Health Dashboard

This dashboard shows the overall health of AWS services across all AWS Regions.

What it provides

- Status of AWS services
- Regional outages
- Service disruptions
- Historical events
- RSS feed subscriptions

Example:

Region: us-east-1

Service: EC2

Status: Service Disruption

This information is public and applies to all AWS customers.

Use Case

You notice your application is slow.

You can check the Service Health Dashboard to determine if AWS is experiencing a service outage in that region.

Exam Keyword

✅ General AWS service status

✅ Regional outages

✅ Public service health information

2. AWS Health Dashboard (Your Account)

Previously known as:

AWS Personal Health Dashboard (PHD)

This dashboard provides personalized information about AWS events affecting your specific account and resources.

What it provides

- Personalized alerts
- Impacted resources
- Remediation guidance
- Scheduled maintenance notifications
- Service availability issues affecting your account

Example

Suppose AWS identifies an issue affecting one of your EC2 instances.

The dashboard might show:

Affected Resource:

EC2 Instance i-123456

Region:

us-east-2

Issue:

Scheduled maintenance required

Recommended Action:

Migrate instance

Unlike the Service Health Dashboard, this is specific to you.

Service Health Dashboard vs Account Health Dashboard

Service Health Dashboard

Shows:

All AWS customers

Example:

EC2 issue in us-east-1

Question answered:

Is AWS having a problem?

AWS Health Dashboard (Your Account)

Shows:

Only your affected resources

Example:

Your EC2 Instance is impacted

Question answered:

Is MY account affected?

Benefits of AWS Health Dashboard

Alerts

Receive notifications when AWS events impact your resources.

Remediation Guidance

AWS provides suggested actions to resolve issues.

Scheduled Changes

Notifications about:

- Planned maintenance
- Infrastructure updates
- Upcoming changes

Event History

Review previous AWS events and incidents.

AWS Organizations Integration

For organizations using AWS Organizations:

AWS Organization

|

Health Dashboard

|

Aggregated Health Data

You can view health events across multiple AWS accounts from a centralized location.

Typical Exam Scenario

Question

AWS is performing maintenance on an EC2 host that affects one of your instances. Which service provides personalized notification and remediation recommendations?

✅ **AWS Health Dashboard**

Question

You want to determine whether AWS is experiencing an outage in a specific region that affects all customers.

✅ **AWS Service Health Dashboard**

AWS Monitoring Services Comparison

Service	Purpose
CloudWatch	Monitor metrics and resources
CloudWatch Logs	Collect and analyze logs
CloudTrail	Audit API calls and account activity
X-Ray	Trace requests across applications
EventBridge	React to events and scheduling
AWS Health Dashboard	AWS service and account health information

Exam Tips

✅ **Service Health Dashboard**

- Public
- Shows health of AWS services
- Regional status information

✅ **AWS Health Dashboard (Personal Health Dashboard)**

- Personalized
- Shows events affecting your resources
- Provides remediation guidance
- Shows scheduled maintenance

✅ Supports AWS Organizations account aggregation

✅ Used for proactive notifications and service-impact alerts

Cloud Monitoring Summary

1. Amazon CloudWatch

Purpose: Monitor AWS resources and applications.

CloudWatch Metrics

Used to track numerical performance data such as:

- CPU Utilization
- Network In/Out
- EBS Read/Write activity
- S3 metrics
- Billing metrics

Key Point: Metrics are time-based measurements.

CloudWatch Alarms

Used to trigger actions when metrics cross a threshold.

Examples:

- Send SNS notifications
- Scale Auto Scaling Groups
- Reboot/stop/recover EC2 instances

Exam Keyword: Monitoring + Alerts

2. Amazon CloudWatch Logs

Purpose: Centralized log management.

Collects logs from:

- EC2 instances
- Lambda functions
- ECS
- Elastic Beanstalk
- CloudTrail
- On-premises servers

Important Fact

EC2 instances do **not** automatically send logs to CloudWatch Logs.

You must:

1. Install CloudWatch Agent
2. Attach an IAM Role with appropriate permissions

Exam Keyword: Log collection and troubleshooting

3. Amazon EventBridge (formerly CloudWatch Events)

Purpose: Event-driven automation.

Can react to:

- AWS service events
- Custom application events
- Scheduled events (cron jobs)

Examples

S3 Upload

|

EventBridge

|

Lambda Function

Every Hour

|

EventBridge Schedule

|

Lambda Function

Exam Keyword: Events, Scheduling, Automation

4. AWS CloudTrail

Purpose: Auditing and tracking API activity.

Records actions performed through:

- AWS Console
- AWS CLI
- AWS SDKs
- AWS Services

Typical Questions

Who deleted an S3 bucket?

Who terminated an EC2 instance?

Who modified an IAM policy?

Answer:

✅ CloudTrail

CloudTrail Insights

Provides automated analysis to detect unusual API activity.

Exam Keyword: Audit, Compliance, API Calls

5. AWS X-Ray

Purpose: Distributed tracing and application troubleshooting.

Helps visualize how requests move through:

- Microservices
- Lambda functions
- APIs
- Databases

Benefits

- Identify bottlenecks
- Find slow services
- Trace errors
- Analyze request paths

Exam Keyword: Distributed Tracing

6. AWS Health Dashboard

Service Health Dashboard

Shows AWS service health for everyone.

Examples:

- Regional outages
- Service disruptions

Question:

Is AWS experiencing an issue?

Answer:

✅ Service Health Dashboard

AWS Health Dashboard (Personal Health Dashboard)

Shows AWS events affecting your account specifically.

Examples:

- Impacted EC2 instances
- Scheduled maintenance
- Personalized alerts
- Remediation guidance

Question:

Is my account affected?

Answer:

✅ AWS Health Dashboard

Quick Comparison

Service	Main Purpose
CloudWatch Metrics	Performance monitoring
CloudWatch Alarms	Notifications and automation
CloudWatch Logs	Log collection
EventBridge	Event routing and scheduling
CloudTrail	Audit API activity
CloudTrail Insights	Analyze unusual API behavior
X-Ray	Distributed request tracing
Service Health Dashboard	Global AWS service status
AWS Health Dashboard	Account-specific AWS issues

Cloud Practitioner

VPC & Networking

VPC Overview

This lecture introduces **Amazon VPC (Virtual Private Cloud)** and explains its importance for AWS networking.

Key Points

- **VPC (Virtual Private Cloud)** is AWS's networking service that allows you to create an isolated virtual network in the cloud.
- VPC is a **complex topic**, especially for:
  - AWS Certified Solutions Architect – Associate (SAA)
  - AWS Certified SysOps Administrator – Associate (SOA)
- For the **AWS Certified Cloud Practitioner (CCP)** exam, you only need a **high-level understanding** of several VPC-related concepts:

1. **VPC** – Your private virtual network in AWS.
2. **Subnets** – Segments of a VPC used to organize resources.
3. **Internet Gateway (IGW)** – Enables internet access for resources.
4. **NAT Gateway** – Allows private resources to access the internet without being directly reachable.
5. **Security Groups** – Instance-level virtual firewalls.
6. **Network ACLs (NACLs)** – Subnet-level security controls.
7. **VPC Flow Logs** – Capture network traffic information.
8. **VPC Peering** – Connects two VPCs together.
9. **VPC Endpoints** – Enables private access to AWS services.
10. **Site-to-Site VPN** – Secure connection between on-premises networks and AWS.
11. **Direct Connect** – Dedicated private connection to AWS.
12. **Transit Gateway** – Central hub for connecting multiple networks and VPCs.

Exam Relevance

- VPC-related topics account for **very few questions (around 1–2 questions)** on the Cloud Practitioner exam.
- The focus is on understanding **what each service does**, not deep technical implementation details.

Hands-On Activity

- The course will explore the **default VPC automatically created by AWS**.
- You **will not build a VPC from scratch** in this section.

Important Takeaway

For the CCP exam, remember the purpose of each networking component at a high level. You do **not** need advanced configuration knowledge.

Simple memory tip:

- **VPC** = Network
- **Subnet** = Network segment
- **IGW/NAT** = Internet connectivity
- **Security Group/NACL** = Security
- **Flow Logs** = Monitoring
- **Peering/Endpoints/VPN/Direct Connect/Transit Gateway** = Connectivity between networks and services

IP Address in AWS

1. Public IPv4 Addresses

- IPv4 is the traditional internet protocol.
- It supports approximately **4.3 billion addresses**.
- A **public IPv4 address** is reachable from anywhere on the internet.

In AWS

- When you launch an EC2 instance, AWS can assign it a public IPv4 address.
- If you **stop** the instance, the public IPv4 is released.
- When you **start** the instance again, AWS typically assigns a **new public IPv4 address**.

Example:

54.210.123.45

2. Private IPv4 Addresses

- Used only within private networks such as an AWS VPC.
- Cannot be accessed directly from the public internet.
- Common private IP ranges include:

10.0.0.0/8

172.16.0.0/12

192.168.0.0/16

Example:

192.168.1.1

In AWS

- Every EC2 instance receives a private IPv4 address.
- This private IP remains the same throughout the instance's lifetime.
- Even if the instance is stopped and restarted, the private IP generally does not change.

3. Elastic IP (EIP)

An **Elastic IP** is a static public IPv4 address.

Benefits

- Remains attached to your account.
- Can be associated with an EC2 instance.
- Keeps the same public IP even after stopping and restarting the instance.

Use Case

Useful when:

- Applications require a fixed public IP.
- DNS records or firewall rules depend on a specific public IP.

4. Public IPv4 Pricing

AWS charges for public IPv4 addresses.

Cost

$0.005 per hour per public IPv4 address

This includes:

- Standard public IPv4 addresses
- Elastic IP addresses

Because IPv4 addresses are limited, AWS encourages customers to adopt IPv6.

5. IPv6 Addresses

IPv6 is the newer version of the Internet Protocol.

Capacity

- Supports approximately:

3.4 × 10^38 addresses

This is an extremely large address space.

Example IPv6 Address

2001:0db8:85a3:0000:0000:8a2e:0370:7334

AWS Characteristics

- IPv6 addresses are public by design.
- There is no private IPv6 address range in the context discussed in this course.
- AWS currently does not charge for IPv6 addresses.

CCP Exam Key Points

Concept	What to Remember
Public IPv4	Internet-accessible address
Private IPv4	Internal VPC-only address
Public IPv4 after restart	Usually changes
Private IPv4 after restart	Typically stays the same
Elastic IP	Fixed public IPv4 address
IPv4 Cost	Charged hourly
IPv6	Vast address space, public, free in AWS

VPC, Subnet, Internet Gateway & NAT Gateways

1. What is a VPC?

VPC (Virtual Private Cloud) is your own private network inside AWS where you deploy resources such as:

- EC2 instances
- Databases
- Load Balancers
- Other AWS services

Key Point

- A VPC belongs to a **single AWS Region**.
- Different Regions have different VPCs.

AWS Region
└── VPC
 ├── EC2
 ├── Database
 └── Load Balancer

2. What is a Subnet?

A **Subnet** is a smaller network inside a VPC.

Characteristics

- A subnet exists in **one Availability Zone (AZ)**.
- It is a partition of the VPC network.

Example:

VPC
 ├── Public Subnet (AZ-A)
 └── Private Subnet (AZ-A)

3. Public vs Private Subnet

Public Subnet

A subnet that can communicate directly with the internet.

Resources commonly placed here:

- EC2 instances with public access
- Load Balancers
- Bastion Hosts

Internet

|

▼

Public Subnet

|

▼

EC2 Instance

Private Subnet

A subnet that cannot be directly reached from the internet.

Resources commonly placed here:

- Databases (RDS)
- Application servers
- Internal services

Internet ❌

Private Subnet

|

▼

Database

Benefits:

- Higher security
- No direct external access

4. Route Tables

A **Route Table** controls how network traffic moves.

It determines:

- Where internet traffic goes
- How subnets communicate
- Whether a subnet is public or private

Think of it as the **GPS for network traffic**.

5. Example of a Complete VPC

Region

└── VPC (CIDR: 10.0.0.0/16)

AZ-1

├── Public Subnet

└── Private Subnet

AZ-2

├── Public Subnet

└── Private Subnet

In this architecture:

- 1 VPC
- 2 Availability Zones
- 4 Subnets
    - 2 Public
    - 2 Private

This design provides:

- High Availability
- Better fault tolerance

6. CIDR Range

A **CIDR block** defines the IP address range available in a VPC.

Example:

10.0.0.0/16

This means the VPC can allocate IP addresses from that range to its resources.

7. Internet Gateway (IGW)

An **Internet Gateway** allows resources inside a VPC to communicate with the internet.

For a Public Subnet

You need:

1. Internet Gateway attached to the VPC
2. Route Table pointing internet traffic to the IGW

EC2

|

Public Subnet

|

Internet Gateway

|

Internet

Exam Tip

A subnet becomes **public** when it has a route to an Internet Gateway.

8. NAT Gateway

Sometimes resources in a private subnet need outbound internet access for:

- OS updates
- Downloading packages
- Accessing AWS services

But they should remain private.

This is where a **NAT Gateway** is used.

How it Works

Private EC2

|

Private Subnet

|

NAT Gateway

|

Internet Gateway

|

Internet

Benefits

✅ Can access the internet

✅ Cannot be directly accessed from the internet

✅ More secure

9. NAT Gateway Location

A NAT Gateway must be deployed in a:

Public Subnet

Then private subnets route their internet-bound traffic to it.

Cloud Practitioner Exam Cheat Sheet

Component	Purpose
VPC	Private AWS network
Subnet	Partition of a VPC
Public Subnet	Accessible from the internet
Private Subnet	Not directly accessible from the internet
Route Table	Controls traffic routing
Internet Gateway	Connects VPC to the internet
NAT Gateway	Allows private resources to access the internet
CIDR	Defines VPC IP address range
AZ	Physical data center location for subnets

Security Groups & Network Access Control List (NACL)

AWS provides two main layers of network security inside a VPC:

1. **Network ACL (NACL)** – Subnet-level firewall
2. **Security Group (SG)** – Instance-level firewall

1. Network ACL (NACL)

A **Network Access Control List (NACL)** acts as a firewall at the **subnet level**.

Key Characteristics

- Applied to an entire subnet.
- Controls traffic entering and leaving the subnet.
- Supports both:
  - **ALLOW** rules ✅
  - **DENY** rules ❌
- Rules are based on **IP addresses**.

Traffic Flow

Internet

|

▼

Network ACL

|

▼

Subnet

|

▼

EC2 Instance

The NACL filters traffic before it reaches any resource in the subnet.

2. Security Groups

A **Security Group** is a firewall attached directly to resources such as EC2 instances.

Key Characteristics

- Applied at the **instance level**.
- Controls traffic to and from an EC2 instance.
- Supports only:
  - **ALLOW** rules ✅

- Can reference:
  - IP addresses
  - Other Security Groups

Traffic Flow

Internet

|

▼

Subnet

|

▼

Security Group

|

▼

EC2 Instance

3. Security Group vs NACL

Most Important CCP Exam Differences

Feature	Security Group	Network ACL
Level	Instance	Subnet
Rules	Allow only	Allow and Deny
Type	Stateful	Stateless

Stateful vs Stateless

Security Group (Stateful)

If incoming traffic is allowed, return traffic is automatically allowed.

Example:

User → EC2 (Allowed)

EC2 → User (Automatically Allowed)

You don't need a separate rule for the response.

Network ACL (Stateless)

Return traffic must be explicitly allowed.

Example:

User → Subnet (Allowed)

EC2 → User (Need separate outbound rule)

Both directions require rules.

4. Default NACL in AWS

When AWS creates a default VPC, it also creates a default Network ACL.

Default Inbound Rules

Rule 100:

ALLOW ALL Traffic

Source: 0.0.0.0/0

Rule *:

DENY ALL

Default Outbound Rules

Rule 100:

ALLOW A*L Traffic

Rule *:

DENY ALL

Because Rule 100 allows everything, all traffic is effectively permitted.

5. Example Custom NACL Rule

Suppose you want to allow only HTTPS traffic:

Rule 200:*ALLOW HTTPS (Port 443)

Source: 0.0*0.0/0

Rule *:

DENY ALL

Result:

✅ HTTPS traffic allowed

❌ HTTP traffic denied

❌ SSH traffic denied

❌ All other traffic denied

CCP Exam Cheat Sheet

Security Group

- Instance-level firewall
- Allow rules only
- Stateful
- Can reference Security Groups
- Protects EC2 instances

Network ACL

- Subnet-level firewall
- Allow and Deny rules
- Stateless
- Uses IP addresses
- Protects entire subnet

VPC Flow Logs & VPC Peering

1. VPC Flow Logs

What are VPC Flow Logs?

VPC Flow Logs capture information about IP traffic flowing through your AWS network.

They can be enabled at three levels:

- **VPC level**
- **Subnet level**
- **Elastic Network Interface (ENI) level**

VPC

├─ Subnet

│ └─ ENI

│ └─ EC2 Instance

Why Use VPC Flow Logs?

They help with:

✅ Monitoring network traffic

✅ Troubleshooting connectivity issues

✅ Security analysis

✅ Auditing network activity

Example Use Cases

- EC2 instance cannot access the internet
- One subnet cannot communicate with another subnet
- Internet users cannot reach an application
- Investigating allowed or blocked traffic

Flow logs help identify the root cause.

Supported AWS Services

VPC Flow Logs can provide visibility into traffic for:

- EC2
- Elastic Load Balancers (ELB)
- RDS
- Aurora

- ElastiCache
- Other AWS resources using network interfaces

Flow Log Destinations

Flow logs can be sent to:

- **Amazon CloudWatch Logs**
- **Amazon S3**
- **Amazon Data Firehose (Kinesis Data Firehose)**

Information Captured

Typical fields include:

Source IP Address

Destination IP Address

Source Port

Destination Port

Protocol

Packets

Bytes

Action (ACCEPT/REJECT)

Start Time

End Time

CCP Exam Tip

VPC Flow Logs = Network Traffic Monitoring and Troubleshooting

2. VPC Peering

What is VPC Peering?

A VPC Peering Connection creates a private connection between two VPCs using AWS's internal network.

VPC A <------> VPC B

Once connected, resources in both VPCs can communicate as if they were on the same network.

Benefits

✅ Private communication

✅ No internet required

✅ Uses AWS backbone network

✅ Works across accounts and regions

Important Requirement

The VPC CIDR ranges must **not overlap**.

✅ Valid:

VPC A: 10.0.0.0/16

VPC B: 172.31.0.0/16

❌ Invalid:

VPC A: 10.0.0.0/16

VPC B: 10.0.0.0/16

Overlapping IP ranges prevent peering.

3. VPC Peering is NOT Transitive

This is a favorite exam question.

Example

VPC A <----> VPC B

VPC A <----> VPC C

Can B communicate with C?

❌ No

Because VPC peering is **not transitive**.

To Enable Communication

You must create another peering connection.

VPC A <----> VPC B

VPC A <----> VPC C

VPC B <----> VPC C

Now all VPCs can communicate.

Cloud Practitioner

Cloud Practitioner Exam Cheat Sheet

Service	Purpose
VPC Flow Logs	Capture and analyze network traffic
Flow Log Levels	VPC, Subnet, ENI
Flow Log Destinations	CloudWatch Logs, S3, Firehose
VPC Peering	Private connection between VPCs
Peering Requirement	Non-overlapping CIDR ranges
Peering Traffic	Uses AWS private network
Peering Limitation	Not transitive

Easy Memory Trick

VPC Flow Logs

Flow Logs = "Who talked to whom?"

Used for:

- Monitoring
- Auditing
- Troubleshooting

VPC Peering

Peering = Private friendship between two VPCs

Remember:

A ↔ B

A ↔ C

Does B ↔ C work?

NO

Peering is NOT transitive.

Most Important CCP Exam Facts

1. **VPC Flow Logs capture network traffic information.**
2. **Flow Logs help troubleshoot connectivity problems.**
3. **Flow Logs can be stored in CloudWatch Logs, S3, or Firehose.**
4. **VPC Peering connects two VPCs privately.**
5. **Peered VPCs cannot have overlapping CIDR ranges.**
6. **VPC Peering is NOT transitive.**

VPC Endpoints – Interface & Gateway (S3 & DynamoDB)

Why Use VPC Endpoints?

Normally, when an EC2 instance accesses AWS services such as:

- Amazon S3
- DynamoDB
- CloudWatch

the traffic may go through public AWS service endpoints.

A **VPC Endpoint** allows resources inside your VPC to connect to AWS services using the **AWS private network** instead.

Benefits

✅ Improved security (no public internet exposure)

✅ Lower latency

✅ Traffic stays within the AWS network

✅ Simpler network architecture

Example

Without a VPC Endpoint:

EC2 Instance

|

▼

Public Internet

|

▼

AWS Service (S3)

With a VPC Endpoint:

EC2 Instance

|

▼

VPC Endpoint

|

▼

AWS Service (S3)

Traffic remains on the AWS backbone network.

Types of VPC Endpoints

There are two types:

1. Gateway Endpoint

Used only for:

- Amazon S3
- Amazon DynamoDB

Example

EC2

|

▼

Gateway Endpoint

|

▼

Amazon S3

or

Exam Tip

Remember:

Gateway Endpoints are available only for **Amazon S3 and DynamoDB**.

2. Interface Endpoint

Used for almost all other AWS services.

Examples:

- CloudWatch
- EC2 API
- SNS
- SQS
- Secrets Manager
- KMS
- Many others

Example

EC2

|

▼

Interface Endpoint

|

▼

CloudWatch

Interface Endpoints are powered by **AWS PrivateLink**.

Service Support

AWS Service	Gateway Endpoint	Interface Endpoint
Amazon S3	✅	✅
DynamoDB	✅	✅
CloudWatch	❌	✅
EC2 API	❌	✅
SNS	❌	✅
SQS	❌	✅

Exam Cheat Sheet

Gateway Endpoint

- Private access to AWS services
- Supports only:
  - S3
  - DynamoDB
- No public internet required

Interface Endpoint

- Private access to AWS services
- Powered by AWS PrivateLink
- Used for almost every AWS service

Easy Memory Trick

Gateway Endpoint

= S3 + DynamoDB only

Interface Endpoint

= Everything else

AWS Cloud Practitioner Key Facts

1. **VPC Endpoints provide private connectivity to AWS services.**
2. **Traffic stays on the AWS network and avoids the public internet.**
3. **Gateway Endpoints are only for Amazon S3 and DynamoDB.**
4. **Interface Endpoints are used for most AWS services.**
5. **Interface Endpoints are powered by AWS PrivateLink.**

Private Link

AWS **PrivateLink** is a service that enables private connectivity between VPCs and services without using the public internet.

The Problem

Suppose a vendor hosts an application in their own AWS account and VPC:

Vendor VPC

└── Application Service

Many customers want to access this service privately from their own VPCs.

A simple solution could be:

- VPC Peering

However, VPC Peering:

- Doesn't scale well for thousands of customers
- Requires network management
- Can become complex and less secure

The Solution: AWS PrivateLink

AWS PrivateLink allows customers to connect privately to a service running in another VPC.

Benefits:

✅ Private communication

✅ No public internet traffic

✅ No VPC Peering required

✅ No Internet Gateway required

✅ No NAT Gateway required

✅ Highly scalable

How It Works

Service Provider (Vendor Side)

The vendor:

1. Hosts an application.
2. Places a **Network Load Balancer (NLB)** in front of the application.
3. Creates a PrivateLink service.

Vendor VPC

Application

|

▼

Network Load Balancer

Customer Side

The customer:

1. Creates an **Interface Endpoint (ENI)**.
2. Connects it to the vendor's PrivateLink service.

Customer VPC

Application

|

▼

Interface Endpoint

|

▼

PrivateLink

Complete Architecture

Customer VPC Vendor VPC

Consumer App

|

▼

Interface Endpoint

|

▼

PrivateLink

|

▼

Network Load Balancer

|

▼

Vendor Service

Traffic remains entirely within the AWS network.

Why Use PrivateLink?

Security

Traffic never traverses the public internet.

Customer VPC

|

AWS Private Network

|

Vendor VPC

Scalability

Instead of creating:

Customer 1 ↔ Vendor

Customer 2 ↔ Vendor

Customer 3 ↔ Vendor

through VPC Peering, the vendor simply exposes one PrivateLink service that many customers can consume.

Simplicity

No need to manage:

- VPC Peering
- Internet Gateways
- NAT Gateways
- Complex route configurations

PrivateLink vs VPC Peering

Feature	PrivateLink	VPC Peering
Private Connectivity	✅	✅
Uses Public Internet	❌	❌
Scales to Many Customers	✅	❌
Requires Route Updates	❌	✅
Requires Full Network Connectivity	❌	✅
Service-Specific Access	✅	❌

Cloud Practitioner Exam Cheat Sheet

AWS PrivateLink

- Provides private access to services across VPCs.
- Uses **Interface Endpoints** on the consumer side.
- Uses a **Network Load Balancer (NLB)** on the provider side.
- Avoids the public internet.
- More scalable than VPC Peering.
- Commonly used by AWS Marketplace vendors.

Direct Connect & Site-to-site VPN

1. Site-to-Site VPN

A **Site-to-Site VPN** creates an **encrypted connection** between your on-premises network and your AWS VPC.

How it Works

On-Premises Data Center

|

| Encrypted VPN Tunnel

|

Public Internet

|

▼

AWS VPC

Characteristics

✅ Encrypted

✅ Quick to set up (minutes)

✅ Lower cost

✅ Uses the public internet

❌ Bandwidth may vary

❌ Less reliable than a dedicated connection

Components Required

Customer Gateway (CGW)

Located on your on-premises side.

Customer Gateway (CGW)

Virtual Private Gateway (VGW)

Located on the AWS side.

Virtual Private Gateway (VGW)

Architecture

Exam Tip

To establish a Site-to-Site VPN, remember:

- **CGW = Customer Gateway**
- **VGW = Virtual Private Gateway**

2. AWS Direct Connect (DX)

AWS Direct Connect provides a **dedicated physical network connection** between your data center and AWS.

How it Works

On-Premises Data Center

|

Dedicated Private Connection

|

▼

AWS

Characteristics

✅ Private connection

✅ High bandwidth

✅ Consistent performance

✅ More reliable

✅ Does not rely on the public internet

❌ More expensive

❌ Takes longer to provision (often weeks or more)

VPN vs Direct Connect

Feature	Site-to-Site VPN	Direct Connect
Uses Public Internet	Yes	No
Encrypted	Yes	Private connection
Setup Time	Minutes	Weeks+
Cost	Lower	Higher
Performance	Variable	Consistent
Reliability	Good	Very High

When to Use Each?

Use Site-to-Site VPN When:

- You need connectivity quickly.
- Budget is important.

- Occasional bandwidth fluctuations are acceptable.

Use Direct Connect When:

- You need dedicated private connectivity.
- High throughput is required.
- Consistent network performance is critical.
- Long-term enterprise workloads are involved.

Cloud Practitioner Exam Cheat Sheet

Site-to-Site VPN

- Connects on-premises networks to AWS.
- Uses the **public internet**.
- Traffic is **encrypted**.
- Requires:
  - Customer Gateway (CGW)
  - Virtual Private Gateway (VGW)
- Quick and inexpensive.

Direct Connect

- Dedicated private physical connection.
- Does not use the public internet.
- Higher performance and reliability.
- More expensive and slower to establish.

Easy Memory Trick

VPN = Fast Setup, Uses Internet

Direct Connect = Dedicated Cable, Private Network

Most Important Exam Facts

1. **Site-to-Site VPN uses the public internet and encryption.**
2. **Direct Connect uses a dedicated private connection.**
3. **VPN is faster to deploy.**
4. **Direct Connect provides more consistent performance.**
5. **CGW + VGW are required for Site-to-Site VPN.**

Client VPN

AWS **Client VPN** allows an individual user's computer to securely connect to an AWS VPC using a VPN connection.

Unlike a **Site-to-Site VPN** (which connects an entire network to AWS), **Client VPN** connects a single user device.

What is AWS Client VPN?

AWS Client VPN is a managed VPN service that enables users to connect securely to:

- AWS VPCs
- On-premises networks
- Private resources

using the **OpenVPN** protocol.

How It Works

Your Laptop
|
| VPN Connection (OpenVPN)
|
Public Internet
|
▼
AWS Client VPN Endpoint
|
▼
AWS VPC
|
├── EC2 Instances
├── Databases
└── Other Resources

After connecting, your computer behaves as if it is part of the VPC network.

Main Use Case

Suppose you have:

VPC

└── Private Subnet

└── EC2 Instance

The EC2 instance has only a **private IP address**.

Without VPN:

❌ You cannot directly access it from your laptop.

With AWS Client VPN:

✅ You connect to the VPC.

✅ You can reach the EC2 instance using its private IP address.

Laptop

|

▼

Client VPN

|

▼

Private EC2 (10.x.x.x)

Access to On-Premises Networks

If the VPC is connected to an on-premises network through a Site-to-Site VPN:

Laptop

|

Client VPN

|

AWS VPC

|

Site-to-Site VPN

|

On-Premises Data Center

Then your laptop can also access:

- On-premises servers
- Internal applications
- Corporate resources

through the same VPN connection.

Benefits

- ✅ Secure remote access
- ✅ Uses OpenVPN
- ✅ Access private EC2 instances
- ✅ Access VPC resources without public IPs
- ✅ Can also access connected on-premises networks
- ✅ Managed by AWS

Cloud Practitioner Exam Cheat Sheet

AWS Client VPN

- Connects **individual users/devices** to AWS.
- Uses **OpenVPN**.
- Provides secure access to VPC resources.
- Allows access to resources through their **private IP addresses**.
- Can also access on-premises networks connected via Site-to-Site VPN.

Transit Gateway Overview

As AWS environments grow, networking can become complex.

You may have:

- Multiple VPCs
- Site-to-Site VPN connections
- Direct Connect connections
- On-premises data centers

Managing direct connections between all of them becomes difficult.

The Problem

Without Transit Gateway:

VPC A ↔ VPC B

VPC A ↔ VPC C

VPC B ↔ VPC C

VPC A ↔ VPN

VPC B ↔ VPN

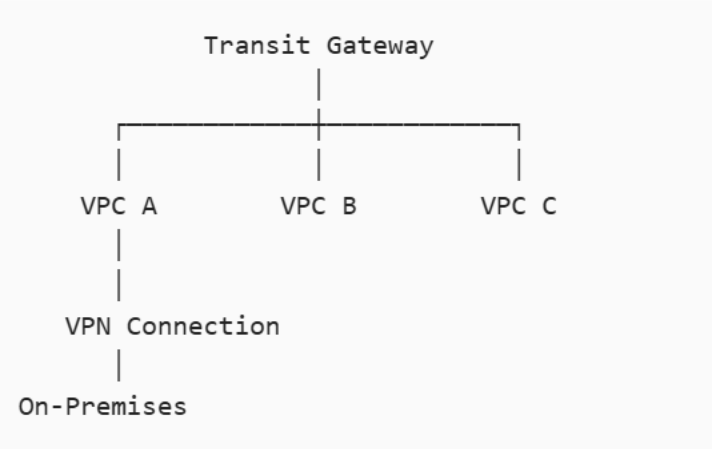
VPC C ↔ VPN

This creates a complex mesh of connections.

The Solution: AWS Transit Gateway

AWS Transit Gateway acts as a **central networking hub**.

Instead of connecting everything to everything else, each network connects only to the Transit Gateway.



This is called a **hub-and-spoke architecture**.

What Can Connect to a Transit Gateway?

Supported Connections

- Amazon VPCs
- Site-to-Site VPNs
- Direct Connect Gateways

VPCs

|

▼

Transit Gateway

▲

|

VPN / Direct Connect

Benefits

✅ Simplifies network architecture

✅ Easier to manage routing

✅ Connects hundreds or thousands of VPCs

✅ Supports hybrid cloud environments

✅ Scales much better than VPC Peering

Transit Gateway vs VPC Peering

VPC Peering

VPC A ↔ VPC B

VPC A ↔ VPC C

VPC B ↔ VPC C

- Many connections
- Difficult to manage at scale

Transit Gateway

Transit Gateway

/ | \

VPC A VPC B VPC C

- One central hub
- Easier routing
- Better scalability

Cloud Practitioner Exam Cheat Sheet

AWS Transit Gateway

- Central hub for networking.
- Connects:
    - Multiple VPCs
    - Site-to-Site VPNs
    - Direct Connect Gateways
- Uses a **hub-and-spoke architecture**.
- Simplifies network management.
- Designed for large-scale environments.

Cloud Practitioner

VPC & Networking Summary

VPC (Virtual Private Cloud)

- Your private network in AWS.
- Regional service (exists within a specific AWS Region).
- Contains subnets and AWS resources.

Subnet

- A partition of a VPC.
- Tied to a single Availability Zone (AZ).
- Can be:
    - **Public Subnet** → Internet accessible.
    - **Private Subnet** → Not directly internet accessible.

Internet Connectivity

Internet Gateway (IGW)

- Connects a VPC to the Internet.
- Required for public subnets.

EC2 → Public Subnet → Internet Gateway → Internet

NAT Gateway

- Allows resources in private subnets to access the Internet.
- Prevents the Internet from initiating connections back.

Private EC2 → NAT Gateway → Internet

Security

Security Group (SG)

- Instance-level firewall.
- Supports **ALLOW rules only**.
- **Stateful** (return traffic automatically allowed).

Network ACL (NACL)

- Subnet-level firewall.
- Supports **ALLOW and DENY rules**.
- **Stateless** (return traffic must be explicitly allowed).

Quick Comparison

Feature	Security Group	NACL
Level	Instance	Subnet
Rules	Allow Only	Allow & Deny
Type	Stateful	Stateless

IP Addresses

Public IPv4

- Internet reachable.
- Usually changes after stop/start.

Private IPv4

- Internal VPC communication.
- Typically remains the same.

Elastic IP

- Fixed public IPv4.
- Useful when a static public IP is required.

IPv6

- Massive address space.
- Public by default.
- Free in AWS.

Monitoring

VPC Flow Logs

Capture network traffic information.

Used for:

- Monitoring
- Troubleshooting
- Security analysis

Can be sent to:

- CloudWatch Logs
- Amazon S3
- Amazon Data Firehose

Connecting VPCs

VPC Peering

Private connection between two VPCs.

Requirements:

- Non-overlapping CIDR ranges.

Limitation:

- **Not Transitive**

A ↔ B

A ↔ C

B ↔ C ? NO

Private Access to AWS Services

VPC Endpoints

Allow access to AWS services without using the public Internet.

Gateway Endpoint

Only for:

- Amazon S3
- DynamoDB

Interface Endpoint

For almost all AWS services:

- CloudWatch
- SNS
- SQS
- Secrets Manager
- EC2 APIs
- etc.

AWS PrivateLink

Provides private access to services hosted in another VPC.

Provider Side

- Network Load Balancer (NLB)

Consumer Side

- Interface Endpoint

Benefits:

- Private communication
- No VPC Peering required
- Scalable for many customers

Hybrid Cloud Connectivity

Site-to-Site VPN

Connects:

On-Premises Network ↔ AWS VPC

Characteristics:

- Uses public Internet
- Encrypted
- Fast to set up

Required Components:

- CGW (Customer Gateway)
- VGW (Virtual Private Gateway)

AWS Direct Connect (DX)

Dedicated physical connection between AWS and your data center.

Characteristics:

- Private connection
- High bandwidth
- Low latency
- Expensive
- Takes longer to provision

Remote User Access

AWS Client VPN

Connects:

Laptop ↔ AWS VPC

Uses:

- OpenVPN

Benefits:

- Access private EC2 instances

- Access private resources using private IP addresses

Large-Scale Networking

AWS Transit Gateway

A central networking hub.

Connects:

- Multiple VPCs
- Site-to-Site VPNs
- Direct Connect Gateways

Architecture:

Transit Gateway

/ | \

VPC1 VPC2 VPC3

|

VPN

|

Data Center

Best choice when:

- Hundreds or thousands of VPCs need connectivity.

Ultimate CCP Memory Map

VPC = Private Network

Subnet = Network Section

Internet Gateway = Public Internet Access

NAT Gateway = Internet Access for Private Resources

Security Group = Instance Firewall (Stateful)

NACL = Subnet Firewall (Stateless)

Flow Logs = Network Traffic Logs

VPC Peering = Connect 2 VPCs

VPC Endpoint = Private Access to AWS Services

PrivateLink = Private Access to Third-Party Services

Site-to-Site VPN = On-Prem ↔ AWS (Internet + Encryption)

Client VPN = Laptop ↔ AWS

Direct Connect = Dedicated Private Link

Transit Gateway = Central Networking Hub

Most Important CCP Exam Associations

- **S3 & DynamoDB → Gateway Endpoint**
- **Instance Firewall → Security Group**
- **Subnet Firewall → NACL**
- **Fixed Public IP → Elastic IP**
- **On-Prem + Public Internet → Site-to-Site VPN**
- **On-Prem + Dedicated Connection → Direct Connect**
- **Many VPCs → Transit Gateway**
- **Private access to AWS services → VPC Endpoint**
- **Private access to third-party services → PrivateLink**
- **Network troubleshooting → VPC Flow Logs**

Security & Compliance

Shared Responsibility Model: Remainders & Examples

The **AWS Shared Responsibility Model** defines who is responsible for security and compliance in the cloud.

AWS Responsibility: Security of the Cloud

AWS is responsible for protecting the infrastructure that runs AWS services, including:

- Physical data centers
- Hardware
- Networking
- Global infrastructure (Regions, Availability Zones, Edge Locations)
- Managed service infrastructure (such as RDS, DynamoDB, S3)

AWS ensures that the cloud platform itself is secure and operational.

Customer Responsibility: Security in the Cloud

Customers are responsible for how they use AWS services, including:

- Managing operating systems on EC2 instances
- Installing security patches on EC2
- Configuring Security Groups and Network ACLs
- Managing IAM users, roles, and permissions
- Protecting application data
- Enabling and managing encryption where required
- Configuring services securely

Shared Controls

Some responsibilities are shared between AWS and customers:

Area	AWS Responsibility	Customer Responsibility
Patch Management	Patches managed services (e.g., RDS)	Patch EC2 operating systems
Configuration Management	Configure AWS infrastructure	Configure AWS resources
Awareness & Training	Train AWS employees	Train customer employees

Example 1: Amazon RDS

AWS Responsibilities

- Manage underlying EC2 instances
- Disable direct SSH access
- Perform OS patching
- Perform database patching
- Maintain storage and hardware

Customer Responsibilities

- Configure Security Groups
- Control database users and permissions
- Decide public/private accessibility
- Configure parameter groups
- Enable encryption if needed

Example 2: Amazon S3

AWS Responsibilities

- Provide durable and scalable storage
- Maintain infrastructure
- Ensure tenant isolation
- Protect customer data from AWS employee access
- Support encryption capabilities

Customer Responsibilities

- Configure bucket permissions
- Create bucket policies
- Manage IAM permissions
- Enable encryption
- Control access to objects and buckets

Exam Tip

Remember:

AWS = Security OF the Cloud
Customer = Security IN the Cloud

Customer Responsibilities

- Data

- Applications
- IAM
- Operating systems
- Firewall configuration
- Encryption settings
- Network traffic protection

AWS Responsibilities

- Hardware
- Networking
- Databases/Compute infrastructure
- AWS-managed software
- Regions, AZs, and Edge Locations

Quick Memory Trick

EC2 = More customer responsibility

- You manage OS, patches, security, and applications.

RDS/S3 = More AWS responsibility

- AWS manages underlying infrastructure, but you still manage access, configuration, and data protection.

This topic is very important for the AWS certification exams and commonly appears in multiple-choice questions.

DDoS Protection: WAF & Shield

What is a DDoS Attack?

DDoS (Distributed Denial of Service) is an attack where thousands or millions of compromised devices (bots) send huge amounts of traffic to a target application.

Result:

- Server becomes overwhelmed.
- Legitimate users cannot access the application.
- Service becomes slow or unavailable.

AWS Services for DDoS Protection

AWS provides multiple layers of protection:

1. **AWS Shield**
2. **AWS WAF**
3. **CloudFront**
4. **Route 53**
5. **Auto Scaling**

1. AWS Shield

AWS Shield is a managed DDoS protection service.

AWS Shield Standard

✅ Free for all AWS customers

✅ Automatically enabled

Protects against common:

- Layer 3 (Network) attacks
- Layer 4 (Transport) attacks

Examples:

- SYN floods
- UDP floods
- Reflection attacks

Exam Tip

Shield Standard is enabled by default for every AWS customer.

AWS Shield Advanced

✅ Optional paid service (~$3,000/month per organization)

Provides:

- Enhanced DDoS protection
- Protection for:
  - EC2
  - Elastic Load Balancer (ELB)
  - CloudFront
  - Route 53
  - Global Accelerator
- 24/7 access to AWS DDoS Response Team (DRT)
- Cost protection during DDoS incidents

Exam Tip

If a question mentions:

- **24/7 expert support**
- **Response team**
- **Advanced DDoS defense**

➡️ Answer: **AWS Shield Advanced**

2. AWS WAF (Web Application Firewall)

WAF protects web applications from Layer 7 attacks.

Layer 7 = Application Layer (HTTP/HTTPS)

Can be attached to:

- Application Load Balancer (ALB)
- CloudFront
- API Gateway

WAF Filtering Rules

WAF can filter requests based on:

IP Addresses

Allow or block specific IPs.

HTTP Headers

Inspect incoming requests.

Request Body

Analyze user input.

Query Strings

Filter URLs and parameters.

Protection Against Common Attacks

SQL Injection (SQLi)

Attackers try to inject SQL commands into applications.

Example:

SQL

' OR 1=1 --

WAF can detect and block it.

Cross-Site Scripting (XSS)

Attackers inject malicious JavaScript into web pages.

WAF can identify and block these requests.

Geo-Match Rules

Block or allow traffic from specific countries.

Example:

- Allow only India and Australia
- Block all other countries

Size Constraints

Block unusually large requests.

Useful against:

- Buffer attacks
- Malicious payloads

Rate-Based Rules

Very important for DDoS protection.

Example:

Limit:

- 100 requests per minute
- 5 requests per second

If exceeded:

- WAF automatically blocks or throttles traffic

Exam Tip

Rate-based rules in WAF help mitigate application-layer DDoS attacks.

3. CloudFront

CloudFront is AWS's Content Delivery Network (CDN).

Benefits:

- Caches content at Edge Locations
- Reduces load on origin servers
- Absorbs large traffic volumes

Combined with AWS Shield:

- Traffic is filtered at edge locations
- Attacks are mitigated before reaching your application

4. Route 53

Route 53 is AWS's managed DNS service.

Benefits:

- Highly scalable
- DDoS protected through AWS Shield
- Can handle massive DNS request volumes

Exam Tip

Route 53 includes Shield protection automatically.

5. Auto Scaling

Even with DDoS protections, traffic may increase dramatically.

Auto Scaling helps by:

- Launching additional EC2 instances
- Handling higher workloads
- Improving availability during traffic spikes

Typical AWS DDoS-Protected Architecture

Users

|

Route 53

(Shield Protected)

|

CloudFront

(Shield Protected)

|

AWS WAF

|

Application Load Balancer

|

Auto Scaling Group

|

EC2 Instances

Exam Cheat Sheet

Service	Purpose
Shield Standard	Free, automatic DDoS protection
Shield Advanced	Advanced DDoS protection + response team
WAF	Layer 7 HTTP/HTTPS protection
CloudFront	Edge protection and content caching
Route 53	DDoS-resistant DNS service
Auto Scaling	Scale resources during traffic spikes

Common Exam Questions

Which AWS service provides free DDoS protection by default?

✅ **AWS Shield Standard**

Which AWS service protects against SQL Injection and XSS?

✅ **AWS WAF**

Which AWS service provides 24/7 DDoS Response Team access?

✅ **AWS Shield Advanced**

Which WAF feature helps mitigate DDoS attacks by limiting requests?

✅ **Rate-Based Rules**

Which AWS services are commonly used together for DDoS protection?

✅ **Shield + WAF + CloudFront + Route 53 + Auto Scaling**

Memory Trick

Network attacks (L3/L4) → Shield
Web attacks (L7) → WAF
Global edge protection → CloudFront & Route 53
Traffic surge handling → Auto Scaling

AWS Network Firewall

What is AWS Network Firewall?

AWS Network Firewall is a managed firewall service that provides protection for an entire **VPC (Virtual Private Cloud)**.

It helps inspect and filter network traffic at multiple layers:

- Layer 3 (Network)
- Layer 4 (Transport)
- Layer 7 (Application)

Exam Tip

If a question asks:

"How can I protect all traffic entering and leaving a VPC?"

The answer is often **AWS Network Firewall**.

Why Use AWS Network Firewall?

Unlike Security Groups and Network ACLs:

- **Security Groups** protect individual resources (EC2, ENIs).
- **Network ACLs** protect individual subnets.
- **AWS Network Firewall** protects the entire VPC's traffic flow.

It provides centralized security enforcement across the VPC.

Traffic That Can Be Inspected

AWS Network Firewall can inspect traffic:

Internet Traffic

- Inbound from the Internet
- Outbound to the Internet

VPN Traffic

- Site-to-Site VPN connections

AWS Direct Connect Traffic

- Traffic between on-premises and AWS

VPC-to-VPC Traffic

- VPC Peering
- Transit Gateway-connected networks

Protection Capabilities

AWS Network Firewall provides:

Stateful Inspection

Tracks active connections and evaluates traffic based on connection state.

Stateless Inspection

Filters packets individually using defined rules.

Layer 7 Filtering

Can inspect application traffic and enforce advanced security policies.

Intrusion Prevention and Detection

Helps identify and block malicious traffic patterns.

Comparison

Service	Protection Scope
Security Group	Instance/ENI level
Network ACL	Subnet level
AWS Network Firewall	VPC level

Example Architecture

Internet

|

AWS Network Firewall

|

VPC

├— Public Subnets

├— Private Subnets

└— Applications

The firewall evaluates traffic before it reaches resources inside the VPC.

Exam Cheat Sheet

Remember:

Security Group

- Resource-level protection
- Stateful

Network ACL

- Subnet-level protection
- Stateless

AWS Network Firewall

- VPC-wide protection
- Layer 3–7 inspection
- Centralized traffic filtering

Common Exam Question

A company wants to inspect and filter all inbound and outbound traffic for an entire VPC. Which AWS service should they use?

✅ **AWS Network Firewall**

Memory Trick

Instance → Security Group
Subnet → Network ACL
VPC → AWS Network Firewall

AWS Firewall Manager

What is AWS Firewall Manager?

AWS Firewall Manager is a security management service that allows you to **centrally configure, manage, and enforce security rules across multiple AWS accounts and resources** within an **AWS Organization**.

Instead of configuring security settings separately in each account, Firewall Manager lets you manage them from one central location.

Main Exam Takeaway

If you see:

"Manage Security Groups across multiple AWS accounts"

✅ **AWS Firewall Manager**

This is one of the most common exam associations.

What Can Firewall Manager Manage?

Firewall Manager can centrally manage policies for:

- **VPC Security Groups**
- **AWS WAF rules**
- **AWS Shield Advanced protections**
- **AWS Network Firewall**
- Other supported AWS security services

Key Benefits

Centralized Management

Manage security policies from a single account instead of configuring each account individually.

Organization-Wide Enforcement

Works with **AWS Organizations** to apply policies across member accounts.

Automatic Protection

Policies are automatically applied to:

- Existing accounts
- New accounts added to the organization
- Existing resources
- New resources created in the future

This ensures consistent security compliance.

Example

Imagine an organization with:

AWS Organization

├— Account A

├— Account B

├— Account C

└— Account D

A security team wants all EC2 instances to have specific Security Group rules.

Without Firewall Manager:

- Configure each account separately.

With Firewall Manager:

- Create one policy centrally.
- AWS automatically enforces it across all accounts.

Firewall Manager vs Security Groups

Service	Scope
Security Group	Individual resource (EC2, ENI)
Network ACL	Subnet
Network Firewall	Entire VPC
Firewall Manager	Multiple AWS accounts across an Organization

Exam Scenarios

Scenario 1

A company has hundreds of AWS accounts and wants identical Security Group policies across all of them.

✅ **AWS Firewall Manager**

Scenario 2

A company wants to centrally manage AWS WAF rules for applications deployed across multiple AWS accounts.

✅ **AWS Firewall Manager**

Scenario 3

New AWS accounts should automatically receive the organization's security policies.

✅ **AWS Firewall Manager**

Memory Trick

Think of security management levels:

Security Group → Resource

Network ACL → Subnet

Network Firewall → VPC

Firewall Manager → Organization

Penetration Testing

What is Penetration Testing?

Penetration Testing (Pen Testing) is the practice of testing the security of your own systems by attempting to identify vulnerabilities before attackers do.

In AWS, customers are allowed to perform penetration testing against certain AWS services without obtaining prior approval.

Services Allowed for Penetration Testing (No Prior Approval Required)

You can perform security assessments and penetration testing on your own resources in services such as:

- Amazon EC2
- NAT Gateway
- Elastic Load Balancer (ELB)
- Amazon RDS
- Amazon Aurora
- Amazon CloudFront
- Amazon API Gateway
- AWS Lambda and Lambda@Edge
- Amazon Lightsail
- AWS Elastic Beanstalk

Exam Tip

Remember:

✅ **Penetration testing on your own AWS resources is generally allowed for specific services without prior authorization.**

Activities NOT Allowed

Certain testing activities are prohibited because they can impact AWS infrastructure or other customers.

DNS Zone Walking

Attempting to enumerate DNS records from Route 53 hosted zones.

❌ Not allowed

Denial of Service (DoS)

Overwhelming systems with traffic.

❌ Not allowed

Distributed Denial of Service (DDoS)

Simulating large-scale attacks against resources.

❌ Not allowed

Simulated DoS / Simulated DDoS

Even if performed against your own systems.

❌ Not allowed

Port Flooding

Sending excessive traffic to ports.

❌ Not allowed

Protocol Flooding

Flooding systems using network protocols.

❌ Not allowed

Request Flooding

Generating excessive requests to overload systems.

❌ Not allowed

Why Are These Prohibited?

These activities can appear as attacks against AWS infrastructure and may affect:

- AWS networks
- AWS security operations
- Other AWS customers

Therefore AWS restricts such testing.

Exam Scenarios

Scenario 1

A company wants to perform penetration testing on its EC2 instances.

✅ Allowed

Scenario 2

A security team wants to simulate a DDoS attack against its AWS-hosted application.

❌ Not allowed

Scenario 3

A company wants to test its API Gateway security configuration.

✅ Allowed

Scenario 4

A company wants to perform DNS zone walking against Route 53.

❌ Not allowed

Exam Cheat Sheet

Allowed

✅ EC2

✅ RDS

✅ Aurora

✅ ELB

✅ NAT Gateway

✅ API Gateway

✅ Lambda / Lambda@Edge

✅ CloudFront

✅ Lightsail

✅ Elastic Beanstalk

Not Allowed

❌ DoS attacks

❌ DDoS attacks

❌ Simulated DoS/DDoS

❌ Port flooding

❌ Protocol flooding

❌ Request flooding

❌ DNS zone walking

Memory Trick

Pen Test = Allowed

Anything that looks like a real attack (DoS/DDoS/Flooding) = Not Allowed

Encryption with KMS and CloudHSM

What is Encryption?

Encryption is the process of converting readable data into an unreadable format using encryption keys.

Only someone with the correct key can decrypt and read the data.

Two Types of Encryption in AWS

1. Encryption at Rest

Encryption at rest protects data that is stored on disk or other storage devices.

Examples:

- Amazon S3 objects
- EBS volumes
- EFS file systems
- RDS databases
- Redshift clusters
- Glacier archives

Example

Data --> Stored in S3 --> Encrypted at Rest

Even if someone gains access to the storage, they cannot read the data without the encryption key.

2. Encryption in Transit

Encryption in transit protects data while it is moving across a network.

Examples:

- On-premises → AWS
- EC2 → DynamoDB
- EC2 → RDS
- User → Website (HTTPS)

Example

EC2 ------Encrypted Connection------> DynamoDB

Common technologies:

- SSL/TLS
- HTTPS

Exam Tip

Best practice is to encrypt data:

✅ At Rest

✅ In Transit

This protects data throughout its lifecycle.

AWS Key Management Service (KMS)

AWS KMS (Key Management Service) is the primary AWS service for encryption key management.

With KMS:

- AWS manages the encryption infrastructure.
- Customers control access to keys.
- AWS services integrate directly with KMS.

Common Services Using KMS

- Amazon S3
- EBS
- RDS
- Redshift
- EFS

Exam Tip

When an exam question asks:

"Which AWS service manages encryption keys?"

✅ **AWS KMS**

Services Supporting KMS Encryption

EBS

Encrypt EBS volumes using KMS.

S3

Enable Server-Side Encryption (SSE).

RDS

Encrypt databases.

Redshift

Encrypt data warehouse clusters.

EFS

Encrypt file systems.

Services with Automatic Encryption

Some AWS services automatically encrypt data.

Examples:

- CloudTrail Logs
- Amazon S3 Glacier
- AWS Storage Gateway

AWS CloudHSM

What is CloudHSM?

AWS CloudHSM provides a dedicated Hardware Security Module (HSM) for managing encryption keys.

Unlike KMS:

- AWS manages the hardware.
- You manage the keys.

KMS vs CloudHSM

KMS

- AWS manages keys and infrastructure.
- Easier to use.
- Fully managed service.

CloudHSM

- AWS manages hardware only.
- Customer manages keys.
- Greater control.
- Suitable for strict compliance requirements.

CloudHSM Features

Dedicated Hardware

Your encryption keys are stored in dedicated HSM hardware.

Customer-Controlled Keys

Only you manage the keys.

Tamper Resistant

Hardware is designed to resist physical attacks.

Compliance

Supports:

FIPS 140-2 Level 3

A high-security compliance standard.

Types of KMS Keys

1. Customer Managed Keys (CMK)

Created and managed by the customer.

You can:

- Enable or disable keys
- Create access policies
- Configure key rotation
- Audit key usage

Exam Tip

Provides the most control within KMS.

2. AWS Managed Keys

Created and managed by AWS on your behalf.

Examples:

aws/s3

aws/ebs

aws/rds

Characteristics:

- Automatically created
- Used by AWS services
- Less administrative work

Exam Tip

If a key name starts with:

aws/

it is an **AWS-managed key**.

3. AWS Owned Keys

Owned completely by AWS.

Characteristics:

- Used internally by AWS services.
- Customers cannot view or manage them.
- No direct control.

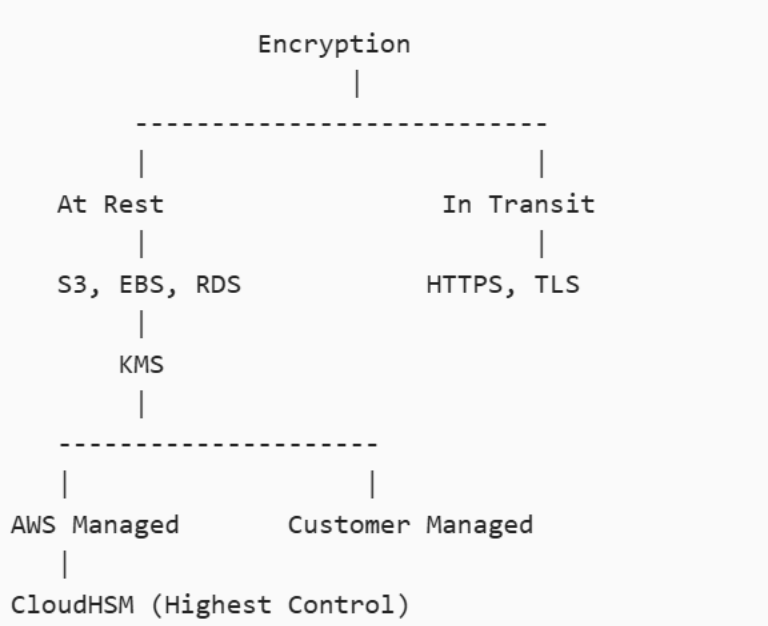
4. CloudHSM Keys (Custom Key Store)

Keys generated and stored inside CloudHSM hardware.

Characteristics:

- Customer controls keys.
- Cryptographic operations occur within HSM devices.
- Highest level of control.

Architecture Overview



Cloud Practitioner

Exam Cheat Sheet

Concept	Remember
Encryption at Rest	Protects stored data
Encryption in Transit	Protects moving data
AWS KMS	Main AWS encryption service
CloudHSM	Customer controls encryption keys
Customer Managed Key	Customer creates and manages
AWS Managed Key	AWS creates and manages
AWS Owned Key	Fully controlled by AWS
CloudHSM Key	Generated and stored in HSM

Most Important Exam Associations

Encrypt S3, EBS, RDS, EFS, Redshift

✅ KMS

Need customer-controlled hardware security module

✅ CloudHSM

Key name starts with aws/

✅ AWS Managed Key

Highest control over keys

✅ CloudHSM / Customer Managed Keys

Data stored on disk

✅ Encryption at Rest

Data moving on a network

✅ Encryption in Transit

AWS Certificate Manager

What is AWS Certificate Manager (ACM)?

AWS Certificate Manager (ACM) is a service that allows you to easily:

- Provision SSL/TLS certificates
- Manage certificates
- Deploy certificates on AWS services
- Automatically renew certificates

ACM simplifies the process of securing websites and applications with **HTTPS**.

Why Use ACM?

SSL/TLS certificates provide:

✅ Encryption in transit

✅ Secure HTTPS connections

✅ Authentication of websites

✅ Protection of data exchanged between users and applications

Example Architecture

```
User
  |
HTTPS
  |
  v
Application Load Balancer
  |
HTTP
  |
  v
EC2 Instances
```

In this setup:

- ACM provides the SSL/TLS certificate.
- The certificate is attached to the Application Load Balancer (ALB).
- Users connect through HTTPS.
- Traffic is encrypted while traveling across the Internet.

ACM Features

Certificate Provisioning

ACM can generate SSL/TLS certificates for your domains.

Example:

www.example.com

api.example.com

Certificate Management

ACM manages the lifecycle of certificates, reducing operational effort.

Automatic Renewal

One of ACM's biggest benefits.

Instead of manually renewing certificates every year:

✅ ACM automatically renews supported certificates before expiration.

Exam Tip

Automatic certificate renewal is a major ACM advantage.

Public and Private Certificates

Public Certificates

Used for public websites.

Examples:

- www.company.com
- app.company.com

✅ Free when issued through ACM

Private Certificates

Used internally within organizations.

Examples:

- Internal applications
- Private APIs
- Corporate services

AWS Services Integrated with ACM

Elastic Load Balancer (ELB)

Most common exam scenario.

Attach certificates directly to:

- Application Load Balancer (ALB)
- Network Load Balancer (TLS listeners)

Amazon CloudFront

Use ACM certificates for secure content delivery.

API Gateway

Secure APIs using HTTPS.

Exam Scenarios

Scenario 1

A company needs HTTPS for its Application Load Balancer and wants automatic certificate renewal.

✅ **AWS Certificate Manager (ACM)**

Scenario 2

A company wants to provision free SSL/TLS certificates for a public website hosted on AWS.

✅ **AWS Certificate Manager (ACM)**

Scenario 3

A company wants automatic management and renewal of website certificates.

✅ **AWS Certificate Manager (ACM)**

Scenario 4

A developer wants to secure a CloudFront distribution with HTTPS.

✅ **AWS Certificate Manager (ACM)**

ACM vs KMS

ACM

- Manages SSL/TLS certificates
- Enables HTTPS
- Provides encryption in transit

KMS

- Manages encryption keys
- Encrypts data at rest
- Used by services like S3, EBS, RDS

Memory Trick

HTTPS Certificate → ACM

Encryption Key → KMS

Exam Cheat Sheet

Service	Purpose
ACM	Provision and manage SSL/TLS certificates
KMS	Manage encryption keys
ACM + ALB	HTTPS for web applications
ACM + CloudFront	Secure content delivery
ACM + API Gateway	Secure APIs
Public ACM Certificates	Free
Automatic Renewal	Supported by ACM

Secret Manager Overview

What is AWS Secrets Manager?

AWS Secrets Manager is a service used to securely store, manage, and retrieve secrets.

Examples of secrets:

- Database usernames and passwords
- API keys
- Application credentials
- Authentication tokens
- OAuth secrets

Why Use Secrets Manager?

Without Secrets Manager:

Application

|

Hardcoded Password

Problems:

- Passwords exposed in code
- Difficult to rotate credentials
- Security risk

With Secrets Manager:

Application

|

Secrets Manager

|

Encrypted Secret

Benefits:

- Secure storage
- Centralized management
- Automatic rotation
- Fine-grained access control

Key Features

1. Secure Secret Storage

Secrets are stored securely and encrypted.

2. Automatic Secret Rotation

One of the most important exam topics.

You can configure secrets to rotate automatically.

Example:

Every 30 days

Every 60 days

Every 90 days

The rotation process can be automated using **AWS Lambda**.

Exam Tip

If a question mentions:

"Automatically rotate database passwords"

✅ **AWS Secrets Manager**

3. Integration with Amazon RDS

Secrets Manager integrates directly with:

- Amazon RDS
- Amazon Aurora

It can:

- Generate database credentials
- Store them securely
- Rotate them automatically

Exam Tip

Questions involving:

- RDS passwords
- Credential rotation
- Database secrets

Almost always point to:

✅ **AWS Secrets Manager**

4. KMS Encryption

Secrets stored in Secrets Manager are encrypted using:

✅ **AWS Key Management Service (KMS)**

This protects secret values at rest.

How Secret Rotation Works

Secret Stored

|

v

Secrets Manager

|

v

Lambda Function

|

v

Generate New Password

|

v

Update Database

The Lambda function performs the rotation automatically according to the configured schedule.

Example

A production RDS database uses:

Username: admin

Password: MyPassword123

Store credentials in Secrets Manager.

Configure rotation:

Rotate every 30 days

Secrets Manager and Lambda automatically:

- Generate a new password
- Update RDS
- Update the stored secret

Applications continue retrieving the latest credential from Secrets Manager.

Pricing

- Paid service
- Typically charged per secret stored
- Additional charges for API requests

Exam Tip

Unlike many AWS security services, Secrets Manager is **not free**.

Secrets Manager vs Parameter Store

A common exam comparison:

AWS Secrets Manager

✅ Secret storage

✅ Automatic rotation

✅ RDS integration

✅ KMS encryption

❌ Paid service

Systems Manager Parameter Store

✅ Configuration storage

✅ Basic secret storage

✅ Lower cost

❌ No built-in automatic secret rotation

Memory Trick

Need rotation? → Secrets Manager

Need simple configuration values? → Parameter Store

Artifact Overview

What is AWS Artifact?

AWS Artifact is a self-service portal that provides access to:

1. **AWS Compliance Reports**
2. **AWS Agreements**

It helps customers obtain documentation needed for:

- Security audits
- Compliance reviews
- Regulatory requirements
- Internal governance

Exam Tip

If a question asks:

"Where can you download AWS compliance reports or audit documents?"

✅ **AWS Artifact**

AWS Artifact Reports

These are compliance and security reports generated by independent third-party auditors.

Examples include:

- SOC Reports (Service Organization Control)
- PCI DSS Reports (Payment Card Industry)
- ISO Certifications (ISO 27001, etc.)

These reports help organizations verify that AWS complies with various industry standards.

Use Cases

- Internal audits
- Risk assessments
- Regulatory compliance
- Security reviews

AWS Artifact Agreements

AWS Artifact also allows customers to review and accept legal agreements.

Examples:

BAA (Business Associate Addendum)

Used for healthcare workloads that must comply with HIPAA requirements.

HIPAA-Related Agreements

Required when handling protected health information (PHI) on AWS.

Other AWS Agreements

Organizations can review, track, and manage agreement status directly from Artifact.

Common Compliance Standards Found in Artifact

SOC Reports

Independent audit reports evaluating AWS controls.

PCI DSS

For organizations processing payment card data.

ISO Certifications

International security and quality standards.

HIPAA Documents

Healthcare compliance documentation and agreements.

Typical Exam Scenarios

Scenario 1

A company auditor needs access to AWS PCI compliance reports.

✅ AWS Artifact

Scenario 2

A healthcare company wants to review and accept a HIPAA Business Associate Agreement (BAA).

✅ AWS Artifact

Scenario 3

A compliance team needs AWS SOC audit reports.

✅ AWS Artifact

Scenario 4

A company needs downloadable security and compliance documentation from AWS.

✅ AWS Artifact

AWS Artifact vs Other Security Services

Service	Purpose
AWS Artifact	Compliance reports and agreements
AWS KMS	Encryption key management
AWS Secrets Manager	Store and rotate secrets
AWS Shield	DDoS protection
AWS WAF	Web application firewall

Exam Cheat Sheet

AWS Artifact Provides:

✅ Compliance reports

✅ SOC reports

✅ PCI reports

✅ ISO certifications

✅ HIPAA/BAA agreements

✅ Audit documentation

✅ On-demand downloads

AWS Artifact Does NOT

❌ Encrypt data

❌ Store secrets

❌ Protect applications

❌ Manage IAM permissions

It is primarily a **compliance and audit documentation portal**.

Memory Trick

Artifact = Audit Documents

Think:

Need compliance reports or legal agreements from AWS?

✅ **AWS Artifact**

GuardDuty Overview

What is Amazon GuardDuty?

Amazon GuardDuty is a threat detection service that continuously monitors your AWS environment for malicious activity and unauthorized behavior.

It uses:

- Machine Learning (ML)
- Anomaly Detection
- AWS threat intelligence feeds
- Third-party threat intelligence

to identify potential security threats.

Exam Tip

If a question asks:

"Which AWS service automatically detects threats, suspicious activity, or compromised accounts?"

✅ **Amazon GuardDuty**

Key Features

Easy to Enable

- One-click activation
- No agents to install
- No infrastructure to manage

Free Trial

- 30-day trial available

Data Sources Monitored by GuardDuty

GuardDuty analyzes multiple AWS logs and data sources.

1. CloudTrail Logs

GuardDuty examines API activity for suspicious operations.

Examples:

- Unauthorized API calls
- Unusual account behavior
- Privilege escalation attempts
- Unexpected resource creation

Management Events

Examples:

- CreateVPC
- CreateSubnet
- CreateUser

Data Events

Examples:

- S3 GetObject
- S3 ListBucket
- S3 DeleteObject

2. VPC Flow Logs

Analyzes network traffic patterns.

Detects:

- Suspicious IP addresses
- Port scanning
- Unusual inbound/outbound traffic
- Communication with known malicious hosts

3. DNS Logs

Monitors DNS requests from EC2 instances.

Can detect:

- Malware communication
- Command-and-control traffic
- Data exfiltration attempts through DNS

Example

An EC2 instance begins sending encoded data within DNS queries.

✅ GuardDuty can identify this as suspicious behavior.

Optional Data Sources

GuardDuty can also analyze additional AWS resources.

Amazon S3 Protection

Detects suspicious S3 access patterns.

Amazon EBS Malware Protection

Scans EBS volumes associated with suspicious workloads.

AWS Lambda Network Activity

Monitors Lambda functions for unusual network behavior.

Amazon RDS & Aurora Login Events

Identifies suspicious database login activity.

Amazon EKS Protection

Examines:

- EKS Audit Logs
- Runtime Monitoring

Detects suspicious Kubernetes activity.

Findings

When GuardDuty detects a threat, it creates a **Finding**.

Examples:

- Compromised instance
- Credential misuse
- Cryptocurrency mining
- Data exfiltration
- Reconnaissance activity

Cryptocurrency Mining Detection

A favorite exam topic.

GuardDuty includes built-in detection for:

✅ Cryptocurrency mining attacks

Example:

An attacker compromises an EC2 instance and installs Bitcoin mining software.

GuardDuty can generate a finding identifying this behavior.

Exam Tip

If you see:

"Detect cryptocurrency mining activity"

✅ Amazon GuardDuty

Integration with EventBridge

GuardDuty findings can automatically generate events in:

✅ Amazon EventBridge

Event Flow

GuardDuty

|

v

Finding Generated

|

v

Amazon EventBridge

|

+--> Lambda

|

+--> SNS

|

+--> Other AWS Services

Automated Responses

Using EventBridge rules, you can:

Invoke Lambda Functions

Example:

- Shut down a compromised instance
- Isolate resources

Send SNS Notifications

Example:

- Email security administrators
- Trigger incident alerts

Architecture Overview

CloudTrail Logs

|

VPC Flow Logs

|

DNS Logs

|

Optional Sources

(S3, EBS, Lambda,

RDS, Aurora, EKS)

|

v

Amazon GuardDuty

|

v

Security Findings

|

v

Amazon EventBridge

|

| |

Lambda SNS

GuardDuty vs Other Security Services

Service	Purpose
GuardDuty	Threat detection and anomaly detection
Shield	DDoS protection
WAF	Web application protection
Inspector	Vulnerability assessment
Security Hub	Centralized security findings
Firewall Manager	Centralized security policy management

Exam Scenarios

Scenario 1

A company wants to detect suspicious API calls and potentially compromised AWS accounts.

- ✅ GuardDuty

Scenario 2

A security team wants machine-learning-based threat detection without installing agents.

- ✅ GuardDuty

Scenario 3

A company wants to detect cryptocurrency mining on EC2 instances.

- ✅ GuardDuty

Scenario 4

A company wants to analyze CloudTrail events, VPC Flow Logs, and DNS logs for threats.

- ✅ GuardDuty

Exam Cheat Sheet

GuardDuty Uses

- ✅ Machine Learning
- ✅ Anomaly Detection
- ✅ Threat Intelligence Feeds

Analyzes

- ✅ CloudTrail Logs
- ✅ VPC Flow Logs
- ✅ DNS Logs
- ✅ S3 Activity (optional)
- ✅ EBS Activity (optional)
- ✅ Lambda Activity (optional)
- ✅ RDS/Aurora Login Activity (optional)
- ✅ EKS Logs (optional)

Integrates With

- ✅ EventBridge
- ✅ Lambda
- ✅ SNS

Detects

- ✅ Unauthorized API calls
- ✅ Suspicious network traffic
- ✅ Compromised instances
- ✅ Credential compromise
- ✅ Data exfiltration
- ✅ Cryptocurrency mining

Inspector Overview

What is Amazon Inspector?

Amazon Inspector is an automated vulnerability management service that continuously scans AWS workloads for security vulnerabilities and unintended network exposure.

It helps identify security issues in:

- **EC2 Instances**
- **Amazon ECR Container Images**
- **AWS Lambda Functions**

Exam Tip

If a question asks:

"Which AWS service scans EC2 instances, container images, and Lambda functions for vulnerabilities?"

✅ **Amazon Inspector**

1. Amazon Inspector for EC2

For EC2 instances, Amazon Inspector works with the **AWS Systems Manager (SSM) Agent**.

It continuously assesses:

Network Exposure

Detects unintended network accessibility, such as:

- Open ports
- Excessive access permissions
- Internet-exposed resources

Operating System Vulnerabilities

Checks installed software and operating systems against known vulnerabilities.

Examples:

- Missing security patches
- Vulnerable packages
- Outdated software

2. Amazon Inspector for Amazon ECR

Inspector can automatically scan container images stored in:

✅ **Amazon Elastic Container Registry (ECR)**

As Docker/container images are pushed to ECR:

- Inspector scans them automatically.

- Detects known software vulnerabilities.
- Identifies insecure packages and dependencies.

Exam Tip

Container image vulnerability scanning → Amazon Inspector

3. Amazon Inspector for AWS Lambda

Inspector also scans:

✅ Lambda functions

It checks:

- Function code dependencies
- Software packages
- Known vulnerabilities (CVEs)

Scanning occurs automatically as functions are deployed.

What Does Inspector Check?

Common Vulnerabilities and Exposures (CVEs)

Inspector compares your workloads against public vulnerability databases.

Examples:

- Operating system vulnerabilities
- Package vulnerabilities
- Dependency vulnerabilities

Exam Tip

Inspector is primarily a **vulnerability assessment service**, not a threat detection service.

Network Reachability (EC2)

Inspector evaluates:

- Which instances are reachable
- Whether resources are unintentionally exposed
- Potential security risks due to open network access

Continuous Scanning

Inspector continuously evaluates resources.

When:

- New vulnerabilities are published

- New CVEs are discovered
- Workloads change

Inspector automatically rescans affected resources.

Risk Scoring

Each finding receives a:

✅ Risk score

This helps security teams:

- Prioritize remediation
- Focus on critical vulnerabilities first

Findings and Integrations

AWS Security Hub

Inspector findings can be sent to:

✅ AWS Security Hub

This provides centralized visibility into security issues.

Amazon EventBridge

Findings can also be sent to:

✅ Amazon EventBridge

This enables automated responses.

Example:

Inspector Finding

|

v

EventBridge

|

| |

Lambda SNS

Possible actions:

- Send alerts
- Open tickets
- Trigger remediation workflows

Inspector vs GuardDuty

A very common exam comparison.

Amazon Inspector

Focuses on:

✅ Vulnerability Assessment

✅ Security Configuration Assessment

✅ EC2 Scanning

✅ ECR Image Scanning

✅ Lambda Vulnerability Scanning

Amazon GuardDuty

Focuses on:

✅ Threat Detection

✅ Suspicious Activity

✅ CloudTrail Analysis

✅ DNS Analysis

✅ VPC Flow Log Analysis

✅ Cryptocurrency Mining Detection

Memory Trick

Inspector = Finds vulnerabilities

GuardDuty = Finds attackers

Architecture Overview

EC2 Instances

|

ECR Images

|

Lambda Functions

|

v

Amazon Inspector

|

v

Security Findings

|

| |

Security Hub EventBridge

Exam Scenarios

Scenario 1

A company wants to identify vulnerabilities on EC2 instances.

✅ Amazon Inspector

Scenario 2

A company wants container images scanned automatically when pushed to ECR.

✅ Amazon Inspector

Scenario 3

A company wants to identify vulnerable package dependencies in Lambda functions.

✅ Amazon Inspector

Scenario 4

A security team wants CVE-based vulnerability scanning.

✅ Amazon Inspector

Exam Cheat Sheet

Amazon Inspector Scans

✅ EC2 Instances

✅ ECR Container Images

✅ Lambda Functions

Detects

✅ CVEs

✅ Vulnerable software packages

- ✅ Missing patches
- ✅ Network exposure
- ✅ Vulnerable dependencies

Integrates With

- ✅ AWS Systems Manager (EC2)
- ✅ AWS Security Hub
- ✅ Amazon EventBridge

Continuous Assessment

- ✅ Automatic rescans when new vulnerabilities are discovered
- ✅ Risk scoring of findings

Config Overview

What is AWS Config?

AWS Config is a service that continuously:

- Records AWS resource configurations
- Tracks configuration changes over time
- Evaluates compliance against predefined rules
- Provides auditing and governance capabilities

Exam Tip

If a question asks:

"Which service keeps a history of resource configurations and checks compliance?"

✅ **AWS Config**

Why Use AWS Config?

AWS Config helps answer questions such as:

- Is SSH (port 22) publicly accessible?
- Does an S3 bucket allow public access?
- Who changed a Security Group?
- When was a resource modified?
- Is a resource compliant with company policies?

Core Functions of AWS Config

1. Configuration Recording

AWS Config records the configuration of AWS resources.

Examples:

- EC2 instances
- Security Groups
- S3 buckets
- VPCs
- Load Balancers
- RDS databases

AWS Config creates a historical record of resource settings.

2. Configuration History

Config stores snapshots of resource configurations over time.

Example:

Day 1: SG allows SSH from company network

Day 10: SG modified to allow SSH from 0.0.0.0/0

Day 15: SG fixed and restricted

AWS Config keeps all these changes.

3. Compliance Monitoring

AWS Config evaluates resources against rules.

Example rule:

restricted-ssh

Checks if port 22 (SSH) is open to everyone.

Result:

- COMPLIANT ✅
- NON_COMPLIANT ❌

AWS Config Rules

Config Rules automatically assess compliance.

Examples

Restricted SSH

Checks if:

0.0.0.0/0 → Port 22

is allowed.

If yes:

❌ Non-compliant

S3 Public Access Checks

Checks whether S3 buckets allow public access.

RDS Public Access Checks

Checks if databases are publicly accessible.

Logging Requirements

Verifies whether logging is enabled.

Exam Tip

AWS Config Rules = Compliance Evaluation

Configuration Timeline

AWS Config provides a history of resource changes.

Example:

Security Group

|

+-- Created

+-- Inbound Rule Added

+-- Rule Removed

+-- Compliance Status Changed

You can view exactly what changed and when.

Compliance Timeline

AWS Config also shows compliance status over time.

Example:

June 1 → Non-Compliant

June 5 → Fixed

June 5 → Compliant

Useful for audits and investigations.

Integration with Amazon S3

AWS Config stores configuration snapshots and history in:

✅ Amazon S3

Benefits:

- Long-term retention
- Audit support
- Historical analysis

Integration with SNS

Config can send notifications when:

- Resources change
- Compliance status changes

- Rules become non-compliant

Using:

✅ Amazon SNS

Example:

Resource Change

|

v

AWS Config

|

v

SNS Notification

|

v

Email Alert

Integration with CloudTrail

Very important exam topic.

AWS Config

Answers:

What changed?

What is the current configuration?

Is the resource compliant?

CloudTrail

Answers:

Who made the change?

When was the API call made?

Which API was used?

Example

Security Group becomes non-compliant.

AWS Config shows:

SSH Rule Added

CloudTrail shows:

IAM User: AdminUser

API: AuthorizeSecurityGroupIngress

Time: 10:45 AM

Multi-Account & Multi-Region Support

Config is:

✅ Regional Service

However, it can aggregate results from:

- Multiple Regions
- Multiple AWS Accounts

This allows centralized compliance reporting.

AWS Config vs CloudTrail

A very common exam comparison.

Service	Purpose
AWS Config	Records resource configurations and compliance
AWS CloudTrail	Records API calls and user activity

Memory Trick

Config = What Changed

CloudTrail = Who Changed It

AWS Config vs Inspector

AWS Config

- Compliance monitoring
- Resource configuration tracking
- Governance

Amazon Inspector

- Vulnerability scanning
- Security assessments
- CVE detection

Typical Exam Scenarios

Scenario 1

A company wants to know whether any Security Groups allow unrestricted SSH access.

✅ AWS Config

Scenario 2

An auditor wants to see how an S3 bucket configuration changed over the past six months.

✅ AWS Config

Scenario 3

A company needs continuous compliance monitoring against security policies.

✅ AWS Config

Scenario 4

A security team wants to know who modified a Security Group.

✅ CloudTrail

(Optionally viewed alongside AWS Config)

Scenario 5

A company wants alerts whenever a resource becomes non-compliant.

✅ AWS Config + SNS

Exam Cheat Sheet

AWS Config Provides

- ✅ Resource inventory
- ✅ Configuration history
- ✅ Configuration timeline
- ✅ Compliance monitoring
- ✅ Config Rules
- ✅ Continuous assessment
- ✅ Multi-account aggregation
- ✅ S3 storage of configuration data
- ✅ SNS notifications

Common Config Rules

- ✅ Restricted SSH access
- ✅ S3 public access checks
- ✅ RDS public access checks
- ✅ Logging checks

Integrations

- ✅ Amazon S3
- ✅ Amazon SNS
- ✅ AWS CloudTrail

Macie Overview

What is Amazon Macie?

Amazon Macie is a fully managed **data security and data privacy service** that uses:

- Machine Learning (ML)
- Pattern Matching

to discover, classify, and protect sensitive data stored in **Amazon S3**.

Exam Tip

If a question asks:

"Which AWS service can identify sensitive data or PII in S3 buckets?"

✅ **Amazon Macie**

What Does Macie Detect?

Macie specializes in finding:

Personally Identifiable Information (PII)

Examples:

- Names
- Email addresses
- Phone numbers
- Passport numbers
- Aadhaar/Social Security numbers
- Credit card numbers

Other Sensitive Data

- Financial information
- Healthcare records
- Credentials
- Confidential business data

How Macie Works

Amazon S3 Bucket

|

v

Amazon Macie

(Machine Learning +

Pattern Matching)

|

v

Sensitive Data Detected

|

v

Findings

|

v

Amazon EventBridge

|

| |

SNS Lambda

Macie scans selected S3 buckets and identifies sensitive content automatically.

Key Features

Sensitive Data Discovery

Automatically locates sensitive information in S3 objects.

Data Classification

Classifies data based on content.

Machine Learning

Detects unusual data patterns and sensitive information.

Security Findings

Generates findings when sensitive data is discovered.

EventBridge Integration

Findings can trigger:

- SNS notifications
- Lambda functions
- Automated remediation workflows

Scope of Macie

Amazon S3 Only

For the AWS Cloud Practitioner exam, remember:

✅ Macie primarily analyzes **Amazon S3 data**

Exam Tip

Macie is **not** a general vulnerability scanner.

It focuses on:

- Data discovery
- Data classification
- Privacy protection

Macie vs Other Security Services

Amazon Macie

Focus:

✅ Sensitive data discovery

✅ PII detection

✅ S3 data classification

Amazon GuardDuty

Focus:

✅ Threat detection

✅ Suspicious activity

✅ Malicious behavior

Amazon Inspector

Focus:

✅ Vulnerability scanning

✅ EC2 assessments

✅ ECR image scanning

✅ Lambda vulnerability checks

Memory Trick

Macie = Finds Sensitive Data

GuardDuty = Finds Threats

Inspector = Finds Vulnerabilities

Typical Exam Scenarios

Scenario 1

A company wants to identify credit card numbers stored in S3 buckets.

✅ Amazon Macie

Scenario 2

An organization needs to discover Personally Identifiable Information (PII) across its S3 environment.

✅ Amazon Macie

Scenario 3

A security team wants automatic classification of sensitive data stored in S3.

✅ Amazon Macie

Scenario 4

Findings about sensitive data should trigger notifications.

✅ Amazon Macie + EventBridge + SNS

Exam Cheat Sheet

Amazon Macie

✅ Scans Amazon S3

✅ Uses Machine Learning

✅ Uses Pattern Matching

✅ Finds PII

✅ Discovers sensitive data

✅ Generates findings

✅ Integrates with EventBridge

✅ Can trigger SNS and Lambda

Security Hub Overview

What is AWS Security Hub?

AWS Security Hub is a centralized security management service that provides a **single dashboard** to view and manage security alerts, compliance status, and findings across multiple AWS accounts and AWS services.

Exam Tip

If a question asks:

"Which AWS service provides a centralized view of security findings across AWS accounts and services?"

✅ **AWS Security Hub**

Why Use Security Hub?

Without Security Hub:

- GuardDuty findings are separate.
- Inspector findings are separate.
- Macie findings are separate.
- Config compliance results are separate.

Security teams must check multiple dashboards.

With Security Hub:

✅ All findings are aggregated into one place.

✅ Centralized visibility.

✅ Easier compliance monitoring.

✅ Faster incident response.

Services Integrated with Security Hub

Security Hub collects findings from many AWS security services.

AWS Native Services

- AWS Config
- Amazon GuardDuty
- Amazon Inspector
- Amazon Macie
- IAM Access Analyzer
- AWS Systems Manager
- AWS Firewall Manager

- AWS Health

Third-Party Security Tools

Security Hub also integrates with AWS Partner solutions.

Exam Tip

Think:

Security Hub = Central Security Dashboard

Key Features

1. Centralized Security Dashboard

Provides a single view of:

- Security alerts
- Vulnerabilities
- Compliance status
- Threat detections

across multiple AWS accounts and regions.

2. Security Findings Aggregation

Collects and normalizes findings from integrated services.

Example:

GuardDuty:

- Suspicious API Call

Inspector:

- Critical Vulnerability

Macie:

- PII Found in S3

Config:

- Non-Compliant Resource

Security Hub:

- Displays all findings together

3. Automated Security Checks

Security Hub automatically evaluates resources against security standards and best practices.

Examples:

- S3 bucket security
- IAM best practices
- Security Group exposure
- Encryption requirements

4. Multi-Account Support

Security Hub works across:

✅ Multiple AWS Accounts

✅ Multiple AWS Regions

Perfect for organizations using AWS Organizations.

Dependency on AWS Config

A very common exam point.

Security Hub Requires AWS Config

Before Security Hub can perform many compliance checks:

✅ AWS Config must be enabled.

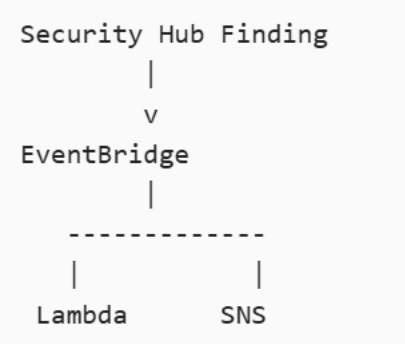
Exam Question

What service must be enabled before Security Hub can perform compliance monitoring?

✅ AWS Config

Integration with EventBridge

Security Hub findings generate events that can be processed automatically.



Use cases:

- Send alerts
- Open tickets
- Trigger automated remediation

Amazon Detective Integration

Security Hub identifies findings.

Amazon Detective helps investigate them.

Workflow

```
Security Hub
     |
     v
Detect Security Issue
     |
     v
Amazon Detective
     |
     v
Investigate Root Cause
```

Exam Tip

Security Hub finds the issue.

Amazon Detective investigates the issue.

Security Hub vs Other Security Services

Security Hub

✅ Aggregates findings

✅ Central dashboard

✅ Compliance visibility

GuardDuty

✅ Threat detection

✅ Suspicious activity analysis

Inspector

✅ Vulnerability scanning

✅ EC2, ECR, Lambda assessment

Macie

✅ Sensitive data discovery

✅ PII detection in S3

Memory Trick

GuardDuty = Threats

Inspector = Vulnerabilities

Macie = Sensitive Data

Security Hub = All Findings in One Place

Architecture Overview

GuardDuty

Inspector

Macie

Config

Firewall Manager

IAM Access Analyzer

Systems Manager

AWS Health

|

v

AWS Security Hub

|

v

Central Dashboard

|

v

EventBridge

|

| |

SNS Lambda

|

v

Amazon Detective

(Investigation)

Typical Exam Scenarios

Scenario 1

A security team wants a single dashboard to view GuardDuty, Inspector, and Macie findings.

✅ AWS Security Hub

Scenario 2

An organization needs centralized security monitoring across multiple AWS accounts.

✅ AWS Security Hub

Scenario 3

A company wants automated security and compliance checks against AWS best practices.

✅ AWS Security Hub

Scenario 4

A security finding should trigger automated remediation through Lambda.

✅ Security Hub + EventBridge + Lambda

Scenario 5

A security analyst wants to investigate the root cause of a finding reported by Security Hub.

✅ Amazon Detective

Exam Cheat Sheet

Security Hub Provides

✅ Centralized security dashboard

✅ Multi-account visibility

✅ Multi-region visibility

- ✅ Security findings aggregation
- ✅ Compliance monitoring
- ✅ Automated security checks
- ✅ EventBridge integration
- ✅ Partner tool integration

Integrates With

- ✅ AWS Config
- ✅ GuardDuty
- ✅ Inspector
- ✅ Macie
- ✅ IAM Access Analyzer
- ✅ Firewall Manager
- ✅ Systems Manager
- ✅ AWS Health
- ✅ Third-party security tools

Related Services

Service	Purpose
Security Hub	Central security dashboard
GuardDuty	Threat detection
Inspector	Vulnerability scanning
Macie	Sensitive data discovery
Detective	Security investigation

Amazon Detective Overview

What is Amazon Detective?

Amazon Detective is a security investigation service that helps security teams quickly **analyze, investigate, and identify the root cause of security findings or suspicious activities** in AWS.

Exam Tip

If a question asks:

"Which AWS service helps investigate the root cause of security findings?"

✅ **Amazon Detective**

Why Do We Need Detective?

Services like:

- Amazon GuardDuty
- Amazon Macie
- AWS Security Hub

can identify security issues and generate findings.

However, they do not always tell you:

- How the incident happened
- Which resources were involved
- What sequence of events occurred
- The root cause

Amazon Detective helps answer these questions.

How Detective Works

Detective automatically collects and analyzes data from:

AWS CloudTrail

Provides:

- API activity
- User activity
- Resource changes

VPC Flow Logs

Provides:

- Network traffic information
- Communication patterns
- Suspicious connections

Amazon GuardDuty

Provides:

- Security findings
- Threat detections
- Anomaly alerts

Key Features

Automated Investigation

No need to manually gather logs from multiple services.

Machine Learning

Uses machine learning to identify relationships and suspicious activities.

Graph-Based Analysis

Creates linked relationships between:

- Users
- IP addresses
- EC2 instances
- AWS resources

to help visualize investigations.

Unified View

Combines data from multiple sources into a single investigation interface.

Example Workflow

GuardDuty

|

v

Suspicious Activity Detected

|

v

Amazon Detective

|

+--> Analyze CloudTrail

+--> Analyze VPC Flow Logs

+--> Analyze Resource Activity

|

v

Root Cause Analysis

Security Investigation Example

A GuardDuty finding reports:

EC2 instance communicating with a malicious IP address

With Detective, you can investigate:

- Which instance was involved
- Which IAM user created it
- What network traffic occurred
- What API calls were executed
- How the compromise happened

Detective vs GuardDuty

Amazon GuardDuty

✅ Detects threats

✅ Generates findings

✅ Identifies suspicious activity

Amazon Detective

✅ Investigates findings

✅ Finds root causes

✅ Provides visual analysis

✅ Helps understand incidents

Memory Trick

GuardDuty = Detects the Problem

Detective = Investigates the Problem

Detective and Security Hub

A common exam architecture:

GuardDuty

Macie

Inspector

|

v

Security Hub

|

v

Amazon Detective

|

v

Root Cause Analysis

- Security Hub aggregates findings.
- Detective investigates them.

Exam Scenarios

Scenario 1

A security analyst wants to determine the root cause of a GuardDuty finding.

✅ Amazon Detective

Scenario 2

A company wants visual investigation tools for suspicious AWS activity.

✅ Amazon Detective

Scenario 3

A team wants to analyze CloudTrail logs, VPC Flow Logs, and GuardDuty findings together.

✅ Amazon Detective

Exam Cheat Sheet

Amazon Detective Uses

- ✅ Machine Learning
- ✅ Graph Analysis
- ✅ Automated Investigation

Data Sources

- ✅ CloudTrail
- ✅ VPC Flow Logs
- ✅ GuardDuty Findings

Purpose

- ✅ Root cause analysis
- ✅ Security investigations
- ✅ Incident analysis
- ✅ Relationship visualization

Amazon Abuse

What is AWS Abuse?

AWS provides a process for reporting resources that are being used for **abusive, malicious, or illegal activities**.

If you suspect that AWS resources are being misused, you can report them to the **AWS Abuse Team**.

Exam Tip

If a question asks:

"Where should a customer report suspected abuse originating from AWS resources?"

✅ **AWS Abuse Team**

Examples of Reportable Abuse

Spam

Examples:

- Unsolicited emails sent from AWS-owned IP addresses
- Email spam campaigns hosted on AWS

✅ Report to AWS Abuse Team

Website or Forum Spam

Examples:

- Automated posting
- Spam comments
- Unwanted advertisements

✅ Report to AWS Abuse Team

Port Scanning

Examples:

- Repeated scanning of network ports
- Attempts to discover open services

✅ Report to AWS Abuse Team

Denial-of-Service (DoS/DDoS) Attacks

Examples:

- Malicious traffic originating from AWS resources

- Attempts to overwhelm systems

✅ Report to AWS Abuse Team

Intrusion Attempts

Examples:

- Unauthorized login attempts
- Brute-force attacks
- Unauthorized access attempts

✅ Report to AWS Abuse Team

Illegal or Copyrighted Content

Examples:

- Pirated software
- Copyright-infringing content
- Illegal hosted material

✅ Report to AWS Abuse Team

Malware Distribution

Examples:

- Virus hosting
- Trojan distribution
- Malicious software downloads

✅ Report to AWS Abuse Team

How to Report Abuse

Option 1: AWS Abuse Reporting Form

Use the online AWS Abuse Reporting process.

Option 2: Email

abuse@amazonaws.com

Exam Scenarios

Scenario 1

A company discovers that an AWS IP address is sending spam emails.

✅ Contact the AWS Abuse Team

Scenario 2

Security engineers detect a DDoS attack originating from AWS resources.

✅ Report to the AWS Abuse Team

Scenario 3

Malware is being distributed through a workload hosted on AWS.

✅ Report to the AWS Abuse Team

Scenario 4

A website hosted on AWS contains illegal copyrighted content.

✅ Contact the AWS Abuse Team

Exam Cheat Sheet

Report These Activities

✅ Spam emails

✅ Website/forum spam

✅ Port scanning

✅ Intrusion attempts

✅ DoS attacks

✅ DDoS attacks

✅ Malware distribution

✅ Illegal/copyrighted content

Contact

✅ AWS Abuse Team

✅ abuse@amazonaws.com

Root User Privileges

What is the AWS Root User?

The **Root User** is the account owner created when an AWS account is first set up.

Characteristics:

- Has **full access** to all AWS services and resources.
- Cannot have permissions restricted by IAM policies.
- Has privileges that even an Administrator IAM user cannot perform.

Exam Tip

Root User = Highest privilege account in AWS

Best Practice

✅ Create an **IAM Administrator User** for daily administration.

✅ Secure the Root User with **MFA**.

✅ Do **not** use the Root User for everyday tasks.

✅ Do **not** use Root User access keys.

Exam Tip

AWS recommends using the Root User **only for tasks that require root privileges**.

Root User-Only Tasks

These are actions that **only the Root User** can perform.

1. Change Account Settings

Only the Root User can:

- Change account name
- Change account email address
- Change root password
- Modify root access keys

✅ Root User required

2. View Certain Tax Invoices

Access to some billing and tax-related documents requires:

✅ Root User

3. Close an AWS Account

If you want to permanently close an AWS account:

✅ Root User required

4. Restore IAM User Permissions

If administrators accidentally remove IAM permissions and lock themselves out:

✅ Root User can recover access

5. Change or Cancel AWS Support Plans

Examples:

- Basic Support
- Developer Support
- Business Support
- Enterprise Support

✅ Root User required

6. Register as a Seller in the Reserved Instance Marketplace

Example:

You purchased a Reserved Instance for:

3 years

After some time you no longer need it.

To sell it in the Reserved Instance Marketplace:

✅ Root User required

Exam Tip

This is a commonly tested root-user privilege.

7. Configure Amazon S3 MFA Delete

MFA Delete adds extra protection before objects can be deleted.

Only:

✅ Root User can enable or disable MFA Delete.

Exam Tip

MFA Delete on S3 → Root User

This is one of the most common exam questions.

8. Modify Certain S3 Bucket Policies

When an S3 bucket policy references an invalid:

- VPC ID
- VPC Endpoint ID

Only:

✅ Root User can edit or delete the bucket policy.

9. Sign Up for AWS GovCloud

AWS GovCloud registration requires:

✅ Root User

Important Exam Items to Memorize

Root User Required

✅ Change account settings

✅ Close AWS account

✅ Restore IAM permissions

✅ Change support plan

✅ Register Reserved Instance Marketplace seller

✅ Enable S3 MFA Delete

✅ Edit problematic S3 bucket policies

✅ Sign up for GovCloud

Root User vs IAM Administrator

Task	Root User	Admin IAM User
Manage EC2	✅	✅
Manage S3	✅	✅
Create IAM users	✅	✅
Change account email	✅	❌
Close AWS account	✅	❌
Enable S3 MFA Delete	✅	❌
Change support plan	✅	❌
Register RI Marketplace seller	✅	❌

Memory Trick

Think:

Account-level actions = Root User

Examples:

- Account settings
- Account closure
- Support plan changes

Think:

Special S3 action = Root User

- MFA Delete

Common Exam Questions

Which account can enable S3 MFA Delete?

✅ Root User

Which account can close an AWS account?

✅ Root User

Which account can change the AWS account email address?

✅ Root User

Which account can register as a seller in the Reserved Instance Marketplace?

✅ Root User

A company locks out all administrators. Who can restore access?

✅ Root User

IAM Access Analyzer

What is IAM Access Analyzer?

IAM Access Analyzer is an AWS security service that helps identify resources that are **shared externally**.

It analyzes resource policies and alerts you when resources can be accessed from outside your defined **Zone of Trust**.

Exam Tip

If a question asks:

"Which AWS service identifies resources shared with external accounts?"

✅ **IAM Access Analyzer**

Why Use IAM Access Analyzer?

Sometimes resources are accidentally exposed through:

- Resource policies
- Cross-account access
- Public access permissions

These exposures can create security risks.

IAM Access Analyzer helps detect them automatically.

Resources Analyzed

IAM Access Analyzer can identify external access to:

✅ Amazon S3 Buckets

✅ IAM Roles

✅ AWS KMS Keys

✅ AWS Lambda Functions

✅ AWS Lambda Layers

✅ Amazon SQS Queues

✅ AWS Secrets Manager Secrets

Exam Tip

Remember that Access Analyzer focuses on **resource sharing and external access**, not vulnerabilities or threats.

Zone of Trust

Before creating an analyzer, you define a **Zone of Trust**.

The zone can be:

Single AWS Account

My Account

or

Entire AWS Organization

AWS Organization

├– Account A

├– Account B

└– Account C

Resources shared outside this trusted boundary generate findings.

How It Works

Example

An S3 bucket grants access to:

My IAM User

My IAM Role

External AWS Account

Public Users

If the Zone of Trust is:

My AWS Account

Then:

✅ IAM User → Trusted

✅ IAM Role → Trusted

❌ External Account → Finding

❌ Public Access → Finding

IAM Access Analyzer reports the external access.

Findings

When external access is detected, Access Analyzer creates a **Finding**.

Example finding:

S3 Bucket

Shared with External Account

or

SQS Queue

Accessible by Everyone

Finding States

Active

External access currently exists.

Archived

Finding is intentionally ignored because access is expected.

Example:

Public Website Bucket

Public access is intentional.

Archive the finding.

Resolved

Access has been removed.

Example:

Public Policy Removed

Access Analyzer automatically marks the finding as resolved.

Example: Public SQS Queue

Bad Policy

Principal: *

Action: SendMessage

This allows anyone to send messages.

IAM Access Analyzer reports:

❌ External Access Detected

After removing the public policy:

✅ Finding becomes Resolved

Archive Rules

You can create automatic rules to archive findings.

Example:

Automatically archive

public website buckets

This reduces noise from expected access patterns.

Cost

✅ IAM Access Analyzer is available at no additional charge for basic external access analysis.

Exam Tip

A common exam statement is:

IAM Access Analyzer helps identify resources shared outside your AWS account or organization.

✅ Correct

IAM Access Analyzer vs Other Security Services

IAM Access Analyzer

Focus:

✅ External access

✅ Resource sharing

✅ Cross-account permissions

AWS Config

Focus:

✅ Configuration tracking

✅ Compliance monitoring

Amazon GuardDuty

Focus:

✅ Threat detection

✅ Suspicious activity

Amazon Inspector

Focus:

✅ Vulnerability scanning

Memory Trick

Service	Purpose
Access Analyzer	Who can access my resources?
Config	How did my resources change?
GuardDuty	Is someone attacking me?
Inspector	Do I have vulnerabilities?

Typical Exam Scenarios

Scenario 1

A company wants to identify S3 buckets shared with external AWS accounts.

✅ IAM Access Analyzer

Scenario 2

A security team wants to find KMS keys accessible outside the organization.

✅ IAM Access Analyzer

Scenario 3

An administrator needs to discover unexpectedly public SQS queues.

✅ IAM Access Analyzer

Scenario 4

A company wants to find resources shared outside its AWS Organization.

✅ IAM Access Analyzer

Exam Cheat Sheet

IAM Access Analyzer

✅ Identifies external access

✅ Analyzes resource policies

✅ Works with AWS Organizations

✅ Creates findings

✅ Supports archive rules

✅ Helps discover public resources

Resources Supported

- ✅ S3 Buckets
- ✅ IAM Roles
- ✅ KMS Keys
- ✅ Lambda Functions
- ✅ Lambda Layers
- ✅ SQS Queues
- ✅ Secrets Manager Secrets

Security & Compliance Summary

- **Shared Responsibility Model**: AWS secures the cloud; customers secure what they put in the cloud.
- **AWS Shield**: DDoS protection. Shield Standard is free; Shield Advanced adds enhanced protection and 24/7 support.
- **AWS WAF**: Protects web applications by filtering HTTP/HTTPS requests (SQL injection, XSS, rate limiting).
- **AWS KMS**: Manages encryption keys.
- **AWS CloudHSM**: Dedicated hardware security modules where **you control the keys**.
- **AWS ACM (Certificate Manager)**: Provides and renews SSL/TLS certificates for HTTPS.
- **AWS Secrets Manager**: Securely stores secrets and automatically rotates passwords (especially for RDS).
- **AWS Artifact**: Download compliance reports (SOC, PCI, ISO) and agreements (HIPAA/BAA).

Threat Detection & Security Monitoring

- **GuardDuty**: Detects threats using CloudTrail, VPC Flow Logs, and DNS logs.
- **Inspector**: Finds vulnerabilities in EC2, ECR container images, and Lambda functions.
- **Macie**: Discovers sensitive data (PII) in S3 buckets.
- **Security Hub**: Central dashboard that aggregates findings from GuardDuty, Inspector, Macie, Config, etc.
- **Detective**: Investigates security findings and helps determine root causes.

Compliance & Governance

- **AWS Config**: Tracks resource configurations and compliance over time.
- **IAM Access Analyzer**: Identifies resources shared outside your account or organization.
- **Firewall Manager**: Centrally manages Security Groups, WAF, Shield, and Network Firewall policies across AWS Organizations.
- **AWS Network Firewall**: Protects an entire VPC with Layer 3–7 traffic inspection.

Other Important Topics

- **AWS Abuse Team**: Report spam, malware, DDoS attacks, port scanning, or illegal content hosted on AWS.
- **Root User Only Tasks**:
    - Change account settings
    - Close AWS account
    - Change/cancel support plan
    - Register as a seller in the Reserved Instance Marketplace

- Enable S3 MFA Delete

Exam Memory Tricks

- **GuardDuty → Threats**
- **Inspector → Vulnerabilities**
- **Macie → Sensitive Data**
- **Security Hub → Central Dashboard**
- **Detective → Investigation**
- **Config → Compliance**
- **Access Analyzer → External Access**
- **Firewall Manager → Organization-wide Security**
- **KMS → Encryption Keys**
- **ACM → HTTPS Certificates**

Machine Learning

Recognition Overview

Amazon Rekognition is an AWS machine learning service that analyzes **images and videos** to identify and recognize:

- Objects
- People
- Text
- Activities
- Scenes
- Faces

You only need a **high-level understanding** of this service for AWS certification exams.

Key Features

1. Object and Scene Detection

Rekognition can identify items and environments in images, such as:

- Person
- Bicycle
- Dog
- Mountain
- Outdoors

2. Facial Analysis

It can analyze facial attributes, including:

- Gender
- Age range
- Emotions (happy, sad, etc.)
- Whether eyes are open/closed
- Whether a person is smiling

3. Face Search and Verification

Useful for:

- Identity verification
- Access control systems
- Creating a database of known faces
- Matching faces against stored images

4. Content Moderation

Automatically detects inappropriate or unsafe content in images and videos.

5. Text Detection (OCR)

Extracts text from images, such as:

- Race bib numbers
- Signs
- License plates
- Documents

6. Celebrity Recognition

Can identify well-known public figures from images or videos.

7. People Pathing

Tracks movement of people through video footage, useful for:

- Sports analytics
- Crowd monitoring
- Security applications

Common Exam Use Cases

Remember these keywords:

- **Image and video analysis**
- **Labeling objects**
- **Content moderation**
- **Text detection (OCR)**
- **Face detection and analysis**
- **Face search and verification**
- **Celebrity recognition**
- **People tracking/pathing**

Transcribe Overview

Amazon Transcribe is an AWS machine learning service that **automatically converts speech into text**.

You provide an audio or video recording, and Transcribe generates a text transcript using **Automatic Speech Recognition (ASR)** technology.

What It Does

Input: Audio/voice recordings

Output: Written text

Example:

Audio:

"Hello, my name is Stephane and I hope you're enjoying the course."

Transcription:

Hello, my name is Stephane and I hope you're enjoying the course.

Key Features

1. Automatic Speech Recognition (ASR)

Amazon Transcribe uses **deep learning-based ASR** to convert speech into text quickly and accurately.

2. Personally Identifiable Information (PII) Redaction

Transcribe can automatically detect and remove sensitive information such as:

- Names
- Phone numbers
- Email addresses
- Credit card numbers
- Social Security numbers

Example:

Original Speech:

"My name is John Smith and my phone number is 123-456-7890."

Transcript with PII Redaction:

"My name is [PII REDACTED] and my phone number is [PII REDACTED]."

This is useful for compliance and privacy requirements.

3. Automatic Language Identification

Transcribe can automatically recognize multiple languages spoken in an audio stream.

For example:

- English
- French
- Spanish
- Other supported languages

Useful for multilingual customer support and international applications.

4. Real-Time Streaming Transcription

Transcribe can process audio as it is being spoken and generate text in real time.

Example:

- Live meetings
- Call centers
- Broadcast captioning

Common Use Cases

Customer Service Call Transcription

Convert customer support conversations into searchable text.

Closed Captioning and Subtitles

Automatically generate captions for:

- Videos
- Webinars
- Training materials

Searchable Media Archives

Create transcripts that make audio and video content searchable.

Meeting Notes

Generate written records of meetings and interviews.

Compliance and Monitoring

Store and analyze conversations while protecting sensitive information through PII redaction.

Exam Keywords

When you see requirements such as:

- Convert speech to text
- Audio transcription
- Call center transcription
- Closed caption generation
- Subtitle generation
- Real-time speech recognition
- PII redaction in audio
- Language identification

➡️ **Amazon Transcribe** is usually the correct AWS service.

Polly Overview

Amazon Polly is the opposite of **Amazon Transcribe**.

- **Amazon Transcribe:** Speech ➜ Text
- **Amazon Polly:** Text ➜ Speech

Polly uses **deep learning** to convert written text into natural-sounding speech, allowing applications to "talk" to users.

What It Does

Input: Text

Output: Audio (spoken speech)

Example:

Text:

"Hello, my name is Stephane and this is a demo of Amazon Polly."

Output: A generated audio file that speaks the text aloud.

Key Features

1. Text-to-Speech (TTS)

Convert text into realistic human speech.

Use cases:

- Voice assistants
- Interactive applications
- Accessibility tools
- Automated announcements

2. Multiple Voices and Languages

Polly supports:

- Multiple languages
- Different accents
- Male and female voices

This allows applications to speak naturally to users around the world.

3. Neural Voices

Uses advanced deep learning models to generate more natural-sounding speech.

Benefits:

- More human-like pronunciation
- Better intonation
- Improved listening experience

4. Standard Voices

Also offers standard synthesized voices that sound more robotic but can be less expensive.

Common Use Cases

Accessibility

Read website content aloud for visually impaired users.

Voice Assistants

Power chatbot and virtual assistant speech.

E-Learning

Convert educational content into audio lessons.

Automated Notifications

Generate spoken announcements and alerts.

Audiobooks

Convert written text into spoken audio.

Exam Keywords

Look for phrases such as:

- Convert text to speech
- Generate audio from text
- Natural-sounding voice generation
- Voice-enabled application
- Read content aloud
- Speech synthesis

➡️ The correct AWS service is **Amazon Polly**.

Cloud Practitioner

Translate Overview

Amazon Translate is a neural machine translation service that **automatically translates text from one language to another**.

It helps businesses localize content for global users without needing human translators for every translation task.

What It Does

Input: Text in one language

Output: Translated text in another language

Example:

English	French	Portuguese	Hindi
Hi, my name is Stephen.	Salut, je m'appelle Stephen.	Olá, meu nome é Stephen.	नमस्ते, मेरा नाम स्टीफन है।

Key Features

1. Language Translation

Translate text between supported languages quickly and accurately using machine learning.

2. Content Localization

Translate:

- Websites
- Mobile applications
- Product descriptions
- User-generated content
- Documents

for international audiences.

3. Large-Scale Translation

Efficiently translate large volumes of text automatically.

4. Neural Machine Translation

Uses advanced deep learning models to produce more natural and context-aware translations than traditional rule-based systems.

Common Use Cases

Website Localization

Offer your website in multiple languages.

Application Internationalization

Translate app content for users worldwide.

Customer Support

Translate support messages and communications.

Content Publishing

Translate articles, blogs, and documentation.

Global E-Commerce

Translate product catalogs and descriptions.

Exam Keywords

If a question mentions:

- Translate text between languages
- Website localization
- Application localization
- Multilingual content
- International users
- Language translation service

➡️ The correct answer is **Amazon Translate**.

Lex & Connect Overview

Amazon Lex

Amazon Lex is the service behind **Amazon Alexa**.

It combines:

- **Automatic Speech Recognition (ASR)** → Converts speech into text.
- **Natural Language Understanding (NLU)** → Understands the meaning and intent of what a user says.

What Lex Does

A user might say:

"I'd like to book an appointment for tomorrow at 3 PM."

Lex can:

1. Convert the speech to text (ASR).
2. Understand the intent ("schedule appointment") using NLU.
3. Extract important information such as date and time.
4. Trigger backend actions (often through AWS Lambda).

Common Use Cases

- Chatbots
- Virtual assistants
- Customer service bots
- Interactive voice response (IVR) systems
- Call center automation

Exam Keywords

Look for:

- Chatbot
- Conversational interface
- Voice bot
- Speech recognition
- Natural language understanding
- Alexa technology

➡️ **Amazon Lex** is usually the correct answer.

Amazon Connect

Amazon Connect is a **cloud-based contact center (call center) service**.

It lets companies:

- Receive customer calls
- Design call flows visually
- Route calls to agents
- Integrate with AWS services
- Integrate with CRM systems

Key Features

- Fully managed contact center
- Visual workflow/contact flow builder
- No infrastructure to manage
- Pay-as-you-go pricing
- Integrates with Lex for intelligent conversations

Common Use Cases

- Customer support centers
- Help desks
- Appointment scheduling
- Sales hotlines
- Automated customer service systems

Exam Keywords

Look for:

- Contact center
- Call center
- Customer support phone system
- Call routing
- Interactive voice response (IVR)

➡️ **Amazon Connect** is usually the correct answer.

How Lex and Connect Work Together

A typical workflow:

1. Customer calls a phone number.
2. **Amazon Connect** receives the call.
3. **Amazon Lex** listens to the conversation.
4. Lex identifies the customer's intent.
5. Lex invokes an AWS Lambda function if needed.
6. The Lambda function performs an action:
    - Schedule a meeting
    - Check an order status
    - Retrieve customer information
    - Update a CRM system
7. Connect responds to the customer.

Example

Customer says:

"Schedule a meeting with Tom tomorrow at 3 PM."

- **Connect** receives the call.
- **Lex** understands the request.
- **Lambda** creates the calendar event.
- Customer receives confirmation.

Quick Comparison

Service	Purpose
Amazon Lex	Build chatbots and voice assistants using ASR and NLU
Amazon Connect	Build cloud-based contact centers/call centers

Comprehend Overview

Amazon Comprehend is AWS's **Natural Language Processing (NLP)** service.

Exam shortcut: If you see **NLP**, think **Amazon Comprehend**.

It uses machine learning to analyze and understand text automatically.

What Is NLP?

Natural Language Processing (NLP) is the ability for computers to understand human language.

Amazon Comprehend helps extract meaning and insights from unstructured text such as:

- Emails
- Customer reviews
- Support tickets
- Social media posts
- Articles
- Documents

Key Features

1. Language Detection

Identifies the language of a text.

Example:

- English
- French
- Spanish
- Hindi

2. Entity Recognition

Extracts important entities from text, such as:

- People
- Organizations
- Locations
- Brands
- Events
- Dates

Example:

"John visited Paris for the AWS Summit."

Comprehend can identify:

- Person: John
- Location: Paris
- Event: AWS Summit

3. Key Phrase Extraction

Identifies important phrases within text.

Example:

"I purchased a new laptop with excellent battery life."

Key phrases:

- new laptop
- excellent battery life

4. Sentiment Analysis

Determines whether text expresses:

- Positive sentiment
- Negative sentiment
- Neutral sentiment
- Mixed sentiment

Example:

"The product is amazing and easy to use."

Result:

- Positive

This is commonly used for customer feedback analysis.

5. Syntax Analysis

Analyzes text structure using:

- Tokenization
- Parts of speech

Helps applications better understand the grammar and meaning of text.

6. Topic Modeling

Automatically discovers topics within large collections of documents.

Example:

Thousands of articles could be grouped into topics such as:

- Technology
- Sports
- Finance
- Healthcare

without manually reviewing each article.

Common Use Cases

Customer Feedback Analysis

Analyze:

- Support emails
- Customer reviews
- Survey responses

to understand customer satisfaction and trends.

Social Media Monitoring

Track customer sentiment about:

- Products
- Services
- Brands

Document Classification

Organize large numbers of documents automatically.

Content Discovery

Group articles and documents by topics.

Exam Keywords

Look for phrases like:

- Natural Language Processing (NLP)
- Analyze text
- Extract entities
- Detect language
- Sentiment analysis
- Topic modeling
- Understand customer feedback
- Find key phrases

➡️ **Amazon Comprehend** is usually the correct answer.

SageMaker AI overview

Amazon SageMaker is a fully managed AWS service that helps **developers and data scientists build, train, tune, and deploy machine learning (ML) models**.

Unlike services such as:

- **Transcribe** (Speech → Text)
- **Polly** (Text → Speech)
- **Translate** (Language Translation)
- **Comprehend** (NLP)

SageMaker is a **general-purpose machine learning platform** used to create custom ML models.

Why SageMaker?

Building ML models normally requires:

- Collecting data
- Preparing and labeling data
- Choosing algorithms
- Training models
- Tuning models
- Deploying models
- Managing infrastructure

SageMaker simplifies this entire process.

Machine Learning Workflow with SageMaker

1. Gather Data

Collect historical data that will be used for predictions.

Example:

To predict exam scores, collect:

- Years of IT experience
- Years of AWS experience
- Course completion percentage
- Practice exam scores
- Actual exam results

2. Label Data

Labeling means providing the correct answers (expected outcomes) for training.

Example:

Experience	Practice Exams	Actual Score
1 year	2	670
3 years	5	850
5 years	8	950

The **Actual Score** becomes the label.

3. Build the Model

Create a machine learning model that learns patterns from the historical data.

The model tries to understand relationships such as:

More practice exams + more AWS experience = higher probability of passing.

4. Train and Tune the Model

Training:

- The model learns from historical data.

Tuning:

- Improve accuracy by adjusting parameters and refining the model.

This is often one of the most computationally intensive parts of ML.

5. Deploy the Model

Once trained, deploy the model so it can make predictions on new data.

Example:

A new student provides:

- 4 years IT experience
- 2 years AWS experience
- 6 practice exams completed

The model predicts:

Expected exam score: 906

This process is called **inference** or **prediction**.

What SageMaker Helps With

SageMaker supports the entire ML lifecycle:

✅ Data preparation

✅ Data labeling

- ✅ Model building
- ✅ Model training
- ✅ Hyperparameter tuning
- ✅ Model deployment
- ✅ Model hosting
- ✅ Predictions and inference

Common Use Cases

Predictive Analytics

- Sales forecasting
- Demand forecasting
- Exam score prediction

Fraud Detection

- Detect suspicious transactions

Recommendation Systems

- Product recommendations
- Movie recommendations

Customer Behavior Analysis

- Churn prediction
- Customer segmentation

Computer Vision & NLP Models

- Create custom image recognition models
- Create custom text analysis models

Exam Keywords

Look for phrases such as:

- Build machine learning models
- Train ML models
- Deploy ML models
- Data scientists
- Machine learning platform
- Predictive analytics
- Custom machine learning solution

➡️ **Amazon SageMaker** is usually the correct answer.

Kendra Overview

Amazon Kendra is a **fully managed, machine learning-powered intelligent search service**.

Its purpose is to help users **search across large collections of documents and find precise answers using natural language queries**.

Exam shortcut: If you see **enterprise document search** or **intelligent search**, think **Amazon Kendra**.

What Does Kendra Do?

Organizations often store information in:

- PDF files
- Microsoft Word documents
- PowerPoint presentations
- HTML pages
- FAQs
- Knowledge bases
- SharePoint repositories
- File systems

Kendra connects to these data sources, indexes the content, and builds a searchable knowledge base.

Natural Language Search

Instead of searching with keywords, users can ask questions in plain English.

Example:

User asks:

"Where is the IT support desk?"

Kendra searches across all indexed documents and returns:

"The IT support desk is located on the 1st floor."

This is much more powerful than traditional keyword-based search.

Key Features

1. Intelligent Document Search

Search through:

- PDFs
- Word documents
- PowerPoint files

- Web pages
- FAQs
- Corporate knowledge repositories

2. Natural Language Queries

Users can ask questions conversationally.

Example:

"How do I reset my company password?"

instead of searching with keywords like:

password reset procedure

3. Machine Learning-Powered Results

Kendra understands context and relevance to provide more accurate answers than traditional search engines.

4. Incremental Learning

Kendra improves search relevance over time by learning from:

- User interactions
- Click behavior
- Feedback

This helps promote the most useful results.

5. Result Customization

Search results can be refined based on:

- Document importance
- Freshness of content
- Custom ranking rules
- Business-specific filters

Common Use Cases

Internal Knowledge Search

Employees can search company documentation and policies.

IT Help Desk

Find answers from technical documentation and FAQs.

Customer Support Portals

Search product manuals and support content.

Enterprise Knowledge Management

Enable quick access to information scattered across many repositories.

Exam Keywords

Look for:

- Enterprise search
- Intelligent search
- Document search
- Search across PDFs and documents
- Knowledge base search
- Natural language search
- Find answers from documents

➡️ **Amazon Kendra** is usually the correct answer.

Amazon Kendra vs Amazon Comprehend

Service	Purpose
Amazon Comprehend	Analyze and understand text (NLP, sentiment, entities, key phrases)
Amazon Kendra	Search and retrieve answers from documents and knowledge bases

Personalize Overview

Amazon Personalize is a fully managed machine learning service that enables developers to build **real-time personalized recommendations** without needing deep machine learning expertise.

Exam shortcut: If you see **recommendation engine** or **personalized recommendations**, think **Amazon Personalize**.

What Does Amazon Personalize Do?

It analyzes user behavior and generates recommendations such as:

- Products to buy
- Movies to watch
- Music to listen to
- Articles to read
- Content to display
- Marketing recommendations

This is the same recommendation technology used by **Amazon.com**.

Example

Suppose a customer purchases:

- Gardening gloves
- Watering cans
- Plant fertilizer

Amazon Personalize might recommend:

- Garden tools
- Flower seeds
- Lawn equipment

based on:

- Purchase history
- Browsing behavior
- User preferences
- Similar users' behavior

Key Features

1. Personalized Recommendations

Provides recommendations tailored to each user.

Examples:

- "Customers like you also bought..."
- "Recommended for you"
- "You may also like"

2. Real-Time Personalization

Recommendations can change instantly based on:

- New purchases
- Recent clicks
- Current browsing activity

3. No ML Expertise Required

Unlike SageMaker, you do **not** need to:

- Build ML algorithms
- Train models manually
- Tune hyperparameters
- Manage ML infrastructure

Amazon Personalize handles the complexity for you.

4. Easy Data Integration

Data can come from:

- Amazon S3
- Application events
- User interactions
- Real-time APIs

5. Multi-Channel Personalization

Recommendations can be delivered through:

- Websites
- Mobile apps
- Emails

- SMS campaigns
- Marketing systems

Common Use Cases

Retail & E-Commerce

Recommend:

- Products
- Accessories
- Related purchases

Example:

"Frequently bought together"

Media & Entertainment

Recommend:

- Movies
- TV shows
- Music
- Podcasts

Example:

"Because you watched..."

Personalized Marketing

Customize:

- Emails
- Promotions
- Advertisements
- Offers

for each customer.

Amazon Personalize vs SageMaker

Service	Purpose
Amazon SageMaker	Build, train, tune, and deploy custom ML models
Amazon Personalize	Ready-made ML service for recommendations and personalization

SageMaker is for creating your own ML models.

Personalize is a pre-built recommendation service.

Exam Keywords

Look for:

- Recommendation engine
- Product recommendations
- Personalized recommendations
- Content recommendations
- Real-time personalization
- "Customers also bought"
- "Recommended for you"
- User preferences

➡️ **Amazon Personalize** is usually the correct answer.

Textract Overview

Amazon Textract is a machine learning service that automatically **extracts text, handwriting, and structured data** from scanned documents.

Exam shortcut: If you see **extract data from documents, forms, invoices, PDFs, or scanned images**, think **Amazon Textract**.

What Does Textract Do?

Textract can read:

- Scanned documents
- PDFs
- Images
- Forms
- Tables
- Handwritten text

and convert them into structured, machine-readable data.

Example

Input:

- Driver's License
- Passport
- Invoice
- Tax Form

Output:

Plain Text

Name: John Smith

Date of Birth: 01/15/1990

Document ID: D12345678

Show more lines

Instead of just returning raw text, Textract understands the document structure.

Key Features

1. Text Extraction (OCR)

Extracts printed text from documents and images.

Example:

A scanned PDF can be converted into searchable text.

2. Handwriting Recognition

Can also recognize handwritten content in many documents.

3. Forms Extraction

Automatically identifies:

- Keys
- Values
- Form fields

Example:

Name: John Smith

Date of Birth: 01/15/1990

Textract understands that "Name" is associated with "John Smith."

4. Table Extraction

Extracts information from tables while preserving rows and columns.

Example:

Item	Quantity	Price
Laptop	2	$1000

5. Structured Data Output

Instead of producing a block of text, Textract returns structured data that applications can easily process.

Common Use Cases

Financial Services

- Invoice processing
- Financial reports
- Expense management
- Loan applications

Healthcare

- Medical records
- Insurance claims
- Patient forms

Government & Public Sector

- Tax forms
- Passports
- Driver licenses
- Identity documents

Business Automation

- Document digitization
- Workflow automation
- Data entry replacement

Textract vs Rekognition

Service	Purpose
Amazon Rekognition	Analyze images and videos (faces, objects, celebrities, activities)
Amazon Textract	Extract text and data from documents

Textract vs Comprehend

Service	Purpose
Amazon Textract	Extract text from documents
Amazon Comprehend	Understand and analyze the meaning of text

A common workflow is:

Document → Textract → Text → Comprehend → Insights

Exam Keywords

Look for phrases such as:

- OCR
- Extract text from documents
- Extract text from PDFs
- Read forms and tables
- Digitize documents
- Process invoices
- Scan identity documents
- Handwriting recognition

➡️ **Amazon Textract** is usually the correct answer.

AWS AI Services Quick Review

Service	Purpose
Amazon Rekognition	Image and video analysis
Amazon Transcribe	Speech → Text
Amazon Polly	Text → Speech
Amazon Translate	Language translation
Amazon Lex	Chatbots and voice interfaces
Amazon Connect	Cloud contact centers
Amazon Comprehend	NLP and text analysis
Amazon SageMaker	Build custom ML models
Amazon Kendra	Intelligent document search
Amazon Personalize	Personalized recommendations
Amazon Textract	Extract text and data from documents

Machine Learning Summary

1. Amazon Rekognition 👁️

Purpose: Image and video analysis

Use Cases:

- Face detection
- Face verification
- Celebrity recognition
- Object and scene detection
- Content moderation
- Text detection in images
- People tracking (pathing)

Keyword: *Analyze images/videos*

2. Amazon Transcribe 🎙️➡️📝

Purpose: Convert speech to text

Features:

- Automatic Speech Recognition (ASR)
- Real-time transcription
- Subtitle generation
- PII redaction
- Language identification

Keyword: *Speech → Text*

3. Amazon Polly 📝➡️🔊

Purpose: Convert text to speech

Use Cases:

- Voice assistants
- Audiobooks
- Accessibility applications
- Automated announcements

Keyword: *Text → Speech*

4. Amazon Translate 🌎

Purpose: Language translation

Use Cases:

- Website localization
- Application localization
- Multilingual content

Keyword: *Translate text between languages*

5. Amazon Lex 🤖

Purpose: Build chatbots and conversational interfaces

Technologies:

- ASR (Automatic Speech Recognition)
- NLU (Natural Language Understanding)

Use Cases:

- Chatbots
- Voice bots
- Virtual assistants

Keyword: *Chatbots*

6. Amazon Connect ☎️

Purpose: Cloud-based contact center

Use Cases:

- Customer service centers
- Call routing
- IVR systems
- Automated customer support

Keyword: *Call center*

7. Amazon Comprehend 📄🧠

Purpose: Natural Language Processing (NLP)

Capabilities:

- Sentiment analysis
- Entity recognition
- Language detection

- Key phrase extraction
- Topic modeling

Keyword: *NLP*

8. Amazon SageMaker 🧪

Purpose: Build, train, tune, and deploy machine learning models

Used By:

- Developers
- Data Scientists

Keyword: *Custom Machine Learning*

9. Amazon Kendra 🔍

Purpose: Intelligent document search

Use Cases:

- Search PDFs
- Search FAQs
- Search Word/PowerPoint documents
- Enterprise knowledge search

Keyword: *Document Search Engine*

10. Amazon Personalize 🎯

Purpose: Personalized recommendations

Use Cases:

- Product recommendations
- Movie recommendations
- Customized marketing
- Content recommendations

Keyword: *Recommendation Engine*

11. Amazon Textract 📑

Purpose: Extract text and structured data from documents

Capabilities:

- OCR
- Forms extraction

- Table extraction
- Handwriting recognition

Use Cases:

- Invoice processing
- Passport processing
- Tax forms
- Medical records

Keyword: *Extract text from documents*

Quick Memory Table

Service	Remember As
Rekognition	Image/Video Analysis
Transcribe	Speech → Text
Polly	Text → Speech
Translate	Translation
Lex	Chatbots
Connect	Call Center
Comprehend	NLP
SageMaker	Build ML Models
Kendra	Document Search
Personalize	Recommendations
Textract	OCR / Document Extraction

Ultimate Exam Mnemonic

"See, Hear, Speak, Translate, Chat, Call, Understand, Build, Search, Recommend, Extract"

- **See** → Rekognition
- **Hear** → Transcribe
- **Speak** → Polly
- **Translate** → Translate
- **Chat** → Lex
- **Call** → Connect
- **Understand** → Comprehend
- **Build** → SageMaker
- **Search** → Kendra
- **Recommend** → Personalize
- **Extract** → Textract

This summary covers virtually all ML services typically tested in the AWS Certified Cloud Practitioner exam. ✅

Cloud Practitioner

Account Management, Billing & Support

Organization Overview

What is AWS Organizations?

- **Global AWS service** used to manage **multiple AWS accounts** centrally.
- Structure:
    - **Management/Master Account** (payer account)
    - **Member/Child Accounts**

Benefits of AWS Organizations

1. Consolidated Billing

- One bill for all accounts.
- The **Master Account pays** for all member accounts.
- No need to configure separate payment methods for each account.

2. Aggregated Usage Discounts

- Usage from all accounts is combined.
- Helps achieve higher usage tiers and better pricing for services like:
    - Amazon EC2
    - Amazon S3

3. Reserved Instance Sharing

- Reserved Instance (RI) discounts can be shared across accounts.
- Maximizes cost savings.

4. Automated Account Creation

- AWS Organizations provides APIs to create accounts programmatically.
- Useful for automatically creating:
    - Sandbox accounts
    - Project accounts
    - Department accounts

5. Central Governance

- Use **Service Control Policies (SCPs)** to restrict permissions across accounts.

Multi-Account Strategy

Organizations often create separate accounts for:

By Department

- Sales
- Finance
- Retail
- HR

By Environment

- Development
- Test
- Production

By Project

- Project A
- Project B
- Project C

Other Reasons

- Regulatory compliance
- Better resource isolation
- Separate service quotas/limits
- Dedicated logging accounts

Organizational Units (OUs)

An **OU (Organizational Unit)** is a logical grouping of accounts.

Example Structure:

```
Root
|
├── Management Account
|
├── Dev OU
|   ├── Dev Account 1
|   └── Dev Account 2
|
└── Prod OU
    ├── Prod Account 1
    ├── Prod Account 2
    |
    ├── Finance OU
    |   └── Finance Account
    |
    └── HR OU
        └── HR Account
```

Service Control Policies (SCPs)

What are SCPs?

Policies used to control the maximum permissions available to accounts within an organization.

Key Exam Points

✅ Applied at:

- OU level
- Account level

✅ Affect:

- IAM Users
- IAM Roles
- Root User of member accounts

❌ Do NOT affect:

- Management (Master) Account
- Service-linked roles

SCP Behavior

Deny Specific Services

Allow everything except DynamoDB:

```
{
"Effect": "Deny",
"Action": "dynamodb:*",
"Resource": "*"
}
```

Allow Only Specific Services (Whitelist)

Allow only EC2 and CloudWatch:

```
{
"Effect": "Allow",
"Action": [
"ec2:*",
"cloudwatch:*"
],
"Resource": "*"
}
```

All other services become unavailable.

Common SCP Use Cases

Restrict Services

Example:

- Block Amazon EMR in production accounts.

Compliance Enforcement

Example:

- Disable non-PCI-compliant services.

Governance

Example:

- Ensure only approved services can be used.

Best Practices

- Use **multi-account architecture**.
- Apply consistent **tagging standards**.
- Enable **CloudTrail** in all accounts.
- Send CloudTrail logs to a **central S3 logging account**.
- Centralize **CloudWatch Logs** in a dedicated logging account.

Exam Tips (Very Important)

⭐ AWS Organizations is a **Global Service**.

⭐ **Consolidated Billing** combines costs across accounts.

⭐ **Reserved Instance discounts are shared** across accounts.

⭐ **SCPs do not apply to the Management/Master Account.**

⭐ SCPs define the **maximum permissions** available in member accounts.

⭐ Even the **Root User** in a member account can be restricted by an SCP.

⭐ Organizational Units (OUs) help apply policies to groups of accounts.

These are the most frequently tested concepts from AWS Organizations on AWS certification exams.

Organizations Consolidated Billing

Consolidated Billing is a feature of **AWS Organizations** that lets multiple AWS accounts be billed together under a single organization.

It provides **two major benefits**:

1. Combined Usage (Shared Discounts)

AWS aggregates the usage of all accounts in the organization.

Volume Pricing Benefits

Many AWS services offer lower prices as usage increases.

Example:

- Account A uses **1 TB**
- Account B uses **1 TB**
- Account C uses **1 TB**
- Account D uses **1 TB**
- Account E uses **1 TB**
- Account F uses **1 TB**

Individually, none of these accounts may reach a discount threshold.

However, with consolidated billing:

- Total usage = **6 TB**
- AWS treats the organization as a whole for eligible volume discounts.
- All accounts benefit from the lower pricing tier.

Exam Tip: AWS may ask about **shared volume discounts** across accounts in an organization.

2. One Bill

Instead of receiving separate invoices for each AWS account:

- The organization receives **one consolidated bill**.
- Easier for accounting and financial tracking.
- Encourages the creation of multiple AWS accounts without increasing billing complexity.

Reserved Instance (RI) Sharing

Another major benefit is **sharing Reserved Instance discounts** across accounts.

Example

Organization contains:

- **Account A**
    - 6 EC2 instances running

    - No Reserved Instances
- **Account B**
    - 3 EC2 instances running
    - 5 EC2 Reserved Instances purchased

Assume all instances are in the same Availability Zone and match RI requirements.

What Happens?

Account B owns **5 Reserved Instances**.

First, the RI discounts apply to:

- 3 running instances in Account B

There are still:

- 2 unused Reserved Instances remaining

Because **RI sharing is enabled**, AWS automatically applies the remaining RI discounts to:

- 2 instances in Account A

Final Result

Instance Type	Count
Covered by RI discount	5
Not covered by RI discount	4

Even though Account B only runs 3 instances, the unused RI capacity is shared with Account A, maximizing savings across the organization.

Savings Plans Sharing

Similar to Reserved Instances:

- Savings Plan discounts can also be shared across accounts in an AWS Organization.
- AWS automatically applies eligible discounts where they provide the greatest benefit.

Important Exam Points

✅ Consolidated Billing provides **one bill** for the entire organization.

✅ Usage from all accounts is combined for **volume pricing discounts**.

✅ **Reserved Instance discounts** can be shared across accounts.

✅ **Savings Plans discounts** can be shared across accounts.

✅ RI discount sharing can be **disabled** for specific accounts, including the management account.

AWS Control Tower Overview

AWS Control Tower is a service that helps you quickly set up and govern a **secure, compliant, multi-account AWS environment** based on AWS best practices.

Instead of manually:

- Creating an AWS Organization
- Creating multiple AWS accounts
- Configuring security settings
- Applying governance policies
- Monitoring compliance

You can use **AWS Control Tower** to automate these tasks with just a few clicks.

Why Use AWS Control Tower?

1. Automated Multi-Account Setup

Control Tower automatically:

- Creates and configures an AWS Organization
- Sets up core accounts (such as management, logging, and audit accounts)
- Establishes a landing zone (a well-architected multi-account environment)

This significantly reduces setup time and operational effort.

2. Governance Through Guardrails

A **Guardrail** is a high-level rule that helps enforce governance across accounts.

Guardrails can:

- Prevent non-compliant actions
- Detect policy violations
- Ensure accounts follow organizational standards

Examples:

- Restrict public access to certain resources
- Enforce encryption requirements
- Prevent disabling security services

3. Continuous Compliance Monitoring

Control Tower provides an interactive dashboard that lets you:

- View compliance status across accounts
- Monitor policy violations

- Track governance posture
- Identify non-compliant resources

This gives administrators a centralized view of the organization.

4. Automated Detection and Remediation

Control Tower can:

- Detect violations of governance policies
- Alert administrators
- Help remediate non-compliant configurations

This reduces the risk of security and compliance drift over time.

Relationship with AWS Organizations

A key exam point is:

AWS Control Tower runs on top of AWS Organizations.

When you deploy Control Tower, it automatically:

- Creates and manages an AWS Organization
- Organizes accounts into Organizational Units (OUs)
- Uses AWS Organizations features for governance

Service Control Policies (SCPs)

Control Tower uses **Service Control Policies (SCPs)** behind the scenes.

SCPs help enforce guardrails by:

- Restricting actions that accounts can perform
- Applying organization-wide governance rules
- Ensuring compliance requirements are followed

Example

An SCP might prevent member accounts from:

- Disabling AWS CloudTrail
- Leaving the organization
- Creating resources in unauthorized regions

Key Components of AWS Control Tower

Landing Zone

A preconfigured, secure multi-account AWS environment.

Organizational Units (OUs)

Used to logically group AWS accounts.

Guardrails

Governance rules that enforce or monitor compliance.

Dashboard

Provides compliance and governance visibility across the environment.

AWS Certification Exam Tips

- ✅ AWS Control Tower is used to create and govern **multi-account AWS environments**.
- ✅ Provides an **easy, automated setup** based on AWS best practices.
- ✅ Uses **AWS Organizations** underneath.
- ✅ Uses **Service Control Policies (SCPs)** to enforce governance.
- ✅ Governance rules are called **Guardrails**.
- ✅ Includes a **compliance dashboard** for monitoring.
- ✅ Helps detect and remediate policy violations.

AWS Resource Access Manager

AWS Resource Access Manager (AWS RAM) is a service that allows you to **securely share AWS resources** that you own with:

- Other AWS accounts
- Accounts within the same AWS Organization
- Organizational Units (OUs)

The main goal is to **avoid creating duplicate resources** and simplify multi-account architectures.

Why Use AWS RAM?

Without AWS RAM:

- Each account would need its own copy of a resource.
- This increases management overhead and costs.

With AWS RAM:

- A resource is created once.
- Multiple accounts can use that same resource.

This leads to:

✅ Reduced resource duplication
✅ Lower operational complexity
✅ Easier multi-account management
✅ Consistent network architecture

Commonly Shared Resources

AWS RAM supports sharing many AWS resources, including:

- VPC Subnets
- AWS Transit Gateway
- Amazon Aurora DB Clusters
- Route 53 Resolver Rules
- License Manager Configurations
- EC2 Capacity Reservations

Exam Tip: Don't memorize every supported resource. Focus on the concept of **resource sharing across accounts**.

Example: Sharing a VPC Subnet

Imagine:

Shared/Owner Account

Owner Account

└── VPC

  ├── Private Subnet A

  ├── Private Subnet B

  ├── Application Load Balancer

  └── Amazon RDS Database

Using AWS RAM, the owner account shares the VPC subnets with:

Account 1

Account 2

Result

Account 1

Can launch:

- EC2 Instances
- Containers
- Other supported resources

inside the shared subnet.

Account 2

Can also launch:

- EC2 Instances
- Application workloads

inside the same shared subnet.

Network Connectivity Benefit

Because all resources reside in the same VPC networking environment:

- EC2 instances from Account 1 can communicate with the RDS database.
- EC2 instances from Account 2 can access the Application Load Balancer.
- No VPC peering or Transit Gateway is required for communication within the shared VPC.

This greatly simplifies deployments.

AWS RAM and AWS Organizations

AWS RAM works particularly well with **AWS Organizations**.

You can share resources with:

- A specific AWS account
- An Organizational Unit (OU)
- The entire Organization

This makes large enterprise environments much easier to manage.

AWS Certification Exam Tips

✅ AWS RAM = **Share resources across AWS accounts**

✅ Helps **avoid resource duplication**

✅ Commonly used for **VPC subnet sharing**

✅ Works with **AWS Organizations**

✅ Shared resources remain owned by the original account

✅ Multiple accounts can use the same infrastructure resources

AWS Service Catalog – Overview

AWS Service Catalog allows organizations to create and manage a catalog of **approved AWS resources** that users can deploy through a self-service portal.

It solves a common problem:

New AWS users often have too many choices and may create resources that don't comply with organizational standards.

Instead of giving users unrestricted AWS access, organizations can provide a controlled set of pre-approved products.

Why Use AWS Service Catalog?

Without Service Catalog:

- Users create resources manually.
- Resources may be configured incorrectly.
- Tagging standards may not be followed.
- Security and compliance requirements may be missed.

With Service Catalog:

- Users choose from approved products.
- Deployments are standardized.
- Security and governance requirements are enforced.
- Self-service is enabled without sacrificing control.

How It Works

Administrators Create Products

A **Product** is typically:

- An AWS CloudFormation template
- With predefined parameters and configurations

Examples:

- EC2 Virtual Machine
- RDS Database
- S3 Storage Solution
- VPC Environment
- Application Stack

Products Are Grouped into Portfolios

A **Portfolio** is a collection of products.

Example:

Development Portfolio

- Linux EC2 Server
- Windows EC2 Server
- Small RDS Database
- S3 Storage Bucket

Data Analytics Portfolio

- Redshift Cluster
- EMR Cluster
- Data Lake Template

Access Is Controlled

Administrators decide:

- Which users, groups, or roles can access each portfolio.
- Which products they are allowed to launch.

This ensures users only see resources relevant to them.

Users Launch Products

Users access the Service Catalog portal and see only the approved products they have permission to use.

For example:

```
Available Products

✓ Standard Linux Server
✓ Production RDS Database
✓ S3 Data Bucket
✓ Three-Tier Web Application
```

The user simply selects a product and launches it.

CloudFormation Handles Deployment

Behind the scenes:

```
User
  ↓
AWS Service Catalog
  ↓
CloudFormation
  ↓
AWS Resources Created
```

CloudFormation automatically provisions all resources according to the approved template.

This guarantees:

- Correct configuration
- Standardized architecture
- Required security settings
- Required tags
- Compliance with organizational policies

Example

Suppose a developer needs an Amazon RDS database.

Without Service Catalog:

- They must know how to configure networking.
- Configure backups.
- Configure encryption.
- Apply tags.
- Set security groups.

This can lead to mistakes.

With Service Catalog:

- The admin creates a pre-approved "RDS Database" product.
- The developer selects it from the catalog.
- CloudFormation deploys it automatically with all required settings.

Result:

✅ Faster deployment

✅ Consistent configuration

✅ Reduced operational risk

Key Components

Product

A deployable AWS resource or application, usually defined by a CloudFormation template.

Portfolio

A collection of related products.

Constraints

Rules applied to products, such as deployment permissions or parameter restrictions.

End Users

Users who launch approved products through the self-service portal.

Benefits

Self-Service

Users can deploy resources without administrator involvement.

Standardization

Every deployment follows approved templates.

Governance

Resources comply with organizational standards.

Compliance

Security, tagging, and operational requirements are automatically enforced.

Faster Provisioning

Users can launch infrastructure in minutes.

AWS Certification Exam Tips

- ✅ AWS Service Catalog provides a **self-service portal** for approved AWS products.
- ✅ Products are typically based on **CloudFormation templates**.
- ✅ Products can include:
  - EC2 instances
  - Databases
  - Storage solutions
  - Complete application stacks
- ✅ Products are grouped into **Portfolios**.
- ✅ Administrators control who can access each portfolio.
- ✅ Deployments are automatically provisioned using **CloudFormation**.
- ✅ Ensures consistent, compliant, and approved infrastructure.

Pricing Models of the Cloud

AWS pricing is based on a few core principles. Understanding **why costs occur** is more important for the exam than memorizing exact prices.

1. AWS Pricing Models

AWS has **four main pricing models**:

1. Pay As You Go

Pay only for the resources you consume.

Benefits:

- No upfront costs
- Start and stop resources anytime
- Highly flexible
- Good for unpredictable workloads

Examples:

- EC2 On-Demand Instances
- Lambda
- S3 storage

2. Save When You Reserve

Commit to using resources for a longer period (typically 1 or 3 years).

Benefits:

- Significant discounts
- Predictable budgeting
- Suitable for steady workloads

Examples:

- EC2 Reserved Instances
- RDS Reserved Instances
- DynamoDB Reserved Capacity
- ElastiCache Reserved Nodes
- Redshift Reserved Nodes

Savings can reach **up to 75%** compared to On-Demand pricing.

3. Pay Less by Using More

AWS offers volume-based discounts.

The more you use:

- The lower the per-unit cost becomes.

Examples:

- S3 storage
- Data transfer
- CloudFront

4. AWS Economies of Scale

As AWS grows and lowers its infrastructure costs, it frequently reduces prices for customers.

Benefits:

- Continuous cost reductions
- Customers benefit automatically

AWS Free Tier

New AWS accounts receive:

- Up to **$200 AWS credits**
- Choice between:
    - Free Plan
    - Paid Plan

Free Plan

- Credits available
- Expires after 6 months or when credits are exhausted
- Account may be closed if not upgraded

Paid Plan

- Same credits
- After credits are exhausted, normal AWS billing begins

Always Free Services

Some services offer free monthly usage forever.

AWS Lambda

- 1 million requests/month

- 400,000 GB-seconds of compute/month

DynamoDB

- 25 GB storage
- Up to 200 million requests/month

EC2 Pricing

EC2 pricing depends on:

- Instance type
- CPU and RAM
- Region
- Operating system
- Additional software/license costs
- Load balancers
- Monitoring options

EC2 On-Demand Instances

Pay only for usage.

Characteristics:

- No commitment
- Flexible
- Billing starts at 60 seconds minimum
- Then per-second billing for Linux/Windows

Best for:

- Short-term workloads
- Testing
- Unpredictable demand

Reserved Instances (RI)

Commit for:

- 1 year
- 3 years

Benefits:

- Up to 75% savings

Payment options:

1. All Upfront
2. Partial Upfront
3. No Upfront

Greater upfront payment = larger discount.

Spot Instances

Use AWS spare capacity.

Benefits:

- Up to 90% cheaper than On-Demand

Drawback:

- AWS can terminate them when capacity is needed elsewhere.

Best for:

- Batch processing
- Big data
- Fault-tolerant workloads

Dedicated Hosts

Physical servers dedicated to one customer.

Benefits:

- Compliance requirements
- Licensing requirements

Available as:

- On-Demand
- Reserved

Savings Plans

Alternative discount model.

Provides lower prices in exchange for a commitment to spend a certain amount per hour.

Lambda Pricing

Lambda charges are based on:

Request Count

You pay per invocation.

Execution Time

Cost depends on:

- Duration
- Allocated memory

Formula:

Cost = Requests + (Duration × Memory)

ECS Pricing

ECS with EC2 Launch Type

No charge for ECS itself.

You only pay for:

- Underlying EC2 instances

ECS with Fargate

No EC2 management.

You pay for:

- CPU allocated
- Memory allocated

Per running container.

S3 Pricing

Pricing depends on:

Storage Class

Examples:

- S3 Standard
- S3 Standard-IA
- S3 One Zone-IA
- S3 Intelligent Tiering
- Glacier
- Glacier Deep Archive

Storage Amount

You pay for:

- Number of objects
- Object size

Higher volumes may receive discounts.

Requests

Charges apply for:

- PUT requests
- GET requests
- Other API requests

Data Transfer

Uploading to S3

Inbound = Free

Downloading from S3

Outbound = Charged

Additional Costs

- Lifecycle transitions
- S3 Transfer Acceleration

EFS Pricing

Elastic File System charges are based on:

- Amount of storage used
- Storage tier
- Lifecycle transitions

Unlike EBS:

Pay for actual usage

EBS Pricing

EBS is provisioned storage.

You pay for:

Volume Size

Example:

100 GB Provisioned

You pay whether you use all 100 GB or not.

Volume Type

Examples:

- GP3 SSD
- Provisioned IOPS SSD
- Magnetic

IOPS

Provisioned IOPS volumes incur additional charges based on IOPS provisioned.

Snapshots

You pay for:

- Snapshot storage
- Snapshot retention

RDS Pricing

RDS costs depend on:

Database Engine

Examples:

- MySQL
- PostgreSQL
- Oracle
- SQL Server

Compute Capacity

- Instance size
- Memory class

Purchase Option

On-Demand

Pay as you use.

Reserved

1-year or 3-year commitment for discounts.

Storage

Charged per GB per month.

Backups

AWS usually provides free backup storage up to the size of your database.

Example:

Database Size = 100 GB

Backup Storage up to 100 GB = Free

Multi-AZ Deployments

Multi-AZ costs more because AWS maintains multiple database instances.

Data Transfer

Inbound = Free

Outbound = Charged

CloudFront Pricing

CloudFront is AWS's CDN.

Pricing depends on:

Geographic Location

Different pricing for:

- North America
- Europe
- Asia
- Australia
- South America
- India

Data Transfer Out

Charged based on amount served to users.

Data In = Free

Data Out = Charged

Requests

You also pay based on:

- HTTPS requests
- HTTP requests

AWS Networking Costs

A very common certification topic.

Traffic Into AWS

Inbound = Free

Always remember:

Data entering AWS is generally free.

Same Availability Zone

Two EC2 instances using private IPs:

Free

Different AZs

Using private IPs:

Using public IPs:

More expensive

Exam Recommendation

Always prefer:

Private IP

Benefits:

- Lower cost
- Better performance
- More secure

Inter-Region Traffic

Traffic between AWS Regions is charged.

Example:

US-East-1 ↔ EU-West-1

Data transfer costs apply.

Important Exam Tips

- ✅ On-Demand = flexibility
- ✅ Reserved Instances = long-term savings
- ✅ Spot Instances = biggest discounts but can be interrupted
- ✅ Savings Plans = commitment-based discounts
- ✅ S3 uploads (inbound) are free
- ✅ S3 downloads (outbound) are charged
- ✅ Data entering AWS is generally free
- ✅ Data leaving AWS is usually charged
- ✅ Use private IPs instead of public IPs for EC2-to-EC2 communication
- ✅ Multi-AZ improves availability but increases cost

Quick Memory Cheat Sheet

```
Pay as you go       → On-Demand
Commit & Save       → Reserved Instances
Huge Discount       → Spot Instances
Serverless Cost     → Lambda Requests + Duration
S3 Upload           → Free
S3 Download         → Charged
Data Into AWS       → Free
Data Out of AWS     → Charged
Private Network     → Cheaper
Multi-AZ            → More Cost, More Availability
```

Exam keyword: AWS charges are generally based on **consumption**, and in most services, **data ingress (into AWS) is free while data egress (out of AWS) is charged.**

Saving Plan Overview

Savings Plans are AWS's flexible pricing model that helps reduce costs by committing to a consistent amount of usage over time.

Instead of reserving specific resources (like Reserved Instances), you simply commit to spending a certain amount **per hour** for either:

- **1 year**
- **3 years**

In return, AWS provides significant discounts compared to On-Demand pricing.

Why Savings Plans?

Reserved Instances vs Savings Plans

Reserved Instances (RI)

- Commit to specific resources
- Less flexible
- Must consider:
    - Instance family
    - Region
    - Instance type
    - Size

Savings Plans

- Commit to a dollar amount per hour
- Much simpler
- More flexible
- AWS automatically applies discounts to eligible usage

Types of Savings Plans

1. EC2 Instance Savings Plan

Provides the highest discount:

✅ Up to **72%** off On-Demand pricing

You commit to:

- A specific EC2 instance family
- A specific AWS Region

Example:

```
Commit:
$10/hour

Region:
us-east-1

Instance Family:
C5
```

AWS applies discounts regardless of:

- Availability Zone
- Instance size
  - C5.large
  - C5.xlarge
  - C5.4xlarge
- Operating System
  - Linux
  - Windows
- Tenancy
  - Shared
  - Dedicated

Example

If you commit:

$10/hour

for 3 years

on C5 instances in us-east-1

You can freely change:

- C5.large → C5.xlarge
- Linux → Windows
- AZ A → AZ B

while still receiving Savings Plan discounts.

2. Compute Savings Plan

The most flexible Savings Plan.

Provides:

✅ Up to **66%** discount

You commit to a spending amount per hour, regardless of:

- Instance family
- Region
- Instance size
- Operating system
- Tenancy
- Compute service

Eligible Services

- EC2
- AWS Fargate
- AWS Lambda

Example:

Committed Spend:

$20/hour

Today:

EC2 = $20/hour

Six months later:

Fargate = $12/hour

Lambda = $8/hour

The Savings Plan still applies.

This flexibility is why many organizations prefer Compute Savings Plans.

3. Machine Learning Savings Plan

Designed specifically for:

- Amazon SageMaker

Benefits:

- Lower SageMaker costs
- Commitment-based discounts

Suitable for organizations running long-term machine learning workloads.

Payment Options

Like Reserved Instances, Savings Plans offer:

All Upfront

Pay everything immediately.

✅ Maximum savings

Partial Upfront

Pay part of the commitment now.

✅ Medium savings

No Upfront

No initial payment.

✅ Most flexible

❌ Lowest discount

Commitment Terms

Available terms:

1-Year Term

- More flexibility
- Smaller discount

3-Year Term

- Less flexibility
- Larger discount

How AWS Applies Savings Plans

AWS automatically applies discounts to eligible workloads.

You don't need to:

- Reconfigure resources
- Assign reservations manually
- Manage instance matching

AWS does it for you.

Finding Recommended Savings Plans

AWS can recommend Savings Plans using:

AWS Cost Explorer

Navigate to:

AWS Cost Management

→ Cost Explorer

→ Savings Plans

AWS analyzes:

- Historical usage
- Compute patterns
- Existing workloads

and recommends:

- EC2 Savings Plans
- Compute Savings Plans
- Commitment amounts

Exam Comparison

Feature	Reserved Instance	Savings Plan
Commit to Resource	Yes	No
Commit to Spend per Hour	No	Yes
Flexible	Limited	Highly Flexible
Easier to Manage	No	Yes
Covers Lambda	No	Compute SP Only
Covers Fargate	No	Compute SP Only

AWS Certification Exam Tips

✅ Savings Plans require a **1-year or 3-year commitment**

✅ Commitment is based on **dollars per hour**

✅ **EC2 Instance Savings Plan** offers the highest EC2 savings (up to 72%)

✅ **Compute Savings Plan** is the most flexible (EC2, Lambda, Fargate)

✅ **Machine Learning Savings Plan** applies to Amazon SageMaker

✅ Payment options:

- All Upfront

- Partial Upfront
- No Upfront

✅ AWS Cost Explorer can recommend appropriate Savings Plans

Quick Memory Trick

Reserved Instance

= Reserve specific resources

Savings Plan

= Commit spending per hour

Show more lines

Most Flexible: Compute Savings Plan

Highest EC2 Discount: EC2 Instance Savings Plan

For SageMaker: Machine Learning Savings Plan

Exam keyword: *"Commit a certain dollar amount per hour for 1 or 3 years and receive automatic discounts"* → **AWS Savings Plans**.

Compute Optimizer Overview

AWS Compute Optimizer is a service that helps you **reduce costs and improve performance** by recommending the most appropriate AWS resources for your workloads.

Instead of manually analyzing resource utilization, Compute Optimizer uses **Machine Learning (ML)** to identify resources that are:

- **Over-provisioned** (too large, wasting money)
- **Under-provisioned** (too small, causing performance issues)
- **Optimally provisioned** (appropriately sized)

Why Use Compute Optimizer?

Many organizations tend to:

- Launch larger instances than necessary
- Allocate excessive storage
- Overestimate resource requirements

This leads to unnecessary costs.

AWS Compute Optimizer analyzes actual usage and recommends better configurations.

Example

Current EC2 Instance:

m5.4xlarge

CPU Utilization: 5%

Memory Utilization: 10%

Recommendation:

m5.large

Result:

✅ Lower monthly cost

✅ Same workload performance

How Compute Optimizer Works

Compute Optimizer uses:

1. Resource Configuration Data

It examines:

- Instance types
- CPU allocation
- Memory allocation

- Storage configuration

2. Amazon CloudWatch Metrics

It analyzes historical metrics such as:

- CPU usage
- Network traffic
- Memory utilization (where available)
- Disk activity

3. Machine Learning

AWS ML models detect usage patterns and generate recommendations for better resource sizing.

Supported Resources

Compute Optimizer can analyze:

EC2 Instances

Recommends:

- Larger instances if resources are insufficient
- Smaller instances if resources are underutilized

Auto Scaling Groups

Analyzes groups and recommends:

- More suitable instance types
- Better capacity configurations

EBS Volumes

Provides recommendations for:

- Volume type
- Volume performance
- Volume size

Example:

gp3 instead of gp2

or

Reduce provisioned capacity

AWS Lambda Functions

Analyzes:

- Allocated memory
- Runtime performance

Recommends more efficient memory settings.

Example:

Current Memory: 2048 MB

Recommended: 1024 MB

This can reduce Lambda costs significantly.

Benefits

Cost Savings

AWS indicates that organizations can potentially reduce costs by up to:

25%

through rightsizing recommendations.

Improved Performance

Compute Optimizer can also identify resources that are too small and causing bottlenecks.

Benefits include:

- Better application responsiveness
- Improved user experience
- Increased workload efficiency

Visibility

Provides a centralized view of:

- Resource utilization
- Cost optimization opportunities
- Performance improvement recommendations

Easy Reporting

Recommendations can be exported to:

Amazon S3

This allows:

- Further analysis
- Reporting
- Integration with cost management processes

Typical Workflow

```
Resource Running
        ↓
CloudWatch Metrics Collected
        ↓
Compute Optimizer Analysis
        ↓
ML-Based Recommendations
        ↓
Review Recommendations
        ↓
Implement Changes
        ↓
Lower Cost / Better Performance
```

AWS Certification Exam Tips

✅ Compute Optimizer uses **Machine Learning**.

✅ Analyzes **CloudWatch metrics** and resource configurations.

✅ Identifies:

- Over-provisioned resources
- Under-provisioned resources

✅ Supports:

- EC2 Instances
- Auto Scaling Groups
- EBS Volumes
- Lambda Functions

✅ Helps reduce costs and improve performance.

✅ Recommendations can be exported to **Amazon S3**.

Quick Memory Trick

Compute Optimizer = "AWS Rightsizing Service"

Think:

Analyze → Recommend → Optimize → Save Money

If an exam question asks:

"Which AWS service uses machine learning and CloudWatch metrics to recommend optimal EC2, EBS, Auto Scaling, or Lambda configurations to reduce cost and improve performance?"

✅ **AWS Compute Optimizer** is the correct answer.

Billing & Costing Tool Overview

AWS provides several tools to help you:

1. **Estimate costs**
2. **Track costs**
3. **Monitor and control costs**

This is a commonly tested area in AWS certification exams.

1. Estimate Costs

AWS Pricing Calculator

Used to estimate the cost of AWS services **before deployment**.

Use Cases

- Estimate monthly AWS bills
- Compare architectures
- Build budgets for projects
- Present cost estimates to stakeholders

Example

You want to estimate the cost of:

- 2 EC2 instances
- 1 RDS database
- 500 GB S3 storage

Use the **AWS Pricing Calculator** to generate an expected monthly cost.

Exam Keyword

"Estimate future AWS costs before launching resources."

✅ **AWS Pricing Calculator**

2. Track Costs

These tools help you understand where money is being spent.

AWS Billing Dashboard

Provides a high-level overview of current AWS charges.

Shows:

- Current month's spending
- Previous bills
- Service-by-service costs

- Billing history

Example

EC2 $500

S3 $100

RDS $200

Useful for quickly checking account spending.

Exam Keyword

"View current AWS charges and billing information."

✅ **Billing Dashboard**

Cost Allocation Tags

Used to categorize and track AWS spending.

Example Tags

Department = Finance

Department = Marketing

Project = MobileApp

Project = Website

AWS can break down expenses based on these tags.

Example

Finance Team = $2,000

Marketing Team = $4,000

Useful for chargeback and cost reporting.

Exam Keyword

"Track costs by department, application, team, or project."

✅ **Cost Allocation Tags**

Cost and Usage Report (CUR)

The most detailed AWS cost report.

Contains:

- Resource-level usage data
- Billing data
- Cost details
- Discounts
- Reserved Instance usage

- Savings Plan usage

Can be delivered to:

Amazon S3

for further analysis.

Exam Keyword

"Most detailed billing and usage report."

✅ **Cost and Usage Report (CUR)**

AWS Cost Explorer

Visualization and analysis tool for AWS spending.

Provides:

- Historical cost trends
- Usage trends
- Forecasting
- Cost breakdowns

Example Questions

- Which service costs the most?
- How has EC2 spending changed over 6 months?
- What will next month's bill likely be?

Features

- Graphs
- Forecasts
- Recommendations
- Savings Plan suggestions

Exam Keyword

"Analyze past spending and forecast future costs."

✅ **Cost Explorer**

3. Monitor and Control Costs

Billing Alarms

Uses Amazon CloudWatch to alert you when charges exceed a threshold.

Example

Alert when monthly bill exceeds $100

AWS sends notifications via:

- Amazon SNS
- Email
- SMS (via SNS)

Exam Keyword

"Receive alerts when spending exceeds a predefined threshold."

✅ **Billing Alarm**

AWS Budgets

Allows you to define spending limits and monitor them.

You can create budgets for:

- Cost
- Usage
- Reserved Instances
- Savings Plans

Example

Monthly Budget = $500

Notifications:

- At 80%
- At 100%
- Above budget

Advanced Feature

AWS Budgets can also trigger actions when thresholds are reached.

Exam Keyword

"Set budgets and receive alerts when approaching limits."

✅ **AWS Budgets**

Quick Comparison

Tool	Purpose
Pricing Calculator	Estimate future costs
Billing Dashboard	View current charges
Cost Allocation Tags	Track spending by project/team
Cost & Usage Report	Detailed billing data
Cost Explorer	Analyze and forecast spending

Billing Alarm	Alert on spending thresholds
AWS Budgets	Set and monitor spending limits

Exam Memory Trick

Estimate

➡️ **Pricing Calculator**

Track

➡️ **Billing Dashboard**

➡️ **Cost Allocation Tags**

➡️ **Cost and Usage Report**

➡️ **Cost Explorer**

Monitor

➡️ **Billing Alarms**

➡️ **AWS Budgets**

Most Common Exam Questions

Question	Answer
Estimate AWS costs before deployment?	Pricing Calculator
View current AWS bill?	Billing Dashboard
Track costs by project or department?	Cost Allocation Tags
Need the most detailed billing report?	Cost and Usage Report
Analyze trends and forecast future costs?	Cost Explorer
Get notified when costs exceed a threshold?	Billing Alarm
Create spending limits and receive alerts?	AWS Budgets

Final Exam Cheat Sheet

Estimate Costs → Pricing Calculator

Track Costs → Billing Dashboard

Cost Allocation Tags

Cost & Usage Report

Cost Explorer

Monitor Costs → Billing Alarms

AWS Budgets

This classification (**Estimate → Track → Monitor**) is the easiest way to remember all AWS billing and costing tools for the exam.

Estimate Costs in the Cloud – Pricing Calculator

The **AWS Pricing Calculator** is a tool used to **estimate the cost of an AWS solution before deploying it**.

It helps answer questions such as:

- How much will my architecture cost per month?
- What will be the yearly cost?
- How much can I save using Reserved Instances or Savings Plans?
- What is the cost impact of adding more resources?

Purpose

Before creating resources in AWS, you can model your architecture and get a cost estimate.

Typical use cases:

- Budget planning
- Cost comparisons
- Architecture design
- Client proposals
- Migration planning

How It Works

1. Open the AWS Pricing Calculator.
2. Add AWS services to an estimate.
3. Configure each service.
4. Specify usage patterns.
5. Review the projected monthly and annual costs.

Example: EC2 Cost Estimate

Suppose you need:

- 4 EC2 instances
- Linux operating system
- t4g.xlarge instances
- 200 GB EBS storage per instance
- 80% monthly utilization

You can configure:

Service: Amazon EC2

Region: us-east-1

Instances: 4

Instance Type: t4g.xlarge

Operating System: Linux

Storage: 200 GB EBS per instance

Utilization: 80%

You can also choose a pricing strategy:

- On-Demand
- Savings Plans
- Reserved Instances

For example:

1-Year EC2 Savings Plan

No Upfront Payment

The calculator will automatically estimate monthly and annual costs.

Example: Load Balancer Estimate

You can also add:

Elastic Load Balancer

Type: Application Load Balancer (ALB)

Quantity: 1

Specify expected usage such as:

- Data processed per hour
- Connections per second
- Rules evaluated

The calculator includes those costs in the final estimate.

Combined Architecture Estimate

Example architecture:

4 EC2 Instances

+

1 Application Load Balancer

+

EBS Storage

The calculator combines all costs and may show:

Monthly Cost Estimate

Annual Cost Estimate

This provides a full picture of expected AWS spending.

Features

Cost Estimation

Estimate costs before deployment.

Compare Pricing Options

Evaluate:

- On-Demand
- Reserved Instances
- Savings Plans

Multi-Service Architectures

Include multiple services:

- EC2
- S3
- RDS
- Lambda
- CloudFront
- Load Balancers
- EKS
- DynamoDB
- Many others

Grouping Resources

Organize estimates into logical groups such as:

Frontend

Backend

Database

Networking

Export and Share

Estimates can be shared with:

- Clients
- Managers
- Architects

- Finance teams

Exam Tips

✅ Used to **estimate future AWS costs**

✅ Helps design and budget AWS architectures

✅ Supports multiple AWS services

✅ Can compare On-Demand vs Reserved vs Savings Plans

✅ Shows monthly and annual estimated costs

✅ Used before deployment, not for tracking actual spending

AWS Billing Tool Comparison

Tool	Purpose
AWS Pricing Calculator	Estimate future costs
Cost Explorer	Analyze historical costs
Cost and Usage Report	Detailed usage and billing data
AWS Budgets	Set spending limits
Billing Dashboard	View current charges

Quick Memory Trick

Before deployment → Pricing Calculator

After deployment → Cost Explorer / Budgets / Billing Dashboard

Exam Keyword

If a question asks:

"Which AWS service helps estimate the cost of a planned AWS architecture before resources are deployed?"

✅ **AWS Pricing Calculator** is the correct answer.

Tracking Costs in the Cloud – Billing Dashboard, Cost Allocation Tags, Reports

The **Billing Dashboard** gives a quick, high-level overview of your AWS costs.

It shows:

- Month-to-date spending
- Previous month's spending
- Forecasted spending
- Cost breakdown by AWS service
- Cost breakdown by account or region

Example

Current Month Cost: $500

Forecasted Cost: $650

EC2: $300

RDS: $120

S3: $80

Best Used For

- Quick cost overview
- Monitoring current spending trends
- Identifying which services are generating costs

Exam Keyword

"View a summary of AWS charges and forecasts."

✅ **Billing Dashboard**

2. Cost Allocation Tags

Cost Allocation Tags help track spending by categorizing resources.

Why Use Tags?

Suppose your organization has multiple teams:

Finance

Marketing

IT

Development

You can tag resources:

Department = IT

Environment = Production

Project = MobileApp

CostCenter = 1234

AWS can then break down costs based on these tags.

Types of Tags

AWS-Generated Tags

Created automatically by AWS.

Examples:

aws:createdBy

These start with the aws: prefix.

User-Defined Tags

Created by administrators.

Examples:

Department = IT

Owner = John

Project = Website

Benefits

- Cost chargeback
- Cost reporting
- Resource organization
- Team-level budgeting

Exam Keyword

"Track costs by department, team, application, or project."

✅ **Cost Allocation Tags**

Resource Groups and Tag Editor

Tags can also be used to organize resources.

For example:

Department = IT

You can create a resource group that automatically contains:

- EC2 instances
- Security groups
- Load balancers
- RDS databases

that share the same tag.

Benefits

- Easier resource management
- Easier reporting
- Cost categorization

3. Cost and Usage Report (CUR)

The **Cost and Usage Report (CUR)** is the most detailed AWS billing report.

What It Contains

- Resource-level usage
- Service-level costs
- Pricing information
- Reservation data
- Savings Plan data
- Cost allocation tags

Granularity

Reports can be generated with:

- Hourly data
- Daily data

Example Information

Service: EC2

Resource:

i-123456

Start Time:

10:00 AM

Usage:

1 hour

Cost:

$0.12

This level of detail is not available in Billing Dashboard.

Integration Options

Reports can be analyzed using:

- Amazon Athena

- Amazon Redshift
- Amazon QuickSight

Reports are typically delivered to:

Amazon S3

Best Used For

- Detailed billing investigations
- Enterprise reporting
- Cost analytics
- Auditing AWS usage

Exam Keyword

"Most comprehensive billing and usage data available in AWS."

✅ **Cost and Usage Report (CUR)**

4. AWS Cost Explorer

Cost Explorer provides a visual interface for cost analysis.

Think of it as:

Billing Dashboard + Analytics + Forecasting

Features

Cost Visualization

Display spending trends using graphs.

Example:

January $200

February $300

March $450

Cost Breakdown

Analyze spending by:

- Service
- Account
- Region
- Tag
- Resource

Resource-Level Visibility

You can identify:

Which EC2 instance costs the most?

Which service drives my bill?

Savings Plan Recommendations

Cost Explorer analyzes usage and recommends:

- Savings Plans
- Potential cost reductions

Forecasting

A very important exam topic.

Cost Explorer can forecast future costs based on historical usage.

Forecast period:

Up to 12 months

Example

Suppose your spending has been:

January $1000

February $1200

March $1400

Cost Explorer estimates what future costs may look like.

Best Used For

- Cost trend analysis
- Budget planning
- Forecasting
- Savings Plan recommendations

Exam Keyword

"Forecast AWS usage and costs up to 12 months into the future."

✅ **Cost Explorer**

Data Exports

AWS also provides **Data Exports**.

Data Exports allow you to:

- Export cost and usage data
- Save it to Amazon S3
- Choose formats like:
    - CSV
    - Parquet

The exported data can then be analyzed using:

- Athena
- Redshift
- QuickSight
- Excel

Comparison of Cost Tracking Tools

Tool	Purpose	Level of Detail
Billing Dashboard	View current costs	High-level
Cost Allocation Tags	Categorize costs	Resource category
Cost & Usage Report (CUR)	Detailed billing analysis	Most detailed
Cost Explorer	Visualization and forecasting	Medium/High
Data Exports	Export cost data for analysis	Detailed

Exam Cheat Sheet

Need a quick billing overview?

✅ Billing Dashboard

Need to track costs by department or project?

✅ Cost Allocation Tags

Need the most detailed billing information?

✅ Cost and Usage Report (CUR)

Need graphs, trends, and forecasting?

✅ Cost Explorer

Need to predict future AWS costs (up to 12 months)?

✅ Cost Explorer

Need raw billing data in S3?

✅ Data Exports / Cost & Usage Report

Memory Trick

Overview → Billing Dashboard

Categorize → Cost Allocation Tags

Deep Analysis → Cost & Usage Report

Visualize → Cost Explorer

Forecast → Cost Explorer

Export Data → Data Exports

Most Detailed Report: Cost and Usage Report (CUR)

Forecasting Tool: Cost Explorer

Cost Categorization Tool: Cost Allocation Tags

Quick Billing View: Billing Dashboard

Monitoring Cost in the Cloud – Billing Alarm & AWS Budgets

After estimating and tracking costs, the next step is **monitoring costs proactively** so you can avoid unexpected AWS bills.

1. Billing Metric (CloudWatch)

AWS automatically publishes a **Billing Metric** to Amazon CloudWatch.

Key Facts

✅ Stored only in **US East (N. Virginia) – us-east-1**

✅ Aggregates costs from **all AWS regions**

✅ Represents **actual incurred charges**

✅ Used as the source for Billing Alarms

Example

CloudWatch may show:

Current AWS Charges = $65

As your spending increases, the metric increases.

2. Billing Alarms

A **Billing Alarm** is a CloudWatch alarm created on top of the Billing Metric.

How It Works

1. CloudWatch tracks AWS charges.
2. You define a threshold.
3. AWS sends a notification when the threshold is exceeded.

Example

Threshold = $70

When spending exceeds $70:

CloudWatch Alarm

↓

SNS Notification

↓

Email/SMS

You receive an alert immediately.

Characteristics of Billing Alarms

Pros

- Simple to configure
- Real-time monitoring of actual charges
- Email notifications via SNS

Cons

- Uses actual costs only
- Cannot forecast future spending
- Less flexible than AWS Budgets

Exam Tip

If the question mentions:

"Create a CloudWatch alarm when AWS charges exceed a certain amount"

✅ **Billing Alarm**

3. AWS Budgets

AWS Budgets is a more advanced cost monitoring service.

It can notify you when:

- Actual spending exceeds a threshold
- Forecasted spending exceeds a threshold

This is a major advantage over Billing Alarms.

Why AWS Budgets Is Better

Billing Alarm:

Actual Cost Only

AWS Budgets:

Actual Cost

+

Forecasted Cost

This means AWS can warn you before you actually exceed your budget.

Budget Types

AWS supports four budget categories.

Cost Budget

Tracks spending.

Example:

Monthly Budget = $100

Usage Budget

Tracks resource consumption.

Examples:

100 GB S3 Storage

1000 EC2 Instance Hours

Reservation Budget

Tracks Reserved Instance utilization and coverage.

Supported services include:

- EC2 Reserved Instances
- RDS Reserved Instances
- Redshift Reserved Nodes
- ElastiCache Reserved Nodes

Savings Plans Budget

Tracks:

- Savings Plan utilization
- Savings Plan coverage

Useful for ensuring your Savings Plans are fully utilized.

Budget Notifications

Each budget can send notifications using Amazon SNS.

Maximum Notifications

Up to 5 SNS notifications per budget

Notifications can trigger:

- Emails
- Lambda functions
- Automated actions

Example Alert Configuration

Actual Cost Alert

Budget = $100

Alert at:

80%

Notification sent when:

Actual Spending = $80

Forecast Alert

Budget = $100

If AWS predicts:

End-of-month Spending = $120

an alert is sent even before the budget is exceeded.

This is a favorite AWS exam topic.

Budget Filters

Budgets can be scoped very precisely.

You can filter by:

- AWS Service
- Linked Account
- Region
- Availability Zone
- Tag
- Purchase Option
- Instance Type
- API Operation

and many other dimensions.

Example

Create a budget only for:

Amazon EC2

or

AWS KMS

or

Environment = Production

This allows very granular cost control.

Integration with Cost Explorer

AWS Budgets integrates directly with Cost Explorer.

You can:

- Analyze spending trends
- Investigate budget overruns
- Drill into detailed cost reports

Billing Alarm vs AWS Budgets

Feature	Billing Alarm	AWS Budgets
Uses CloudWatch Billing Metric	✅	❌
Actual Cost Alerts	✅	✅
Forecasted Cost Alerts	❌	✅
Budget Tracking	❌	✅
Service Filtering	Limited	✅
Reservation Tracking	❌	✅
Savings Plan Tracking	❌	✅

Exam Cheat Sheet

Need a CloudWatch alarm when spending exceeds a threshold?

✅ **Billing Alarm**

Need alerts based on forecasted spending?

✅ **AWS Budgets**

Need budget tracking for EC2, RDS, Redshift Reserved Instances?

✅ **Reservation Budget**

Need Savings Plan utilization tracking?

✅ **Savings Plans Budget**

Need email notifications at 80% or 100% of budget?

✅ **AWS Budgets + SNS**

Where is the Billing Metric stored?

✅ **us-east-1**

What can forecast costs and trigger alerts before overspending?

✅ **AWS Budgets**

Quick Memory Trick

Billing Alarm = Actual Spending Alert

AWS Budgets = Actual + Forecasted Spending Alert

Most important exam point:

If the question mentions **forecasted costs** or **forecasted budget overruns**, the correct answer is **AWS Budgets**, not Billing Alarms.

AWS Cost Anomaly Detection

AWS Cost Anomaly Detection is a cost-monitoring service that uses **Machine Learning (ML)** to automatically identify unusual spending patterns in your AWS account.

Unlike Billing Alarms or Budgets, you **do not need to define thresholds manually**.

AWS learns your normal spending behavior and alerts you when something appears abnormal.

Why Use Cost Anomaly Detection?

Normally, you might not know:

- An EC2 instance was accidentally left running
- A Lambda function is looping endlessly
- Data transfer costs suddenly increased
- A new service is generating unexpected charges

Cost Anomaly Detection automatically detects these situations.

How It Works

Step 1: Learn Historical Patterns

AWS analyzes your historical:

- Costs
- Resource usage
- Spending trends

Using ML, it learns what "normal" looks like for your environment.

Step 2: Detect Anomalies

AWS continuously monitors spending and looks for:

One-Time Cost Spikes

Example:

Daily spending: $50

Today: $700

AWS detects the spike and flags it as an anomaly.

Continuous Cost Increases

Example:

Day 1: $50

Day 2: $70

Day 3: $90

Day 4: $120

Even though no single day is extreme, AWS detects the unusual upward trend.

What Can It Monitor?

AWS Cost Anomaly Detection can monitor:

AWS Services

Example:

EC2

RDS

Lambda

CloudFront

S3

AWS Accounts

Including:

- Management Account
- Member Accounts in AWS Organizations

Cost Allocation Tags

Example:

Department = Finance

Project = MobileApp

Environment = Production

Cost Categories

Custom business groupings of AWS costs.

Example:

Development

Production

Analytics

Root Cause Analysis

One of the most useful features is automatic root cause analysis.

Instead of simply saying:

Your costs increased.

AWS provides details such as:

Service: Amazon EC2

Region: us-east-1

Increase: +$500

Likely Cause: Large EC2 instance launched

This makes troubleshooting much faster.

Notifications

AWS can send anomaly notifications using **Amazon SNS**.

You can choose:

Individual Alerts

Receive an alert whenever an anomaly is detected.

Anomaly Detected

↓

SNS

↓

Email/SMS

Summary Alerts

Receive:

- Daily summaries
- Weekly summaries

This is useful for finance teams that don't want real-time notifications.

Cost Anomaly Detection vs Other Cost Tools

Service	Purpose
Billing Alarm	Alert on actual cost threshold
AWS Budgets	Alert on actual or forecasted budgets
Cost Explorer	Analyze and forecast costs
Cost Anomaly Detection	Automatically detect unusual spending patterns

Exam Tips

- ✅ Uses **Machine Learning**
- ✅ Learns historical spending patterns automatically
- ✅ No need to define spending thresholds
- ✅ Detects:
  - One-time cost spikes
  - Continuous cost increases
- ✅ Supports monitoring:
  - AWS Services
  - Member Accounts
  - Cost Allocation Tags
  - Cost Categories
- ✅ Provides **Root Cause Analysis**
- ✅ Sends notifications through **Amazon SNS**

Quick Memory Trick

Billing Alarm

Threshold-Based

AWS Budgets

Budget-Based

Cost Anomaly Detection

Machine Learning-Based

AWS Cost Management Cheat Sheet

Requirement	AWS Service
Estimate future costs	Pricing Calculator
View current AWS bill	Billing Dashboard
Track costs by project/team	Cost Allocation Tags
Most detailed billing report	Cost & Usage Report
Forecast costs (up to 12 months)	Cost Explorer
Alert when bill exceeds threshold	Billing Alarm
Alert on actual or forecasted budget	AWS Budgets
Detect unusual spending automatically	AWS Cost Anomaly Detection

Exam Keyword

If a question says:

"Use machine learning to automatically detect unusual AWS spending patterns and identify the root cause without configuring thresholds."

✅ **AWS Cost Anomaly Detection** is the correct answer.

AWS Service Quotas

AWS Service Quotas is a service that helps you monitor, manage, and request increases for AWS service limits (quotas).

AWS services have default limits to protect resources and ensure fair usage. These limits are known as **Service Quotas**.

Why Service Quotas Matter

Every AWS account has quotas such as:

- Number of EC2 instances
- Number of VPCs
- Number of Lambda concurrent executions
- Number of EBS volumes
- API request limits

If you reach a quota, AWS may prevent you from creating additional resources until:

- Resources are deleted, or
- The quota is increased

Example: AWS Lambda

Suppose your account has a Lambda concurrency quota.

Lambda Concurrent Executions Limit = 1,000

If your application suddenly scales and reaches:

950 concurrent executions

you may want to receive an alert before hitting the limit.

AWS Service Quotas can help monitor this usage and notify you.

CloudWatch Integration

One of the key features of Service Quotas is its integration with **Amazon CloudWatch**.

You can create:

CloudWatch Alarms

based on quota utilization.

Example

Quota: 1,000 Lambda Concurrent Executions

Alarm Threshold: 80%

Alert at: 800 concurrent executions

When utilization reaches the threshold:

CloudWatch Alarm

↓

SNS Notification

↓

Email / SMS

This allows proactive monitoring before workloads fail due to quota limits.

Requesting Quota Increases

Another important feature is the ability to request quota increases directly from the Service Quotas console.

Example

Current quota:

EC2 Instances = 20

Business requirement:

Need 100 instances

You can submit a quota increase request directly from Service Quotas.

AWS reviews the request and may increase the limit.

What Can You Do When Approaching a Quota?

When alerts are received, you can choose to:

Option 1: Request a Quota Increase

Increase the limit so additional resources can be created.

Option 2: Reduce Resource Usage

Investigate whether unnecessary resources are running and terminate them.

Example:

Unused Lambda executions

Unused EC2 instances

Excessive API activity

Benefits of AWS Service Quotas

Centralized Quota Management

Manage quotas across many AWS services from a single console.

Monitoring

Track quota utilization before limits are reached.

Alerting

Create CloudWatch alarms for quota thresholds.

Self-Service Quota Requests

Request increases without opening separate support cases for many services.

Prevent Service Disruptions

Avoid failures caused by reaching resource limits.

Typical Workflow

AWS Service
↓
Quota Utilization
↓
Service Quotas
↓
CloudWatch Alarm
↓
SNS Notification
↓
Take Action
├─ Increase Quota
└─ Reduce Usage

AWS Certification Exam Tips

- ✅ AWS limits are called **Service Quotas**.
- ✅ Service Quotas provides a centralized view of AWS service limits.
- ✅ Integrates with **CloudWatch Alarms**.
- ✅ Sends notifications when usage approaches a quota.
- ✅ Allows direct requests for **quota increases**.
- ✅ Helps monitor limits such as:
  - Lambda concurrent executions
  - EC2 limits
  - VPC limits

- Many other AWS service quotas

Quick Memory Trick

Service Quotas = Monitor + Alert + Increase Limits

Need to view AWS limits?

→ Service Quotas

Need alerts when approaching limits?

→ Service Quotas + CloudWatch

Need a quota increase?

→ Service Quotas

Exam Keyword

If a question asks:

"Which AWS service provides a centralized way to monitor service limits, create alerts when approaching those limits, and request quota increases?"

✅ **AWS Service Quotas** is the correct answer.

AWS Trusted Advisor

AWS Trusted Advisor is an AWS service that analyzes your AWS environment and provides **best-practice recommendations** to help improve:

- Cost efficiency
- Performance
- Security
- Reliability
- Service limit management
- Operational excellence

Think of it as an **automated AWS consultant** that continuously reviews your account and suggests improvements.

Key Characteristics

✅ Fully managed AWS service

✅ No installation required

✅ Scans your AWS account automatically

✅ Provides recommendations and warnings

✅ Helps optimize resources and reduce risk

What Does Trusted Advisor Check?

Trusted Advisor performs checks across **six categories**:

1. Cost Optimization

Helps identify opportunities to save money.

Examples:

- Underutilized EC2 instances
- Idle load balancers
- Unused EBS volumes
- Unused Elastic IP addresses

Goal: Reduce AWS spending.

2. Performance

Recommends changes that can improve performance.

Examples:

- Service configuration improvements

- Performance-related optimizations

Goal: Improve application responsiveness and efficiency.

3. Security

Identifies potential security risks.

Examples:

- S3 buckets with public access
- Open security groups
- IAM security recommendations
- Public EBS snapshots
- Public RDS snapshots

Goal: Improve account security posture.

4. Fault Tolerance

Helps increase availability and resiliency.

Examples:

- Insufficient redundancy
- Backup recommendations
- Multi-AZ recommendations

Goal: Improve reliability and reduce downtime risk.

5. Service Limits

Monitors AWS service quotas and limits.

Examples:

- EC2 limits
- Auto Scaling Group limits
- CloudFormation stack limits
- DynamoDB limits

Goal: Prevent service disruptions caused by quota exhaustion.

6. Operational Excellence

Provides operational best-practice recommendations.

Examples:

- Infrastructure management improvements

- Governance recommendations
- Operational best practices

Goal: Improve AWS operations and management.

Example Security Findings

Trusted Advisor might identify:

Public S3 Bucket

Warning: S3 bucket allows public access

You can then verify whether this is intentional.

Open Security Groups

Security Group: 0.0.0.0/0 access on port 22

Trusted Advisor flags the rule for review.

Public Snapshots

EBS Snapshot Public

RDS Snapshot Public

These may expose sensitive data and should be investigated.

Core Checks vs Full Checks

AWS provides different levels of Trusted Advisor checks depending on your support plan.

Core Checks

Available to all AWS customers.

Includes basic recommendations such as:

- Security checks
- Service limit checks

Full Checks

Require:

- Business Support Plan
- Enterprise Support Plan

Provides access to:

- All Cost Optimization checks
- All Performance checks

- All Fault Tolerance checks
- All Operational Excellence checks
- Advanced Security checks

Programmatic Access

With:

- Business Support
- Enterprise Support

You can access Trusted Advisor programmatically using the:

AWS Support API

This allows automation and integration with operational workflows.

Trusted Advisor vs Service Quotas

Service Quotas

Focuses on:

Monitoring limits

Requesting quota increases

CloudWatch alerts

Trusted Advisor

Focuses on:

Best-practice recommendations

Cost savings

Security reviews

Performance improvements

Service limit visibility

Trusted Advisor vs Compute Optimizer

Compute Optimizer

Uses Machine Learning to recommend:

- EC2 instance sizing
- EBS sizing
- Lambda memory optimization

Trusted Advisor

Provides broader account-level recommendations:

- Cost
- Security
- Reliability
- Service limits

Exam Tips

✅ Trusted Advisor provides **account assessments and recommendations**.

✅ No software installation is required.

✅ Checks are grouped into **6 categories**:

- Cost Optimization
- Performance
- Security
- Fault Tolerance
- Service Limits
- Operational Excellence

✅ Detects:

- Public S3 buckets
- Public RDS snapshots
- Public EBS snapshots
- Open security groups
- Other AWS best-practice violations

✅ Full set of checks requires:

- Business Support Plan
- Enterprise Support Plan

✅ Programmatic access is available through the **AWS Support API**.

Quick Memory Trick

Trusted Advisor = AWS Best-Practice Advisor

Cost Optimization

Performance

Security

Fault Tolerance

Service Limits

Operational Excellence

Remember:

"Trusted Advisor checks your AWS account and tells you what can be improved."

Common Exam Question

Which AWS service provides automated recommendations for cost optimization, security, fault tolerance, service limits, performance, and operational excellence?

✅ **AWS Trusted Advisor**.

Support Plans for AWS

AWS offers several support plans that provide different levels of technical assistance, guidance, and response times.

Support Plan Hierarchy

Basic Support (Free)

↓

Business Support+

↓

Enterprise Support

↓

Unified Operations

As you move up the tiers:

- More AWS expertise becomes available
- Faster response times
- More proactive guidance
- Access to dedicated support personnel

1. Basic Support Plan (Free)

Included with every AWS account.

Features

✅ 24/7 access to:

- Customer Service
- AWS Documentation
- Whitepapers
- AWS Support Forums

✅ AWS Personal Health Dashboard

✅ Trusted Advisor Core Checks (7 checks)

Examples:

- Security Group checks
- Service limits
- Public snapshots

Best For

- Learning AWS
- Personal projects

- Non-production workloads

2. Business Support+ Plan

Designed for **production workloads**.

Includes everything in Basic Support plus:

Technical Support

✅ 24/7 access to AWS Cloud Support Engineers

Through:

- Phone
- Chat
- Web support

Unlimited Support

✅ Unlimited cases

✅ Unlimited contacts

Faster Response

For **Business-Critical System Down** issues:

Response Time < 30 minutes

Trusted Advisor

✅ Full Trusted Advisor checks

✅ Trusted Advisor API access

Generative AI Assistance

✅ Real-time contextual support assistance

Third-Party Software Support

AWS can help with:

- Ubuntu
- Linux distributions
- Common third-party software

running on AWS infrastructure.

Best For

- Production applications
- Growing companies
- Organizations needing direct AWS engineering support

3. Enterprise Support Plan

Designed for **business-critical workloads**.

Includes everything in Business Support+ plus:

Technical Account Manager (TAM)

✅ Dedicated Technical Account Manager

A TAM helps:

- Guide AWS adoption
- Review architectures
- Optimize operations
- Coordinate AWS resources

Faster Response Times

For **Production-Critical Issues**:

Response Time < 15 minutes

AWS Security Incident Response

✅ Access to AWS Security Incident Response Team

Helps with:

- Security investigations
- Recovery assistance
- Security best practices

Business Reviews

✅ Regular reviews with AWS experts

AWS Countdown

✅ TAM-led event readiness support

Examples:

- Black Friday
- Product launches
- Major migrations

AWS helps prepare your environment for critical events.

Best For

- Large enterprises
- Critical production systems
- Organizations requiring strategic AWS guidance

4. Unified Operations Support

Highest support tier.

Designed for **mission-critical workloads**.

Includes everything from Business Support+ and Enterprise capabilities, plus additional specialized personnel.

Architecture Guidance

✅ Application Architecture Guidance

AWS experts provide:

- Deep architecture reviews
- Workload assessments
- Optimization strategies

Specialized Teams

You gain access to experts such as:

TAM

Technical Account Manager

DSE

Domain Specialist Engineer

SBAS

Senior Billing & Account Specialist

IME

Incident Management Engineer

Migration Specialists

On-demand migration support

SSE

Specialist Support Engineer

Advanced Programs

✅ AWS Countdown Premium

✅ Customer Incident Response Team (CIRT)

Operational Reviews

✅ Critical workload reviews

✅ Operational procedure reviews

Best For

- Mission-critical systems
- Large enterprises

- Highly regulated industries
- Complex global AWS environments

AWS Personal Health Dashboard

Available even with the Basic plan.

Provides:

✅ Personalized service health information

✅ Alerts when your resources are impacted

Example:

RDS maintenance event

EC2 degradation affecting your instance

Networking issue affecting your VPC

Unlike the public AWS Service Health Dashboard, it shows events that specifically affect your resources.

Trusted Advisor by Support Plan

Support Plan	Trusted Advisor Access
Basic	Core checks only
Business+	Full checks + API access
Enterprise	Full checks + API access
Unified Operations	Full checks + API access

Response Time Comparison

Plan	Critical Response Time
Basic	No technical support
Business+	< 30 minutes
Enterprise	< 15 minutes
Unified Operations	Highest priority support

Exam Tips

Want free support?

✅ Basic Support

Need technical support for production workloads?

✅ Business Support+

Need a dedicated Technical Account Manager (TAM)?

✅ Enterprise Support

Need mission-critical architecture guidance and specialist teams?

✅ Unified Operations

Need full Trusted Advisor checks?

✅ Business Support+ or higher

Need AWS Security Incident Response assistance?

✅ Enterprise Support or higher

Quick Exam Cheat Sheet

Basic

→ Free

→ Customer Service

→ Personal Health Dashboard

→ 7 Trusted Advisor Checks

Business+

→ Production Workloads

→ Full Trusted Advisor

→ 24/7 Cloud Support Engineers

→ <30 min critical response

Enterprise

→ Dedicated TAM

→ Security Incident Response

→ AWS Countdown

→ <15 min critical response

Unified Operations

→ Mission-Critical Workloads

→ Architecture Guidance

→ Specialist Engineers

→ Countdown Premium

→ Customer Incident Response Team

Most Common Exam Questions

Need full Trusted Advisor checks?

✅ Business Support+ or higher

Need a TAM (Technical Account Manager)?

✅ Enterprise Support

Need help for a major launch event?

✅ Enterprise Support (AWS Countdown)

Need mission-critical operational guidance and specialist teams?

✅ Unified Operations

Need only customer service, documentation, and basic health monitoring?

✅ Basic Support Plan.

Account Best Practices Summary

- **AWS Organizations** → Manage multiple AWS accounts centrally.
- **SCPs (Service Control Policies)** → Restrict permissions across accounts.
- **AWS Control Tower** → Quickly set up and govern secure multi-account environments.
- **Tags & Cost Allocation Tags** → Organize resources and track costs.
- **IAM Best Practices**:
  - Enable **MFA**
  - Follow **Least Privilege**
  - Use strong password policies
  - Enable password rotation
- **AWS Config** → Track resource configurations and compliance changes.
- **CloudFormation** → Automate deployments across accounts and regions.
- **Trusted Advisor** → Get recommendations for cost, security, performance, and reliability.
- **CloudTrail** → Audit and record all API activity.
- **CloudWatch Logs / S3** → Store and monitor application and service logs.
- **AWS Service Catalog** → Let users deploy approved, preconfigured resources.
- **If an account is compromised**:
  - Change the root password
  - Delete/rotate access keys and credentials
  - Contact AWS Support immediately

Exam Quick Cheat Sheet

Multi-Account Management → AWS Organizations

Account Restrictions → SCPs

Easy Setup & Governance → Control Tower

Cost Tracking → Tags / Cost Allocation Tags

Compliance Tracking → AWS Config

Infrastructure as Code → CloudFormation

Audit API Calls → CloudTrail

Recommendations → Trusted Advisor

Approved Deployments → Service Catalog

Security → MFA + Least Privilege

Billing Summary

Cost Optimization

- **AWS Compute Optimizer** → Uses machine learning to recommend better EC2, EBS, Auto Scaling, and Lambda configurations to reduce costs and improve performance.

Cost Estimation

- **AWS Pricing Calculator** → Estimates the cost of an AWS architecture before deployment.

Cost Tracking

- **Billing Dashboard** → High-level view of current charges, month-to-date costs, and forecasts.
- **Cost Allocation Tags** → Track costs by project, department, environment, or application.
- **Cost and Usage Report (CUR)** → Most detailed billing and usage report available.
- **Cost Explorer** → Visualize, analyze, and forecast AWS spending (up to 12 months ahead).

Cost Monitoring

- **Billing Alarms** → CloudWatch alarms based on billing metrics (stored in **us-east-1**) that notify you when spending exceeds a threshold.
- **AWS Budgets** → Set cost, usage, Reserved Instance, or Savings Plan budgets and receive alerts based on actual or forecasted spending.

Cost Savings

- **Savings Plans** → Commit to a specific hourly spend for 1 or 3 years and receive discounts on AWS usage.

Anomaly Detection

- **AWS Cost Anomaly Detection** → Uses machine learning to detect unusual spending patterns and provide root-cause analysis.

Service Limits

- **AWS Service Quotas** → Monitor AWS service limits, create alerts when approaching quotas, and request quota increases.

Exam Cheat Sheet

Reduce Costs → Compute Optimizer

Estimate Costs → Pricing Calculator

View Current Bill → Billing Dashboard

Track Costs by Tags → Cost Allocation Tags

Detailed Billing Data → Cost & Usage Report

Analyze & Forecast → Cost Explorer

Billing Threshold Alert→ Billing Alarms

Budget Monitoring → AWS Budgets

Long-Term Discounts → Savings Plans

Detect Unusual Spend → Cost Anomaly Detection

Monitor Service Limits → Service Quotas

Most Important Exam Associations

- **Forecast future costs** → Cost Explorer
- **Most detailed billing report** → Cost & Usage Report
- **ML-based unusual spend detection** → Cost Anomaly Detection
- **Alerts on forecasted budget overruns** → AWS Budgets
- **Monitor and increase AWS limits** → Service Quotas
- **Estimate architecture cost before deployment** → Pricing Calculator
- **Optimize resources and reduce cost** → Compute Optimizer

Advanced Utility

Security Token Services (STS) Overview

WS STS (Security Token Service) is a service that provides **temporary security credentials** to access AWS resources.

What STS Provides

Instead of using permanent credentials, STS generates:

- **Access Key ID**
- **Secret Access Key**
- **Session Token**

These credentials are **temporary** and automatically expire after a specified time.

How It Works

User/Application

|

| AssumeRole

AWS STS

|

| Temporary Credentials

AWS Resources

1. User assumes an IAM role.
2. AWS STS issues temporary credentials.
3. User accesses AWS resources using those credentials.
4. Credentials expire automatically.

Common Use Cases

1. IAM Roles

Used when a user, application, or service needs temporary access to AWS resources.

Example:

- Assume an Admin role in another account.

2. Cross-Account Access

Allows users in one AWS account to access resources in another AWS account using a role.

3. Identity Federation

Users authenticate through an external identity provider such as:

- Microsoft Entra ID (Azure AD)

- Okta
- Active Directory

STS then provides temporary AWS credentials.

4. EC2 Instance Roles

When an EC2 instance has an IAM role attached:

EC2 Instance

↓

AWS STS

↓

Temporary Credentials

AWS automatically refreshes the credentials behind the scenes.

Benefits

✅ More secure than long-term credentials

✅ Credentials expire automatically

✅ Supports cross-account access

✅ Supports federated users

✅ Enables IAM roles

Exam Tip

If the question mentions:

"Temporary AWS credentials"

"AssumeRole"

"Cross-account access"

"Federated user access"

"EC2 IAM Role credentials"

The answer is usually:

✅ **AWS STS (Security Token Service)**

One-Line Memory Trick

STS = Temporary credentials for IAM roles and cross-account/federated access.

Cognito Overview

Amazon Cognito is AWS's service for **user authentication, authorization, and user management** for **web and mobile applications**.

Why Use Cognito?

If your application has:

- Hundreds
- Thousands
- Millions

of users, you **should not create IAM users** for them.

IAM users are intended for:

- AWS administrators
- Employees
- Developers using AWS resources

Application users should instead be managed by **Amazon Cognito**.

How Cognito Works

Web App / Mobile App

|

Amazon Cognito

|

User Database

- Users sign up and sign in through Cognito.
- Cognito manages user accounts and passwords.
- Your application receives authentication tokens after successful login.

Social Login Support

Cognito supports sign-in using:

- Google
- Facebook
- Apple
- Amazon
- Other identity providers

Example:

Login with Google

Login with Facebook

Login with Apple

Users can authenticate without creating a separate account.

Main Benefits

- ✅ User sign-up and sign-in
- ✅ User profile management
- ✅ Social media login integration
- ✅ Scales to millions of users
- ✅ Supports web and mobile applications
- ✅ Integrates with AWS services

Typical Architecture

User

|

Web/Mobile App

|

Amazon Cognito

|

Authenticated User

|

AWS Services

Exam Tips

IAM vs Cognito

IAM

- AWS administrators
- Employees
- Internal AWS access

Cognito

- Application users
- Customers
- Mobile app users

- Website users

When to Choose Cognito?

If a question says:

"Millions of users need to sign into a mobile or web application."

✅ **Amazon Cognito**

If a question says:

"Employees need access to AWS resources."

✅ **IAM Users / IAM Roles**

One-Line Memory Trick

Amazon Cognito = User authentication and identity management for web and mobile app users.

Directory Services Overview

What is Microsoft Active Directory (AD)?

Microsoft Active Directory (AD) is a directory service that runs on Windows Server and provides **centralized identity and access management**.

It stores and manages objects such as:

- Users
- Computers
- Printers
- File shares
- Security groups

Main Benefit

Users can log in once and access company resources across the organization.

Example:

Username: John

Password: ********

↓

Active Directory verifies identity

↓

Access company laptop, printers, file shares, etc.

AWS Directory Service

AWS provides **Directory Service** to integrate or extend Microsoft Active Directory into AWS.

1. AWS Managed Microsoft AD

A fully managed Microsoft Active Directory in AWS.

Features:

✅ Microsoft AD compatible

✅ MFA support

✅ Can establish trust with on-premises AD

✅ AWS manages the infrastructure

Use when:

You want a full Active Directory running in AWS.

2. AD Connector

AD Connector is a proxy.

AWS Applications

↓

AD Connector

↓

On-Premises AD

Features:

✅ No directory stored in AWS

✅ Users remain in on-premises AD

✅ Supports MFA

✅ Forwards authentication requests

Use when:

You already have Active Directory on-premises and want AWS resources to use it.

3. Simple AD

A lightweight, AWS-managed directory service.

Features:

✅ AD-compatible

✅ Standalone

❌ Not Microsoft Active Directory

❌ Cannot establish trust with on-premises AD

Use when:

You need a simple directory in AWS without integrating with existing AD.

Exam Tips

If you hear:

"Microsoft Active Directory"

or

"Extend Active Directory into AWS"

✅ **AWS Directory Service**

Quick Comparison

Service	Description
AWS Managed Microsoft AD	Full Microsoft AD in AWS
AD Connector	Proxy to on-premises AD
Simple AD	Lightweight standalone directory

Exam Cheat Sheet

Centralized User Management → Active Directory

Microsoft AD in AWS → AWS Managed Microsoft AD

Use Existing On-Prem AD → AD Connector

Simple Cloud Directory → Simple AD

Active Directory on AWS → AWS Directory Service

One-Line Memory Trick

AWS Directory Service = AWS solution for Microsoft Active Directory integration and management.

AWS IAM Identity Center

AWS IAM Identity Center (formerly **AWS Single Sign-On (AWS SSO)**) enables users to **sign in once and access multiple AWS accounts and applications**.

Key Benefit: Single Sign-On (SSO)

Instead of managing separate usernames and passwords for multiple AWS accounts:

One Login

↓

IAM Identity Center

↓

Multiple AWS Accounts

Users authenticate once and gain access to all authorized AWS accounts and applications.

What Can It Provide Access To?

- ✅ Multiple AWS Accounts in an AWS Organization
- ✅ AWS Management Console
- ✅ Business Applications
- ✅ SAML 2.0 Applications
- ✅ EC2 Windows Instances

Identity Sources

IAM Identity Center can use:

Built-in Identity Store

AWS stores users and groups directly in IAM Identity Center.

External Identity Providers

Integrates with:

- Microsoft Active Directory
- Okta
- OneLogin
- Other SAML 2.0 identity providers

Example

User logs in once

↓

IAM Identity Center Portal

↓

Account A

Account B

Account C

Account D

The user can switch between accounts without needing separate credentials for each account.

Why Use IAM Identity Center?

- Centralized user management
- Simplified access control
- Easier administration across AWS Organizations
- Better user experience with one login

Exam Tips

If the question says:

"Users need one login to access multiple AWS accounts."

✅ **AWS IAM Identity Center**

If the question says:

"Provide Single Sign-On (SSO) across AWS accounts."

✅ **AWS IAM Identity Center**

If the question says:

"Integrate AWS access with Active Directory, Okta, or OneLogin."

✅ **AWS IAM Identity Center**

Quick Comparison

IAM Users

One user per AWS account

IAM Identity Center

One login across multiple AWS accounts

Exam Cheat Sheet

Single Sign-On (SSO) → IAM Identity Center

Former Name → AWS SSO

Multiple AWS Accounts Access → IAM Identity Center

Centralized User Management → IAM Identity Center

AD / Okta Integration → IAM Identity Center

One Login for AWS Organization → IAM Identity Center

One-Line Memory Trick

IAM Identity Center = One login to access multiple AWS accounts and applications. ✅

Advanced Identity Summary

1. IAM (Identity and Access Management)

Used to manage identities and permissions **within a single AWS account**.

- Create IAM users, groups, and roles
- Assign permissions using policies
- Follow security best practices (MFA, least privilege)

Use for: Employees and administrators who need AWS access.

2. AWS Organizations

Used to manage **multiple AWS accounts centrally**.

- Group accounts into Organizational Units (OUs)
- Apply Service Control Policies (SCPs)
- Centralized governance and billing

Use for: Multi-account AWS environments.

3. AWS STS (Security Token Service)

Provides **temporary security credentials**.

- Temporary Access Key
- Temporary Secret Key
- Session Token

Common uses:

- AssumeRole
- Cross-account access
- Federated users
- EC2 instance roles

Key concept: Temporary, limited-privilege access.

4. Amazon Cognito

Provides authentication and user management for:

- Web applications
- Mobile applications

Features:

- User sign-up/sign-in
- Social login (Google, Facebook, Apple, etc.)

- Scales to millions of users

Use for: Application customers, not AWS administrators.

5. AWS Directory Service

Integrates **Microsoft Active Directory (AD)** with AWS.

Options include:

- AWS Managed Microsoft AD
- AD Connector
- Simple AD

Use for: Windows environments and Active Directory integration.

6. AWS IAM Identity Center (Formerly AWS SSO)

Provides **Single Sign-On (SSO)** across:

- Multiple AWS accounts
- AWS Organizations
- Business applications
- SAML 2.0 applications

Can integrate with:

- Microsoft Active Directory
- Okta
- OneLogin

Key concept: One login for multiple AWS accounts and applications.

Exam Cheat Sheet

Manage Users & Permissions → IAM

Manage Multiple Accounts → AWS Organizations

Temporary Credentials → AWS STS

Mobile/Web App Users → Amazon Cognito

Microsoft AD Integration → AWS Directory Service

Single Sign-On (SSO) → IAM Identity Center

Memory Trick

Employees → IAM

App Users → Cognito

AD Users → Directory Service

Temporary Access → STS

Multiple Accounts → Organizations

One Login → IAM Identity Center

This is the complete **Advanced Identity Services** summary commonly tested in the AWS Certified Cloud Practitioner exam. ✅

Cloud Practitioner

Other Services

Other Servies - Section Intro

This section covers **miscellaneous AWS services** that:

- Don't fit neatly into previous categories
- Occasionally appear on the AWS Certified Cloud Practitioner exam
- Are generally tested at a **high-level definition** rather than deep technical detail

Key Points

✅ Lectures will be short and concise

✅ Usually no hands-on demonstrations

✅ Focus on understanding **what the service does**

✅ Rare but possible exam topics

✅ No final summary lecture for this section

Exam Strategy

For these services, don't worry about learning every feature.

Instead, remember:

Service Name → Primary Purpose

For example:

Cognito → User authentication for apps

STS → Temporary credentials

Trusted Advisor → AWS best-practice recommendations

Compute Optimizer → Resource optimization recommendations

Takeaway

For the Cloud Practitioner exam, these services are typically tested with straightforward questions such as:

"Which AWS service provides X functionality?"

So focus on learning **one clear definition and primary use case** for each service. That is usually enough to answer exam questions correctly. ✅

Workspaces Overview

Amazon WorkSpaces is a fully managed **Desktop as a Service (DaaS)** solution that lets you provision **Windows or Linux virtual desktops in the AWS Cloud**.

What Problem Does It Solve?

It replaces traditional **Virtual Desktop Infrastructure (VDI)** that would otherwise need to be managed on-premises.

Instead of providing physical laptops or maintaining VDI servers, organizations can provide cloud-hosted desktops.

Key Features

✅ Fully managed Windows and Linux desktops

✅ Scales from a few users to thousands

✅ Pay-as-you-go pricing

✅ Secure and integrates with AWS services

✅ Accessible from anywhere

✅ Supports corporate applications and data

How It Works

User

|

Internet

|

Amazon WorkSpaces

|

Corporate Applications & Data

A user can connect from:

- Home
- Office
- Remote location

and access their cloud desktop securely.

Common Use Cases

- Remote workforce
- Work-from-home employees

- Temporary contractors
- Call centers
- Virtual training environments
- Secure access to corporate resources

Exam Tip: Minimize Latency

To reduce latency, deploy WorkSpaces **close to your users**.

Example:

California Users → WorkSpaces in US Region

Paris Users → WorkSpaces in Europe Region

General AWS Rule

To minimize latency, deploy resources as close as possible to end users.

This applies not only to WorkSpaces but to most AWS applications and services.

Exam Keywords

If you see:

- **Desktop as a Service (DaaS)**
- **Cloud-hosted Windows desktop**
- **Cloud-hosted Linux desktop**
- **Virtual Desktop Infrastructure (VDI) replacement**
- **Remote desktop solution**

✅ **Amazon WorkSpaces**

One-Line Memory Trick

Amazon WorkSpaces = Managed Windows/Linux virtual desktops in the AWS Cloud (DaaS).

💻☁️

AppStream 2.0 Overview

Amazon AppStream 2.0 is a fully managed **application streaming service** that allows users to run desktop applications from a **web browser** without installing them locally.

What Does It Do?

Instead of giving users an entire virtual desktop, AppStream 2.0 streams **individual applications**.

Examples:

- Blender
- Photoshop
- Eclipse
- Firefox
- OpenOffice
- Engineering/CAD applications

User

|

Web Browser

|

Amazon AppStream 2.0

|

Application Runs in AWS

Key Features

✅ Stream applications through a web browser

✅ No software installation required

✅ Works on almost any device

✅ Fully managed by AWS

✅ Pay only for usage

✅ Can assign different CPU, RAM, and GPU resources per application

AppStream 2.0 vs WorkSpaces

Amazon WorkSpaces

Provides a complete virtual desktop:

Windows Desktop

or

Linux Desktop

Users connect to a remote desktop and can run multiple applications.

Use case: Full desktop replacement.

Amazon AppStream 2.0

Provides only the application.

Web Browser

↓

Single Application

No full desktop is exposed to users.

Use case: Deliver a specific application through a browser.

Quick Comparison

Feature	WorkSpaces	AppStream 2.0
Service Type	Desktop as a Service (DaaS)	Application Streaming
User Gets	Full Desktop	Single Application
Access Method	Remote Desktop Client	Web Browser
OS Exposure	Windows/Linux Desktop	No Desktop
Best For	Remote Workstations	Streaming Applications

When to Use AppStream 2.0

- Provide browser-based access to desktop software
- Deliver graphics-intensive applications
- Avoid installing software on user devices
- Support users on different operating systems
- Provide secure application access

Exam Keywords

If you see:

- **Application streaming**
- **Run desktop apps in a web browser**
- **No need for a full virtual desktop**
- **Stream Photoshop, Blender, CAD applications**

- **Browser-based desktop applications**

✅ **Amazon AppStream 2.0**

If you see:

- **Full Windows desktop**
- **Virtual Desktop Infrastructure (VDI)**
- **Desktop as a Service (DaaS)**

✅ **Amazon WorkSpaces**

Memory Trick

WorkSpaces → Full Desktop

AppStream → Single Application

Amazon AppStream 2.0 = Stream desktop applications through a web browser without providing a full desktop. 🚀

IoT Core Overview

AWS IoT Core is a managed cloud service that allows **Internet of Things (IoT) devices** to securely connect to AWS and exchange data.

What is IoT?

IoT (Internet of Things) refers to internet-connected devices that collect, send, and receive data.

Examples:

- Connected cars 🚗
- Smart lights 💡
- Smart refrigerators 🧊
- Sensors 🌡️
- Industrial machines 🏭
- Smart home devices 🏠

What Does AWS IoT Core Do?

AWS IoT Core acts as a gateway between IoT devices and AWS services.

IoT Devices

↓

AWS IoT Core

↓

AWS Services

(Lambda, S3, DynamoDB, SageMaker, etc.)

It enables devices to:

✅ Connect securely to AWS

✅ Send and receive data

✅ Communicate with applications

✅ Scale to billions of devices and trillions of messages

Key Features

Secure Connectivity

Devices connect securely using authentication and encryption.

Device Communication

Supports publish/subscribe (pub-sub) messaging between:

- Devices
- Applications
- Cloud services

Massive Scale

Can handle:

- Billions of devices
- Trillions of messages

AWS Integration

Works with services such as:

- AWS Lambda
- Amazon S3
- DynamoDB
- Amazon SageMaker
- CloudWatch

Example

Smart Car

Connected Car

↓

AWS IoT Core

↓

AWS Cloud

The car sends:

- Location data
- Speed data
- Diagnostics

AWS can then:

- Store data in S3
- Analyze data using SageMaker
- Trigger actions using Lambda

Common Use Cases

- Smart homes
- Connected vehicles

- Industrial IoT
- Smart cities
- Asset tracking
- Remote monitoring

Exam Tips

If you see:

- **Internet of Things (IoT)**
- **Connected devices**
- **Sensors sending data to AWS**
- **Billions of devices**
- **Device-to-cloud communication**

✅ **AWS IoT Core**

One-Line Memory Trick

AWS IoT Core = Securely connect and manage IoT devices in the AWS Cloud. 🌐 📱 🚗 💡

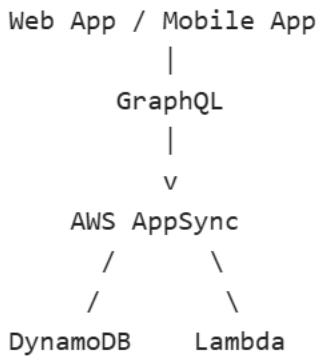
AppSync

AWS AppSync is a managed service used to build **backend APIs for web and mobile applications** using **GraphQL**.

What Does AppSync Do?

It helps applications:

- Store data
- Retrieve data
- Synchronize data in real time
- Work offline and sync later



Key Technology: GraphQL

AppSync is built around **GraphQL** (originally developed by Facebook).

GraphQL allows applications to:

- Request exactly the data they need
- Reduce unnecessary data transfer
- Simplify API development

Exam Tip

If a question mentions:

"GraphQL"

The answer is often:

✅ **AWS AppSync**

Key Features

Real-Time Data Updates

Supports GraphQL subscriptions for real-time updates.

Examples:

- Chat applications
- Live dashboards
- Multiplayer games
- Collaboration tools

Offline Data Synchronization

Applications can:

1. Continue working while offline
2. Sync data automatically when connectivity returns

Useful for mobile apps.

AWS Integrations

AppSync integrates directly with:

- **Amazon DynamoDB**
- **AWS Lambda**
- Other AWS services

This makes backend development easier.

Built-In Security

Supports authentication and authorization mechanisms to secure API access.

Common Use Cases

- Mobile app backends
- Web application backends
- Real-time applications
- Social networking apps
- Collaboration platforms
- Chat applications

AppSync and AWS Amplify

AWS Amplify can use **AppSync** behind the scenes to create GraphQL APIs quickly.

AWS Amplify

↓

AWS AppSync

↓

DynamoDB / Lambda

Exam Keywords

If you see:

- **GraphQL**
- **Real-time data synchronization**
- **Mobile/web application backend**
- **Offline synchronization**
- **GraphQL subscriptions**

✅ **AWS AppSync**

One-Line Memory Trick

AWS AppSync = GraphQL backend service for web and mobile applications with real-time and offline data synchronization. 🚀

Amplify

AWS Amplify is a set of tools and services that helps developers **build, deploy, and manage full-stack web and mobile applications** on AWS.

Think of it as:

"A development platform that simplifies building web and mobile apps using AWS services."

What Can Amplify Handle?

Amplify can manage:

- ✅ Authentication
- ✅ Storage
- ✅ APIs (REST & GraphQL)
- ✅ CI/CD deployments
- ✅ Analytics
- ✅ Monitoring
- ✅ Machine Learning integrations
- ✅ Real-time features (Pub/Sub)

How It Works

Developer

|

AWS Amplify

|

AWS Services

Amplify automatically configures AWS services behind the scenes.

Examples:

Feature Needed	AWS Service Used
User Login	Amazon Cognito
GraphQL API	AWS AppSync
REST API	API Gateway + Lambda
Storage	Amazon S3
Database	DynamoDB
AI/ML	Amazon SageMaker

Amplify Studio

Amplify Studio provides a visual interface to configure:

- Authentication
- Databases
- APIs
- Storage
- Functions
- User interfaces

without manually wiring all AWS services together.

AWS Amplify vs AWS AppSync

AWS AppSync

Focuses on:

GraphQL APIs

Real-time Sync

Offline Sync

AWS Amplify

Focuses on:

Entire Application Development

Frontend + Backend

Deployment + CI/CD

Authentication + APIs + Storage

AppSync can be used inside Amplify.

Common Use Cases

- Mobile applications
- React applications
- Angular applications
- Vue applications
- Full-stack web applications
- Startup MVPs
- Rapid application development

Exam Tips

If you see:

"Build a full-stack web or mobile application quickly"

✅ AWS Amplify

If you see:

"GraphQL backend"

✅ AWS AppSync

If you see:

"Authentication for app users"

✅ Amazon Cognito

If you see:

"Amplify automatically configures Cognito, AppSync, S3, DynamoDB, Lambda"

✅ AWS Amplify

Quick Memory Trick

Cognito → User Authentication

AppSync → GraphQL Backend

Amplify → Full-Stack App Platform

One-Line Memory Trick

AWS Amplify = The easiest way to build, deploy, and manage full-stack web and mobile applications on AWS. 🚀 📱 💻

AWS Infrastructure Composer

AWS Infrastructure Composer is a **visual drag-and-drop tool** that helps you design and build **serverless applications** on AWS and automatically generate **Infrastructure as Code (IaC)**.

What Does It Do?

Instead of writing CloudFormation or SAM templates manually, you can:

1. Drag AWS services onto a canvas.
2. Connect them visually.
3. Configure their properties.
4. Automatically generate CloudFormation code.

Example Architecture

API Gateway

|

Lambda Functions

|

DynamoDB

Using Infrastructure Composer, you visually create this architecture and AWS generates the corresponding infrastructure code.

Key Features

Visual Design

Create architectures by dragging and dropping resources such as:

- Lambda
- API Gateway
- DynamoDB
- Kinesis
- Amplify
- SQS
- SNS
- Many other AWS services

Infrastructure as Code Generation

Automatically generates:

- CloudFormation templates
- AWS SAM-compatible templates

Visualize Existing Templates

You can also:

CloudFormation Template

↓

Import into Composer

↓

Visual Diagram

This helps understand existing architectures.

Configure Relationships

Example:

Kinesis Stream

↓

Lambda Function

You can visually connect resources and define how they interact.

Benefits

- ✅ Faster serverless application design
- ✅ No need to be a CloudFormation expert
- ✅ Visual view of resource relationships
- ✅ Generates CloudFormation automatically
- ✅ Can import existing CloudFormation/SAM projects
- ✅ Simplifies Infrastructure as Code creation

Common Use Cases

- Designing serverless applications
- Learning AWS architectures
- Creating CloudFormation templates visually
- Visualizing existing CloudFormation stacks
- Rapid prototyping of AWS solutions

Exam Tips

If you see:

"Visual drag-and-drop tool for building serverless architectures"

✅ **AWS Infrastructure Composer**

If you see:

"Generate CloudFormation from a visual workflow"

✅ **AWS Infrastructure Composer**

If you see:

"Visualize existing CloudFormation or SAM templates"

✅ **AWS Infrastructure Composer**

Quick Comparison

CloudFormation

→ Write IaC manually

Infrastructure Composer

→ Design visually and generate IaC

One-Line Memory Trick

AWS Infrastructure Composer = Drag-and-drop visual builder for serverless applications that generates CloudFormation infrastructure as code. 🎨☁️🚀

Device Farm Overview

AWS Device Farm is a fully managed testing service that lets you test **mobile and web applications** on **real devices and browsers** hosted by AWS.

What Problem Does It Solve?

Applications may behave differently on:

- Different phones
- Different tablets
- Different operating systems
- Different screen sizes
- Different browsers

Instead of buying and managing hundreds of devices, AWS provides a **Device Farm** of real devices for testing.

How It Works

Your App

|

AWS Device Farm

|

+--> Real Android Devices

+--> Real iPhones/iPads

+--> Real Tablets

+--> Real Desktop Browsers

You upload your application and test scripts, and Device Farm runs the tests on multiple devices simultaneously.

Key Features

✅ Test on real mobile devices

✅ Test on real desktop browsers

✅ Run tests in parallel

✅ Faster bug detection

✅ Configure device settings such as:

- GPS
- Language
- Wi-Fi

- Bluetooth

✅ Receive:

- Test reports
- Logs
- Screenshots
- Performance data

Example Use Case

You're an Android developer and want to ensure your app works on:

- Samsung devices
- Google Pixel devices
- Different Android versions
- Various screen sizes

Instead of purchasing all these devices, you upload your app to **Device Farm** and test it across AWS's real device fleet.

Benefits

- Catch bugs before release
- Improve application quality
- Test across many device types
- Reduce testing infrastructure costs
- Speed up QA processes through parallel testing

Exam Keywords

If you see:

- **Test mobile applications on real devices**
- **Test across many phones and tablets**
- **Real device testing**
- **Mobile app compatibility testing**
- **Application testing on multiple browsers/devices**

✅ **AWS Device Farm**

Quick Comparison

AWS Device Farm

→ Test apps on real devices

AWS WorkSpaces

→ Virtual desktops

Amazon AppStream 2.0

→ Stream applications

AWS Amplify

→ Build web/mobile apps

One-Line Memory Trick

AWS Device Farm = Test web and mobile applications on real AWS-hosted phones, tablets, and browsers. 📱 🧪 💻

Provide your feedback on BizChat

AWS Backup Overview

AWS Backup is a fully managed service that allows you to **centrally manage and automate backups** across multiple AWS services.

What Does AWS Backup Do?

- Automates backups using backup plans
- Supports **on-demand** and **scheduled** backups
- Supports **Point-in-Time Recovery (PITR)**
- Manages backup retention and lifecycle policies
- Supports **cross-region** and **cross-account** backups
- Integrates with **AWS Organizations**

How It Works

AWS Backup
|
Backup Plan
(Frequency + Retention)
|
Resources
├– EC2
├– EBS
├– RDS
├– DynamoDB
├– Aurora
├– EFS
├– FSx
└– Storage Gateway
|
Backups Stored and Managed

Key Features

Backup Plans

Define:

- Backup frequency (daily, weekly, monthly)
- Retention period

- Lifecycle policies

Point-in-Time Recovery (PITR)

Recover data to a specific point in time.

Cross-Region Backup

Region A

↓

Region B

Improves disaster recovery capabilities.

Cross-Account Backup

Account A

↓

Account B

Provides additional protection against account issues.

Supported Services

Common supported services include:

- Amazon EC2
- Amazon EBS
- Amazon RDS
- Amazon Aurora
- Amazon DynamoDB
- Amazon EFS
- Amazon FSx
- AWS Storage Gateway

Benefits

✅ Centralized backup management

✅ Automated backups

✅ Consistent backup policies

✅ Easier compliance and governance

✅ Disaster recovery support

✅ Cross-account and cross-region protection

Exam Tips

If you see:

"Centrally manage backups across AWS services"

✅ **AWS Backup**

If you see:

"Automate backup schedules and retention policies"

✅ **AWS Backup**

If you see:

"Cross-account or cross-region backups"

✅ **AWS Backup**

If you see:

"Point-in-Time Recovery (PITR)"

✅ Often associated with **AWS Backup** (and supported services like DynamoDB/RDS).

One-Line Memory Trick

AWS Backup = Centralized, automated backup management for AWS resources. 💾☁️

Cloud Practitioner

Disaster Recovery Strategies

The exam mainly expects you to know the **cost order** and **basic characteristics** of each DR strategy.

Strategy	Cost	Description	Recovery Time
Backup & Restore	Lowest	Data is backed up to the cloud. During a disaster, restore the data and recreate the application.	Slowest
Pilot Light	Low-Medium	Critical core components (typically a database) run in the cloud with minimal resources. Application servers are not fully running.	Faster than Backup & Restore
Warm Standby	Medium-High	A complete but scaled-down version of the application runs in the cloud. Scale it up during a disaster.	Fast
Multi-Site / Hot Site	Highest	Full production environment runs in multiple locations at full capacity and is ready immediately.	Fastest

1. Backup & Restore

- Only backups are stored.
- Application is not running.
- During a disaster:
    - Restore data.
    - Rebuild or redeploy application.
- **Cheapest option** because you're mainly paying for storage.

✅ **Exam Tip:** If asked for the **least expensive DR strategy**, choose **Backup & Restore**.

2. Pilot Light

- Essential components are always running.
- Example:
    - Database is running in the cloud.
    - Application servers are not running.
- During a disaster:
    - Launch application servers.
    - Scale infrastructure as needed.

✅ More expensive than Backup & Restore, but recovery is faster.

3. Warm Standby

- Entire application exists in the cloud.
- Runs at reduced size/capacity.
- During a disaster:
    - Scale resources up to production size.

✅ Faster recovery than Pilot Light. ✅ Higher cost because the full application is already deployed.

4. Multi-Site / Hot Site

- Full production environment is already running.
- Traffic can be redirected immediately.
- Minimal or near-zero downtime.

✅ **Most expensive option** ✅ **Fastest recovery**

Cost Order (Important for Exam)

Cheapest → Most Expensive

Backup & Restore
↓
Pilot Light
↓
Warm Standby
↓
Multi-Site / Hot Site

AWS Multi-Region Disaster Recovery

A common AWS DR setup:

Primary Region (us-east-1)
|
Disaster
|
Route 53 Failover
|
Secondary Region (eu-west-2)

- If one AWS Region fails, traffic can be redirected to another Region using **Amazon Route 53**.
- This provides regional disaster recovery and higher availability.

Key Exam Memory Trick

- **Cheapest:** Backup & Restore
- **Most Expensive:** Multi-Site / Hot Site
- **Fastest Recovery:** Multi-Site / Hot Site
- **Slowest Recovery:** Backup & Restore

That's the core AWS CCP knowledge typically tested for Disaster Recovery strategies.

AWS Elastic Disaster Recovery (RDS)

AWS Elastic Disaster Recovery (AWS DRS) is a service that helps you recover servers running anywhere into AWS during a disaster.

Formerly known as **CloudEndure Disaster Recovery**, it was acquired by AWS and renamed to **AWS Elastic Disaster Recovery (DRS)**.

What Does AWS DRS Do?

AWS DRS continuously replicates your servers to AWS so that if a disaster occurs, you can quickly recover them in AWS with minimal downtime and data loss.

It protects:

- Physical servers
- Virtual machines (VMs)
- Cloud-based servers
- Databases (Oracle, MySQL, SQL Server, etc.)
- Enterprise applications (SAP, custom applications, etc.)

How AWS DRS Works

1. Install Replication Agent

A replication agent is installed on your source servers.

Source Server

|

AWS Replication Agent

|

Continuous Replication

2. Continuous Block-Level Replication

The agent continuously copies disk changes to AWS.

- Operating System files
- Application files
- Database files
- User data

are replicated at the **block level**.

Server Disk Changes

↓

Continuous Replication

↓

AWS Staging Area

3. Low-Cost Staging Area in AWS

Replicated data is stored in a staging environment using:

- Low-cost EC2 instances
- Amazon EBS volumes

This keeps costs lower than running a full duplicate environment.

Corporate Data Center

↓

AWS Staging Area

(EBS + Small EC2)

4. Disaster Occurs → Failover

If the source environment fails:

- Launch production-ready EC2 instances
- Attach replicated EBS volumes
- Bring applications online

Recovery can happen within minutes.

Disaster

↓

Failover

↓

Production EC2 Instances

↓

Application Running in AWS

5. Failback

Once the original environment is available again:

- Synchronize changes
- Move workloads back

This process is called **Failback**.

AWS Recovery Environment

↓

Failback

↓

Original Data Center

Key Terms

Replication

Continuous copying of server data into AWS.

Failover

Switching operations from the failed environment to AWS.

Failback

Returning operations from AWS back to the original environment.

Benefits of AWS DRS

- ✅ Continuous replication
- ✅ Low Recovery Point Objective (RPO)
- ✅ Low Recovery Time Objective (RTO)
- ✅ Supports physical, virtual, and cloud servers
- ✅ Reduces downtime
- ✅ Cost-effective staging environment
- ✅ Protects against:
  - Data center failures
  - Hardware failures
  - Natural disasters
  - Ransomware attacks

Exam Takeaways

Remember these points:

- **AWS DRS = Disaster Recovery service**
- **Formerly CloudEndure Disaster Recovery**
- Uses **continuous block-level replication**
- Replicates servers into a low-cost AWS staging area
- Supports physical, virtual, and cloud servers
- **Failover** = move to AWS
- **Failback** = move back to original environment
- Recovery can occur within **minutes**

One-Line Exam Answer

AWS Elastic Disaster Recovery (DRS) continuously replicates servers to AWS and enables fast failover and failback for disaster recovery with minimal downtime and data loss.

AWS Data Sync

AWS DataSync is a service used to **transfer large amounts of data between on-premises storage and AWS storage services**.

What AWS DataSync Does

DataSync helps you:

- Migrate data to AWS
- Replicate data between environments
- Synchronize files on a schedule
- Automate ongoing data transfers

Supported AWS destinations include:

- **Amazon S3**
- **Amazon EFS (Elastic File System)**
- **Amazon FSx for Windows File Server**

Key Exam Concept: Incremental Transfers

The most important exam keyword is:

Incremental

How it works:

1. First transfer = **Full data load**
2. Subsequent transfers = **Incremental changes only**

Initial Transfer

↓

Full Copy of Data

Future Transfers

↓

Only New/Changed Files

(Incremental)

✅ If an exam question mentions:

- "incremental synchronization"
- "scheduled file transfers"
- "ongoing replication of files"

Think **AWS DataSync**.

How DataSync Works

On-Premises Environment

You deploy a **DataSync Agent** on-premises.

On-Premises Storage

↓

DataSync Agent

AWS DataSync Service

The agent communicates securely with the managed AWS DataSync service.

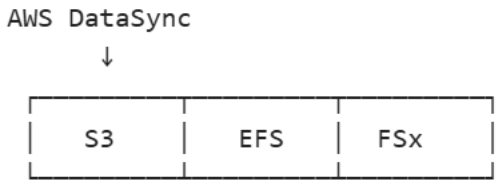
DataSync Agent

↓

AWS DataSync

Target Storage

Data is transferred to:



Scheduling

DataSync tasks can be scheduled:

- Every hour
- Every day
- Every week
- Custom schedules

This makes it useful for regular synchronization and backups.

Typical Use Cases

Data Migration

Data Center

↓

Amazon S3

Move large datasets into AWS.

Hybrid Storage Synchronization

On-Premises Files

↔

Amazon EFS

Keep data synchronized across environments.

File Replication

Corporate File Server

↓

Amazon FSx

Regularly replicate Windows file shares.

DataSync vs DRS

Service	Purpose
AWS DataSync	Move and synchronize files/data
AWS Elastic Disaster Recovery (DRS)	Recover entire servers during disasters

DataSync = Data Transfer

DRS = Server Recovery

Exam Tips

Choose AWS DataSync When:

✅ Large-scale data migration

✅ Scheduled synchronization

✅ Incremental file transfers

✅ Moving data to S3, EFS, or FSx

✅ Hybrid on-premises to AWS storage

Don't Choose DataSync When:

❌ Recovering entire servers

❌ Disaster recovery of applications

❌ Continuous server replication

For those scenarios, choose **AWS Elastic Disaster Recovery (DRS)**.

Quick Memory Trick

DataSync = "Sync Data"

On-Premises

↓

DataSync Agent

↓

AWS DataSync

↓

S3 / EFS / FSx

CCP Exam Keywords

- Large amounts of data
- Data migration
- Synchronization
- Scheduled transfers
- DataSync Agent
- Amazon S3
- Amazon EFS
- Amazon FSx
- **Incremental transfers** (most important keyword)

One-Line Exam Answer

AWS DataSync is a service that transfers and synchronizes large amounts of data between on-premises storage and AWS storage services (S3, EFS, FSx), using an initial full transfer followed by incremental updates.

Cloud Practitioner

Cloud Migration Strategies – The 7Rs

The **7 Rs of Cloud Migration** are strategies organizations use when moving applications and workloads to the cloud.

Quick Exam Summary

Strategy	Meaning	Key Idea
Retire	Remove	Turn off applications you no longer need
Retain	Keep	Leave workloads where they are
Relocate	Move Infrastructure	Move without changing the application
Rehost	Lift & Shift	Move application as-is to AWS
Replatform	Lift, Tinker & Shift	Use managed AWS services with minimal changes
Repurchase	Drop & Shop	Replace with a SaaS product
Refactor / Re-architect	Cloud-Native Redesign	Redesign application for cloud benefits

1. Retire

What it means

Applications or servers that are no longer needed are decommissioned.

Benefits

- Saves money
- Reduces maintenance effort
- Reduces security risk
- Simplifies migration

Example

Old Reporting System

↓

Not Used Anymore

↓

Retire

Exam Keyword

✅ **Turn off unused applications**

2. Retain

What it means

Keep the workload where it currently runs.

Reasons

- Regulatory requirements
- Data residency concerns
- Unresolved dependencies
- No business value in migrating

Example

Sensitive Database

↓

Remain On-Premises

Exam Keyword

✅ **Keep on-premises**

3. Relocate

What it means

Move infrastructure without changing applications.

Typical example:

- VMware on-premises
- VMware Cloud on AWS

Example

VMware Data Center

↓

VMware Cloud on AWS

Exam Keyword

✅ **Move infrastructure with minimal changes**

4. Rehost (Lift and Shift)

What it means

Move applications exactly as they are to AWS.

No redesign. No optimization.

Example

On-Prem VM

↓

Amazon EC2

Common Service

✅ AWS Application Migration Service (MGN)

Benefits

- Fast migration
- Low effort
- Quick cloud adoption

Exam Keywords

✅ Lift and Shift

✅ Migrate as-is

✅ EC2 migration

5. Replatform (Lift and Reshape)

What it means

Make small optimizations while migrating.

Application architecture remains mostly unchanged.

Examples

Database Migration

Database Server

↓

Amazon RDS

Application Migration

Application

↓

Elastic Beanstalk

Benefits

- Managed services
- Automated backups
- High availability
- Less operational burden

Exam Keywords

✅ Managed Services

✅ Amazon RDS

✅ Elastic Beanstalk

✅ Small changes only

6. Repurchase (Drop and Shop)

What it means

Replace an existing application with a SaaS product.

Examples

Existing App	SaaS Replacement
CRM	Salesforce
HR System	Workday
CMS	Drupal Cloud

Example

On-Prem CRM

↓

Salesforce

Benefits

- Fast deployment
- Reduced management
- No infrastructure maintenance

Exam Keywords

✅ SaaS

✅ Replace existing software

✅ Salesforce / Workday

7. Refactor / Re-Architect

What it means

Redesign the application to fully leverage cloud-native services.

This requires the most work but usually provides the greatest long-term benefits.

Typical Changes

- Monolith → Microservices
- Servers → Serverless
- Traditional storage → Amazon S3

- Manual scaling → Auto Scaling

Example

Monolithic Application

↓

Microservices

↓

Lambda + API Gateway + DynamoDB

Benefits

- Higher scalability
- Better performance
- Increased agility
- Improved resilience
- Reduced operational overhead

Exam Keywords

✅ Cloud-Native

✅ Microservices

✅ Serverless

✅ Most effort

✅ Greatest cloud benefit

Effort Comparison

Lowest Effort

↓

Retain

Retire

Relocate

Rehost

Replatform

Repurchase

Refactor/Re-Architect

↑

Highest Effort

AWS CCP Exam Memory Trick

If the application remains unchanged

→ Rehost (Lift & Shift)

If moving to RDS or Beanstalk

→ Replatform

If replacing with Salesforce or Workday

→ Repurchase

If redesigning into microservices/serverless

→ Refactor / Re-Architect

If shutting it down

→ Retire

If keeping it on-premises

→ Retain

If moving VMware workloads without changes

→ Relocate

One-Line Exam Answer

The **7 Rs of Cloud Migration** are **Retire, Retain, Relocate, Rehost, Replatform, Repurchase, and Refactor/Re-Architect**, representing different strategies for moving workloads to the cloud based on cost, effort, and business requirements.

Cloud Practitioner

Application Discovery Service & Application Migration Service

This lecture covers **two important AWS migration services**:

1. **AWS Application Discovery Service (ADS)** → Plan migrations
2. **AWS Application Migration Service (MGN)** → Perform migrations

1. AWS Application Discovery Service

Before moving applications to AWS, organizations need to understand:

- What servers they have
- How heavily those servers are used
- Which applications depend on each other

AWS Application Discovery Service helps collect this information.

Purpose

✅ Assess on-premises environments

✅ Discover applications and dependencies

✅ Gather utilization metrics

✅ Plan migration strategy

Information Collected

The service helps identify:

- Server inventory
- CPU utilization
- Memory usage
- Disk usage
- Application dependencies
- Network connections

This information helps determine:

- What to migrate
- Migration order
- Resource sizing in AWS

Discovery Methods

A. Agentless Discovery

Uses a **Connector**.

Provides:

- Virtual machine inventory
- Configuration details
- CPU usage
- Memory usage
- Disk utilization

VMware Environment

↓

Discovery Connector

↓

AWS Application Discovery Service

B. Agent-Based Discovery

Install an **Application Discovery Agent** inside servers.

Provides everything from agentless discovery plus:

- System configuration
- Running processes
- Detailed network connections
- Dependency mapping

Server

↓

Discovery Agent

↓

Application Discovery Service

Exam Tip

Agent-based discovery = more detailed information

especially:

- Running processes
- Network dependencies
- Application relationships

AWS Migration Hub

Discovery results can be viewed through:

AWS Migration Hub

Migration Hub provides:

- Centralized migration dashboard
- Discovery information
- Migration tracking
- Progress monitoring

Application Discovery Service

↓

Migration Hub

↓

Migration Planning

Exam Keyword

✅ **Migration Hub = Central location for migration tracking**

2. AWS Application Migration Service (MGN)

Once you know what to migrate, you can move applications to AWS using:

AWS Application Migration Service (MGN)

Formerly:

CloudEndure Migration

Purpose

MGN simplifies:

- Physical server migration
- Virtual machine migration
- Cloud-to-cloud migration

into AWS.

Migration Type Supported

Rehost (Lift-and-Shift)

The application is moved with minimal changes.

On-Premises Server

↓

AWS Application Migration Service

↓

Amazon EC2

Exam Keyword

✅ **Lift-and-Shift**

✅ **Rehosting**

✅ **Minimal application changes**

How MGN Works

Step 1: Install Replication Agent

A replication agent is installed on source servers.

Source Server

↓

Replication Agent

Step 2: Continuous Replication

The agent continuously copies disk data to AWS.

Server Disk

↓

Continuous Replication

↓

AWS Staging Area

Replicated components include:

- Operating system
- Applications
- Databases
- Configuration
- Data

Step 3: Low-Cost Staging Area

AWS stores replicated data in:

- Low-cost EC2 instances
- Amazon EBS volumes

AWS Staging Environment

- Low-cost EC2

- EBS Volumes

This reduces migration costs before cutover.

Step 4: Cutover

When ready:

- Launch production EC2 instances
- Attach replicated data
- Switch traffic to AWS

Staging

↓

Cutover

↓

Production EC2

Exam Keyword

✅ **Cutover** = Final switch to AWS production environment

Benefits of MGN

✅ Lift-and-shift migration

✅ Continuous replication

✅ Minimal downtime

✅ Reduced migration effort

✅ Supports many operating systems

✅ Supports physical, virtual, and cloud servers

✅ Automated migration process

Application Discovery Service vs Application Migration Service

Service	Purpose
Application Discovery Service	Discover and assess environments before migration
Application Migration Service (MGN)	Actually migrate servers to AWS
Migration Hub	Track migration progress centrally

Easy Memory Trick

Discover → Plan

Migrate → Move

Hub → Monitor

CCP Exam Keywords

AWS Application Discovery Service

- Migration assessment
- Dependency mapping
- Server utilization
- Agentless Discovery
- Discovery Agent
- Migration planning

AWS Migration Hub

- Central dashboard
- Migration tracking

AWS Application Migration Service (MGN)

- Formerly CloudEndure Migration
- Lift-and-Shift
- Rehosting
- Continuous replication
- Cutover
- Minimal downtime

One-Line Exam Answers

AWS Application Discovery Service:

Collects server utilization and dependency information to help plan migrations to AWS.

AWS Application Migration Service (MGN):

Performs lift-and-shift migrations by continuously replicating source servers and launching them on AWS with minimal downtime.

AWS Migration Hub:

Provides a central dashboard to track and monitor AWS migration projects.

AWS Migration Evaluator

AWS Migration Evaluator helps organizations build a **data-driven business case** for migrating to AWS.

Think of it as a service that answers:

"If we move to AWS, what will it cost, what will we save, and what is the best migration plan?"

Purpose of Migration Evaluator

Before migrating, organizations want to understand:

- Current infrastructure costs
- Server inventory
- Resource utilization
- Dependencies between workloads
- Expected AWS costs
- Potential savings

Migration Evaluator collects and analyzes this information.

How It Works

1. Discover Current Environment

Migration Evaluator gathers information about your on-premises infrastructure.

This includes:

- Servers
- Applications
- Resource utilization
- Infrastructure footprint
- Dependencies

Option A: Agentless Collector

Install the **Agentless Collector** to automatically discover infrastructure.

On-Premises Environment

↓

Agentless Collector

↓

Migration Evaluator

Option B: Data Import

If you already have infrastructure data, you can import it.

Existing Data

↓

Template

↓

Migration Evaluator

No discovery agent is required in this approach.

Analysis Process

Current State Assessment

Migration Evaluator analyzes:

- Current workloads
- Resource consumption
- Server dependencies
- Existing infrastructure costs

Current Environment

↓

Baseline Assessment

Define Target AWS Environment

The service helps map workloads to AWS services and resources.

Current State

↓

Target AWS State

Migration Planning

Migration Evaluator can help develop:

- Migration strategies
- Sizing recommendations
- Cost projections

Key Outputs

1. Cost Analysis

Provides customized cost estimates for AWS.

Current Cost

vs

Projected AWS Cost

2. Savings Opportunities

Identifies potential:

- Infrastructure savings
- Operational savings
- Licensing optimization

3. Business Case

Creates a business justification for migration.

This helps stakeholders answer:

- Why migrate?
- How much will it cost?
- How much can we save?
- What is the expected ROI?

4. Migration Recommendations

Provides guidance on:

- Right-sizing resources
- Migration planning
- AWS target architecture

Expert Guidance

AWS specialists can help:

- Review findings
- Validate assumptions
- Improve the migration business case

Migration Evaluator vs Other Migration Services

Service	Main Purpose
Migration Evaluator	Build business case and cost analysis
Application Discovery Service	Discover servers and dependencies
Migration Hub	Track migration progress
Application Migration Service (MGN)	Perform lift-and-shift migration

Easy Memory Trick

Migration Evaluator = Cost + Savings + Business Case

Discovery Service = What do I have?

Migration Evaluator = What will it cost?

MGN = Move it to AWS

CCP Exam Keywords

Look for:

- ✅ Business case
- ✅ Cost analysis
- ✅ Cost savings
- ✅ Migration planning
- ✅ Agentless Collector
- ✅ Current-state assessment
- ✅ AWS cost estimation
- ✅ Data-driven migration decisions

One-Line Exam Answer

AWS Migration Evaluator helps organizations assess their current infrastructure, estimate AWS migration costs and savings, and build a data-driven business case for moving workloads to AWS.

AWS Migration Hub

What is AWS Migration Hub?

AWS Migration Hub is a **centralized dashboard** that helps organizations:

- Discover applications and servers
- Assess migration readiness
- Plan migrations
- Track migration progress
- Manage modernization projects

Think of it as the **single place to manage and monitor AWS migrations**.

Why Use Migration Hub?

When migrating hundreds of servers and applications, you may use multiple AWS migration tools.

Migration Hub brings everything together into one view.

Application Discovery Service

|

Application Migration Service (MGN)

|

Database Migration Service (DMS)

|

Migration Hub

|

Centralized Tracking

Key Capabilities

1. Inventory Collection

Migration Hub can collect information about:

- Servers
- Applications
- Dependencies
- Migration status

This helps you understand what needs to be migrated.

✅ Exam Keyword: **Inventory Data**

2. Assessment and Planning

Migration Hub helps with:

- Migration assessment
- Migration planning
- Workload analysis
- Migration prioritization

You can evaluate workloads before moving them to AWS.

✅ Exam Keyword: **Assess and Plan Migrations**

3. Migration Tracking

Migration Hub provides a single place to monitor:

- Migration progress
- Completed migrations
- In-progress migrations
- Application status

✅ Exam Keyword: **Track Migrations**

4. Migration Orchestration

A feature called **AWS Migration Hub Orchestrator** provides:

- Pre-built migration workflows
- Automation of migration tasks
- Guided migration processes

This reduces manual effort.

AWS Migration Hub Orchestrator

Migration Hub Orchestrator helps automate migrations using templates.

Supported migration scenarios include:

- SAP workloads
- Microsoft SQL Server
- Enterprise applications

Benefits

✅ Automated migration workflows

✅ Reduced operational effort

✅ Faster migrations

✅ Best-practice guidance

Integrations

Migration Hub works with several AWS migration services.

AWS Application Discovery Service

Used for:

- Discovery
- Dependency mapping
- Application assessment

AWS Application Migration Service (MGN)

Used for:

- Lift-and-shift migrations
- Server migrations

AWS Database Migration Service (DMS)

Used for:

- Database migration
- Continuous replication

Discovery Service

↓

Migration Hub

↓

MGN / DMS

↓

AWS

Additional Benefits

Migration Hub can help:

Right-Size Workloads

Analyze current workloads and recommend appropriate AWS resources.

Coordinate Multiple Tools

Manage migrations performed by different AWS migration services from one dashboard.

Application Modernization

Support modernization initiatives as applications are gradually moved and improved in AWS.

Exam Tips

If the question asks for:

"A central place to monitor migration projects"

✅ **AWS Migration Hub**

"A centralized dashboard for migration tracking"

✅ **AWS Migration Hub**

"Assess, plan, and track migrations"

✅ **AWS Migration Hub**

"Orchestrate migrations using templates"

✅ **Migration Hub Orchestrator**

Migration Services Quick Comparison

Service	Purpose
Migration Hub	Central dashboard for migration management
Application Discovery Service	Discover servers and dependencies
Migration Evaluator	Build migration business case and cost estimates
Application Migration Service (MGN)	Lift-and-shift server migrations
Database Migration Service (DMS)	Database migrations

CCP Exam Memory Trick

Discovery Service = What do I have?

Migration Evaluator = What will it cost?

MGN = Move servers

DMS = Move databases

Migration Hub = Manage everything

One-Line Exam Answer

AWS Migration Hub is a centralized service used to discover, assess, plan, orchestrate, and track application and server migrations to AWS across multiple migration tools such as MGN, DMS, and Application Discovery Service.

AWS Fault Injection Simulator

What is AWS Fault Injection Simulator (FIS)?

AWS Fault Injection Simulator (FIS) is a managed service that allows you to perform **fault injection experiments** on AWS workloads.

It helps test how applications behave when failures occur.

FIS is AWS's service for implementing **Chaos Engineering**.

What is Chaos Engineering?

Chaos Engineering is the practice of intentionally introducing failures into a system to verify that it can withstand unexpected events.

Examples of failures:

- EC2 instance crashes
- High CPU utilization
- Memory exhaustion
- Network interruptions
- ECS task failures
- Database failures

Normal Application

↓

Introduce Failure

↓

Observe Behavior

↓

Improve Resilience

Why Use AWS FIS?

Organizations use FIS to:

✅ Test application resilience

✅ Validate disaster recovery procedures

✅ Identify hidden bugs

✅ Discover performance bottlenecks

✅ Improve reliability and availability

✅ Verify monitoring and alerting systems

How AWS FIS Works

Step 1: Create an Experiment

Define what type of failure should occur.

Examples:

- Stop EC2 instances
- Restart resources
- Inject network latency
- Cause database disruptions

AWS FIS

↓

Experiment Definition

Step 2: Select Target Resources

Choose resources to test.

Examples:

- EC2
- ECS
- EKS
- RDS

Experiment

↓

Target Resources

Step 3: Run Experiment

AWS introduces controlled failures.

FIS

↓

Disruption

↓

Application

Step 4: Monitor Results

Monitor system behavior using:

- Amazon CloudWatch

- Amazon EventBridge
- AWS X-Ray

Disruption

↓

CloudWatch Metrics

↓

Analyze Results

Step 5: Improve the Application

Based on findings:

- Fix bottlenecks
- Add redundancy
- Improve monitoring
- Enhance recovery processes

Supported AWS Services (Examples)

Amazon EC2

Examples:

- Stop instances
- Reboot instances
- Terminate instances

Amazon ECS

Examples:

- Stop ECS tasks
- Simulate task failures

Amazon EKS

Examples:

- Terminate Kubernetes pods
- Test cluster resiliency

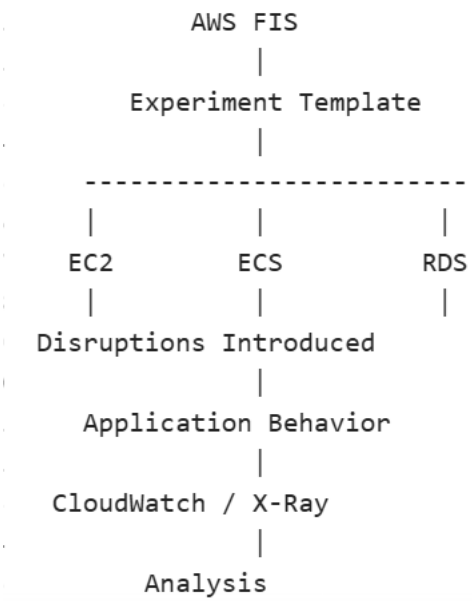
Amazon RDS

Examples:

- Simulate database failures

- Validate failover behavior

Simple Architecture



Benefits

Improves Resilience

Confirms applications can survive failures.

Tests Recovery Procedures

Validates:

- Auto Scaling
- Failover mechanisms
- Disaster recovery processes

Finds Hidden Issues

Reveals:

- Performance bottlenecks
- Single points of failure
- Monitoring gaps

Safe and Controlled

Experiments are planned and controlled, reducing risk.

Exam Tips

If the question asks:

"Which AWS service helps perform chaos engineering?"

✅ **AWS Fault Injection Simulator (FIS)**

"Which service intentionally injects failures into AWS workloads?"

✅ **AWS FIS**

"How can you test application resiliency against disruptions?"

✅ **AWS FIS**

"Which service helps identify hidden reliability issues before production failures occur?"

✅ **AWS FIS**

AWS CCP Keywords

Remember these keywords:

- Chaos Engineering
- Fault Injection
- Experiment Templates
- Application Resilience
- Performance Bottlenecks
- Controlled Failures
- EC2 Disruptions
- ECS Task Failures
- RDS Failures
- CloudWatch Monitoring

One-Line Exam Answer

AWS Fault Injection Simulator (FIS) is a managed service that uses Chaos Engineering techniques to inject controlled failures into AWS workloads, helping organizations test resiliency, identify bottlenecks, and improve application reliability.

AWS Step Function

What is AWS Step Functions?

AWS Step Functions is a **serverless workflow orchestration service** that lets you coordinate multiple AWS services into a visual workflow.

It is commonly used to orchestrate:

- AWS Lambda functions
- Amazon ECS tasks
- AWS Batch jobs
- Amazon EC2 operations
- API Gateway calls
- SQS queues
- Many other AWS services

Think of it as a **workflow engine** that manages the execution order of tasks.

Why Use Step Functions?

Without Step Functions:

Application

↓

Lambda 1

↓

Custom Code

↓

Lambda 2

↓

More Custom Logic

↓

Lambda 3

You must write code to manage:

- Sequencing
- Retries
- Error handling
- Timeouts
- Parallel execution

With Step Functions:

Step Function Workflow

↓

Lambda 1

↓

Lambda 2

↓

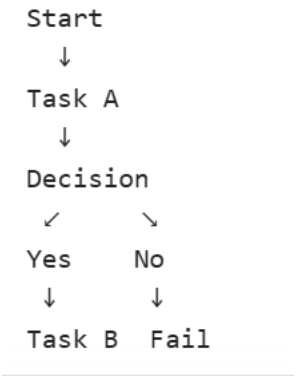
Lambda 3

AWS manages the workflow logic for you.

Key Features

1. Visual Workflows

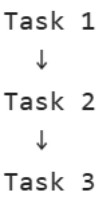
Workflows are represented as a state machine (graph).



This makes complex business processes easier to understand and manage.

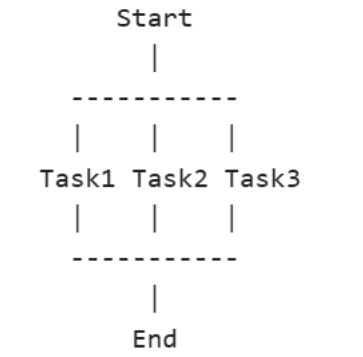
2. Sequential Execution

Run tasks one after another.



3. Parallel Execution

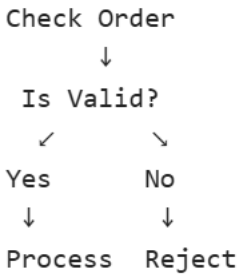
Run multiple tasks at the same time.



Useful for faster processing.

4. Conditional Logic

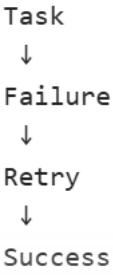
Perform different actions depending on results.



5. Error Handling & Retries

Built-in support for:

- Automatic retries
- Exception handling
- Recovery workflows



6. Timeouts

Prevent workflows from running indefinitely.

```
Task
 ↓
Timeout Reached
 ↓
Error Handling
```

7. Human Approval Steps

A workflow can pause and wait for human input.

Example:

```
Generate Report
        ↓
Manager Approval
      ↙   ↘
Approve  Reject
   ↓       ↓
Continue  Stop
```

This is useful for approval processes and governance workflows.

Integrations

Step Functions integrates with many AWS services.

Examples:

- AWS Lambda
- Amazon EC2
- Amazon ECS
- AWS Batch
- API Gateway
- Amazon SQS
- Amazon SNS
- DynamoDB
- SageMaker
- On-premises systems

Exam Tip

If you see a question about:

"Coordinating multiple AWS services in a workflow"

The answer is usually:

✅ **AWS Step Functions**

Common Use Cases

Order Processing

```
Receive Order
      ↓
Validate Order
      ↓
Process Payment
      ↓
Ship Product
      ↓
Notify Customer
```

Data Processing

```
Receive Data
      ↓
Transform Data
      ↓
Analyze Data
      ↓
Store Results
```

Web Applications

```
User Request
      ↓
Authenticate User
      ↓
Process Request
      ↓
Return Response
```

Approval Workflows

```
Create Request
      ↓
Manager Approval
      ↓
Execute Action
```

Step Functions vs Lambda

Service	Purpose
AWS Lambda	Executes code
AWS Step Functions	Orchestrates workflows and coordinates services

Easy memory trick:

Lambda = Do Work

Step Functions = Coordinate Work

CCP Exam Keywords

Look for:

- ✅ Serverless workflow
- ✅ Orchestration
- ✅ State machine
- ✅ Visual workflow
- ✅ Sequential steps
- ✅ Parallel execution
- ✅ Conditional branching
- ✅ Error handling
- ✅ Human approval
- ✅ Coordinate Lambda functions

One-Line Exam Answer

AWS Step Functions is a serverless workflow orchestration service that uses visual state machines to coordinate AWS services, providing sequencing, parallel execution, conditions, retries, error handling, and human approval workflows.

AWS Ground Station

What is AWS Ground Station?

AWS Ground Station is a **fully managed service** that allows you to:

- Control satellite communications
- Receive satellite data
- Process satellite data in AWS
- Scale satellite operations

It eliminates the need to build and manage your own physical satellite ground stations.

The Problem It Solves

Traditionally, organizations operating satellites needed:

- Physical antenna infrastructure
- Ground stations around the world
- Specialized networking equipment
- Satellite communication expertise

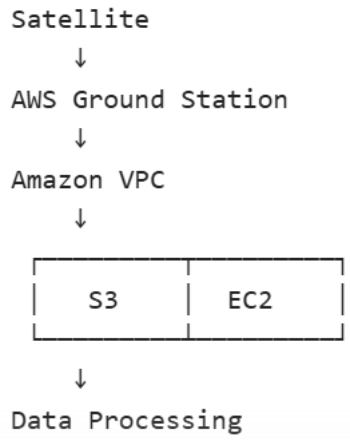
AWS Ground Station provides this infrastructure as a managed AWS service.

How It Works

```
Satellite
     ↓
AWS Ground Station
     ↓
AWS Region / VPC
     ↓
S3 / EC2 / Analytics Services
`
```

1. A satellite passes over an AWS Ground Station.
2. AWS Ground Station communicates with the satellite.
3. Data is downloaded.
4. Data is delivered directly into your AWS environment.

Data Flow Example



The satellite data can be stored in:

- **Amazon S3**
- **Amazon EC2**
- Other AWS analytics and processing services

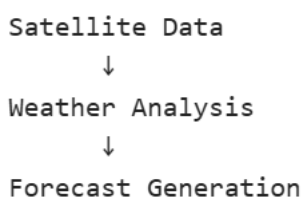
Key Benefit

✅ Download satellite data directly into AWS within seconds.

This allows organizations to immediately process imagery, telemetry, and communications data using AWS services.

Common Use Cases

Weather Forecasting



Earth/Surface Imaging

Satellites capture:

- Land imagery
- Ocean imagery
- Environmental data

The data can be stored in S3 and analyzed using AWS services.

Communications

Used for transmitting and receiving communications through satellites.

Video Broadcasting

Satellite video feeds can be received and processed in AWS.

Exam Tips

If the question mentions:

- Satellite communications
- Receiving satellite data
- Managing satellite ground stations
- Downloading data from satellites into AWS

✅ **AWS Ground Station**

AWS Ground Station vs Other AWS Services

Service	Purpose
AWS Ground Station	Receive and process satellite data
Amazon S3	Store data
Amazon EC2	Process data
AWS Step Functions	Orchestrate workflows
AWS DataSync	Transfer on-premises data

CCP Exam Keywords

Remember these keywords:

- Satellite communications
- Ground stations
- Satellite operations
- Download satellite data
- AWS VPC
- Amazon S3
- Amazon EC2
- Weather forecasting
- Surface imaging
- Communications
- Video broadcast

One-Line Exam Answer

AWS Ground Station is a fully managed service that enables satellite communication, allowing organizations to receive, process, and analyze satellite data directly within AWS using services such as Amazon S3 and Amazon EC2.

AWS Pinpoint

What is Amazon Pinpoint?

Amazon Pinpoint is a **scalable marketing and customer engagement service** that allows businesses to send communications to customers through multiple channels.

Supported channels include:

- Email
- SMS
- Push notifications
- Voice messages
- In-app messaging

Think of Pinpoint as AWS's **customer engagement and marketing campaign platform**.

Main Purpose

Amazon Pinpoint helps you:

✅ Engage customers

✅ Send targeted communications

✅ Run marketing campaigns

✅ Send transactional notifications

✅ Analyze customer responses

✅ Personalize messages

Communication Flow

```
Application
      ↓
Amazon Pinpoint
      ↓
Email / SMS / Push / Voice
      ↓
Customers
      ↓
Replies & Events
      ↓
SNS / Firehose / CloudWatch
```

Key Features

1. Customer Segmentation

You can divide customers into groups based on:

- Location
- Behavior
- Demographics
- Purchase history

Example:

```
Customers
    ↓
Segments
    ↓
VIP Customers
New Users
Inactive Users
```

2. Personalized Messaging

Send customized content to each customer group.

Example:

Hi John, your order has shipped.

instead of

Dear Customer, your order has shipped.

3. Campaign Management

Pinpoint manages:

- Campaign creation
- Audience targeting
- Message templates
- Scheduling
- Delivery

Create Campaign

↓

Choose Audience

↓

Send Messages

↓

Track Results

4. Massive Scale

Amazon Pinpoint can send:

✅ Billions of messages per day

making it suitable for large enterprises.

Common Use Cases

Marketing Campaigns

Promotion

↓

Bulk Email Campaign

Example:

- Product launches
- Sales promotions
- Customer engagement campaigns

Transactional SMS

Order Created

↓

SMS Notification

Examples:

- Order confirmations
- Delivery updates
- Appointment reminders

Push Notifications

Mobile App

↓

Push Notification

Examples:

- Special offers
- User engagement alerts

Voice Notifications

System Event

↓

Automated Voice Call

Event Tracking

Pinpoint tracks communication events such as:

- Delivered messages
- Failed deliveries
- Opens
- Clicks
- Replies

Events can be sent to:

- Amazon SNS
- Amazon Kinesis Data Firehose
- Amazon CloudWatch Logs

Customer Response

↓

Pinpoint Events

↓

SNS / Firehose / CloudWatch

This enables analytics and automation.

Amazon Pinpoint vs Amazon SNS vs Amazon SES

Amazon SNS

Used for:

- Notifications
- Pub/Sub messaging
- Simple messaging

You manage:

- Recipients
- Scheduling
- Delivery logic

Amazon SES

Used for:

- Email sending
- Transactional emails
- Bulk email delivery

You manage:

- Audience
- Email campaigns
- Scheduling

Amazon Pinpoint

Provides:

- Customer segmentation
- Personalization
- Campaign management
- Templates
- Delivery schedules
- Analytics

AWS manages much more of the marketing workflow.

Comparison

Service	Best For
SNS	Notifications and Pub/Sub messaging
SES	Sending emails
Pinpoint	Full marketing campaigns and customer engagement

Easy Memory Trick

SNS = Notify

SES = Email

Pinpoint = Marketing Platform

Exam Tips

Choose **Amazon Pinpoint** when the question mentions:

- ✅ Marketing campaigns
- ✅ Customer engagement
- ✅ SMS campaigns
- ✅ Push notifications
- ✅ Personalized messaging
- ✅ Customer segmentation
- ✅ Communication analytics
- ✅ Multi-channel messaging

Do **not** choose Pinpoint when the requirement is simply:

- Sending an email → SES
- Sending basic notifications → SNS

CCP Exam Keywords

Remember:

- Customer engagement
- Marketing campaigns
- SMS messaging
- Email campaigns
- Push notifications
- Voice messaging
- In-app messaging
- Segmentation
- Personalization
- Analytics
- Billions of messages

One-Line Exam Answer

Amazon Pinpoint is a scalable customer engagement service used to create targeted marketing campaigns and send personalized email, SMS, push notifications, voice, and in-app messages while tracking customer responses and engagement.

Cloud Practitioner

AWS Architecture & Ecosystem

AWS Whitepapers Well-Architected Framework

General Cloud Architecture Principles

1. **Stop Guessing Capacity**
    - Use **Auto Scaling** to match resources with actual demand.
    - Scale up/down dynamically instead of provisioning for peak capacity.
2. **Test at Production Scale**
    - Cloud resources can be created quickly and temporarily.
    - Perform load and stress testing using production-sized environments.
3. **Automate Everything**
    - Use **Infrastructure as Code (IaC)** such as **AWS CloudFormation**.
    - Automation makes deployments, testing, and architectural experimentation easier.
    - Platform-as-a-Service offerings (e.g., **AWS Elastic Beanstalk**) help speed up experimentation.
4. **Design for Evolution**
    - Architectures should adapt to changing business requirements.
    - Initial cloud migrations may be “lift-and-shift,” but workloads should later be optimized using cloud-native services (e.g., serverless).
5. **Make Data-Driven Decisions**
    - Base architectural choices on actual usage patterns, performance metrics, and business data.
    - Avoid assumptions and guessing.
6. **Improve Through Game Days**
    - Simulate failures, traffic spikes, and disaster scenarios.
    - Example: **Netflix Chaos Monkey**, which randomly terminates servers to test system resilience.

AWS Cloud Design Principles

1. Scalability

- **Vertical Scaling**: Increase the size of a server.
- **Horizontal Scaling**: Add more servers/resources.

2. Disposable Resources

- Infrastructure should be replaceable at any time.

- Always:
    - Back up data.
    - Store configurations externally.
    - Be able to rebuild environments quickly.

3. Automation

- Use:
    - Serverless services
    - Infrastructure as Code
    - Auto Scaling
    - Managed services

4. Loose Coupling

- Avoid monolithic applications.
- Break applications into smaller components.
- Use services like:
    - **Amazon SNS**
    - **Amazon SQS**
- Failures in one component should not impact others.

5. Think Services, Not Servers

- Don't rely solely on EC2.
- Prefer managed AWS services whenever possible:
    - Databases (RDS, DynamoDB)
    - Serverless (Lambda)
    - Messaging services
    - Storage services

AWS Well-Architected Framework – 6 Pillars

1. Operational Excellence

- Run and monitor systems effectively.
- Continuously improve processes and operations.

2. Security

- Protect systems, data, and assets.
- Implement identity management, detection, and protection mechanisms.

3. Reliability

- Ensure systems recover from failures.

- Design for fault tolerance and resilience.

4. Performance Efficiency

- Use resources efficiently.
- Select the right AWS services and technologies.

5. Cost Optimization

- Avoid unnecessary expenses.
- Continuously review and optimize spending.

6. Sustainability

- Minimize environmental impact.
- Use resources efficiently to reduce energy consumption.

Exam Tip

Remember the **6 AWS Well-Architected Pillars**:

O S R P C S

- **O**perational Excellence
- **S**ecurity
- **R**eliability
- **P**erformance Efficiency
- **C**ost Optimization
- **S**ustainability

These six pillars form the foundation of good architecture on AWS and generally work together rather than competing against each other.

Pillar 1: Operational Excellence

Operational Excellence is the ability to **run, monitor, and continuously improve systems** to deliver business value effectively.

Design Principles of Operational Excellence

1. Perform Operations as Code

- Treat operational procedures like software code.
- Automate infrastructure and operations.
- Use **Infrastructure as Code (IaC)** tools such as:
    - AWS CloudFormation

Benefits:

- Consistency
- Repeatability
- Reduced human error

2. Make Frequent, Small, Reversible Changes

- Deploy changes in small increments.
- Reduce risk and simplify rollback if issues occur.
- Avoid large, infrequent releases.

Why?

- Smaller changes are easier to test, troubleshoot, and recover from.

3. Refine Procedures Frequently

- Continuously improve operational processes.
- Regularly review runbooks and playbooks.
- Ensure team members are trained on updated procedures.

4. Anticipate Failure

- Assume failures will occur.
- Design systems and processes to handle them gracefully.
- Conduct testing and simulations before failures happen.

5. Learn From Failures

- Analyze incidents and outages.

- Perform root-cause analysis.
- Use lessons learned to improve systems and processes.

6. Use Managed Services

- Reduce operational overhead by leveraging AWS-managed services.
- AWS handles:
    - Infrastructure maintenance
    - Scaling
    - Patching
    - Availability

7. Implement Observability

Gain actionable insights into:

- Performance
- Reliability
- Cost
- Application behavior

Monitor systems continuously to detect issues early.

Operational Excellence Lifecycle in AWS

1. Prepare

Establish standards and infrastructure before deployment.

Activities

- Create runbooks
- Define operational standards
- Perform test deployments
- Build infrastructure as code

Relevant AWS Services

- **AWS CloudFormation**
    - Automates infrastructure provisioning.
- **AWS Config**
    - Evaluates resource compliance and configuration standards.

2. Operate

Run workloads efficiently and automate manual processes.

Goals

- Automate operations
- Release quickly
- Reduce manual intervention
- Monitor continuously

Relevant AWS Services

AWS CloudFormation

- Consistent deployments.

AWS Config

- Continuous compliance monitoring.

AWS CloudTrail

- Records AWS API activity.
- Tracks who changed what and when.

Amazon CloudWatch

- Metrics
- Logs
- Alarms
- Dashboards

AWS X-Ray

- Distributed request tracing.
- Helps identify performance bottlenecks and application errors.

3. Evolve

Continuously improve and modernize workloads.

Goals

- Deploy faster
- Deliver small changes frequently
- Improve operational processes

Relevant AWS Services

AWS CodeCommit

- Source code repository.

AWS CodeBuild

- Build automation.

AWS CodeDeploy

- Automated deployments.

AWS CodePipeline

- CI/CD orchestration.

AWS CloudFormation

- Version-controlled infrastructure updates.

Exam-Focused Summary

Key Design Principles

✅ Operations as code
✅ Small, reversible changes
✅ Refine procedures frequently
✅ Anticipate failure
✅ Learn from failures
✅ Use managed services
✅ Implement observability

Common AWS Services Associated with Operational Excellence

Area	AWS Services
Infrastructure as Code	CloudFormation
Compliance	AWS Config
Audit Tracking	CloudTrail
Monitoring	CloudWatch
Application Tracing	X-Ray
CI/CD	CodeCommit, CodeBuild, CodeDeploy, CodePipeline

Quick Memory Tip

Operational Excellence = Prepare → Operate → Evolve

- **Prepare:** CloudFormation, Config
- **Operate:** CloudTrail, CloudWatch, X-Ray
- **Evolve:** CodeCommit, CodeBuild, CodeDeploy, CodePipeline

This pillar focuses on **automation, monitoring, continuous improvement, and learning from failures** to efficiently deliver business value.

Pillar 2: Security

Definition

Security is the ability to protect information, systems, and assets while delivering business value through **risk assessments** and **risk mitigation strategies**.

The goal is not just compliance—it is to **reduce business risk**, prevent breaches, and ensure continuous protection of applications and data.

Security Design Principles

1. Implement a Strong Identity Foundation

Control who can access what.

Best Practices

- Centralize identity management.
- Grant only required permissions (**Principle of Least Privilege**).
- Use temporary credentials rather than long-term credentials.
- Enable Multi-Factor Authentication (MFA).

AWS Services

- **IAM**
- **AWS STS**
- **MFA**
- **AWS Organizations**

2. Enable Traceability

Monitor and record actions throughout your AWS environment.

Benefits

- Audit activities
- Detect suspicious behavior
- Investigate incidents
- Automate security responses

AWS Services

- **CloudTrail** (API logging)
- **CloudWatch** (metrics, logs, alarms)
- **AWS Config** (resource compliance)

3. Apply Security at All Layers

Use a **defense-in-depth** strategy.

Secure every layer:

- Edge
- Network
- VPC
- Subnet
- Load Balancer
- Compute Instances
- Operating System
- Applications
- Data

If one layer is compromised, other layers continue providing protection.

4. Automate Security Best Practices

Security should be automated whenever possible.

Examples

- Automatic compliance checks
- Automated patching
- Automated alerts
- Automated remediation

Automation reduces:

- Human error
- Operational overhead
- Response time

5. Protect Data in Transit and at Rest

Data in Transit

Use encryption while data travels across networks.

Examples:

- HTTPS
- TLS/SSL

Data at Rest

Encrypt stored data.

Examples:

- EBS encryption
- S3 encryption
- RDS encryption

AWS Services

- **AWS KMS**
- **S3 Encryption**
- **EBS Encryption**
- **RDS Encryption**

6. Keep People Away from Data

Reduce direct human access to sensitive data.

Why?

Human access increases risk:

- Accidental deletion
- Data leakage
- Insider threats

Best Practice

Automate data processing and access wherever possible.

7. Prepare for Security Events

Assume security incidents will eventually happen.

Activities

- Run simulations
- Create incident response plans
- Automate detection and recovery

Goal:

- Detect quickly
- Investigate rapidly
- Recover efficiently

Security Pillar Mapped to AWS Services

Identity and Access Management

AWS IAM

- Users

- Groups
- Roles
- Policies

AWS STS

- Temporary credentials

MFA

- Additional authentication factor

AWS Organizations

- Centralized account management

Detective Controls

Detect and investigate suspicious activity.

AWS Config

- Compliance monitoring
- Configuration tracking

AWS CloudTrail

- Logs API calls
- Security auditing

Amazon CloudWatch

- Metrics monitoring
- Alarms
- Log analysis

Infrastructure Protection

Protect resources from external and internal threats.

Amazon CloudFront

- Global content delivery
- First layer of defense against attacks

Amazon VPC

- Network isolation
- Security Groups
- Network ACLs

AWS Shield

- DDoS protection

AWS WAF

- Web Application Firewall
- Filters malicious web traffic

Amazon Inspector

- Vulnerability assessment
- Security findings

Data Protection

Protect sensitive information.

AWS KMS

- Encryption key management

Amazon S3

Supports:

- SSE-S3
- SSE-KMS
- SSE-C
- Client-Side Encryption

Other Protected Services

Elastic Load Balancer

- HTTPS/TLS support

Amazon EBS

- Encryption at rest

Amazon RDS

- Encryption at rest
- SSL/TLS support

Incident Response

Respond quickly when security incidents occur.

IAM

- Revoke access
- Disable compromised users
- Restrict permissions

AWS CloudFormation

- Rebuild infrastructure quickly

- Disaster recovery

CloudWatch Events / EventBridge

- Automated alerts
- Automated remediation workflows

Exam-Focused Summary

Security Design Principles

✅ Strong identity foundation

✅ Enable traceability

✅ Apply security at all layers

✅ Automate security best practices

✅ Protect data in transit and at rest

✅ Keep people away from data

✅ Prepare for security events

Key AWS Services to Remember

Security Area	AWS Services
Identity & Access	IAM, STS, MFA, Organizations
Auditing & Monitoring	CloudTrail, CloudWatch, Config
Network Protection	VPC, WAF, Shield, CloudFront
Vulnerability Assessment	Inspector
Encryption	KMS, S3 Encryption, EBS, RDS
Incident Response	IAM, CloudFormation, EventBridge/CloudWatch

Quick Memory Tip

Think of Security as:

Identify → Detect → Protect → Encrypt → Respond

- **Identify:** IAM, STS, MFA
- **Detect:** CloudTrail, CloudWatch, Config
- **Protect:** VPC, WAF, Shield, Inspector
- **Encrypt:** KMS, S3, EBS, RDS
- **Respond:** IAM, CloudFormation, EventBridge

This pillar focuses on ensuring that AWS workloads are protected through **identity management, monitoring, layered defenses, encryption, and incident response readiness**.

Pillar 3: Reliability

Reliability is the ability of a system to:

- Recover from infrastructure or service failures.
- Dynamically acquire resources to meet demand.
- Handle disruptions such as misconfigurations and transient network issues.
- Continue operating when failures occur.

In simple terms:

Reliability ensures that your application remains available and functional despite failures or changing workloads.

Design Principles of Reliability

1. Test Recovery Procedures

Don't assume recovery will work—test it.

Best Practices

- Simulate failures.
- Test backup restoration.
- Practice disaster recovery scenarios.
- Conduct regular recovery drills.

Goal

Verify that systems can recover quickly and correctly.

2. Automatically Recover from Failure

Failures are inevitable in cloud environments.

Best Practices

- Detect failures automatically.
- Replace failed resources automatically.
- Use self-healing architectures.

Example

If an EC2 instance fails:

- Auto Scaling detects the failure.
- A new instance is launched automatically.

3. Scale Horizontally

Instead of relying on a single large server:

- Add more servers when demand increases.
- Remove servers when demand decreases.

Benefits

- Higher availability
- Better fault tolerance
- Improved scalability

Example:

1 Large Server ❌

vs

10 Smaller Servers ✅

If one server fails, the application continues running.

4. Stop Guessing Capacity

Avoid manually estimating future resource needs.

Use:

- Auto Scaling
- Monitoring
- Load-based scaling

Benefits

- Better user experience
- Reduced outages
- Cost efficiency

5. Manage Change Through Automation

Manual changes increase risk.

Use Automation For:

- Infrastructure deployment
- Scaling
- Configuration management
- Rollbacks

Automation improves:

- Consistency
- Speed
- Reliability

Reliability and AWS Services

1. Foundations of Reliability

These services provide a strong operational foundation.

AWS IAM

Controls permissions and reduces the risk of accidental or malicious changes.

Amazon VPC

Provides a secure and reliable networking foundation.

Service Quotas (Service Limits)

Every AWS service has limits.

Examples:

- Number of VPCs
- Number of EC2 instances
- Number of Elastic IPs

Monitor these limits to prevent service disruptions.

AWS Trusted Advisor

Helps identify:

- Service quota risks
- Reliability issues
- Best-practice recommendations

2. Change Management

Manage growth and operational changes safely.

AWS Auto Scaling

Automatically adjusts capacity based on demand.

Benefits:

- Improved availability
- No capacity guessing
- Reduced operational effort

Amazon CloudWatch

Monitors:

- CPU utilization
- Memory metrics
- Application metrics

- Database performance

Provides alarms and dashboards for proactive action.

AWS CloudTrail

Tracks:

- API activity
- User actions
- Configuration changes

Useful for troubleshooting and auditing.

AWS Config

Tracks resource configurations and compliance over time.

3. Failure Management

Prepare for disasters and recover quickly.

Backups

Regular backups ensure data recovery after failures.

Common AWS Backup Targets

- EBS Snapshots
- RDS Backups
- DynamoDB Backups
- EFS Backups

AWS CloudFormation

Infrastructure as Code enables rapid recovery.

If infrastructure is lost:

- Recreate entire environments automatically.
- Restore consistent configurations.

Amazon S3

Highly durable storage for:

- Backups
- Application assets
- Disaster recovery data

Amazon S3 Glacier

Low-cost archival storage for long-term backup retention.

Best for:

- Compliance archives
- Historical backups
- Rarely accessed data

Amazon Route 53

Highly available managed DNS service.

Reliability Features

- Health Checks
- Failover Routing
- Multi-region routing

Example:

Primary Region

↓ (fails)

Route 53 detects failure

↓

Traffic redirected

↓

Secondary Region

This helps implement disaster recovery and failover strategies.

Exam-Focused Summary

Reliability Design Principles

✅ Test recovery procedures

✅ Automatically recover from failure

✅ Scale horizontally

✅ Stop guessing capacity

✅ Manage change through automation

Key AWS Services for Reliability

Reliability Area	AWS Services
Access Control	IAM
Networking	VPC
Service Limits	Service Quotas
Best Practice Checks	Trusted Advisor
Dynamic Scaling	Auto Scaling
Monitoring	CloudWatch
Auditing	CloudTrail
Configuration Tracking	Config
Disaster Recovery	Backups
Infrastructure Recovery	CloudFormation
Backup Storage	S3, Glacier
DNS Failover	Route 53

Quick Memory Tip

Think:

Reliable = Recover + Scale + Automate

Recover

- Backups
- S3
- Glacier
- Route 53
- CloudFormation

Scale

- Auto Scaling

Monitor

- CloudWatch
- CloudTrail
- Config

Foundation

- IAM
- VPC

- Service Quotas
- Trusted Advisor

Core idea: Reliability is about ensuring workloads continue to operate and recover automatically despite failures, traffic spikes, or infrastructure disruptions.

Pillar 4: Performance Efficiency

Performance Efficiency is the ability to use computing resources efficiently to meet system requirements and maintain that efficiency as demand changes and technologies evolve.

In simple terms:

Use the right resources, at the right time, in the most efficient way possible.

Design Principles of Performance Efficiency

1. Democratize Advanced Technologies

Take advantage of AWS-managed technologies instead of building everything yourself.

Examples

- AWS Lambda
- Amazon Aurora
- Machine Learning services
- Analytics services

Benefit

AWS makes advanced technologies accessible without requiring deep infrastructure expertise.

2. Go Global in Minutes

AWS allows rapid deployment across multiple Regions worldwide.

Benefits

- Lower latency
- Better user experience
- Global scalability

Example AWS Services

- CloudFormation
- CloudFront
- Route 53
- Global AWS Regions

3. Use Serverless Architectures

Reduce the need to manage servers.

Examples

- AWS Lambda
- DynamoDB

- API Gateway
- Fargate

Benefits

- Automatic scaling
- Reduced operational overhead
- Pay only for usage

4. Experiment More Often

Cloud resources can be provisioned quickly and temporarily.

Best Practices

- Test new architectures
- Benchmark performance
- Try different service combinations
- Validate scalability assumptions

Goal

Continuously improve application performance.

5. Mechanical Sympathy

Understand how AWS services work and choose the most suitable service for your workload.

Examples

- When to use RDS vs Aurora
- When to use EC2 vs Lambda
- When to use EBS vs S3
- When to use ElastiCache

The better you understand AWS services, the better your architectural decisions will be.

Performance Efficiency and AWS Services

1. Selection

Choose the appropriate AWS service for each workload.

AWS Auto Scaling

- Automatically adjusts compute resources based on demand.
- Improves performance during traffic spikes.

AWS Lambda

- Serverless execution.

- Automatic scaling.
- No server management.

Amazon EBS

- Block storage for EC2.
- Different performance options available.

Examples:

- gp3/gp2 SSD volumes
- io1/io2 Provisioned IOPS volumes

Amazon S3

- Highly scalable object storage.
- Virtually unlimited capacity.

Amazon RDS

- Managed relational databases.
- Supports performance tuning and read scaling.

Amazon Aurora

- Higher performance alternative to standard RDS engines for many workloads.

2. Review

Continuously review and improve architecture.

AWS CloudFormation

- Standardized infrastructure deployment.
- Makes performance testing and redesign easier.

AWS News Blog

- Stay updated with:
    - New AWS services
    - Performance enhancements
    - Best practices
    - Feature releases

3. Monitoring

Measure actual system performance.

Amazon CloudWatch

Monitor:

- CPU utilization

- Memory usage
- Network throughput
- Application metrics
- Database performance

CloudWatch Features

- Metrics
- Dashboards
- Alarms
- Logs

Monitoring allows data-driven performance optimization.

AWS Lambda Metrics

Track:

- Invocation count
- Duration
- Error rates
- Throttling

Helps optimize serverless workloads.

4. Trade-Offs

Performance decisions often involve trade-offs.

Amazon RDS vs Amazon Aurora

RDS

- Lower cost
- Simpler workloads

Aurora

- Higher performance
- Better scalability
- Potentially higher cost

ElastiCache

Benefit

- Extremely fast data access

Trade-Off

- Data can become stale or outdated.

AWS Snowball

Benefit

- Very large-scale data transfer

Trade-Off

- Physical device shipping takes time.

Amazon CloudFront

Benefit

- Faster global content delivery

Trade-Off

- Cached content may not update immediately everywhere.

Exam-Focused Summary

Performance Efficiency Design Principles

- ✅ Democratize advanced technologies
- ✅ Go global in minutes
- ✅ Use serverless architectures
- ✅ Experiment more often
- ✅ Mechanical sympathy

Key AWS Services

Area	AWS Services
Scaling	Auto Scaling
Serverless	Lambda
Storage	EBS, S3
Databases	RDS, Aurora
Monitoring	CloudWatch
Caching	ElastiCache
Content Delivery	CloudFront
Data Transfer	Snowball
Infrastructure as Code	CloudFormation

Quick Memory Tip

Think:

Select → Monitor → Optimize → Scale

Select

- Lambda
- EBS
- S3
- RDS
- Aurora

Monitor

- CloudWatch

Optimize

- ElastiCache
- CloudFront

Scale

- Auto Scaling
- Serverless Services

Core Exam Takeaway

Performance Efficiency is about:

- Choosing the right AWS services.
- Monitoring performance continuously.
- Leveraging serverless and managed services.
- Scaling efficiently.
- Making informed trade-offs between performance, cost, latency, and operational complexity.

Pillar 5: Cost Optimization

Cost Optimization is the ability to run systems and deliver business value at the **lowest possible cost** while avoiding unnecessary spending.

In simple terms:

Achieve the required business outcomes while spending only what is necessary.

Design Principles of Cost Optimization

1. Adopt a Consumption Model

Pay only for the resources you use.

Examples

AWS Lambda

- Pay only when code executes.
- No requests = No compute charges.

Amazon RDS

- Pay for the database instance even when it's idle.

Benefit

Avoid paying for unused capacity.

2. Measure Overall Efficiency

Continuously monitor resource usage and spending.

Questions to Ask

- Are instances underutilized?
- Are resources oversized?
- Can workloads be optimized?

AWS Service

- **Amazon CloudWatch**

Monitor:

- CPU usage
- Memory utilization
- Network traffic
- Resource consumption

3. Stop Spending Money on Undifferentiated Heavy Lifting

Instead of managing:

- Data centers
- Hardware
- Cooling
- Power
- Physical security

AWS manages the infrastructure, allowing teams to focus on applications and business value.

4. Analyze and Attribute Expenditure

Understand exactly where money is being spent.

Best Practice

Use **Tags** on AWS resources.

Example:

- Environment = Production
- Team = Finance
- Application = CRM

This enables:

- Cost tracking
- Chargeback/showback
- ROI analysis

5. Use Managed Services

Managed services often reduce:

- Operational costs
- Maintenance effort
- Staffing requirements

Because AWS operates at massive scale, services are usually more cost-effective than self-managed solutions.

Examples

- RDS
- DynamoDB
- Lambda
- S3
- Fargate

Cost Optimization and AWS Services

Cost Awareness

AWS Budgets

Set budgets and receive alerts when spending exceeds thresholds.

AWS Cost Explorer

Analyze:

- Historical costs
- Spending trends
- Forecasted costs

Cost and Usage Reports (CUR)

Provides detailed billing information for deep analysis.

Reserved Instance Reporting

Track whether purchased Reserved Instances are being fully utilized.

Cost-Effective Resources

Spot Instances

Benefits:

- Up to significant discounts compared to On-Demand pricing.
- Ideal for fault-tolerant workloads.

Trade-Off:

- AWS can reclaim instances when capacity is needed.

Reserved Instances

Best for:

- Long-running workloads
- Predictable usage patterns

Benefits:

- Lower pricing compared to On-Demand instances

Amazon S3 Glacier

Designed for:

- Long-term archival storage
- Infrequently accessed data

Benefit:

- Very low storage cost

Match Supply with Demand

Avoid overprovisioning resources.

AWS Auto Scaling

Automatically:

- Adds capacity when demand increases
- Removes capacity when demand decreases

AWS Lambda

Naturally follows demand:

- No idle servers
- Automatic scaling

Optimize Over Time

Cost optimization is not a one-time activity.

AWS Trusted Advisor

Provides recommendations including:

- Idle resources
- Underutilized instances
- Cost-saving opportunities

Cost and Usage Reports

Help identify:

- Cost trends
- Expensive workloads
- Wasteful resources

AWS News & Feature Updates

New AWS features can reduce costs.

Example: Application Load Balancers gained native HTTP → HTTPS redirection, eliminating the need for additional redirect servers and reducing infrastructure costs.

Exam-Focused Summary

Cost Optimization Principles

✅ Adopt a consumption model

✅ Measure overall efficiency

- ✅ Stop spending on data center operations
- ✅ Analyze and attribute expenditure
- ✅ Use managed services

Important AWS Services

Cost Area	AWS Services
Budget Tracking	AWS Budgets
Cost Analysis	Cost Explorer
Detailed Billing	Cost & Usage Reports
Cost Recommendations	Trusted Advisor
Scaling	Auto Scaling
Serverless	Lambda
Cost Savings	Reserved Instances, Spot Instances
Low-Cost Storage	S3 Glacier

Common Cost Optimization Techniques

Compute

- Use Auto Scaling
- Use Spot Instances
- Use Reserved Instances for predictable workloads
- Shut down unused resources

Storage

- Move archives to Glacier
- Use S3 lifecycle policies

Databases

- Right-size instances
- Use on-demand capacity where appropriate
- Monitor utilization

Monitoring

- Track usage with CloudWatch
- Review Cost Explorer regularly
- Use Trusted Advisor recommendations

Quick Memory Tip

Think:

Measure → Optimize → Save

- **Measure:** CloudWatch, Cost Explorer, Budgets
- **Optimize:** Auto Scaling, Lambda, Trusted Advisor
- **Save:** Reserved Instances, Spot Instances, Glacier

Core Exam Takeaway

Cost Optimization is about **paying only for what you use, selecting the most cost-effective resources, leveraging managed services, matching capacity to demand, and continuously monitoring and improving spending over time.**

Provide your feedback on BizChat

Pillar 6: Sustainability

Sustainability focuses on minimizing the environmental impact of running cloud workloads while still meeting business and technical requirements.

In simple terms:

Achieve business goals using the least amount of energy and resources possible.

This pillar encourages organizations to improve efficiency, reduce waste, and take advantage of AWS innovations that lower environmental impact.

Sustainability Design Principles

1. Understand Your Impact

Measure and monitor the environmental impact of workloads.

Goals

- Define sustainability objectives.
- Establish performance indicators.
- Measure progress over time.
- Evaluate improvements continuously.

2. Maximize Utilization

Underutilized resources waste energy and increase costs.

Best Practices

- Right-size workloads.
- Remove idle resources.
- Use Auto Scaling.
- Match capacity to actual demand.

3. Anticipate and Adopt New, Efficient Hardware

AWS regularly introduces more energy-efficient infrastructure.

Examples

- New EC2 instance generations
- Graviton-based processors
- Storage optimizations

Using newer services often delivers:

- Better performance
- Lower cost

- Improved energy efficiency

4. Use Managed Services

Managed services allow AWS to optimize infrastructure across many customers.

Benefits

- Higher resource utilization
- Reduced operational waste
- Greater efficiency at cloud scale

Examples

- Lambda
- DynamoDB
- Aurora
- Fargate
- S3

5. Reduce Downstream Impact

Optimize workloads so that customers and applications consume fewer resources.

Examples

- Reduce data transfers
- Use caching
- Improve application efficiency
- Minimize unnecessary processing

This reduces overall energy consumption across the entire system.

AWS Services Supporting Sustainability

Compute Optimization

Auto Scaling

Ensures resources match actual demand.

Benefits:

- Avoids idle resources
- Reduces wasted compute capacity

AWS Lambda

Serverless compute service.

Benefits:

- No idle servers
- Automatically scales
- Only consumes resources when executing

AWS Fargate

Serverless container platform.

Benefits:

- No infrastructure management
- Efficient resource allocation

AWS Graviton Instances

ARM-based AWS processors.

Benefits:

- Better performance-per-watt
- Lower power consumption
- Improved cost efficiency

Spot Instances

Uses spare AWS capacity.

Benefits:

- Reduces wasted infrastructure
- Lower cost
- Better resource utilization

Storage Optimization

Amazon S3 Glacier

Designed for long-term archival storage.

Benefits:

- Lower storage costs
- More efficient storage usage

Amazon EFS Infrequent Access (EFS-IA)

Moves less frequently accessed files into a lower-cost storage tier.

EBS Cold HDD (sc1)

Suitable for:

- Infrequently accessed workloads
- Archival storage

Provides lower-cost storage with reduced performance requirements.

Storage Lifecycle Management

S3 Lifecycle Policies

Automatically move objects between storage classes.

Example:

S3 Standard

↓

S3 Standard-IA

↓

S3 Glacier

↓

S3 Glacier Deep Archive

This helps ensure data stays in the most efficient storage tier.

S3 Intelligent-Tiering

Automatically moves objects between tiers based on access patterns.

Benefits:

- Cost savings
- Improved efficiency
- Reduced administrative effort

Amazon Data Lifecycle Manager

Automates:

- Snapshot creation
- Backup retention
- Lifecycle management

Database and Network Efficiency

Amazon RDS Read Replicas

Benefits:

- Distribute read traffic
- Improve performance
- Optimize resource usage

Amazon Aurora Global Database

Provides efficient global read access while minimizing replication overhead.

DynamoDB Global Tables

Supports globally distributed applications with optimized data access.

Amazon CloudFront

Caches content closer to users.

Benefits:

- Reduced latency
- Fewer requests to origin servers
- Reduced network traffic
- Improved efficiency

Exam-Focused Summary

Sustainability Design Principles

- ✅ Understand environmental impact
- ✅ Maximize utilization
- ✅ Adopt efficient hardware and software
- ✅ Use managed services
- ✅ Reduce downstream impact

Key AWS Services

Area	AWS Services
Compute Efficiency	Auto Scaling, Lambda, Fargate
Efficient Hardware	Graviton Instances
Spare Capacity Usage	Spot Instances
Storage Optimization	Glacier, EFS-IA, EBS Cold HDD

Lifecycle Management	S3 Lifecycle, Intelligent-Tiering, Data Lifecycle Manager
Database Efficiency	RDS Read Replicas, Aurora Global Database, DynamoDB Global Tables
Content Delivery	CloudFront

Quick Memory Tip

Think:

Measure → Optimize → Modernize

Measure

- Cost Explorer
- Monitoring tools

Optimize

- Auto Scaling
- Lambda
- Fargate
- Storage Tiering

Modernize

- Graviton
- Managed Services
- Global Database Services

Complete Well-Architected Framework Pillars

1. **Operational Excellence** – Run and improve systems effectively.
2. **Security** – Protect systems and data.
3. **Reliability** – Recover from failures and scale automatically.
4. **Performance Efficiency** – Use resources efficiently.
5. **Cost Optimization** – Deliver value at the lowest cost.
6. **Sustainability** – Minimize environmental impact and maximize resource efficiency.

Mnemonic: OSRPCS

Operational Excellence
Security
Reliability
Performance Efficiency
Cost Optimization
Sustainability

AWS Well-Architected Tool

What is the AWS Well-Architected Tool?

The **AWS Well-Architected Tool** helps you review workloads against the **AWS Well-Architected Framework's 6 pillars** and identify areas for improvement.

Purpose

- Evaluate cloud architectures.
- Identify risks and weaknesses.
- Receive AWS best-practice recommendations.
- Generate improvement plans and reports.
- Track architectural improvements over time.

The 6 Pillars Reviewed

The tool evaluates workloads against:

1. Operational Excellence
2. Security
3. Reliability
4. Performance Efficiency
5. Cost Optimization
6. Sustainability

How the Well-Architected Tool Works

Step 1: Define a Workload

Provide information such as:

- Workload Name
- Description
- Owner
- Environment (Production, Pre-Production, etc.)
- AWS Regions
- AWS Account IDs

Example:

Demo Workload

Environment: Production

Region: us-east-1

Step 2: Apply a Lens

A **Lens** is a set of questions tailored to a specific type of workload.

AWS provides several lenses, including:

- AWS Well-Architected Framework Lens
- Serverless Lens
- SaaS Lens
- Financial Services Lens
- Custom Lenses

Default Lens:

- AWS Well-Architected Framework Lens

Step 3: Answer Questions

The tool asks questions for each pillar.

Examples:

Security

- How do you protect data in transit?
- How do you manage identities?

Reliability

- How do you recover from failures?
- How do you back up data?

Cost Optimization

- How do you monitor spending?
- Are resources right-sized?

You can:

- Select answers
- Add notes
- Mark questions as not applicable

Step 4: Review Results

After completing the assessment, the tool evaluates answers and identifies:

- High Risk Issues (HRI)
- Medium Risk Issues (MRI)

Step 5: Generate Improvement Plan

The tool provides:

✅ Improvement recommendations

✅ AWS documentation links

✅ Best-practice guidance

✅ Action plans

Milestones

The tool allows you to create **Milestones**.

Purpose

Capture the state of a workload at a specific point in time.

Example:

Milestone 1: Initial Review

Milestone 2: After Security Improvements

Milestone 3: Production Readiness Review

This helps track architectural progress over time.

Key Outputs

Risk Reports

- High Risk Issues
- Medium Risk Issues

Improvement Plans

- Recommended actions
- AWS best practices

Documentation Links

- Related whitepapers
- Service recommendations
- Best-practice references

Exam Tips

What does the AWS Well-Architected Tool do?

✅ Reviews workloads against the Well-Architected Framework.

✅ Helps identify architectural risks.

✅ Provides improvement recommendations.

✅ Generates reports and milestones.

✅ Supports multiple lenses.

Key Terms to Remember

Workload

The application or system being reviewed.

Lens

A collection of questions focused on a specific architecture type.

HRI

High Risk Issue

MRI

Medium Risk Issue

Milestone

A snapshot of an assessment at a specific point in time.

Quick Exam Summary

AWS Well-Architected Tool = Assess → Identify Risks → Get Recommendations → Improve Architecture

Process:

Create Workload

↓

Apply Lens

↓

Answer Questions

↓

Review Risks (HRI/MRI)

↓

Generate Improvement Plan

↓

Create Milestones

Most likely exam question

Q: What is the purpose of the AWS Well-Architected Tool?

A: It helps evaluate AWS workloads against the six pillars of the AWS Well-Architected Framework, identifies high- and medium-risk issues, and provides recommendations to improve architecture according to AWS best practices.

AWS Customer Carbon Footprint Tool

What is the AWS Customer Carbon Footprint Tool?

The **AWS Customer Carbon Footprint Tool** helps customers:

- Track carbon emissions generated by AWS usage.
- Measure environmental impact over time.
- Review carbon savings achieved by using AWS.
- Forecast and monitor sustainability progress.
- Support corporate ESG (Environmental, Social, Governance) and sustainability initiatives.

Purpose

This tool aligns with the **Sustainability Pillar** of the AWS Well-Architected Framework by helping organizations understand and reduce their cloud-related carbon footprint.

Key Capabilities

1. Track Carbon Emissions

Provides visibility into the estimated carbon emissions associated with AWS workloads.

You can view:

- Total carbon emissions
- Emissions trends over time
- Historical usage data

2. Measure Carbon Savings

The tool estimates carbon savings from using AWS infrastructure compared to traditional on-premises environments.

Benefits include:

- More efficient data centers
- Shared infrastructure
- Renewable energy initiatives

3. Analyze Emissions by Service

Identify which AWS services contribute most to emissions.

Examples:

- Amazon EC2
- Amazon RDS
- Amazon EBS

- Other AWS services

This helps identify optimization opportunities.

4. View Emissions by Geography

Monitor emissions across:

- AWS Regions
- Geographic locations

Useful for organizations with global operations.

5. Monitor Sustainability Progress

Track:

- Environmental impact over time
- Emission reduction efforts
- Progress toward sustainability targets

6. Path to 100% Renewable Energy

AWS provides insights into its transition toward renewable energy usage.

The tool displays:

- Renewable energy progress
- Related sustainability metrics
- Long-term trends

Where to Find It

AWS Console Navigation

Billing and Cost Management

→

Customer Carbon Footprint Tool

Easier Method

Use the AWS Console Search Bar:

Search: Carbon

Then select:

Customer Carbon Footprint Tool

Information Available in the Dashboard

Time Range Selection

Choose:

- Start Month
- End Month

to analyze a specific period.

Emissions Summary

Displays:

- Total carbon emissions
- Overall environmental impact

Savings Information

Shows savings:

- By Geography
- By AWS Service

Emissions Trends

Visual charts display:

- Historical emissions
- Carbon footprint evolution over time

Renewable Energy Progress

Shows AWS's progress toward renewable energy goals.

Relationship to Sustainability Pillar

The tool directly supports the Well-Architected Framework's:

Sustainability Pillar

By helping organizations:

✅ Understand environmental impact

✅ Measure efficiency

✅ Reduce emissions

✅ Track sustainability goals

✅ Make data-driven optimization decisions

Exam Tips

What does the AWS Customer Carbon Footprint Tool do?

Answer: It helps customers measure, review, and track carbon emissions generated by their AWS usage and supports sustainability reporting and optimization efforts.

Key Benefits

- Tracks carbon emissions
- Measures carbon savings
- Provides emissions by service and geography
- Enables sustainability reporting
- Supports environmental goals

Quick Exam Summary

AWS Customer Carbon Footprint Tool = Measure + Track + Optimize Carbon Emissions

It helps organizations:

Understand Emissions

↓

Track Over Time

↓

Measure Savings

↓

Support Sustainability Goals

One-Line Exam Answer

The AWS Customer Carbon Footprint Tool provides visibility into the carbon emissions associated with AWS usage, helping organizations track sustainability metrics, carbon savings, and progress toward environmental goals.

Cloud Practitioner

AWS Cloud Adoption Framework (CAF)

What is AWS CAF?

AWS Cloud Adoption Framework (CAF) is an AWS guidance framework (whitepaper/e-book) that helps organizations:

- Plan cloud adoption.
- Execute digital transformation initiatives.
- Identify organizational and technical gaps.
- Accelerate business outcomes using AWS.

It combines AWS best practices learned from thousands of customer cloud adoption journeys.

Key Point: CAF is a framework, not an AWS service.

CAF Structure

CAF consists of:

1. **Organizational Capabilities**
2. **Six Perspectives**

These perspectives help organizations prepare for successful cloud adoption.

The 6 CAF Perspectives

Business Perspectives

1. Business

Ensures cloud investments support business goals and accelerate digital transformation.

Focus:

- Strategy
- Innovation
- Business outcomes
- Value creation

2. People ⭐ Exam Favorite

Acts as the **bridge between business and technology**.

Focus:

- Organizational culture
- Leadership
- Workforce transformation
- Skill development

- Change management

Exam Tip:

If a question asks which perspective serves as the bridge between business and technology, the answer is **People**.

3. Governance

Ensures cloud initiatives are controlled and aligned with organizational goals.

Focus:

- Risk management
- Compliance
- Financial management
- Portfolio management

Technical Perspectives

4. Platform

Provides the cloud foundation needed for workloads and applications.

Focus:

- Cloud architecture
- Data architecture
- Platform engineering
- CI/CD
- Modern application development

5. Security

Ensures confidentiality, integrity, and availability of workloads and data.

Focus:

- IAM
- Threat detection
- Data protection
- Infrastructure security
- Incident response

6. Operations

Ensures cloud services are delivered reliably and efficiently.

Focus:

- Monitoring
- Incident management
- Patch management
- Configuration management
- Performance management

Easy Mnemonic

Remember:

BPG PSO

- **B**usiness
- **P**eople
- **G**overnance
- **P**latform
- **S**ecurity
- **O**perations

Or:

Business People Govern Platforms Securely Operate

Capability Examples by Perspective

Business

Examples:

- Strategy Management
- Portfolio Management
- Innovation Management
- Product Management
- Data Science
- Business Insight

People

Examples:

- Culture Evolution
- Cloud Fluency
- Workforce Transformation

- Organizational Design
- Leadership

Governance

Examples:

- Risk Management
- Cloud Financial Management
- Benefits Management
- Data Governance
- Program Management

Platform

Examples:

- Platform Architecture
- Data Architecture
- Platform Engineering
- Data Engineering
- CI/CD
- Application Development

Security

Examples:

- Security Governance
- IAM
- Threat Detection
- Vulnerability Management
- Data Protection
- Incident Response

Operations

Examples:

- Observability
- Event Management
- Change Management

- Patch Management
- Availability Management
- Application Management

CAF Transformation Domains

CAF also defines four transformation domains.

1. Technology

Focus:

- Infrastructure modernization
- Application modernization
- Data and analytics modernization

Example:

- Migrating legacy applications to AWS.

2. Process

Focus:

- Digitization
- Automation
- Business optimization

Examples:

- Analytics-driven decisions
- Machine learning solutions
- Automated workflows

3. Organization

Focus:

- People
- Structure
- Operating model

Examples:

- Product-focused teams
- Agile methodologies
- DevOps culture

Common Exam Topic

4. Products

Focus:

- New business models
- New products and services
- New revenue streams

Examples:

- SaaS offerings
- Digital products
- Subscription services

CAF Transformation Phases

There are **4 phases** you must know for the exam.

1. Envision

Goal:

- Identify business opportunities.
- Build the case for cloud adoption.

Outcome:

- Vision for transformation.

2. Align

Goal:

- Evaluate the organization against the six CAF perspectives.
- Identify capability gaps.

Outcome:

- Action plan.

3. Launch

Goal:

- Implement pilot projects.
- Deliver initial business value.

Outcome:

- Production-ready pilot workloads.

4. Scale

Goal:

- Expand successful cloud initiatives.
- Achieve organization-wide benefits.

Outcome:

- Large-scale cloud adoption.

Simple Flow

Envision

↓

Align

↓

Launch

↓

Scale

Remember:

- **Envision** = Find opportunities
- **Align** = Find gaps and create plan
- **Launch** = Deploy pilots
- **Scale** = Expand adoption

Exam-Focused Summary

Six CAF Perspectives

✅ Business

✅ People (Bridge between business and technology)

✅ Governance

✅ Platform

✅ Security

✅ Operations

Four Transformation Domains

✅ Technology

✅ Process

✅ Organization

- ✅ Products

Four Transformation Phases

- ✅ Envision
- ✅ Align
- ✅ Launch
- ✅ Scale

Most Important Exam Facts

1. **CAF is a framework/whitepaper, not a service.**
2. **People Perspective = Bridge between business and technology.**
3. **Six Perspectives:** Business, People, Governance, Platform, Security, Operations.
4. **Transformation Phases:** Envision → Align → Launch → Scale.
5. **Transformation Domains:** Technology, Process, Organization, Products.

Right Sizing

What is Right Sizing?

Right Sizing is the process of selecting the **correct AWS resource type and size** that meets performance and capacity requirements at the **lowest possible cost**.

Goal

Provide enough resources for the workload without:

- Overprovisioning (too large → waste money)
- Underprovisioning (too small → poor performance)

Why is Right Sizing Important?

In AWS, you can easily:

- Scale up
- Scale down
- Change instance types

Therefore, choosing the biggest instance available is usually **not the best approach**.

Benefits

✅ Lower costs

✅ Better resource utilization

✅ Improved efficiency

✅ Reduced waste

✅ Better alignment with actual workload needs

Key Principle

Start Small and Scale Up

Instead of:

Launch largest instance

↓

Pay high costs

Use:

Launch smaller instance

↓

Monitor usage

↓

Scale if needed

Because AWS resources are elastic, you can adjust them over time.

Right Sizing is a Continuous Process

Right sizing is **not a one-time activity**.

Workload requirements change due to:

- Growth in users
- New features
- Seasonal demand
- Business changes

Therefore, resource usage should be reviewed regularly.

Example

An EC2 instance running at:

- 5% CPU usage for months

might be oversized and can potentially be downsized.

When Should You Perform Right Sizing?

1. Before Cloud Migration

A common mistake is:

On-Premises Server

↓

Move to AWS

↓

Choose largest EC2 instance

This often leads to unnecessary costs.

Best Practice

Before migration:

- Analyze current workloads
- Measure actual utilization
- Select appropriate AWS resources

2. Continuously After Migration

Once in AWS:

- Monitor performance

- Review utilization regularly
- Adjust resources as needed

Think of right sizing as an ongoing optimization process.

AWS Tools That Help with Right Sizing

Amazon CloudWatch

Provides metrics such as:

- CPU utilization
- Network usage
- Disk activity
- Memory metrics (with custom monitoring)

Helps determine whether resources are overutilized or underutilized.

AWS Cost Explorer

Provides:

- Cost analysis
- Utilization recommendations
- Rightsizing recommendations

Helps identify opportunities to reduce costs.

AWS Trusted Advisor

Provides recommendations for:

- Underutilized EC2 instances
- Idle resources
- Cost optimization opportunities

Third-Party Optimization Tools

Organizations may also use:

- Cloud management platforms
- FinOps tools
- Resource optimization tools

for advanced rightsizing recommendations.

Exam Tips

Key Ideas to Remember

- ✅ Match resources to workload requirements
- ✅ Start small and scale up
- ✅ Avoid overprovisioning
- ✅ Perform right sizing before cloud migration
- ✅ Continue right sizing after migration
- ✅ Use CloudWatch, Cost Explorer, and Trusted Advisor

Quick Exam Summary

Right Sizing = Selecting the smallest resource that safely meets workload requirements, then continuously monitoring and adjusting to achieve the lowest possible cost while maintaining performance.

Helpful Memory Phrase

"Start Small → Monitor → Right Size → Save Money"

Common Exam Answer

Which AWS tools help with right sizing?

- Amazon CloudWatch
- AWS Cost Explorer
- AWS Trusted Advisor

These tools help identify underutilized or oversized resources and recommend cost-effective alternatives.

AWS Ecosystem

AWS provides a large ecosystem of resources, support options, training, partners, and marketplaces to help customers build, learn, and operate on AWS.

1. AWS Learning Resources

AWS Blog

Provides:

- New feature announcements
- Service updates
- Best practices
- Architecture guidance
- Technical deep-dives

Useful for staying current with AWS innovations.

AWS Community Forums

A place where you can:

- Ask questions
- Share solutions
- Learn from other AWS builders
- Discuss AWS services

AWS Whitepapers & Guides

In-depth documentation covering:

- Security
- Architecture
- Best practices
- Migrations
- Cost optimization
- Industry solutions

Example

- AWS Well-Architected Framework Whitepaper
- AWS CAF Whitepaper

AWS Solutions Library (formerly Quick Starts)

Provides pre-built, AWS-vetted solutions and architectures.

Examples:

- Live Streaming on AWS
- WordPress Deployment
- Data Lakes
- Analytics Platforms

Benefits:

✅ Ready-made architectures

✅ Best-practice implementations

✅ CloudFormation-based deployments

✅ Faster implementation

2. AWS Support Plans

AWS offers multiple support levels.

Developer Support

Best for:

- Development and testing environments

Includes:

- Business-hours email support
- Cloud Support Associates

Response Times:

- General Guidance: < 24 hours
- System Impaired: < 12 hours

Business Support

Best for:

- Production workloads

Includes:

- 24/7 phone support
- Email support
- Chat support
- Access to Cloud Support Engineers

Response Times:

- System Impaired: < 4 hours

- Production System Down: < 1 hour

Enterprise Support

Highest support tier.

Includes:

Technical Account Manager (TAM)

A designated AWS expert familiar with your environment.

Concierge Support Team

Helps with:

- Billing
- Account management
- Best practices

Response Time:

- Business-Critical System Down: < 15 minutes

Support Plan Quick Comparison

Feature	Developer	Business	Enterprise
Email Support	✅	✅	✅
Phone Support	❌	✅	✅
Chat Support	❌	✅	✅
TAM	❌	❌	✅
Concierge Team	❌	❌	✅
Critical Response Time	12 hrs	1 hr	15 min

3. AWS Marketplace

What is AWS Marketplace?

A digital catalog containing thousands of solutions from third-party software vendors.

Think of it as:

An app store for AWS solutions.

What Can You Buy?

AMIs

Preconfigured machine images.

Examples:

- Security-focused Linux AMIs
- Databases
- Monitoring software

CloudFormation Templates

Pre-built production-ready architectures.

SaaS Solutions

Examples:

- Security software
- Monitoring tools
- Collaboration platforms

Containers

Prebuilt container images for ECS/EKS workloads.

Marketplace Benefits

✅ Fast deployment

✅ AWS billing integration

✅ Trusted vendors

✅ Production-ready software

Selling on Marketplace

Organizations can also become AWS Marketplace sellers and offer their own products.

4. AWS Training and Certification

AWS offers multiple learning options.

Digital Training

Online self-paced learning.

Benefits:

- Learn anywhere
- Flexible schedule

- Beginner to advanced courses

Classroom Training

Available:

- In person
- Virtual instructor-led

Private Training

Customized AWS training for organizations.

Government Training Programs

Specialized AWS training and certification paths for government organizations.

Enterprise Training Programs

Designed for large organizations adopting AWS at scale.

AWS Academy

AWS Academy helps universities teach cloud computing.

Benefits:

- AWS curriculum
- Hands-on labs
- Cloud skills development

Goal:

- Prepare students for cloud careers.

5. AWS Professional Services

AWS Professional Services Organization

A global team of AWS experts.

They help customers:

- Migrate workloads
- Design architectures
- Modernize applications
- Accelerate cloud adoption

They work directly with customer teams.

6. AWS Partner Network (APN)

What is APN?

AWS Partner Network is a global community of AWS-approved partners.

APN Technology Partners

Provide:

- Hardware solutions
- Software solutions
- Integrations
- Connectivity solutions

Focus:

- Technology products

APN Consulting Partners

Provide:

- Consulting services
- Architecture guidance
- Cloud implementation
- Migration expertise

Focus:

- Professional services

APN Training Partners

Deliver:

- AWS training
- Certification preparation
- AWS education programs

AWS Competency Program

Recognition awarded to partners who demonstrate:

- Technical expertise
- AWS specialization
- Proven customer success

Examples:

- Security competency
- Migration competency
- Data Analytics competency

APN Navigate Program

Helps AWS partners:

- Improve capabilities
- Grow AWS expertise
- Achieve AWS competencies

Essentially:

A program that helps partners become better AWS partners.

Exam-Focused Summary

AWS Ecosystem Components

Learning Resources

✅ AWS Blog

✅ Community Forums

✅ Whitepapers & Guides

✅ Solutions Library

Support Plans

✅ Developer

✅ Business

✅ Enterprise

Marketplace

✅ AMIs

✅ SaaS Products

✅ Containers

✅ CloudFormation Templates

Training

✅ Digital Training

✅ Classroom Training

✅ AWS Academy

Professional Services

✅ AWS Professional Services Organization

Partner Network (APN)

✅ Technology Partners

✅ Consulting Partners

✅ Training Partners

✅ Competency Program

✅ Navigate Program

Most Important Exam Facts

AWS Solutions Library

- Prebuilt, AWS-vetted architectures and solutions.

AWS Marketplace

- Buy third-party software and solutions.

AWS Professional Services

- AWS experts help customers implement AWS projects.

APN Consulting Partners

- Help customers build and migrate workloads.

APN Technology Partners

- Provide hardware/software products.

APN Training Partners

- Deliver AWS training.

Enterprise Support

- Includes a **Technical Account Manager (TAM)** and **15-minute response time** for business-critical issues.

Quick Memory Aid

Learn → Support → Buy → Build

- **Learn:** Blog, Whitepapers, Academy
- **Support:** Developer, Business, Enterprise
- **Buy:** Marketplace
- **Build:** Professional Services & APN Partners

AWS re:Post

What is AWS re:Post?

AWS re:Post is an AWS-managed **community Q&A platform** where users can:

- Ask technical AWS questions
- Receive answers from the community
- Learn AWS best practices
- Participate in AWS-focused groups and discussions
- Share knowledge and build reputation

It serves as the successor to the original AWS forums.

Think of **AWS re:Post** as AWS's version of Stack Overflow for cloud-related questions.

Key Features

Community Q&A

Users can:

- Post questions
- Answer questions
- Comment on discussions
- Vote on helpful answers

Questions are often reviewed by AWS experts and experienced community members.

Expert-Reviewed Answers

Responses may be:

- Community-generated
- Expert-reviewed
- AWS-reviewed

This helps improve answer quality and reliability.

Build Reputation

Active contributors can earn reputation by:

- Providing useful answers
- Participating in discussions
- Helping other AWS users

Higher reputation helps establish expertise within the AWS community.

AWS Groups and Discussions

Users can join specialized communities around:

- AWS services
- Architectures
- Industry solutions
- Certifications
- Best practices

Premium Support Integration

For customers with AWS support plans:

If a question posted on re:Post does not receive an adequate community response, it may be routed to AWS Support Engineers for assistance.

This provides additional access to AWS expertise.

When to Use AWS re:Post

Use re:Post for:

✅ Technical questions

✅ Best practices

✅ Architecture guidance

✅ Learning AWS services

✅ Troubleshooting advice

✅ Community discussions

When NOT to Use AWS re:Post

AWS re:Post is **not appropriate for:**

Time-Sensitive Issues

Example:

- Production application is down
- Critical business outage

Use AWS Support instead.

Proprietary or Account-Specific Actions

Examples:

- Billing issues
- Security incidents
- Account access problems
- Confidential workload details

These should be handled through AWS Support channels.

AWS re:Post vs AWS Support

AWS re:Post

- Community-based
- Public questions
- Knowledge sharing
- Best practices
- Not guaranteed immediate response

AWS Support

- Direct AWS assistance
- Private support cases
- Time-sensitive issues
- Production-impacting incidents
- Account-specific help

Exam Tips

What is AWS re:Post?

Answer: An AWS-managed community forum and Q&A service that provides crowdsourced and expert-reviewed answers to technical AWS questions.

Key Point

re:Post is for general technical questions and community support, not for urgent production issues or proprietary account-specific requests.

Quick Exam Summary

AWS re:Post

✅ Community Q&A platform

✅ AWS-managed service

✅ Expert-reviewed answers

✅ Reputation system

✅ Best-practice discussions

✅ Similar to Stack Overflow

Not Intended For

❌ Urgent production outages

❌ Time-sensitive requests

❌ Confidential account issues

❌ Proprietary support cases

One-Line Exam Answer

AWS re:Post is AWS's community-driven question-and-answer platform where users can ask technical AWS questions, receive expert-reviewed responses, share knowledge, and learn best practices, but it is not designed for urgent or account-specific support needs.

Cloud Practitioner

AWS Knowledge Center

What is AWS Knowledge Center?

The **AWS Knowledge Center** is a self-service knowledge base available through **AWS re:Post** that provides answers to common AWS questions, troubleshooting guidance, and best practices.

Think of it as:

A collection of AWS-curated FAQs, troubleshooting articles, and how-to guides.

Purpose

The Knowledge Center helps users:

- Troubleshoot AWS issues
- Find solutions to common problems
- Learn AWS best practices
- Follow step-by-step resolution procedures
- Quickly resolve frequently encountered AWS scenarios

Types of Content

The Knowledge Center contains articles covering many AWS services and topics, such as:

Popular AWS Services

- Amazon EC2
- Amazon S3
- Amazon RDS
- AWS Lambda
- Amazon VPC
- Amazon CloudFront

Other Categories

- Analytics
- Application Integration
- Security
- Networking
- Storage
- Databases
- Monitoring

Typical Article Structure

A Knowledge Center article usually includes:

Question

Example:

Why isn't Amazon S3 replication working between my buckets?

Resolution

Provides:

- Possible causes
- Troubleshooting steps
- Configuration checks
- Recommended solutions

Supporting Information

- AWS documentation links
- Best practices
- Additional references

Search and Filtering Features

Users can:

✅ Search by keywords

✅ Browse by AWS service

✅ Filter by tags

✅ View popular articles

✅ View newly created articles

✅ Explore troubleshooting categories

Relationship with AWS re:Post

The AWS Knowledge Center is part of the **AWS re:Post ecosystem**.

Difference

AWS re:Post

- Community Q&A
- User-generated questions
- Expert-reviewed answers

AWS Knowledge Center

- Official AWS troubleshooting articles
- Frequently asked questions
- Curated best-practice guidance

When to Use AWS Knowledge Center

Use it when you need:

- Troubleshooting guidance
- Answers to common AWS questions
- Configuration help
- Service best practices
- Step-by-step resolutions

Exam Tips

What is AWS Knowledge Center?

Answer: A collection of official AWS troubleshooting articles, FAQs, and best-practice guidance available through AWS re:Post.

Common Exam Scenario

A customer wants to find solutions for a common AWS issue without opening a support case. What should they use?

✅ **AWS Knowledge Center**

Quick Exam Summary

AWS Knowledge Center

✅ Part of AWS re:Post

✅ Official AWS knowledge base

✅ Troubleshooting articles

✅ Frequently asked questions

✅ Best practices and resolutions

✅ Searchable by service and topic

One-Line Exam Answer

AWS Knowledge Center is a repository of AWS-curated troubleshooting articles, FAQs, and best practices available through AWS re:Post, designed to help customers resolve common AWS issues and learn recommended solutions.

AWS Managed Services (AMS)

What is AWS Managed Services (AMS)?

AWS Managed Services (AMS) is **not an AWS managed service like RDS or Lambda**.

Instead, it is a **managed operations offering provided by AWS experts** who help customers operate and manage their AWS environments.

Think of it as:

A team of AWS professionals that manages your AWS infrastructure on your behalf.

Purpose of AMS

AMS helps organizations:

- Operate AWS environments securely
- Improve reliability and availability
- Reduce operational overhead
- Focus on business goals instead of infrastructure management

It is particularly useful for organizations that:

- Lack AWS expertise
- Need operational support
- Want AWS best practices implemented
- Require 24/7 operations management

What Does AMS Do?

The AMS team manages day-to-day infrastructure operations, including:

Monitoring

- Infrastructure monitoring
- Performance monitoring
- Incident monitoring

Patch Management

- OS patching
- Security updates
- Maintenance activities

Backup Services

- Backup management
- Recovery planning
- Data protection

Security Operations

- Security monitoring
- Security controls
- Best-practice implementation

Change Management

- Infrastructure changes
- Configuration management
- Operational governance

Availability Management

- Maintaining uptime
- Reliability improvements
- Operational support

Key Benefits

Improved Security

AMS follows AWS security best practices and continuously monitors environments.

Automation

Many operational tasks are automated, reducing manual work and human error.

Stronger Compliance

Helps organizations maintain operational and compliance standards.

Reduced Operating Costs

AWS experts manage infrastructure efficiently, reducing internal operational effort.

Simplified Management

Organizations can focus on applications and innovation rather than infrastructure operations.

Frictionless Innovation

Teams spend less time maintaining systems and more time delivering business value.

AMS Availability

The AMS team provides:

- ✅ 24/7 support
- ✅ 365 days a year operations
- ✅ Continuous monitoring and management

AMS Lifecycle Support

AMS helps customers through multiple stages of cloud adoption:

1. Establish a Baseline

Create a secure and operationally sound AWS environment.

2. Build or Migrate

Support application deployment and cloud migration initiatives.

3. Sustain

Maintain ongoing operations and governance.

4. Operate

Manage production environments continuously.

How Do You Get AMS?

AMS is **not activated from the AWS Console**.

Instead:

1. Contact AWS Sales or AWS representatives.
2. Engage the AWS Managed Services team.
3. AWS helps design and manage the operational model.

AWS Managed Services vs AWS Managed Service (RDS, Lambda, etc.)

AWS Managed Services (AMS)

- Team of AWS experts
- Operational service
- Human and automated management
- Supports entire AWS environments

AWS Managed Services like RDS

- AWS manages underlying infrastructure
- No dedicated operations team
- Product/service offering

Exam Tip: Don't confuse **AMS** with AWS managed services such as RDS, DynamoDB, Lambda, or S3.

Exam-Focused Summary

AWS Managed Services (AMS)

✅ Team of AWS experts

✅ Manages AWS infrastructure and operations

- ✅ 24/7/365 support
- ✅ Handles monitoring, patching, backups, security, and maintenance
- ✅ Helps reduce operational overhead
- ✅ Improves security, reliability, and compliance
- ✅ Engaged through AWS, not directly from the console

One-Line Exam Answer

AWS Managed Services (AMS) is a service provided by AWS experts that operates and manages customers' AWS infrastructure, handling monitoring, patching, backups, security, and operational tasks to improve reliability, security, and efficiency.